CGE-P LAB NOTES

The Complete Curriculum

Every guide in one volume: the Plain English companion, the curriculum overview, twelve labs and the capstone brief.

Each section is identical to its standalone PDF.

For reading. Code lines wrap to fit the page; copy code from the matching file in docs/, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

Contents

CGE-P LAB NOTES

The Plain English Guide

Read this before the labs, or alongside them when something looks like magic.

The labs tell you what to type. This tells you what it means, where every piece came from, and which choices are actually yours to make. Nothing in the curriculum should be a string you paste because a guide said so. If you find something here that is still mysterious, that is a bug in this document, not a gap in you.

For reading. Code lines wrap to fit the page; copy code from docs/00_00_plain_english_guide.md, not the PDF.

GRC Engineering Club

Companion to Labs 0.1 through 7.1

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/00_00_plain_english_guide.md` in your clone, not from the PDF.

Contents

1. What the whole thing is for

Here is the entire curriculum in one sentence:

> **Build a system where an auditor can verify your security controls without talking to you, and without trusting you.**

That is it. Everything else is detail.

Think about how compliance normally works. An auditor asks for evidence. Someone takes a screenshot of a console. The screenshot goes into a folder. Six months later nobody can tell whether the setting is still there, whether the screenshot was taken from production or a test account, or whether it was edited. The auditor's only real option is to trust the person who handed it over.

Every piece of this course attacks one weakness in that story:

Weakness in the screenshot approach	What the course does instead	Which chapter
"Was it configured correctly?"	Write it as code, so the configuration *is* the document	Ch 2
"Is it still configured correctly?"	Check it automatically on every change	Ch 3

Weakness in the screenshot approach	What the course does instead	Which chapter
"Who checked, and when?"	Run the check in a pipeline that records itself	Ch 4
"Could someone have edited the evidence?"	Sign it and store it where deletion is impossible	Ch 4
"Is anything else broken?"	Turn on the platform's own monitoring	Ch 5
"Which control does this even satisfy?"	Publish a machine-readable map from control to evidence	Ch 6

By the end, an auditor reads one JSON file, follows a link, runs one command, and sees `CHAIN INTACT`. You are not in the room. That is the deliverable.

Why it is built on Amazon and Google

The controls are the constant. The resources are not. `SC-28` means "protect data at rest" whether you are on AWS, GCP, Azure, or a server in a closet. But the *code* that satisfies it looks completely different on each.

Doing the same control twice, in two clouds, is the fastest way to internalize that the control ID is the durable thing and the resource type is disposable. It is also why your Rego policies in Chapter 3 need separate AWS and GCP variants that share one control ID.

2. The vocabulary

Plain definitions for the words that get used as though everyone already knows them.

Control. A requirement, written down, that reduces risk. "Data must be encrypted at rest" is a control. It is not a technology. Twelve different technologies could satisfy it.

Control family. NIST groups controls by two-letter prefix. You will meet these:

Prefix	Family	Rough meaning
`AC`	Access Control	Who can reach what
`AU`	Audit and Accountability	Logging, and doing something with the logs
`CM`	Configuration Management	Known-good settings, and knowing what you have
`CP`	Contingency Planning	Backup and recovery
`IA`	Identification and Authentication	Proving who you are
`RA`	Risk Assessment	Scanning, finding weaknesses
`SC`	System and Communications Protection	Encryption, boundaries

Prefix	Family	Rough meaning
`SI`	System and Information Integrity	Making sure things have not been tampered with
`SR`	Supply Chain Risk Management	Trusting the things you install

Control ID. `SC-28` is the 28th control in the System and Communications Protection family. `SC-28(1)` with a number in parentheses is a **control enhancement**, a stricter version of the base control. `SC-28` says protect data at rest. `SC-28(1)` says do it with cryptography specifically.

Framework. A curated list of controls someone decided you need. NIST 800-53 is a catalog of about a thousand controls. HIPAA, SOC 2, and CMMC are different lists for different purposes. They overlap heavily, because there are only so many ways to secure a system.

Catalog, in OSCAL terms. The full library of controls, published as a file. NIST publishes 800-53 as JSON. You link to it; you do not copy it.

Profile. A subset of a catalog. "These 12 controls out of the thousand are the ones that apply to us."

Baseline. A pre-made profile. NIST publishes Low, Moderate, and High baselines. This course builds its own small profile instead, because a Moderate baseline is around 300 controls and that is not a lab.

Preventive vs detective control. A preventive control stops the bad thing from happening. A detective control tells you it happened. A validation rule that rejects a bad value at plan time is preventive. A Security Hub finding three hours after deployment is detective. Preventive is stronger and usually cheaper; detective catches what prevention missed. You want both, and you should always be able to say which of yours is which.

Evidence. Something that demonstrates a control is working. A screenshot is weak evidence. A signed JSON file whose hash you can recompute is strong evidence. The difference is whether verifying it requires trusting anybody.

Attestation. A statement that something is true, in a form a machine can read. When Lab 2.3 outputs `encryption_algorithm = "aws:kms"`, that is an attestation.

Chain of custody. Four properties, together: it has not been altered (integrity), you know who made it (authenticity), you know when (timeliness), and nothing was quietly removed (completeness). Lab 4.4 implements all four, and most real-world pipelines implement two.

Infrastructure as Code (IaC). Describing your servers, buckets, and permissions in text files instead of clicking in a console. The text file becomes reviewable, versionable, and diffable, which is why it can serve as evidence.

Policy as Code. Writing your rules as a program that inspects a proposed change and approves or rejects it. Your compliance requirements become software that runs.

Drift. When reality stops matching the code. Someone changed a setting in the console; your Terraform no longer describes what exists.

Idempotent. Running it twice produces the same result as running it once. Terraform is idempotent: apply the same code repeatedly and nothing further changes.

Confused deputy. An attack where you trick a trusted service into doing something on the attacker's behalf. The service has permission; the attacker does not; the attacker uses the service as a puppet. This is why so many policies in these labs carry `aws:SourceArn` and `aws:SourceAccount` conditions.

Pinning. Naming an exact version of something you depend on, rather than a range or a moving label. A commit SHA for a GitHub Action, an exact version in `requirements.txt`, a provider hash in `.terraform.lock.hcl`. It buys you reproducibility and protection from an upstream changing under you, and it costs you the upstream's fixes until you deliberately move. See section 5's note on Chapter 4: pinning without a refresh process trades one risk for a quieter one.

Blast radius. How much breaks when something goes wrong. A sandbox account has a small blast radius, which is why Lab 0.1 insists on one.

3. The machinery around the labs

None of this is compliance. It is the scaffolding the labs stand on, and it is where most people lose their first evening, so it is worth understanding rather than pattern-matching.

3.1 The container, and why there is one

The labs use ten command-line tools. Installing them by hand means ten downloads, each with a different naming convention, and the instructions are written for one operating system on one processor architecture. Every Mac sold since 2020 uses a different architecture from the one those instructions assume, so every download line is subtly wrong on a Mac and completely wrong on Windows.

A **container** is a packaged filesystem with the tools already in it. You run it, and you are in a shell where `terraform` and `aws` already exist at the exact versions this curriculum was written against. Nothing is installed on your own machine, and nothing you do inside it can install anything on your own machine either.

Two ways to get one, from the same definition:

- **A Codespace** is that container running on GitHub's machines, with VS Code in a browser tab. Nothing is installed locally at all.
- **Locally**, Docker runs the same container on your machine and VS Code attaches to it.

The thing worth knowing, because it looks like magic and is not: the repository folder is *mounted* into the container rather than copied. Files you edit inside are your real files. Delete the container and your work is untouched.

Your cloud credentials work the other way round, and deliberately so. The container gets its own `~/.aws` on a separate volume rather than seeing yours. That is the safer arrangement: a real `~/.aws` collects every profile you have ever configured, work and production included, and sharing it with a container shares all of them with everything running in there. The container needs one sandbox account, so it gets exactly that.

The cost is that you log in twice, once on your machine and once inside, and that is the correct trade rather than an inconvenience to engineer away.

3.2 Why logging in from a container is different

`aws login` and `gcloud auth login` both work by opening a web browser. A container has no browser and no way to open one on your machine, so both hang or fail in ways that do not mention browsers at all.

Every such tool has a flag for this, because the problem is universal:

Command	Flag
`aws login`	`--remote`
`gcloud auth login`	`--no-launch-browser`
`gcloud auth application-default login`	`--no-launch-browser`

They all do the same thing: print a URL for you to open yourself, then wait while you paste back the code it gives you. The gcloud pair catches people because both commands need the flag, and the second is easy to forget.

3.3 How Terraform finds its inputs

A Terraform variable declared without a `default` has to come from somewhere. There are three somewheres, and knowing all three explains most confusing behavior:

How	Looks like	When it is used
On the command line	`-var project_name=cgep-lab`	One-off, explicit, easy to forget
A file it reads automatically	`terraform.tfvars`	Per workspace
An environment variable	`TF_VAR_project_name=cgep-lab`	Everywhere, once

The third is why this curriculum uses a file called `cgep.env`. `project_name` is an input to four different labs and `aws_region` to five. Typing them into each is how you end up with a bucket in the wrong region, or two labs that disagree about what your project is called, and neither mistake announces itself.

So you set them once, in a file you `source` at the start of a session, and Terraform picks them up on its own. There is no magic in it: Terraform reads any environment variable beginning `TF_VAR_` and strips the prefix to get the variable name.

If Terraform asks you for a value interactively, that is the tell. It means none of the three routes supplied it. Answering the prompt works once and teaches you nothing; the fix is to work out which route you meant to use.

`cgep.env` is deliberately not committed. It names your account, your project and your repository. It is configuration rather than a credential, but it is still yours.

3.4 Why the copy has to be yours

The capstone is graded on a public GitHub repository that you submit by URL. So your work has to live under your account from the first commit.

Cloning someone else's repository gives you their code pointed at *their* remote. Everything works right up until your first `git push`, which is refused, usually at the moment you have something worth keeping.

Use this template creates a fresh repository under your account with no relationship to the original. **Fork** does the same job but records that it came from somewhere else, which is fine for contributing back and slightly odd for work you are presenting as your own.

`git remote -v` tells you where a push would go. If your username is not in that URL, you are about to be refused.

Two things that are separate and are routinely confused: **your git identity** (`git config user.name` and `user.email`, which is what commits are labeled with) and **your authentication** (which is what proves you may push). Setting the first does not give you the second. Password authentication over HTTPS stopped working years ago, so `gh auth login` is the short path, and inside a container you want its device-code option for the reason in 3.2.

3.5 Repository variables are not secrets

GitHub gives a repository two places to keep values, and the difference matters more than it looks.

A **variable** is configuration. You can read it back, it shows up in logs, and that is a feature when a workflow fails and you need to see what it actually used. A role ARN and a bucket name belong here: they *identify* things, they do not grant access to them.

A **secret** is write-only and masked in logs. That is correct for a credential and unhelpful for anything else. Putting non-secrets in `secrets` is a common habit, and it quietly teaches that everything is equally sensitive, which is precisely how genuine credentials end up handled casually.

The stronger point is what the pipeline lab stores in neither: **no access key, anywhere**. The OIDC trust means GitHub proves which repository and which branch is asking, and AWS hands back credentials that last minutes. There is no long-lived secret to leak, rotate, or accidentally print. A pipeline that pastes an access key into `secrets` is doing the same job far less safely, and the difference is the entire lesson of Lab 4.3.

3.6 What a saved plan is, and why it goes stale

Most labs run `terraform plan -out=tfplan` and then `terraform apply tfplan`, rather than a bare `apply`. The reason is worth understanding, because it is the same reason it sometimes refuses to run.

A bare `terraform apply` computes a plan and immediately carries it out. Passing a saved plan splits that in two: the plan is a **file recording exactly which API calls Terraform intends to make**, and apply performs precisely those and nothing else. What you reviewed is what runs. In a pipeline that difference is the whole control, and it is why Lab 4.3 gates on a plan rather than on an apply.

The cost is that a saved plan is a promise about a particular state. If the state moves after the plan is written, the promise no longer holds, and Terraform says so:

```
Error: Saved plan is stale
The given plan file can no longer be applied because the state was changed
by another operation after the plan was created.
```

That is the safety feature working, not a fault. Something touched the state between your plan and your apply: another `terraform` command in that workspace, a colleague, a pipeline, or simply a `terraform output` or `refresh` that recorded a data source. Terraform cannot tell a harmless change from a dangerous one, so it refuses rather than guessing.

The fix is always the same: plan again, read it again, apply again. Nothing is broken and nothing is lost. If the new plan differs from the old one, that difference is exactly what you were being protected from, so read it rather than skim it.

Two habits that avoid it: do not leave a saved plan sitting for hours before applying it, and do not run Terraform against the same workspace from two places at once.

3.7 What is safe to publish

Your evidence *is* your portfolio, so most of it is meant to be public. Three things to be deliberate about:

Terraform state is the real risk. State records every attribute the provider stored, including ones marked sensitive, so a workspace that manages a database password has that password in its state. This is exactly the trap Lab 2.5 is built around. Look before you publish, and publish on purpose.

Your account ID will be in there, in every ARN, unavoidably. It is not a credential and publishing it is not a breach, but it does help someone enumerate your roles. Decide once whether you are comfortable with that; if not, keep the repository private and share it with reviewers directly.

Bucket names are globally unique and yours. Publishing them tells the world which buckets to probe. The controls you built in Lab 2.3 are exactly what makes that survivable, which is worth saying in your write-up rather than hiding.

4. The decoder ring

This section exists because of one line in the brief for this guide: nothing should be a mystery as to where it comes from. Every odd-looking string in the labs is below, with its source.

4.1 Amazon Resource Names (ARNs)

An ARN is AWS's universal address for a thing. The shape:

```
arn:partition:service:region:account-id:resource
 |     |        |       |        |         |
 |     |        |       |        |         +-- what it is
 |     |        |       |        +------------ whose it is (12 digits)
 |     |        |       +---------------------- where it lives
 |     |        +------------------------------ which AWS service
 |     +--------------------------------------- aws, aws-us-gov, or aws-cn
 +--------------------------------------------- always the literal "arn"
```

So why do S3 buckets look like this, with two empty fields?

```
arn:aws:s3:::my-bucket-name
```

Because **S3 bucket names are globally unique across every AWS account on Earth.** There is only one `my-bucket-name` anywhere. It needs no region and no account number to be unambiguous. That emptiness is not a typo, it is a statement about how S3 naming works.

Compare a KMS key, which is regional and account-scoped, so every field is populated:

```
arn:aws:kms:us-east-1:123456789012:key/1234abcd-12ab-34cd-56ef-1234567890ab
```

Why the labs use `arn:${local.partition}:...` instead of typing `arn:aws:`. Most of the world is in the `aws` partition. GovCloud is `aws-us-gov` and China is `aws-cn`, and an ARN with the wrong partition silently matches nothing. The `data "aws_partition" "current" {}` data source asks AWS which partition you are actually in. It costs nothing and makes the code correct everywhere.

4.2 `arn:aws:iam::123456789012:root` does not mean the root user

This is the single most misread string in the curriculum.

In a **KMS key policy** or a **bucket policy**, `arn:aws:iam::ACCOUNT_ID:root` means **"the AWS account itself,"** as a container. It is shorthand for "any principal in this account that also has IAM permissions allowing the action." It does **not** mean the root login.

That is why the labs put this in every key policy:

```
statement {
  sid       = "EnableAccountRootAdmin"
  effect    = "Allow"
  actions   = ["kms:*"]
  resources = ["*"]
  principals {
    type        = "AWS"
    identifiers = ["arn:aws:iam::123456789012:root"]
  }
}
```

Without it, the key is only usable by whoever the policy explicitly names, and **there is no way to add anybody later**, because changing the key policy requires permission that only the key policy can grant. You have created a key nobody can administer. AWS lets you do this. It is not recoverable.

Meanwhile, in Lab 0.1 when we say "harden the root user, enable MFA, never use it again," that is the actual root login, a different thing entirely, identified by an email address rather than an ARN.

And that second meaning has a trap. `aws login` reuses whatever console session you happen to be signed into, and it will offer you the **root** session without flagging that as a bad idea. Accept it and every `terraform apply` in this curriculum runs as root: unrestricted, unable to be constrained by any IAM policy, and impossible to scope down afterward.

It also quietly voids the exercise. Lab 4.3's OIDC role is scoped so CI can plan but not apply. Lab 2.5's vault denies `s3:DeleteBucket` to everyone except the account root. Lab 2.2 grants the account root key administration as a safety net. Operate as root by default and all three become decorative, and you spend four chapters generating evidence that controls work when nothing was ever subject to them.

Sign in as a normal IAM user first, then check:

```
aws sts get-caller-identity
```

If the `Arn` ends in `:root`, stop and fix it before applying anything.

4.3 Service principals

Strings like `logging.s3.amazonaws.com` are **service principals**: the identity a piece of AWS assumes when it acts on your behalf.

The ones in this curriculum:

Principal	Who it is	Appears in
`logging.s3.amazonaws.com`	S3's server access logging system, writing your log files	Lab 2.3
`cloudtrail.amazonaws.com`	CloudTrail, writing trail files	Lab 5.2
`cloud-storage-analytics@google.com`	Google's equivalent, a fixed group, not a service account you create	Lab 2.4

Where do you find these? You do not derive them or guess them. AWS documents the correct principal in the page for each feature, and using the wrong one produces a permission error rather than a security hole. The Google one is a literal, unchanging group address that Google publishes.

The point to internalize: when a bucket policy says "allow `logging.s3.amazonaws.com` to PutObject," you are not granting access to a person or a role. You are granting it to a robot inside AWS that will act only when a specific feature is switched on.

4.4 Condition keys

Conditions are how an IAM policy says "only when." There are two kinds.

Global condition keys start with `aws:` and work on any service:

Key	Plain meaning	Where used
`aws:SecureTransport`	Was this request made over HTTPS? `false` means plain HTTP	The TLS deny in every bucket policy
`aws:SourceArn`	Which specific resource caused a service to make this call	Confused-deputy protection
`aws:SourceAccount`	Which account owns the thing that caused the call	Confused-deputy protection
`aws:PrincipalArn`	Who is asking	The "only root may delete this bucket" rule in Lab 2.5

Service-specific keys start with the service name:

Key	Where it comes from
`s3:x-amz-server-side-encryption`	Literally the HTTP header `x-amz-server-side-encryption` on the upload request
`s3:x-amz-server-side-encryption-aws-kms-key-id`	The header naming which KMS key

That second group is worth pausing on: those keys exist because S3's API is HTTP, and the condition is inspecting a header on the actual request. It is not abstract. If you ran the upload with `curl`, you would be setting that header yourself.

The trap this creates. When a Deny statement uses `StringNotEquals` on a header, and the request does not send that header at all, IAM evaluates "not equals" as **true**, so the Deny fires. That is why Lab 2.3 warns that a plain `aws s3 cp` gets rejected: the file would have been encrypted anyway by the bucket default, but the request did not *say* so, and the policy demands that it say so.

4.5 The GitHub OIDC strings

Chapter 4 replaces stored AWS keys with a trust relationship. Three strings do the work.

`https://token.actions.githubusercontent.com` is **GitHub's OIDC issuer**. It is the URL where GitHub publishes the keys that prove a token really came from GitHub Actions. AWS fetches from it to verify signatures.

`sts.amazonaws.com` is the **audience**: who the token is intended for. A token minted for one audience cannot be replayed against another. AWS refuses tokens not addressed to it.

`repo:OWNER/REPO:ref:refs/heads/main` is the **subject claim**, a string GitHub builds and puts inside the token. Its format is GitHub's, not AWS's, and it varies by trigger:

Trigger	`sub` looks like
Push to a branch	`repo:OWNER/REPO:ref:refs/heads/main`
Pull request	`repo:OWNER/REPO:pull_request`
Deployment environment	`repo:OWNER/REPO:environment:production`
Tag	`repo:OWNER/REPO:ref:refs/tags/v1.0`

This is why the trust policy uses `StringLike` with `repo:OWNER/REPO:*`: it covers every trigger. And it is why the capstone can bind the *apply* role to `:environment:production` specifically, so only a job running in a protected environment gets those credentials. **The wildcard you must never write is `repo:*:*`, which trusts every repository on GitHub.**

The `thumbprint_list` value `6938fd4d98bab03faadb97b34396831e3780aea1` is a fingerprint of GitHub's certificate chain. AWS now maintains this trust internally for the GitHub provider, so the value is no longer load-bearing, but the argument is still required. It is one of the few genuinely vestigial strings in the curriculum, and worth knowing is vestigial so you do not worry about rotating it.

4.6 `resources = ["*"]` inside a key policy

This confuses everyone exactly once.

In an IAM policy attached to a user or role, `Resource: "*"` means "every resource in the account." Alarming.

In a **KMS key policy**, the policy is attached to one specific key, and `"*"` means **"this key."** It cannot mean anything else, because a key policy has no reach beyond its own key. Seeing `kms:*` on `"*"` in a key policy is normal and correct.

4.7 S3 setting names

Value	What it actually does
`BucketOwnerEnforced`	ACLs are off entirely. Only IAM and bucket policies decide access. **The modern default and what the labs use.**
`BucketOwnerPreferred`	ACLs still work, but new objects belong to the bucket owner. Tempting, because it lets you apply a log-delivery ACL.
`ObjectWriter`	The uploader owns what they upload. The old default, and a way to end up with objects you cannot read in your own bucket.
`AES256`	S3 manages the key. Free, no configuration, nothing for you to rotate or audit.
`aws:kms`	A KMS key manages it. Costs $1/month, and you control rotation, policy, and who can decrypt.
`GOVERNANCE`	Object Lock retention that a sufficiently privileged caller can bypass
`COMPLIANCE`	Object Lock retention that **nobody** can bypass, including the account root, until it expires

Value	What it actually does
`GLACIER_IR`	"Instant Retrieval" archive storage. Cheaper to store, costs more per read, no retrieval delay. Right for logs.

4.8 Terraform syntax that looks like magic

Where do attribute names come from? When you write `aws_s3_bucket.primary.arn`, the `.arn` is defined by the **provider schema**, not by Terraform. The AWS provider publishes what each resource exports. You can read it locally:

```
terraform providers schema -json | jq '.provider_schemas[].resource_schemas.aws_s3_bucket.block.attributes | keys'
```

That is the authoritative answer to "what can I put after the dot," and it beats guessing.

`aws_s3_bucket.primary.id` vs `.bucket` vs `.arn`. For this resource, `id` and `bucket` are both the bucket name and are interchangeable. `arn` is the full ARN. Different resources make different choices about what `id` means, which is why the labs prefer the explicitly named attribute where one exists.

`~> 5.0` is the "pessimistic constraint operator." It means "at least 5.0, and let the rightmost specified part increase, but nothing beyond." So `~> 5.0` allows any 5.x and refuses 6.0. `~> 5.100.0` allows 5.100.1 but not 5.101.0. It exists because major versions carry breaking changes and minor ones should not.

`local.` vs `var.` vs `data.`

Prefix	What it is
`var.`	An input somebody gives you
`local.`	A value you computed from other values
`data.`	A fact you looked up from the provider, not something you created

`data "aws_caller_identity" "current" {}` asks AWS "who am I?" and returns your account ID. It creates nothing. It exists so you never hardcode your account number.

`depends_on`. Terraform normally works out order by itself: if resource B mentions resource A, A goes first. `depends_on` is for the case where the ordering is real but invisible, because B never mentions A. In Lab 2.3, the logging configuration needs the log bucket's *policy* to exist first, but it only references the *bucket*. Terraform cannot see that, so you tell it. Most `depends_on` in the wild is cargo cult; the two in these labs are not.

`one([for ... ])`. Terraform models some nested blocks as a **set**, and sets have no order, so you cannot ask for element zero. The `for` turns the set into an ordered list, then `one()` pulls out the single element and **raises an error if there is more than one**. The alternative, `tolist(...)[0]`, silently picks an arbitrary one. Since this value goes into a compliance attestation, silently picking is the worse failure.

`coalesce`, `merge`, `alltrue`. `coalesce(a, b)` returns the first that is neither null nor empty. `merge(x, y)` combines maps with **y winning on conflicts**, which is exactly why Lab 2.4 writes `merge(var.labels, local.required_labels)` and a consumer cannot override a compliance label. `alltrue([...])` is true only if every element is.

`filter {}`**, empty.** In a lifecycle rule this means "apply to every object." The provider requires either a filter or a prefix, and an empty filter is how you say "no filtering."

4.9 Rego syntax

`package compliance.sc28` is a namespace. When Conftest runs with `--namespace compliance.sc28`, that string must match exactly. That is the whole relationship.

`deny contains msg if { ... }` builds a **set** of denial messages. Every combination of values that satisfies the body adds one message. An empty set means no violations. It reads strangely at first because you are not writing "if bad then fail," you are describing what a violation *looks like* and letting the engine find all of them.

`input` is whatever JSON you fed in. In this course that is `terraform show -json`, so `input.planned_values.root_module.resources` walks the structure of a Terraform plan.

`import rego.v1` opts into the modern syntax. In OPA 1.0 and later this is the default and the line is harmless; in OPA 0.x it is required. Writing it explicitly means the same policy runs on both.

`# METADATA` blocks are not comments to OPA. It parses them as structured annotations you can extract with `opa inspect --annotations`. That is how Lab 6.1 generates OSCAL from your policies instead of retyping every control ID.

4.10 Sigstore

Signing usually means managing a private key, which means protecting it forever. Sigstore's keyless flow avoids that:

1. Your CI job proves who it is with an OIDC token, the same mechanism as the AWS trust in Chapter 4.
2. **Fulcio**, a certificate authority, issues a certificate that is valid for about ten minutes, containing your identity.
3. You sign, and the key is discarded.
4. **Rekor**, a public append-only transparency log, records the signature and the time.

Nobody stores a long-lived key, and the timestamp lives in a public log outside your infrastructure. That last part is the important one: **an attacker with full administrative access to your AWS account still cannot forge or backdate a Rekor entry**, because Rekor is not in your AWS account.

The `.sig.bundle` file packs the signature, the certificate, and the log entry together so verification needs one file.

"Containing your identity" is worth reading twice. In the pipeline, that identity is the repository, workflow and ref, which is what you want: it names the process that produced the artifact rather than whoever happened to be at a keyboard. But if you run `cosign sign-blob` from your own machine, it is the email address of the account you log in with, and it goes into Rekor.

Rekor being append-only is the property that makes signatures trustworthy, and it is also the property that means **you cannot take that back**. An email address you publish there is public permanently. Cosign warns you before it proceeds, and the warning is literal rather than boilerplate.

So sign from CI, where the identity is a workflow. If you want to try it by hand, use an address you are content to have publicly associated with a throwaway lab artifact forever. This is the rare case where the immutability the whole curriculum is built around works against you.

4.11 OSCAL

Five document types. You will use three.

Type	Plain English
Catalog	The dictionary of all controls. NIST publishes it; you link to it.
Profile	Which controls you picked.
Component Definition	How your thing satisfies those controls, with links to evidence.
SSP	The whole system's story. Out of scope here.
Assessment Plan/Results	The auditor's side. Out of scope here.

Why v4 UUIDs specifically. OSCAL requires the version-4 (random) format, recognizable by the `4` starting the third group: `xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx`. Generate them with `python3 -c "import uuid; print(uuid.uuid4())"`. Hand-written ones fail validation with an unhelpful regex error.

`implementation-status` is where honesty lives:

- `implemented` means your code does it and you can point at the line.
- `partial` means some of it, and you can say which part is missing.
- `inherited` means the cloud provider does it and you rely on that.

`inherited` is not a cop-out. On GCP, encryption in transit is genuinely provided by the platform because the storage API refuses plain HTTP. Writing `implemented` there would be a small lie that collapses the moment somebody asks to see the code.

4.12 How your credentials actually reach Terraform

This one is worth its own entry because the error message points nowhere near the cause.

The AWS CLI and Terraform do **not** share a credential mechanism. The CLI understands several ways of holding a session; Terraform's provider is built on the Go SDK and understands a much shorter list. When they disagree, the CLI works perfectly and Terraform claims you have no credentials at all.

`aws login` writes exactly this and nothing else:

```
[default]
login_session = <token reference>
region        = us-east-1
```

There is no `~/.aws/credentials` file. `login_session` is a CLI construct, the provider has never heard of it, so the provider walks its chain, finds nothing, and reaches its last resort: asking the EC2 instance metadata service whether it is running on an EC2 box with a role attached. It is not, so that call times out:

```
Error: No valid credential sources found
Error: failed to refresh cached credentials, no EC2 IMDS role found,
       operation error ec2imds: GetMetadata, request canceled,
       context deadline exceeded
```

Nothing is wrong with EC2. That is simply where the search gave up, three steps after the actual problem. The same shape of error appears with IAM Identity Center, for the same reason.

The fix is to hand the provider a profile that asks the CLI for credentials each time it needs them. In `~/.aws/config`:

```
[profile cgep]
region = us-east-1
credential_process = aws configure export-credentials --profile default --format process
```

Then, in every new terminal:

```
export AWS_PROFILE=cgep
```

`credential_process` is a plain part of the AWS SDK contract: the SDK runs the command, reads JSON with an access key, a session token and an `Expiration` back from it, and runs it again once that expiration passes. Terraform never learns what `aws login` is, and never needs to.

There is an obvious-looking alternative worth explaining, because you will see it recommended: `eval "$(aws configure export-credentials --format env)"`, which copies today's credentials into the shell. It works, briefly. The credentials it copies are a snapshot with roughly a fifteen-minute life, and nothing refreshes them. Worse, environment variables sit *above* profiles in the credential chain, so once that snapshot expires the shell is stuck in a state that logging in again does not repair: `aws login` reports success, Terraform keeps saying `ExpiredToken`, and the two facts look impossible together. They are not. The provider is still reading the dead variables and never reaches the profile. The cure is `unset`, not another login.

The profile approach has no snapshot to go stale, which is the entire reason this course uses it.

Two smaller things worth knowing before they worry you. Run any command in the second or two right after `aws login` and you may see *Credentials were refreshed, but the refreshed credentials are still expired*; the CLI hands your prompt back before its own cache settles, so just run it again. And when the session does lapse, log in with `aws login --profile default`, not a bare `aws login`: with `AWS_PROFILE=cgep` set, a bare login targets `cgep` and the CLI refuses, because it will not overwrite a profile that resolves credentials through a process.

The general rule: **any credential source that is not a static key file needs this bridge.** Which is every source you should actually be using. A static key file avoids the problem by being the thing you were trying not to have.

5. The numbers

Every constant in the curriculum, and where it came from.

`byte_length = 4` on the random suffix. Four bytes render as 8 hexadecimal characters, giving about 4.3 billion possibilities. Enough that no two learners collide; short enough to fit the name budget below.

21 characters maximum for `project_name`. This is derived, not chosen. S3 allows 63 characters. The longest name the labs build is `project` + `-` + `staging` + `-data-` + 8 hex = 21 + 1 + 7 + 6 + 8 = **43**, comfortably inside 63 with room for a longer suffix later. **Change the naming scheme and this number has to move.**

63 characters is S3's hard limit, and it comes from DNS: bucket names originally had to work as hostnames, and a DNS label maxes out at 63 characters.

`7776000s` for KMS rotation on GCP. That is 90 days in seconds. Google's API takes a duration string, so you do the arithmetic: 90 × 24 × 60 × 60.

7 to 30 days for the AWS KMS deletion window. AWS's allowed range. It exists because deleting a key destroys every piece of data it protects, permanently, and AWS wants a window in which you can change your mind. **The key bills the full $1/month throughout that window.**

90 days for access log retention, **400 days** for CloudTrail. Ninety is a common operational default. Four hundred is deliberate: a full year plus a margin, so that when an auditor asks in month thirteen about something from month one, the logs are still there.

$1 per month per KMS key. AWS's published price, charged whether or not you use the key. It is the largest recurring cost in the curriculum, and it is what buys you SC-12 and SC-13. An AWS-managed key is free and satisfies neither, because you can neither set its rotation policy nor read its key policy.

$0.10 per 100,000 events for CloudTrail data events. Management events are free; data events (who read which object) are not. This is why Lab 5.2 scopes them to the evidence vault only.

`0x2B` and `0x5A`. These are the ASCII codes for `+` and `Z`, and they explain a piece of Lab 4.4 that is subtler than it first looks. AWS returns retention dates ending `+00:00`; `date -u` produces dates ending `Z`. Comparing those as strings *looks* unsafe.

Test it and it holds up. Both strings share a fixed-width, zero-padded ISO prefix, so a lexicographic comparison of `YYYY-MM-DDTHH:MM:SS` is the same as a chronological one. The formats only diverge at the character after the seconds, and that character is only consulted when both sides are the *same second*, where "not in the future" is the right answer either way. Two hundred thousand generated cases produce zero disagreements.

What breaks it is a **non-UTC offset**. `2026-08-18T16:00:00-05:00` is an hour ahead of `2026-08-18T20:00:00Z`, and compares as expired. So the string test is correct by coincidence, not by construction: it depends on AWS returning UTC.

The lesson is not "string comparison is a bug." It is that code can be right for a reason you did not choose, and the fix is to make the reason explicit. Comparing epoch seconds is correct whatever format arrives.

Three of four public access block flags is not enough. Not a number so much as a shape. The four flags are two questions crossed with two states:

	Blocks *new*	**Neutralizes** *existing*
ACLs	`block_public_acls`	`ignore_public_acls`
Policies	`block_public_policy`	`restrict_public_buckets`

Leave one off and you have either an existing public grant still live or an open door for a new one.

6. The labs in plain English

Lab 0.1: Prerequisites

What it does. Sets up a throwaway AWS account and a GCP project, gets you credentials, installs about ten tools, and puts a spending cap in place.

Why it exists. Because the labs now create KMS keys that bill real money, and because the single most common way to lose an afternoon is an authentication error that looks like something else.

The credential decision. Three ways to let your laptop talk to AWS, and they are not equal. `aws login` converts the console session you already have into short-lived credentials and writes no lasting secret; it is the best default. IAM Identity Center gives the same property with more setup and is right for org-managed accounts. An IAM user access key is a permanent secret sitting in a file, and it is the credential type Chapters 4 and 5 exist to argue against. Whichever you pick, see section 3.12 for why Terraform will not see it until you export it.

The one thing not to skip: the budget alarm. It is the only step that has ever saved anyone money.

Lab 2.2: Remote state backend

What it does. Creates one S3 bucket whose job is to hold Terraform's memory.

What Terraform state actually is. When Terraform creates a bucket, it writes down what it made, so next time it can tell the difference between "create this" and "this already exists." That memory is a JSON file called state. Without it, Terraform has amnesia and tries to create everything again.

Why it has to be remote. Your laptop has the state file. Your CI runner does not. So the CI runner thinks nothing exists and proposes to build everything from scratch. Putting state in S3 lets both read the same memory.

Why it is treated as a secret. State contains **every attribute of every resource, in plaintext**, including database passwords and API keys. If you mark something `sensitive` in Terraform, that only hides it from the terminal output. It is still sitting in the state file in the clear. Read access to state is read access to your secrets.

The chicken-and-egg. You need a bucket to store state, and creating the bucket produces state. The answer: a small separate workspace that keeps its state locally and gets committed to git, because that particular state describes only a bucket and a key, with nothing secret in it. The lab shows you how to verify that claim rather than trust it.

Lab 2.3: Your first compliant resource

What it does. Builds two S3 buckets: one for data, one that receives the access logs of the first. Both are locked down eleven different ways.

Why two buckets. Logs should not live in the thing they are logging. If someone compromises the data bucket and can also rewrite its logs, the logs are worthless.

What each piece is for, in one line each:

Piece	Plain English
KMS key	An encryption key you own and can rotate, rather than one Amazon manages invisibly
Server-side encryption config	"Encrypt everything in this bucket with that key"
Versioning	Keep old copies, so a delete or overwrite is recoverable
Public access block	Four separate switches, all off, meaning nothing here goes public
Ownership controls	Turn off the old ACL permission system entirely
TLS deny policy	Refuse any request that arrives over plain HTTP
Encryption deny policy	Refuse any upload that does not explicitly ask for your key
Lifecycle rules	Delete logs after 90 days, so storage does not grow forever
Access logging	Write a record of every request into the other bucket
Tags	Four labels on everything, so you can list your whole environment with one API call

The tag that matters most. `ComplianceScope = "cge-p-lab"` lets you answer "what is in scope for this audit?" with a command instead of a spreadsheet. That is a genuinely different way to run compliance and it is the quiet point of the whole tagging exercise.

The one that will surprise you. After applying, `aws s3 cp file.txt s3://bucket/` **fails**. That is the encryption deny policy working. You have to say which key you want:

```
aws s3 cp file.txt s3://bucket/ --sse aws:kms --sse-kms-key-id "$KMS_ARN"
```

Whether that friction is worth the enforcement is a real decision, and defending your answer is exactly what the capstone asks for.

Lab 2.4: Modules

What it does. Repackages the same idea on Google Cloud as a reusable component, then uses it twice with different settings.

What a module is. A folder of infrastructure code with a defined set of inputs and outputs. Like a function. You hardcode the security settings inside where nobody can reach them, and expose only the business settings.

The design idea worth stealing. Required labels are merged **on top of** the consumer's labels, so a consumer can add labels but cannot override the compliance ones. That asymmetry, rather than documentation or code review, is what makes the floor a floor.

The structural lesson. This module deliberately contains no `provider` block, and Lab 2.3 deliberately does. A module with its own provider cannot be reused across two regions or two accounts. Lab 2.3's "primitive" is really a root module wearing the wrong word, and noticing that is the point.

Lab 2.5: The evidence vault

What it does. Creates a bucket where deletion is impossible, and a script that packages evidence and puts it there.

Object Lock in plain English. Normally a bucket owner can delete anything. Object Lock makes S3 itself refuse. In `GOVERNANCE` mode a sufficiently privileged caller can override it. In `COMPLIANCE` mode nobody can, including the account root, until the clock runs out.

The trap. `COMPLIANCE` mode with a long retention creates a bucket you will pay for and cannot empty for the entire duration. That is correct behavior and an expensive mistake in a lab. Use `GOVERNANCE` with one day while learning.

The trap. It is natural to capture raw Terraform state into this vault, since it describes what is actually deployed. Combine "state contains secrets in plaintext" with "this bucket cannot be deleted from" and you have made a secret permanently undeletable, in the bucket you hand to auditors. The lab captures the plan instead, and refuses to capture state into a COMPLIANCE vault without an explicit override.

A subtlety worth internalizing. Deleting an object without naming a version *appears* to work. It creates a "delete marker" that hides the object. The object is still there. An auditor who does not know this thinks evidence was destroyed; an engineer who does knows where to look.

Lab 3.3 and 3.4: Policy as code

What they do. Write programs that read a proposed infrastructure change and refuse it if it violates a control.

Why this is different from a scanner. A scanner tells you what is wrong after it exists. This runs on the *plan*, before anything is created. Nothing bad ever gets built.

Why Rego feels weird. Rego does not ask "is this OK?" It asks "show me every way this is broken," and returns a set. You describe the shape of a violation, and the engine finds all instances. It is closer to a database query than to an `if` statement.

Mutation testing, the most valuable idea in these two labs. A policy with a typo in the resource type matches nothing, denies nothing, and passes every test you wrote. It is a control that *cannot fail*, which means it is not a control. So you break each policy on purpose and require the tests to notice. If a test suite still passes with the logic inverted, that policy is decorative.

The same idea in one sentence: **a green pipeline looks identical whether your controls passed or never ran.**

The related trap. Running GCP policies against an AWS plan reports zero failures, because the rules match nothing. Zero failures and zero coverage look the same in the output. That is why Lab 3.4's gate script fails loudly when it produces no results at all.

Lab 4.3 and 4.4: The pipeline

What they do. Move the checks into GitHub Actions so they run on every proposed change, then sign the results and file them in the vault.

What OIDC replaces. The old way: create an access key, paste it into GitHub Secrets, hope it never leaks, remember to rotate it. The new way: GitHub proves the job's identity to AWS, AWS hands back credentials valid for an hour. **Nothing long-lived exists to leak.**

Why actions are pinned to a commit hash instead of `@v4`. A tag is a movable pointer. Whoever controls the repository, or whoever compromises their account, can move `v4` to point at different code. A commit hash cannot be moved. This is a real attack that has really happened.

And the part nobody mentions: a pin is a maintenance obligation. A tag floats, so it silently collects fixes and silently collects compromises. A hash is frozen, so it collects neither, including the upstream's runtime migrations. Eventually you are running something the platform is deprecating underneath you.

This curriculum's own CI showed it. Every job passed, and every job carried `Node.js 20 is deprecated ... being forced to run on Node.js 24`. Nothing was broken; the pins were simply old enough that GitHub was migrating them off their target runtime, and the only signal was a warning inside a green run.

So pin, and pair it with something that raises the version: Dependabot, Renovate, a quarterly review, or a CI check that fails when a pin falls too far behind. **A pin without a refresh process does not remove the supply chain risk, it swaps it for an obsolescence risk that is quieter.** That is usually still the right trade. Make it knowingly, and write down who checks.

Why the pass/fail decision moves to the last step. You want evidence collected even when the gate fails. **A failed run is precisely the run whose evidence you will want later.**

Branch protection is what makes it a control. Without it, a red check is a suggestion anybody can merge past. With `enforce_admins: true`, it is enforcement. That toggle is the difference between CM-3 and a habit.

What signing adds that Object Lock does not. Object Lock proves the file has not changed. It does not prove who made it. Someone with admin in your account could create a *different* bucket, put a fabricated bundle in it, and point a careless auditor there. The signature ties the bundle to a specific repository and workflow, and the record of it lives in a public log outside your account.

Lab 5.2 and 5.4: The platform's own tools

What they do. Turn on the monitoring each cloud already provides, and then actually read the output.

The three services, plainly. CloudTrail records who did what. Config records what things looked like over time. Security Hub collects findings and normalizes them.

Where AU-6 finally gets earned. Collecting logs is `AU-3`. Keeping them the right length is `AU-11`. **Reviewing** them is `AU-6`, and that requires something that reads them, on a schedule, with a human whose name you can say. Lab 5.2 sets up Athena so you can query the trail. If you run those queries once by hand, the honest claim is `partial`, not `implemented`.

The GCP difference. Google's Org Policy rejects bad configuration at the API call. Not a finding afterward: the action does not happen. That is the strongest kind of control, and also the one that can break your engineering team on the day you enable it, which is why the lab teaches audit mode first and enforcement second.

Lab 6.1: OSCAL

What it does. Writes down, in a format a machine can read, which controls your thing satisfies and where the proof is.

Why bother. Because it turns an audit from a meeting into a traversal. The assessor opens your file, sees `SC-28`, follows the link, verifies the signature, and moves on.

The generation idea. Your Rego policies already declare which control they enforce. Rather than retyping those IDs into the OSCAL by hand, the lab extracts them programmatically. **Anywhere a mapping is typed twice is a place it will drift.**

Why the verification script matters. `trestle validate` checks that your document is well-formed. It does not check that a single link works. A component definition whose evidence URLs all 404 validates perfectly and proves nothing. That script found a real error while this curriculum was being written: the NIST catalog tag pinned in the first draft did not exist.

7. Every decision you have to make

The labs pick sensible defaults. These are the ones you should think about rather than accept.

Encryption: AWS-managed key or your own?

	`AES256` (S3-managed)	`aws:kms` (your key)
Cost	Free	$1/month per key
Rotation	Amazon's business	Yours, on your schedule
Who can decrypt	Anyone with S3 permission	Anyone with S3 **and** KMS permission
Audit trail	None for key use	Every use logged in CloudTrail
Cross-account sharing	Simple	Requires key policy work

Choose your own key when you are handling regulated data, need to prove rotation, need a second lock separate from S3 permissions, or expect an assessor to ask about key custody.

Choose AES256 when it is a genuine sandbox, cost matters more than control, or the data is not sensitive. Be aware that Trivy and most benchmarks flag AES256 at HIGH, so you will be explaining the choice.

Object Lock: GOVERNANCE or COMPLIANCE?

GOVERNANCE for anything you might need to clean up, and for all lab work. **COMPLIANCE** when a regulator requires that evidence be genuinely undeletable, and you have accepted that "genuinely" includes you.

Ask yourself one question: *if I put the wrong file in here, am I comfortable that it stays for the full retention period?* If the answer is no, you want GOVERNANCE.

Where do logs go: server access logs or CloudTrail data events?

	S3 server access logs	CloudTrail data events
Cost	Free (storage only)	$0.10 per 100k events
Delivery	Best effort, can drop records	Guaranteed
Timing	Hours	Minutes
Format	Space-separated text	JSON
Integrity proof	None	Digest files

Use CloudTrail data events for the buckets that matter, especially the evidence vault, where "who read this" is itself an audit question. **Use server access logs for everything else**, because free and imperfect beats expensive and perfect at scale.

Should evidence bundles include Terraform state?

No, by default. State holds secrets in plaintext and the vault cannot be emptied. The plan file tells an assessor what was configured, which is what they actually read.

Yes, if you have deliberately confirmed the workspace holds no secrets, and you need to demonstrate current deployed reality rather than intent. Say so in your write-up either way; a reviewer will ask.

Which framework for the capstone?

Pick	If
HIPAA Security Rule	The scenario's PHI angle interests you. Smallest control set, most prescriptive language, easiest to be thorough about.
SOC 2	You want the one you are most likely to meet commercially. Trust Services Criteria are broader and vaguer, which means more judgment and more to defend.
CMMC Level 2	You are aiming at federal work. Most prescriptive, maps closest to NIST 800-171, largest volume.

There is no wrong answer, and the grading is on how well you defend the choice. Pick the one you can argue about for a page.

Apply automatically on merge, or require manual approval?

Automatic is the stronger demonstration of engineering maturity and the harder thing to build safely. **Manual approval** is what most regulated organizations actually do.

Either is acceptable. What is not acceptable is automatic apply using the same credentials as the plan step. Split the roles.

One AWS account or two?

Two (workload plus a separate evidence account) is the real answer, because same-account evidence can be destroyed by whoever compromised the account. **One** is fine for a 30-day capstone. Name the gap in your write-up and you have handled it correctly.

8. How to find the answer yourself

The goal is that you stop needing this document. When something is unfamiliar:

A Terraform attribute you do not recognize:

```
terraform providers schema -json | jq '.provider_schemas[].resource_schemas.RESOURCE_NAME'
```

What a plan is actually doing:

```
terraform show -json tfplan | jq '.planned_values.root_module.resources[] | {address, type}'
```

Why an IAM decision went the way it did. The IAM Policy Simulator in the console, or `aws iam simulate-principal-policy`. IAM's rule is simple and worth memorizing: **an explicit Deny always wins, then an explicit Allow, and otherwise the answer is no.**

What a control actually says. The NIST catalog is a JSON file you can grep:

```
curl -s https://raw.githubusercontent.com/usnistgov/oscal-content/v1.2.1/nist.gov/SP800-53/rev5/json/
NIST_SP-800-53_rev5_catalog.json \
  | jq '.. | objects | select(.id? == "sc-28") | {id, title}'
```

What a Rego rule is doing:

```
opa eval -d policies -i plan.json 'data.compliance.sc28_aws' --format=pretty
```

Drop the `.deny` to see every rule in the package, including the helpers.

Whether a tool version is current:

```
curl -s https://api.github.com/repos/OWNER/REPO/releases/latest | jq -r .tag_name
```

What you actually have deployed, using the tag from Lab 2.3:

```
aws resourcegroupstaggingapi get-resources \
  --tag-filters Key=ComplianceScope,Values=cge-p-lab --profile cgep-lab
```

Read the cleanup first

Most instructions can be followed top to bottom. A few in this curriculum cannot, because they build things designed to resist deletion, and that resistance does not care that you were only practising.

`COMPLIANCE`-mode Object Lock cannot be bypassed by anyone, including the account root. A locked GCS retention policy holds a bucket until every object ages out. KMS keys in both clouds enter a deletion window rather than disappearing, and keep billing throughout. A Workload Identity pool you delete blocks its own name for thirty days.

None of that is a flaw. Storage you can quietly empty is not evidence, and a key you can delete in a panic is not custody. It does mean the last section of a lab sometimes constrains the first, so **read a lab's Cleanup section before you run its Apply**. Labs 2.4, 2.5 and 5.4 say so at the top, because those are the ones where the cost of not doing it is measured in months.

The three habits

If the details fade, keep these.

1. Watch every control deny something. Configuration proves intent. A refusal proves enforcement. Every lab ends by making a control say no, on purpose, and that transcript is better evidence than any settings dump.

2. Break your checks to prove they work. An unfired control and an absent control look identical in a green pipeline. Mutation-test your policies. Delete the policy directory and confirm the gate goes red. Tamper with a bundle and confirm verification fails.

3. Claim only what you can show. `partial` that you can defend beats `implemented` that falls apart in one question. The most common overclaim in this field is AU-6, audit review, asserted on the strength of having collected the logs. Collecting is AU-3 and AU-11. Reviewing is a person, a cadence and a record.

Revision history

These notes

- New document. The labs say what to type; this explains what it means, traces every constant and magic string to its source, and lays out which choices are genuinely yours.
- Section 3 covers the scaffolding rather than the compliance: the container and why browser logins fail inside it, the three places Terraform looks for an input, why your copy of the repository has to be yours, why repository variables are not secrets, and what is safe to publish. Most first evenings are lost in there rather than in the labs.

The official labs

Did not exist.

CGE-P LAB NOTES

CGE-P lab notes, lab by lab

Eleven labs and a capstone brief. Every lab produces an artifact the capstone consumes.

For reading. Code lines wrap to fit the page; copy code from docs/README.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/README.md` in your clone, not from the PDF.

Where these live. Upstream, the guides are `guides/*.md` in `GRCEngClub/cgep-labs` and the companion code is `reference/lab-X-Y/`. These drafts sit in `docs/` so you can build the reference workspaces from them before deciding what to upstream. Every code block here is written to pass that repo's CI: `terraform fmt -check -recursive`, `terraform validate -backend=false`, `opa test`, and `trestle validate`.

These notes differ from the official labs in one governing way: **a lab may only cite a control it actually implements.** Where a citation and the code did not line up, the code was written or the claim was moved to the lab that earns it. Where a mapping is arguable, the lab says so and defends it, because teaching students that control mappings are arguments rather than lookups is the single most transferable thing in this course.

Start here

The Plain English Guide is the companion to everything below. The labs tell you what to type; that guide explains what it means, where every magic string and constant came from, and which choices are actually yours. Read it first, or keep it open alongside the labs. It is not a lab and creates nothing.

Reading order

#	Lab	Cloud	Produces
0.0	Plain English Guide	none	Concepts, decoder ring, decision tables. **New in v2.** Not a lab.
0.1	Prerequisites & Credentials	both	Sandbox account, short-lived credentials, toolchain, **budget alarms**. **New in v2.**
2.2	Remote State Backend	AWS	S3 + DynamoDB state backend. **New in v2.**
2.3	First Compliant Resource	AWS	`compliant-s3` primitive with CMK, TLS enforcement, lifecycle.
2.4	Modules for Compliance	GCP	Reusable module discipline, CMEK, consumer pattern.
2.5	IaC as Compliance Evidence	AWS	Object Lock evidence vault + `capture-evidence.sh`.
3.3	Writing Rego	GCP	Policy library with metadata and test fixtures.
3.4	Conftest + Terraform	AWS	AWS policy variants + `policy-gate.sh`.
4.3	GRC Evidence Pipeline	AWS	`.github/workflows/grc-gate.yml`.

#	Lab	Cloud	Produces
4.4	Chain of Custody	AWS	Cosign signing wired into the vault.
5.2	AWS Security Services	AWS	CloudTrail, Config, Security Hub, Athena review.
5.4	GCP Security Services	GCP	Org Policy, Workload Identity Federation, Data Access logs.
6.1	Introduction to OSCAL	none	Component definition + profile, validated.
7.1	Capstone Brief	AWS	The graded deliverable.

Read the starter's gaps early

The capstone hands you a real, deliberately broken application: `GRCEngClub/cgep-app-starter`. Its `GAPS.md` names eight compliance gaps you are asked to close, and the whole point is that you are governing code you did not write, which is harder and more realistic than governing your own.

Read that file before Lab 2.3, and skim the starter's Terraform with it. Ten minutes, no setup, nothing to deploy. Every lab after it changes character: you stop learning SC-8 as an idea and start recognizing the bucket that is missing it. Labs 2.3, 3.4 and 5.2 carry callouts naming which gaps that lab's work addresses.

You do not need to fix all eight. Five, closed with depth and traceable through your OSCAL, is what passes.

One repository, built up

The labs are cumulative. Each one produces something the next consumes, and by Lab 6.1 you are holding the whole structure the capstone is graded on. The full tree, with which lab creates each part, is in Lab 0.1 Step 12d.

The short version of the chain:

Lab	Produces	Consumed by
2.2	the state backend	every workspace after it, and Lab 4.3's pipeline
2.3	the `compliant-s3` pattern	2.5's vault, 5.2's buckets, the capstone
2.4	the GCP module and its attestation	6.1's OSCAL component
2.5	the evidence vault and capture script	4.3, 4.4, 5.2's data events, 6.1's links
3.3	the policy library and catalog	3.4's gate, 4.3's CI, 6.1's requirements
3.4	`policy-gate.sh`	4.3, which shells out to it rather than reimplementing
4.3	the pipeline	4.4, which adds signing to it

Lab	Produces	Consumed by
4.4	signing and verification	6.1's evidence links, the capstone's chain of custody
5.2 / 5.4	the cloud baselines	the capstone's Layer 1
6.1	the OSCAL component and profile	the capstone's write-up

Nothing is discarded between labs. If a step tells you to delete something, it is cleanup of a cloud resource that costs money, never of an artifact you built.

The shared baseline

Every S3 bucket built anywhere in this curriculum enforces the same floor. Labs do not re-argue it; they cite it. If you change something here, it changes everywhere, which is the point.

Property	Resource	Control	Why it is in the floor
Customer-managed KMS key, rotation on	`aws_kms_key`	SC-12, SC-13	Key custody and rotation are yours, not the platform's.
SSE-KMS with bucket keys	`aws_s3_bucket_server_side_encryption_configuration`	SC-28, SC-28(1)	Bucket keys cut KMS request cost by up to 99%, and are **required** when the bucket receives S3 server access logs.
Versioning	`aws_s3_bucket_versioning`	CP-9, SI-7	Prior object states survive deletion and overwrite.
All four public-access-block flags	`aws_s3_bucket_public_access_block`	AC-3	Two axes (ACL vs policy) by two states (new vs existing).
ACLs disabled	`aws_s3_bucket_ownership_controls` = `BucketOwnerEnforced`	AC-3, AC-6	AWS default since April 2023. An ACL-enabled bucket is itself a CIS and Security Hub finding.
Deny non-TLS	`aws_s3_bucket_policy` (`aws:SecureTransport`)	SC-8, SC-8(1)	**The gap the official labs never close anywhere.**
Deny wrong-key uploads	`aws_s3_bucket_policy` (`s3:x-amz-server-side-encryption`)	SC-28	Turns encryption from a default into an enforced condition.
Lifecycle rules	`aws_s3_bucket_lifecycle_configuration`	AU-11, SA-9	Retention is a control, and unbounded logs are an unbounded bill.

Property	Resource	Control	Why it is in the floor
Four required tags	provider `default_tags`	CM-6, CM-8	Boundary enumeration by API call instead of by spreadsheet.

Two properties are **not** in the floor, deliberately, and each lab that needs them says so: Object Lock (Lab 2.5, evidence only) and cross-account log delivery (documented, not built, see Lab 5.2).

What changed, and why

Read this if you have taken the official labs. Everything here is a correction rather than a preference.

1. A state backend now exists (new Lab 2.2). The official labs build no backend anywhere, yet Lab 4.3's pipeline ran `terraform init` on a fresh runner and the capstone required `Apply` on merge to `main`. With local state, that runner starts with empty state, so the first apply after a merge either collides with existing resources or duplicates them. The capstone's Layer 3 was not completable as written. Lab 2.2 fixes it before anything depends on it.

2. Lab 2.3's log bucket is now versioned. The official architecture prose says "both buckets enforce ... versioning" while its code versioned only the primary. The bucket holding audit records was less protected than the bucket holding data. Verified against their own resource count: 11 resources, no `aws_s3_bucket_versioning.log`.

3. TLS enforcement exists (SC-8). They never use `aws:SecureTransport` in any lab, and never cited SC-8. Encryption at rest was covered in Chapter 2; encryption in transit was covered nowhere. This is also a `tfsec` HIGH, meaning their own Chapter 4 gate would fail their own Chapter 2 artifact.

4. AWS KMS is taught before the capstone requires it. They teach CMEK only in Lab 2.4, on GCP. The capstone's Layer 1 required AWS CMKs with rotation. Students met `aws_kms_key` for the first time in a graded deliverable.

5. Log delivery uses a bucket policy, not an ACL. They set `BucketOwnerPreferred` and applied a `log-delivery-write` ACL, re-enabling ACLs on the log bucket to do it. That works, and it trips Security Hub's "S3 general purpose buckets should have ACLs disabled." These notes use the modern grant to `logging.s3.amazonaws.com` with `aws:SourceArn` and `aws:SourceAccount` confused-deputy conditions. The `depends_on` dance disappears with it.

6. AU-6 is no longer claimed by Lab 2.3. AU-6 is audit *review, analysis, and reporting*. Shipping logs into a bucket is AU-3 plus AU-11. They claim AU-6 in a lab where nothing reads the logs. These notes move the claim to Lab 5.2, which now stands up an Athena table over the trail and runs a query, because that is what AU-6 costs.

7. Versioning's control mapping is defended, not asserted. They map versioning to CM-6. These notes map it to CP-9 and SI-7, and on log buckets to AU-9, and explains why CM-6 was the weaker argument.

8. CloudTrail data events are on for the evidence vault (Lab 5.2). They say "we do not enable data events here," which left object-level access to the evidence vault unaudited. The vault is the one bucket where you must know who read what. Cost is called out explicitly.

Cost and time

This costs more than the official labs. That is the honest consequence of "production grade," and every lab states its own number. Running the whole curriculum and destroying same-day:

Lab	Marginal cost	Time
0.1	free (sets the budget cap)	90 min
2.2	under $0.01	20 min
2.3	~$1/mo per KMS key, prorated to cents	60-75 min
2.4	~$0.06 per key version per month	45-60 min
2.5	under $0.01	45 min
3.3, 3.4	free (plan only)	105 min
4.3, 4.4	free	150 min
5.2	$1-8 if left running a month	75 min
5.4	pennies	75-90 min
6.1	free	60-75 min

Destroy same-day and the whole curriculum lands under $5. Leave Security Hub and a few KMS keys running for a month and it is $10-15. KMS keys bill $1/month each whether you use them or not, and a scheduled key deletion still bills through its waiting period.

Conventions

- **Run every command inside the container**, if you set one up in Lab 0.1. That is where the toolchain and your cloud logins live. Running on your own machine means installing the ten tools and logging in a second time, which is a separate setup rather than a shortcut.
- `evidence/` **lives at the repository root**, not inside the workspace you happen to be in. Capture blocks anchor to it with `EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-X-Y"`, so they work from any directory.
- **Read a lab's Cleanup section before you apply it.** Several build things that cannot be undone on a lab timescale: Object Lock retention in `COMPLIANCE` mode, a locked GCS retention policy, KMS keys in either

cloud, a federation pool's 30-day soft delete. Labs 2.4, 2.5 and 5.4 carry a callout naming theirs. Reading two paragraphs ahead is cheaper than waiting a year for a bucket to become deletable.

- Every lab states cost and time before the first command.
- Every control citation appears as a comment at the exact line that enforces it.
- Every lab ends with cleanup that actually works, verified against versioned and Object-Locked buckets.
- Placeholders are `<your-sandbox>`, `<your-vault>`, `your-gcp-project`. Substitute yours.
- No lab asks you to run something it has not told you the cost of.

CGE-P LAB NOTES

Lab 0.1: Prerequisites and Credential Setup

Do this once, before Lab 2.2. It takes about 90 minutes and it is the difference between labs that work and an afternoon of authentication errors.

Three things happen here: you build a sandbox where mistakes are cheap, you set up credentials that are short-lived by default, and you put a spending cap on it before you deploy anything that bills.

For reading. Code lines wrap to fit the page; copy code from docs/00_01_prerequisites.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/00_01_prerequisites.md` in your clone, not from the PDF.

Read this first: these labs cost money

This curriculum uses customer-managed KMS keys, which is where most of the cost comes from and is the point. Rough numbers for a single learner:

If you	Expect
Destroy each lab the same day	Under **$5** for the whole curriculum
Leave the state backend up for a month (recommended)	**$1/month** for its CMK
Leave Security Hub + Config + a few keys running a month	**$10 to $15**
Forget to destroy Lab 5.2 for a quarter	**$30 to $45**

Two specific traps:

- **KMS keys bill $1/month each, whether used or not**, and they keep billing through the 7-to-30 day deletion window after you schedule deletion. You cannot delete one immediately. Each lab tells you which keys it creates.
- **CloudTrail data events** bill per event. Lab 5.2 scopes them to one bucket for this reason. Point them at a busy bucket and the number changes character.

Set the budget alarm in Step 5 before you apply anything. It is not optional, and it is the only step here that has ever saved anyone real money.

Learning objectives

- Stand up an AWS sandbox account isolated from anything you care about.
- Authenticate with short-lived credentials instead of long-lived access keys, and explain why that distinction is a control.
- Stand up a GCP project with the right APIs and both authentication modes.
- Install and verify the full toolchain.
- Cap your own spending before you can hurt yourself.

Part 1: The AWS sandbox

Step 1 Use a separate account

Not your production account. Not the account with your domain in it. A separate AWS account, because every lab ends with `terraform destroy` and the blast radius of a mistake should be a sandbox.

Two ways to get one:

Approach	Do this if
New member account in an Organization (recommended)	You have or can create an AWS Organization. Gives you a clean account with consolidated billing and the option to attach an SCP later.
A brand new standalone account	You have no Organization and do not want one. Works fine. You manage its billing separately.

If you have no AWS account at all

Sign up at **https://aws.amazon.com** and choose **Create an AWS Account**. You will need three things:

- **An email address** nobody has used for an AWS account before. If you plan to make more than one account later, use the `you+cgep-lab@example.com` form now, because AWS requires a unique address per account and most mail providers deliver plus-addressed mail to the same inbox.
- **A payment card.** AWS asks for one even on the free tier and places a small temporary authorization on it, usually about a dollar, which it releases.
- **A phone** for verification.

Choose the **Basic support plan**, which is free. Everything in this curriculum works on it.

The email and password you sign up with become the **root user** of the new account. That account can do anything, including closing itself and changing billing, so Step 2 immediately locks it down and you never use it again for day to day work.

About the free tier. New accounts get twelve months of free-tier allowances plus some always-free services. It does not cover everything this curriculum uses. KMS keys are about a dollar per month each and are not free-tier eligible, so Step 5's budget alarm is doing real work, not ceremony.

If you already have an AWS Organization

Creating a member account in an existing Organization:

```
aws organizations create-account \
  --email you+cgep-lab@example.com \
  --account-name "CGE-P Lab Sandbox" \
  --profile YOUR_MGMT_PROFILE
```

```
# Poll until State is SUCCEEDED, then note the AccountId
aws organizations list-create-account-status \
  --states SUCCEEDED --profile YOUR_MGMT_PROFILE
```

The `you+cgep-lab@example.com` form matters: AWS requires a unique email per account, and most mail providers deliver plus-addressed mail to the same inbox.

> **A caution about Organizations.** A member account created this way has an `OrganizationAccountAccessRole` your management account can assume, and no root password until you reset it. That is convenient and it is also why Lab 5.2 may find Config blocked by a service control policy. Both are normal.

Step 2 Harden the root user

Do this before anything else, in the console, signed in as root:

1. **Enable MFA on the root user.** A hardware key or an authenticator app.
2. **Delete any root access keys.** There should be none. If there are, delete them; nothing should ever use root programmatically.
3. **Set a strong unique password** in your password manager.
4. Then sign out of root and do not use it again. Everything below uses a role.

That is IA-2(1), AC-6(5), and IA-5 in three minutes, and it is the first thing any assessor checks.

Step 3 Create a non-root IAM user

Do this before you touch credentials, because the credential step will otherwise hand you root and not warn you.

In the console: **IAM → Users → Create user**.

- Name it `cgep-lab`.
- Tick **"Provide user access to the AWS Management Console"**.
- Set a password. Untick the force-reset box if you would rather not.
- Permissions: **Attach policies directly** → `AdministratorAccess`. This is a sandbox; scoping it properly is a week of policy authoring and teaches you nothing this curriculum is about.

Then **Security credentials → Assign MFA device** on that user. If you put a hardware key on root in Step 2, register the same one here. A single key enrols against multiple identities, and AWS allows eight devices per user.

Do not create an access key. A console password is all you need. Step 4 turns the resulting console session into short-lived credentials.

Now sign out of root and sign back in as `cgep-lab` at `https://<ACCOUNT_ID>.signin.aws.amazon.com/console`.

Step 4 Credentials: three ways, ranked

	What it is	Secret on disk	Verdict
`aws login`	Converts your existing console session into temporary credentials that auto-refresh	None	**Use this**
IAM Identity Center	Federated sign-in with short-lived session credentials	None	Right for org-managed accounts, more setup
IAM user access key	A permanent `AKIA…` key and secret	Yes, forever	Last resort

The third option is the credential type Labs 4.3 and 5.4 exist to eliminate. Creating one to work through a curriculum about credential hygiene is a choice you will have to defend to yourself.

The recommended path

```
aws login
```

It prompts for a region (`us-east-1`), opens a browser, and asks which console session to use.

> **Read the session it offers.** `aws login` reuses whatever console session you are signed into, and it will happily offer you **root** without flagging that as dangerous. Root cannot be constrained by any IAM policy and cannot be scoped down afterward. If the browser shows `root`, click **"Sign into new session"** and sign in as the `cgep-lab` user from Step 3.

Then confirm which identity you actually got:

```
aws sts get-caller-identity
```

You want an `Arn` ending in `:user/cgep-lab`. **If it ends in `:root`, stop and redo the sign-in before applying anything.** Half of what this curriculum builds is meaningless under root: Lab 4.3's OIDC role is scoped so CI can plan but not apply, Lab 2.5's vault denies `s3:DeleteBucket` to everyone except the account root, and Lab 2.2 grants the account root key administration as a safety net. Run everything as root and all three become decorative.

`aws login` writes no `~/.aws/credentials` file. It stores a session reference and refreshes automatically while the refresh token is valid. When it lapses, run `aws login` again.

The step everyone misses

Terraform cannot read what `aws login` writes. Nor what Identity Center writes. Both store a session reference; the provider's Go SDK wants materialized credentials. It will tell you this in the least helpful way available:

```
Error: No valid credential sources found
Error: failed to refresh cached credentials, no EC2 IMDS role found,
       operation error ec2imds: GetMetadata, request canceled,
       context deadline exceeded
```

Nothing is wrong with EC2. That is simply where the credential chain gave up. The bridge is a profile that resolves credentials by shelling out to the AWS CLI. Add this to `~/.aws/config`:

```
[profile cgep]
region = us-east-1
credential_process = aws configure export-credentials --profile default --format process
```

Then, once per terminal:

```
export AWS_PROFILE=cgep
```

That is the whole setup. `credential_process` is part of the AWS SDK contract, not a Terraform feature, so every tool in this course honors it.

One consequence to learn now rather than mid-lab: **re-login with `aws login --profile default`.** With `AWS_PROFILE=cgep` exported, a bare `aws login` targets `cgep` and the CLI stops you:

```
An error occurred (Configuration): Profile 'cgep' is already configured
with Credential Process credentials.
```

That is correct behavior, not a broken profile. `aws login` writes credentials into a profile, and it will not overwrite one that resolves them through a process. The session lives in `default`, so that is what you refresh. `cgep` reads through to it and needs no maintenance.

Why a profile and not an environment variable. The obvious-looking alternative is `eval "$(aws configure export-credentials --format env)"`, which copies the current credentials into the shell. It works for about fifteen minutes and then becomes the problem. Those variables are a *snapshot*: nothing updates them when the session lapses, and environment variables outrank profiles in the credential chain, so a shell holding an expired snapshot cannot be repaired by running `aws login` again. You get `ExpiredToken` from a terminal where `aws login` just reported success, which is a genuinely confusing place to be.

`credential_process` has no snapshot. Terraform re-runs the command whenever it needs credentials, so refreshing the session with `aws login` is picked up by the next Terraform command automatically. Nothing to re-export, nothing to remember.

The rule generalizes: any credential source that is not a static key file needs this bridge, which is every source you should be using.

If you already ran the `eval` line in a terminal, that shell still has the snapshot and will keep failing. Clear it before setting the profile:

```
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION
export AWS_PROFILE=cgep
```

If you must use an access key

Only when Identity Center is unavailable and `aws login` is not an option.

```
aws configure --profile cgep-lab   # paste key and secret, region us-east-1
chmod 600 ~/.aws/credentials
```

Understand what you just made: a long-lived secret, in plaintext, valid until you delete it. Rotate or remove it when the course ends:

```
aws iam list-access-keys --user-name cgep-lab
aws iam delete-access-key --user-name cgep-lab --access-key-id AKIA...
```

Never commit it. Install a guard before you can:

```
pip install detect-secrets && detect-secrets scan > .secrets.baseline
```

Step 5 The budget alarm (do not skip)

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

cat > /tmp/budget.json <<EOF
{
  "BudgetName": "cgep-lab-monthly",
  "BudgetLimit": { "Amount": "25", "Unit": "USD" },
  "TimeUnit": "MONTHLY",
  "BudgetType": "COST"
}
EOF

cat > /tmp/notifications.json <<'EOF'
[
  {
    "Notification": {
      "NotificationType": "ACTUAL",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 50,
      "ThresholdType": "PERCENTAGE"
```

```
    },
    "Subscribers": [
      { "SubscriptionType": "EMAIL", "Address": "you@example.com" }
    ]
  },
  {
    "Notification": {
      "NotificationType": "FORECASTED",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 100,
      "ThresholdType": "PERCENTAGE"
    },
    "Subscribers": [
      { "SubscriptionType": "EMAIL", "Address": "you@example.com" }
    ]
  }
]
EOF

aws budgets create-budget \
  --account-id "$ACCOUNT_ID" \
  --budget file:///tmp/budget.json \
  --notifications-with-subscribers file:///tmp/notifications.json
```

Two alerts: one when you have actually spent half of $25, one when the forecast says you will exceed it. The forecast alert is the useful one, because it fires while you can still act.

Budgets are a billing API and may need to run against your management account in an Organization. If you get `AccessDeniedException`, set it there instead, scoped to the sandbox account.

Step 6 Pick a region and stay in it

Every lab uses `us-east-1`. Use the same one everywhere: the state backend in Lab 2.2 and every workspace that reads it must agree, and a region mismatch produces `NoSuchBucket` errors that look like permissions errors.

If you use a different region, change it in Lab 2.2 first and carry it through consistently.

Part 2: The GCP project

Labs 2.4, 3.3, and 5.4 are GCP. You can complete the AWS track and the capstone without them, but the cross-cloud lesson is most of the point of the curriculum.

Step 7 Create the project and attach billing

If you have no Google Cloud account at all

Go to **https://cloud.google.com** and click **Get started for free**. You sign in with a Google account, so if you already have Gmail you have the identity half already. You still need:

- **A payment card**, for the same reason as AWS. Google places a small temporary authorization and releases it.
- **A phone** for verification.

New accounts get a trial credit, $300 at the time of writing, valid for ninety days. That covers this entire curriculum many times over. Google will not charge you when the trial ends unless you explicitly upgrade to a paid account, so the worst case is that things stop working rather than that you get a bill.

Two things that confuse people, and they are worth getting straight now:

A **billing account** is where the payment method lives. A **project** is where resources live. They are separate objects, and one billing account can pay for many projects. Creating a project does not attach billing automatically, which is why Step 7 links them explicitly below.

There is also an **organization**, which you get only with Google Workspace or Cloud Identity. A personal account has none, and that is fine: every GCP lab here works at project scope. Lab 5.4 says so explicitly.

When signup finishes you will have a billing account, usually named *My Billing Account*. Find its ID under **Billing** in the console, or with `gcloud billing accounts list` once the CLI is installed. It looks like `01DBB6-ECB143-6DDE9B`.

Creating the project

```
gcloud auth login

PROJECT_ID="cgep-lab-$(date +%s | tail -c 6)"   # must be globally unique
gcloud projects create "$PROJECT_ID" --name="CGE-P Lab"

gcloud billing accounts list
gcloud billing projects link "$PROJECT_ID" --billing-account=XXXXXX-XXXXXX-XXXXXX
gcloud config set project "$PROJECT_ID"
```

Billing must be attached or API calls fail with errors that do not mention billing.

Step 8 Enable the APIs

```
gcloud services enable \
  cloudkms.googleapis.com \
  iam.googleapis.com \
  iamcredentials.googleapis.com \
```

```
  cloudresourcemanager.googleapis.com \
  orgpolicy.googleapis.com \
  storage.googleapis.com \
  logging.googleapis.com \
  sts.googleapis.com
```

`orgpolicy` and `sts` are needed for Lab 5.4 and are easy to forget. Enabling takes a minute or two to propagate.

Step 9 The two authentications, which are not the same thing

This is the single most common GCP confusion, and it costs people an hour each time.

```
gcloud auth login                       # authenticates the gcloud CLI as you
gcloud auth application-default login   # writes Application Default Credentials
```

You need both. `gcloud auth login` authenticates the `gcloud` command. Terraform's google provider does not use that; it uses Application Default Credentials, a separate credential file written by the second command. Run only the first and every `gcloud` command works while every `terraform apply` fails with a confusing authentication error.

Verify both:

```
gcloud auth list                                 # your account, active
gcloud auth application-default print-access-token | head -c 20; echo "..."
```

ADC tokens expire and **Terraform will not refresh them**. When you see `reauth related error (invalid_rapt)`, run `gcloud auth application-default login` again. This is normal and will happen several times during the course.

Authenticating from a container. Both commands try to open a browser, and a container does not have one. Add `--no-launch-browser` to each:

```
gcloud auth login --no-launch-browser
gcloud auth application-default login --no-launch-browser
```

Each prints a URL to open on your own machine and waits for the code it gives you back. This is the GCP counterpart of `aws login --remote`, and you need it for both commands, not just the first.

Step 10 Roles

If you created the project you are already Owner, which is sufficient. If someone else grants you access, you need:

```
for ROLE in roles/storage.admin roles/cloudkms.admin \
            roles/orgpolicy.policyAdmin roles/iam.workloadIdentityPoolAdmin \
            roles/logging.admin roles/iam.serviceAccountAdmin; do
  gcloud projects add-iam-policy-binding "$PROJECT_ID" \
    --member="user:you@example.com" --role="$ROLE"
done
```

`roles/orgpolicy.policyAdmin` and `roles/iam.workloadIdentityPoolAdmin` are **not** included in Owner in all configurations, and Lab 5.4 fails without them with a `PERMISSION_DENIED` that does not say which role is missing.

Step 11 A GCP budget alert

```
gcloud billing budgets create \
  --billing-account=XXXXXX-XXXXXX-XXXXXX \
  --display-name="cgep-lab-monthly" \
  --budget-amount=25USD \
  --threshold-rule=percent=0.5 \
  --threshold-rule=percent=0.9 \
  --threshold-rule=percent=1.0,basis=forecasted-spend
```

Needs `roles/billing.admin` on the billing account. If you do not have it, set the budget in the console; do not skip it. Lab 5.4's Data Access logs are the line item that can surprise you, at $0.50/GB ingested.

Part 3: GitHub, and your own copy of this

Everything you build gets committed, and the capstone is graded on a **public GitHub repository**, submitted as a URL plus the commit SHA you want scored. So the work has to live under your account from the first lab, not in a clone of someone else's repository that you cannot push to.

Step 12 Your account and your copy

If you do not have a GitHub account, sign up at **https://github.com**. Free is enough for everything here, including Codespaces and Actions on a public repo.

Then make your own copy of this repository. Do not just clone it: a clone points at someone else's remote and your first `git push` will be refused.

Use the template. On the repository page choose **Use this template > Create a new repository**. Name it whatever you want your portfolio to be called, and make it **public** if you intend to submit it for the capstone. This gives you a clean repository with no fork relationship, which is what you want for work you are presenting as your own.

If the template button is not offered, **Fork** does the same job; it just shows as forked from the original.

Now clone **your** copy, not this one:

```
git clone https://github.com/YOUR_USER/YOUR_REPO.git
cd YOUR_REPO
```

Check where a push would go. The URL should have your username in it:

```
git remote -v
```

Step 12b Tell git who you are

Commits carry a name and an email. If you have never set them, git either refuses to commit or attributes your work to something unhelpful.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the same address your GitHub account knows about, or GitHub will not link the commits to you and your contribution graph will stay empty.

Inside a container these settings do not carry over from your machine, so set them there too the first time.

Step 12c Authenticate so you can push

The easiest route, and `gh` is already in the container:

```
gh auth login
```

Choose **GitHub.com**, then **HTTPS**, and let it authenticate git for you. From a container with no browser, pick the device-code option it offers rather than the browser one.

Verify:

```
gh auth status
```

Password authentication over HTTPS has not worked for years. If you skip this and try to push, you get an authentication failure that does not explain that a token, not a password, is what is wanted.

Step 12d What you are building

The labs are not eleven separate exercises. They are one repository, built up a piece at a time, and the capstone is what it looks like finished. Nothing is thrown away between labs, and nothing later goes hunting in a previous lab's folder for something it needs.

```
your-repo/
├── terraform/
│   ├── bootstrap/                  Lab 2.2   the state backend, keeps local state
│   ├── primitives/
│   │   ├── compliant-s3/           Lab 2.3   the pattern every bucket reuses
│   │   ├── compliant-gcs-bucket/   Lab 2.4   the same idea on GCP
│   │   └── evidence-vault/         Lab 2.5   Object Lock storage
│   └── baselines/
│       ├── aws/                    Lab 5.2   CloudTrail, Config, Security Hub
│       └── gcp/                    Lab 5.4   Org Policy, federation, audit logs
├── policies/                       Lab 3.3, extended by Lab 3.4
│   ├── *.rego
│   └── tests/
├── scripts/
│   ├── capture-evidence.sh         Lab 2.5
│   ├── mutation-test.sh            Lab 3.3
│   ├── policy-gate.sh              Lab 3.4
│   ├── verify-evidence.sh          Lab 4.4
│   ├── gen-oscal-requirements.py   Lab 6.1
│   └── verify-oscal-graph.sh       Lab 6.1
├── queries/                        Lab 5.2   the Athena SQL that earns AU-6
├── oscal/
│   ├── components/                 Lab 6.1
│   └── profiles/                   Lab 6.1
├── .github/workflows/grc-gate.yml  Lab 4.3, extended by Lab 4.4
├── evidence/
│   └── lab-2-2/ ... lab-6-1/       every lab deposits its proof here
└── cgep.env                        your values, gitignored
```

Read it as a dependency graph rather than a filing system. Lab 2.2's backend is where every later workspace keeps its state. Lab 2.3's primitive is the pattern Lab 2.5's vault and the capstone's buckets both reuse. Lab 3.3's policies are what Lab 3.4 gates on and Lab 4.3 runs in CI. Lab 6.1's OSCAL component points at evidence produced by all of them. **Each lab consumes what the last one produced**, which is why the order in the reading list is not a suggestion.

Your evidence goes to your repository, not to anyone else's. Because you took a copy in Part 3, the `evidence/` you build belongs to you: `git push` goes to your remote, and nothing you capture can reach the repository you copied from. That is the whole reason for taking a copy rather than cloning someone else's.

If you are reading this in the source repository rather than your own copy, note that it deliberately ships **no** evidence. A template hands the next person a snapshot, and one built from someone else's account IDs and bucket names would be worse than an empty one.

A note on this repository's own layout. These notes keep a worked copy of every lab under `reference/lab-2-2/`, `reference/lab-2-3/` and so on, one directory per lab so each can be run and tested independently. That is a reference implementation, not the shape you are building. When a guide says `terraform/primitives/compliant-s3`, that is your repository; when a command says `../lab-2-3`, that is the reference copy. Both work, because Terraform cares about finding a workspace rather than where you keep it. Pick one and stay in it.

Part 4: The toolchain

Ten tools, each with its own installer. There are two ways to get them, and the first one is strongly preferred unless you specifically want the practice.

Step 13a The container (recommended)

The repository ships a devcontainer with the whole toolchain pinned to the versions this course was written against. It builds natively on both `amd64` and `arm64`, which matters: every Mac sold since 2020 is `arm64`, and the manual instructions below are `amd64` only.

There are two ways to run it from the same definition. Pick one.

Option 1: in your browser, nothing installed

On the repository page on GitHub, click **Code**, choose the **Codespaces** tab, then **Create codespace on main**. That is the whole thing. GitHub builds the container on its own machines and gives you VS Code in a browser tab.

This is the answer if you are on Windows, on a work laptop you cannot install software on, or you simply do not want ten new binaries on your machine. The free tier covers far more than this course needs.

Option 2: on your own machine

You need three pieces first:

1. **Docker.** Install Docker Desktop and start it. It must be *running*, not merely installed; the whale icon should be in your menu bar or system tray.
2. **VS Code.**
3. **The Dev Containers extension.** In VS Code open the Extensions panel (`Ctrl+Shift+X`, or `Cmd+Shift+X` on a Mac), search for **Dev Containers**, and install the one published by Microsoft. Its identifier is `ms-vscode-remote.remote-containers`.

You should already have your own copy from Part 3. If you skipped it, go back: cloning this repository directly leaves you unable to push, and your portfolio needs somewhere to live.

Now the step that catches almost everyone:

Open the repository folder itself, on its own. File > Open Folder, then select the folder that contains `README.md` and `.devcontainer`. Not the folder above it, and not a saved multi-root workspace containing several projects.

VS Code only offers to reopen in a container when `.devcontainer` sits at the top level of what you have open. Open your whole projects directory, or a workspace holding six repositories, and it will not find the configuration and will not offer anything.

With the folder open, a prompt appears in the bottom right: **"Folder contains a Dev Container configuration file. Reopen folder to develop in a container."** Click **Reopen in Container**.

If the prompt does not appear, press `Ctrl+Shift+P` (`Cmd+Shift+P` on a Mac), type **Dev Containers: Reopen in Container**, and press Enter.

If VS Code offers "Add Dev Container Configuration Files", say no. That means it did not find the existing configuration, and it is offering to create a brand new empty one. Accepting gives you a second, generic container that does not have any of the tools in it. Close the dialog and check you opened the repository folder on its own, per the box above.

The first start takes a few minutes. It is building the image and pulling roughly 3 GB of tooling. A long quiet pause is the build working, not a hang; later starts reuse the image and open in seconds. You can watch it by clicking **Starting Dev Container (show log)** in the notification.

The container has its own `~/.aws`, separate from your machine's, kept on a named volume so it survives rebuilds. It starts empty, so the container writes the `[profile cgep]` stanza for you the first time it starts. It writes no credentials: you still run `aws login` yourself, and inside a container that means `aws login --remote --profile default`.

Log in from inside the container, not on your machine. This is the right way round, and the reasoning is worth following because the tempting alternative is the insecure one.

The alternative is to log in on your machine and bind-mount `~/.aws` into the container with something like `-v ~/.aws:/home/vscode/.aws`. It is a common pattern and you should not use it here. Your real `~/.aws` accumulates every profile you have ever configured, including work and production accounts. Bind mounting it hands all of them to whatever runs in that container, which includes Terraform providers and any tool a lab downloads. The container needs one sandbox account, so give it exactly that and nothing else.

A dedicated volume is therefore the smaller blast radius, not the larger one.

Know what is in that volume, though. `aws login` caches more than the fifteen minute session. Alongside the access token it stores a refresh token that outlives it, and a DPoP key that binds the two together. The binding means a stolen refresh token is not usable on its own, which is a real improvement over a bearer token, but the pair sitting together in one volume is still a credential. Treat it like one:

```
aws logout --profile default     # clears the cached login
```

And when you are finished with the course entirely, remove the volumes rather than leaving them on the machine:

```
docker volume rm cgep-aws cgep-gcloud
```

What is deliberately absent from all of this is a long-lived access key. There is no `aws_access_key_id` anywhere in the container, on your machine, or in the image. Re-authenticating every fifteen minutes is mildly annoying and it is the reason a leak of this volume expires on its own.

When it finishes, the green box at the bottom left of VS Code reads **Dev Container: CGE-P Labs**, and a terminal there starts you in `/workspaces/cgep-labs-study-notes`. That path is the same folder as on your own machine: edits inside the container are edits to your real files, and your git history is the same one.

Either way you land in a shell with `terraform`, `aws`, `gcloud`, `opa`, `conftest`, `trivy`, `cosign`, `trestle`, `jq` and `gh` already present and on your `PATH`. Skip to Step 14 and verify. `AWS_PROFILE` is already set to `cgep`; you still create the profile itself in Step 4 and still run `aws login`, because the container deliberately contains no credentials.

Logging in from a container. `aws login` opens a browser, and a container does not have one. Use `aws login --remote --profile default`, which prints a URL to open on your own machine and prompts for the code it shows you. Everything else behaves identically. Your `~/.aws` and gcloud config live on named volumes, so rebuilding the container does not cost you another login.

The same applies on the GCP side, and there it bites twice, because `gcloud auth login` and `gcloud auth application-default login` each open a browser separately. Add `--no-launch-browser` to both. This is covered in Step 9.

When the container does not cooperate

"Reopen in Container" is not offered anywhere. Three causes, in order of likelihood. You opened the wrong folder, so re-read the box above. The Dev Containers extension is not installed. Or you have a multi-root workspace open, which you can tell because the Explorer heading reads something like `MYSTUFF (WORKSPACE)` instead of the repository name.

It offers to add configuration files instead. Same cause, same fix. Say no.

An error mentioning Docker, the daemon, or a socket. Docker Desktop is not running, or cannot be reached. Start it and wait for the whale icon to settle. On Linux, note that a VS Code installed as a snap or flatpak is sandboxed and sometimes cannot see Docker Desktop's socket at all; if that is your situation, the `.deb`/`.rpm` build of VS Code avoids it.

You want a shell without VS Code. Running the image directly works, but a bare `docker run` gives you a container with none of your files in it and nothing that survives exit. You have to mount the repository and the credential volumes yourself:

Build it once, giving it a name you can refer to:

```
docker build -t cgep-labs .devcontainer
```

Then run it, from the repository folder:

```
docker run --rm -it \
  -v "$PWD":/workspaces/cgep-labs-study-notes \
  -v cgep-aws:/home/vscode/.aws \
  -v cgep-gcloud:/home/vscode/.config/gcloud \
  -w /workspaces/cgep-labs-study-notes \
  -e AWS_PROFILE=cgep \
  cgep-labs
```

`--rm` throws the container away when you exit, which is fine: your files are on your machine and your logins are in the two named volumes, so nothing you care about lives in the container itself.

Everything works but your cloud login is gone. Check you are using the named volumes above. Without them each container starts with an empty `~/.aws` and you will be logging in every time.

Step 13b Installing by hand

Do this if you want to know exactly what lands on your machine, which is a reasonable thing to want in a security course.

This path is Ubuntu 24.04 on `amd64` only. Every download below names `linux_amd64` explicitly. On an Apple Silicon Mac or an ARM machine each one is a different filename, and on Windows the shell itself differs; use the container, or WSL2 with Ubuntu, rather than translating fourteen URLs by hand.

`~/.local/bin` must be on your `PATH`; every install below is sudo-free.

```
mkdir -p ~/.local/bin
case ":$PATH:" in *":$HOME/.local/bin:"*) : ;; *) echo 'export PATH="$HOME/.local/bin:$PATH"' >>
~/.bashrc && export PATH="$HOME/.local/bin:$PATH" ;; esac
```

Terraform (check the current version at `https://checkpoint-api.hashicorp.com/v1/check/terraform`):

```
TF_VERSION=1.15.8
cd /tmp
curl -fsSLO "https://releases.hashicorp.com/terraform/${TF_VERSION}/terraform_${TF_VERSION}_linux_amd6
4.zip"
curl -fsSLO "https://releases.hashicorp.com/terraform/${TF_VERSION}/terraform_${TF_VERSION}_SHA256SUMS
"
sha256sum --ignore-missing -c "terraform_${TF_VERSION}_SHA256SUMS"
unzip -o "terraform_${TF_VERSION}_linux_amd64.zip" terraform -d ~/.local/bin
```

Note `unzip ... terraform -d`, naming the one file you want. Unzipping the whole archive drops a `LICENSE.txt` into your bin directory.

Verify the signature too. You are taking a course about supply chain evidence:

```
curl -fsSL https://www.hashicorp.com/.well-known/pgp-key.txt | gpg --import
curl -fsSLO "https://releases.hashicorp.com/terraform/${TF_VERSION}/terraform_${TF_VERSION}_SHA256SUMS
.sig"
gpg --verify "terraform_${TF_VERSION}_SHA256SUMS.sig" "terraform_${TF_VERSION}_SHA256SUMS"
```

`Good signature from "HashiCorp Security..."` is what you want. The "key is not certified" warning is expected and only means you have not assigned local trust.

AWS CLI v2. Do not install this from `apt`; the packaged `awscli` is v1, and `aws configure export-credentials` is v2-only.

```
cd /tmp
curl -fsSL "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip
./aws/install --install-dir ~/.local/aws-cli --bin-dir ~/.local/bin
```

Add `--update` to that last command to upgrade later.

gcloud. The widely-cited `curl https://sdk.cloud.google.com | bash` is interactive and stops to ask questions. Use the tarball with explicit flags instead:

```
cd /tmp
curl -fsSL https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-linux-
x86_64.tar.gz -o gcloud.tar.gz
tar -xzf gcloud.tar.gz -C ~/
~/google-cloud-sdk/install.sh --quiet --path-update true --usage-reporting false --command-completion
true
exec -l $SHELL
```

About 92MB. `--path-update true` appends to your shell profile, so start a new shell before `gcloud` resolves.

OPA, Conftest, Trivy, Cosign, gh. These versions were current as of this writing. Check for newer with the loop underneath before you install.

```
cd /tmp
curl -fsSL https://github.com/open-policy-agent/opa/releases/download/v1.19.1/opa_linux_amd64_static -
o ~/.local/bin/opa
chmod +x ~/.local/bin/opa

curl -fsSL https://github.com/open-policy-agent/conftest/releases/download/v0.69.0/
conftest_0.69.0_Linux_x86_64.tar.gz | tar -xz -C ~/.local/bin conftest

curl -fsSL https://github.com/aquasecurity/trivy/releases/download/v0.74.0/
trivy_0.74.0_Linux-64bit.tar.gz | tar -xz -C ~/.local/bin trivy

curl -fsSL https://github.com/sigstore/cosign/releases/download/v3.1.3/cosign-linux-amd64 -o ~/.local/
bin/cosign
chmod +x ~/.local/bin/cosign

curl -fsSL https://github.com/cli/cli/releases/download/v2.98.0/gh_2.98.0_linux_amd64.tar.gz | tar -
xz -C ~/.local/bin --strip-components=2 gh_2.98.0_linux_amd64/bin/gh
```

```
for r in open-policy-agent/opa open-policy-agent/conftest aquasecurity/trivy sigstore/cosign cli/cli;
do
  printf '%-30s %s\n' "$r" "$(curl -sS "https://api.github.com/repos/$r/releases/latest" | jq -r .tag_
name)"
done
```

Each of those tar extractions names the single binary to pull out. Extracting the whole archive scatters `LICENSE` and `README` files into your bin directory.

OPA 1.x. OPA 1.0 made the `rego.v1` syntax the default. Every policy in this curriculum declares `import rego.v1` explicitly, which is valid in 1.x and required by 0.x, so the library runs on both. Inherited v0-syntax policies need `--v0-compatible` or a migration.

Cosign 3.x. Cosign went 2.x to 3.x. The flags Lab 4.4 depends on (`sign-blob --bundle`, and `verify-blob --bundle --certificate-identity-regexp --certificate-oidc-issuer`) are all present in 3.1.3, so the lab is unaffected. Confirm with `cosign verify-blob --help` if you install a later major version.

trestle. Ubuntu 24.04 and other recent distributions enforce PEP 668, so plain `pip install --user` fails with `error: externally-managed-environment`. Use `pipx`:

```
sudo apt install -y pipx jq git unzip curl   # most are present already
pipx install compliance-trestle
```

`pipx` gives trestle its own virtualenv and links the binary into `~/.local/bin`. If you would rather not use pipx: `python3 -m venv ~/.venvs/trestle && ~/.venvs/trestle/bin/pip install compliance-trestle`, then link the binary onto your `PATH`.

Check which OSCAL version your trestle targets. It decides what you write in Lab 6.1.

```
trestle version
# Trestle version v5.0.0 based on OSCAL version 1.2.1
```

Your component definition, your profile, and the NIST catalog you import must all agree on that number. Lab 6.1 uses **1.2.1** to match trestle 5.x. If your trestle reports something different, use what it reports; a mismatch is the most common `trestle validate` failure.

`gh`, the GitHub CLI, per **cli.github.com**, then `gh auth login`.

Step 14 Verify the whole toolchain

This one ships as a file, so you can run it instead of pasting it:

```
bash reference/lab-0-1/scripts/check-prereqs.sh
```

It is reproduced here so you can read what it checks before you run it.

```
#!/usr/bin/env bash
# scripts/check-prereqs.sh
# scripts/check-prereqs.sh
FAIL=0
check() { # check TOOL COMMAND
  if ! command -v "$1" >/dev/null 2>&1; then
    printf '  %-12s MISSING\n' "$1"; FAIL=1; return
  fi
  # eval, because some tools need a pipeline to yield a legible version.
  # cosign prints an ASCII banner, so `head -1` alone reported banner art and
  # the one tool whose whole job is supply-chain integrity was the one this
  # script never actually verified.
  out=$(eval "$2" 2>&1 | head -1)
  if [ -z "$out" ]; then
    # Installed but silent. Treat that as a failure: a check that prints
    # nothing and passes is the thing this course keeps warning about.
```

```
    printf '  %-12s INSTALLED, but reported no version\n' "$1"; FAIL=1
  else
    printf '  %-12s %s\n' "$1" "$out"
  fi
}

echo "=== toolchain ==="
check terraform "terraform version"
check aws       "aws --version"
check gcloud    "gcloud version"
check opa       "opa version"
check conftest  "conftest --version"
check trivy     "trivy --version"
check cosign    "cosign version 2>&1 | grep -iE \"gitversion\""
check trestle   "trestle version"
check jq        "jq --version"
check gh        "gh --version"

echo "=== AWS ==="
aws sts get-caller-identity --profile "${AWS_PROFILE:-cgep}" 2>&1 | head -4 || FAIL=1

echo "=== GCP ==="
gcloud config get-value project 2>&1 | head -1
gcloud auth application-default print-access-token >/dev/null 2>&1 \
  && echo "  ADC: OK" || { echo "  ADC: MISSING (run: gcloud auth application-default login)";
FAIL=1; }

echo
[[ $FAIL -eq 0 ]] && echo "PREREQS OK" || { echo "PREREQS INCOMPLETE"; exit 1; }
```

```
chmod +x scripts/check-prereqs.sh && bash scripts/check-prereqs.sh
```

`PREREQS OK` means you are ready for Lab 2.2.

Part 5: Set your values once

Across the course you supply the same handful of values over and over. `project_name` is asked for by four labs, `aws_region` by five. Typing them each time is how you end up with a bucket in the wrong region, or two labs that disagree about what your project is called.

Terraform reads any environment variable named `TF_VAR_<variable>` as an input. So you can set every one of them once, in a file you source at the start of each session, and stop passing them to individual commands entirely.

Step 15 Create `cgep.env`

In the repository root:

```
cat > cgep.env <<'EOF'
# Sourced at the start of every session. Not committed: it names your account,
# your project, and your repository.

# ---- AWS, needed from Lab 2.2 onward ----
export AWS_PROFILE=cgep
export AWS_REGION=us-east-1
export TF_VAR_aws_region="$AWS_REGION"

# 3 to 21 lowercase letters, digits and hyphens. Becomes your bucket prefix,
# so it has to be globally unique-ish. Your own name or initials work well.
export TF_VAR_project_name=cgep-lab

# dev, staging or prod. Drives the Environment tag.
export TF_VAR_environment=dev

# ---- GCP, needed for Labs 2.4, 3.3 and 5.4 ----
# Uncomment and fill this in when you reach Lab 2.4.
#
# It is commented out rather than left as an empty export, and that matters.
# Terraform stops on a variable that was never set, with "No value for required
# variable". It ACCEPTS an empty string and builds from it. An empty github_org
# produces the OIDC condition "repo:/your-repo:*", which no real token can ever
# match, so the role is simply un-assumable and CI reports "invalid identity
# token" instead of telling you the value is blank.
# export TF_VAR_gcp_project=your-project-id
export TF_VAR_gcp_region=us-central1
if [ -n "${TF_VAR_gcp_project:-}" ]; then
  export CLOUDSDK_CORE_PROJECT="$TF_VAR_gcp_project"
fi

# ---- GitHub, needed for Labs 4.3, 4.4 and 5.4 ----
# Same rule: uncomment these when you reach Lab 4.3, do not leave them blank.
# export TF_VAR_github_org=your-github-username
# export TF_VAR_github_repo=your-repo-name

# ---- Filled in by labs as you finish them. Leave these alone for now. ----
# Lab 2.2 gives you the state backend:
# export TF_VAR_state_bucket=
# export TF_VAR_state_kms_arn=
# Lab 2.5 gives you the evidence vault:
# export TF_VAR_evidence_vault_arn=
EOF
```

Fill in the blanks you know, leave the rest. Then make sure it is never committed, because it names your account and your repository:

```
grep -qxF 'cgep.env' .gitignore || echo 'cgep.env' >> .gitignore
```

Step 16 Source it, every session

```
source cgep.env
```

That is the first command of every working session, before any `terraform` or `aws` command. A convenient check that it worked:

```
echo "profile=$AWS_PROFILE project=$TF_VAR_project_name region=$AWS_REGION"
```

From here on, `terraform plan` and `terraform apply` find their inputs on their own. **You do not need a `terraform.tfvars` and you do not need `-var` flags.** Where a lab shows those, they are the alternative for anyone not using this file; if you sourced `cgep.env` you can leave them off.

In the container this is already done for you. The devcontainer's `postStartCommand` adds the line on every start, pointing at wherever your repository is actually mounted. That placement is deliberate: `~/.bashrc` lives in the image rather than in a mounted volume, so a container rebuild throws it away, and `postStartCommand` runs again afterwards and puts it back.

Outside a container, add it yourself. Derive the path rather than typing one: your repository is named whatever you named it when you created it from the template, so a hardcoded path is a line that silently does nothing.

```
REPO=$(git rev-parse --show-toplevel)
grep -q cgep.env ~/.bashrc \
  || echo "if [ -f \"$REPO/cgep.env\" ]; then . \"$REPO/cgep.env\"; fi" >> ~/.bashrc
```

The `if` form matters more than it looks. Writing `[ -f ... ] && . ...` leaves your shell's exit status at 1 whenever the file is absent, which shows up in prompts that display it and in any script that checks `$?` early.

> **When Terraform prompts you for a variable, that is this.** A fresh shell, or a rebuilt container, and nothing sourced. Answering the prompt works once and teaches you nothing. Press Ctrl+C, source the file, and run it again.

Step 17 Add values as labs produce them

Some values do not exist until a lab creates them. Lab 2.2 produces the state bucket and its key; Lab 2.5 produces the evidence vault. Each of those labs tells you to append its outputs, in this shape:

```
cd reference/lab-2-2
# Delete any previous value first, so re-running a lab updates cgep.env rather
# than stacking a second export that silently shadows the first. This runs
# before the redirect on purpose: sed -i replaces the file, and an already-open
# >> would keep writing to the old inode.
sed -i '/^export TF_VAR_state_bucket=/d;/^export TF_VAR_state_kms_arn=/d' ../../cgep.env
```

```
{
  echo "export TF_VAR_state_bucket=$(terraform output -raw state_bucket)"
  echo "export TF_VAR_state_kms_arn=$(terraform output -raw state_kms_key_arn)"
} >> ../../cgep.env
cd ../..
source cgep.env
```

Reading them out of `terraform output` rather than copying them from your terminal is deliberate. Both strings contain your account ID and a random suffix, and transcription is exactly where this goes wrong.

Part 6: Habits that will save you

Set `AWS_PROFILE` at the start of every session, which `cgep.env` does for you. The `[profile cgep]` entry points at `--profile default`, because that is where `aws login` writes. If you configured a named login profile instead, point it there.

Never export credentials into the shell. The one habit that will cost you an afternoon is `eval "$(aws configure export-credentials --format env)"`. Environment variables outrank profiles, so the moment that snapshot expires your shell is stuck: `aws login` succeeds and Terraform keeps failing, because the provider never looks past the dead variables. If a shell is already in that state, `unset` the four variables rather than trying to refresh them.

Check which identity you are actually using, especially after a re-login:

```
aws sts get-caller-identity --query Arn --output text
```

An `Arn` ending in `:root` means stop and re-authenticate as your IAM user.

Never commit credentials. The repository's `.gitignore` excludes `*.tfvars`, `*.tfstate`, `backend.hcl` and `cgep.env` for this reason. Add a scanner before you need one.

Your evidence is your portfolio, so commit it. Every lab's submission checklist asks for files under `evidence/lab-X-Y/`, and the capstone is graded on a public repository submitted by URL and commit SHA. Those files are the point. They are deliberately not ignored.

Three things to be deliberate about before you make that repository public:

- **Terraform state is the one real risk.** State records every attribute the provider stored, including ones marked sensitive, which is the trap Lab 2.5 is built around. `evidence/**/state.json` is gitignored for that reason. When a checklist asks for it, look at the file first and then add it on purpose:

  ```
  grep -iE 'password|secret|private_key|token' evidence/lab-2-3/state.json
  git add -f evidence/lab-2-3/state.json
  ```

  For Lab 2.3 that file holds no secrets, which is why the checklist asks for it. Do not assume the same of a workspace that manages a database or a key pair.

- **Your account ID will be in there**, in every ARN, and there is no avoiding it if you publish evidence at all. It is not a credential and it is not a breach, but it does help someone enumerate your roles. Decide once whether you are comfortable with that; if you are not, keep the repository private and share it with reviewers directly.
- **Bucket names are globally unique and yours.** Publishing them tells the world which buckets to probe. The controls you built are exactly what makes that survivable, which is worth saying in your write-up rather than hiding.

Destroy what you finish, except the Lab 2.2 state backend, which everything else depends on. Destroy that last, after Lab 6.1, and only after every other workspace is gone. Deleting the state bucket while resources still exist strands them: no state means Terraform can no longer destroy them, and you are deleting things by hand in the console.

Check your bill weekly:

```
aws ce get-cost-and-usage \
  --time-period Start=$(date -u -d '30 days ago' +%Y-%m-%d),End=$(date -u +%Y-%m-%d) \
  --granularity MONTHLY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE \
  --query 'ResultsByTime[].Groups[?Metrics.UnblendedCost.Amount>`0.01`].
[Keys[0],Metrics.UnblendedCost.Amount]' \
  --output table
```

Cost Explorer must be enabled once in the console before this API returns data, and it can take 24 hours to populate.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** Two different causes. If it appears seconds after `aws login`, it is a transient cache race: `aws login` returns your prompt as soon as it writes the profile, while the CLI's own cache still holds the previous entry. Re-run the command. If it persists, your shell is holding an expired snapshot from `export-credentials --format env`, which outranks the profile: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION` and re-run.
- **`No valid credential sources found`** plus **`no EC2 IMDS role found`** from Terraform, while `aws sts get-caller-identity` works fine. Nothing is wrong with EC2; that is just where the credential chain gave up. Neither `aws login` nor Identity Center writes credentials the provider can read. Set up the `credential_process` profile from Part 1 and `export AWS_PROFILE=cgep`. **This is the most common error in the entire course.**
- **`failed to find SSO session section`** from Terraform. Same cause, different wording, same fix.
- **`ExpiredToken`** mid-lab. Your session lapsed. Re-run `aws login` (or `aws sso login` on the Identity Center path), then re-export. Short-lived credentials expiring is the feature, not a fault.
- **`aws: command not found`** after installing. `~/.local/bin` is not on `PATH`, or you need a new shell.
- **`aws configure export-credentials: Invalid choice.`** You have AWS CLI v1. Install v2; do not use `apt`.

- `reauth related error (invalid_rapt)` from Terraform on GCP. Re-run `gcloud auth application-default login`.
- `PERMISSION_DENIED` **on Org Policy or WIF** despite being Owner. Owner does not always include `roles/orgpolicy.policyAdmin` or `roles/iam.workloadIdentityPoolAdmin`. Grant them explicitly.
- `billing account not found`**.** The project has no billing attached. `gcloud billing projects link`.
- **Budget creation returns** `AccessDeniedException`**.** Budgets are a billing API. In an Organization, create it from the management account.

Teardown, when you finish the course

In this order:

1. Destroy every lab workspace except the Lab 2.2 bootstrap.
2. Destroy the Lab 2.2 bootstrap last.
3. Check for orphans: `aws resourcegroupstaggingapi get-resources --tag-filters Key=ComplianceScope,Values=cge-p-lab`. This is what the `ComplianceScope` tag was for all along, and finding your own strays with it is a good final exercise.
4. Schedule KMS key deletion for anything left. Keys bill through their waiting period.
5. GCP: `gcloud projects delete "$PROJECT_ID"`. Deleting the project is the cleanest teardown, and it is 30-day recoverable.
6. If you created a dedicated AWS member account, close it. AWS retains it in `SUSPENDED` for 90 days.
7. Delete any IAM user access keys you created in Path B.

Leave the budget alarm in place until the account is closed. It is the last thing that will tell you about something you forgot.

Revision history

These notes

- New document. Sandbox setup, credentials, toolchain and budget alarms were previously scattered across each lab's prerequisites.
- `aws login` is the recommended credential path, ahead of IAM Identity Center, with access keys demoted to last resort.
- Creating a non-root IAM user comes before the credential step, because `aws login` will otherwise offer the root session without flagging it.
- Terraform is bridged to `aws login` with a `credential_process` profile rather than by exporting credentials into the shell. The profile resolves credentials per command, so a re-login is picked up automatically; an exported snapshot goes stale after about fifteen minutes and then outranks every attempt to fix it.

The official labs

Did not exist. Prerequisites were listed per lab.

CGE-P LAB NOTES

Lab 2.2: The Remote State Backend

Terraform state is the most sensitive artifact in this entire course and the one nobody thinks about. It holds every attribute of every resource you have ever created, in plaintext, including values you marked `sensitive`. By default it sits on your laptop. This lab moves it somewhere defensible, and in doing so unblocks something you would not otherwise discover is broken until Chapter 7.

For reading. Code lines wrap to fit the page; copy code from docs/02_02_remote_state_backend.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/02_02_remote_state_backend.md` in your clone, not from the PDF.

Why this lab exists

Chapter 4 builds a pipeline that runs `terraform init` on a fresh GitHub Actions runner. The capstone requires that pipeline to run `terraform apply` on merge to `main`.

A fresh runner has no state. With a local backend, `terraform plan` on that runner reports every resource as "to be created," because as far as it knows, nothing exists. The first apply after a merge either collides with the resources you already made (`BucketAlreadyExists`) or quietly builds a second copy of your infrastructure.

There is no way to complete the capstone's Layer 3 without this. Build it now, once, and every later lab inherits it.

Learning objectives

- Explain why Terraform state is a confidentiality problem, not just a bookkeeping file.
- Stand up an S3 backend with native locking, KMS encryption, and versioning.
- Solve the bootstrap paradox: creating the bucket that will store the state that describes the bucket.
- Migrate an existing local state into the backend without losing it.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- AWS account with permissions to create S3 buckets, a KMS key, and a DynamoDB table in `us-east-1`.
- Terraform `>= 1.10`. Native S3 state locking (`use_lockfile`) landed in 1.10; the DynamoDB path below is the fallback for older versions.
- AWS CLI v2, with the `credential_process` profile from Lab 0.1 and `export AWS_PROFILE=cgep` set in this shell.
- 20 minutes.

Estimated time & cost

- Time: 20 minutes.
- Cost: under $0.01 for the bucket. **$1/month for the KMS key**, billed whether or not you use it, and it keeps billing through the deletion waiting period after you schedule it for deletion. The optional DynamoDB table is on-demand billing and costs effectively nothing at lab volume.

Unlike the other labs, **do not destroy this one same-day.** Every subsequent lab stores its state here. Destroy it at the end of the course, after Lab 6.1.

Concept: what is actually in a state file

Run this against any workspace you have already applied:

```
terraform state pull | jq '.resources[].instances[].attributes | keys' | head -40
```

Every attribute. Not a summary, not a hash: the values. For an RDS instance that includes `password`. For an IAM access key it includes `secret`. For a Lambda it includes the environment variables.

Three consequences worth stating in your own words before you continue:

1. `sensitive = true` **is a display control, not a security control.** It redacts CLI output. The value sits in state in plaintext regardless.
2. **Read access to state is equivalent to read access to your secrets.** Treat the state bucket as a secrets store, because that is what it is.
3. **Write access to state is equivalent to control of your infrastructure.** An attacker who can edit state can make Terraform delete resources it no longer believes it owns, or adopt resources it never created.

That is why this bucket gets the full baseline plus a deliberately narrow bucket policy.

Architecture

```
bootstrap/  (local state, committed, run once)
     |
     | creates
     v
+-----------------------------------------+
|  S3 bucket: <project>-tfstate-<suffix>    |
|    SSE-KMS (CMK, rotation on)   SC-28     |
|    versioning ON                CP-9      |
|    public access block x4       AC-3      |
|    deny non-TLS                 SC-8      |
|    lifecycle: 90d noncurrent    AU-11     |
+-----------------------------------------+
     ^
     | backend "s3" { use_lockfile = true }
     |
every other lab in this course
```

The bootstrap workspace keeps local state forever. That is not a mistake; see Step 4.

Step-by-step walkthrough

Step 1 The bootstrap paradox

You need a bucket to store state. Creating that bucket produces state. Where does *that* state go?

Three real answers:

Approach	How it works	Trade-off
Local bootstrap state, committed	A tiny separate workspace with a local backend creates the bucket. Its state file is committed to git.	The bootstrap state describes only a bucket, a key, and a table. Nothing secret. This is what we do.
Bootstrap, then migrate its own state in	Create the bucket, then point the bootstrap workspace at itself.	Elegant, and it makes the bucket undeletable by Terraform without a dance. Common in production.
ClickOps the bucket once	Create it by hand, never manage it in Terraform.	Honest, and it means your most security-critical bucket is the one resource with no code review. Reject this.

We take the first. The bootstrap state contains no secrets, and keeping it local and committed means a new learner clones the repo and can see exactly what was created.

Step 2 Write the bootstrap workspace

```
mkdir -p terraform/bootstrap && cd terraform/bootstrap   # reference layout: cd reference/lab-2-2
touch main.tf variables.tf outputs.tf
```

```
# main.tf
terraform {
  required_version = ">= 1.10"
  required_providers {
    aws    = { source = "hashicorp/aws", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
  # No backend block. This workspace is deliberately local.
}

provider "aws" {
  region = var.aws_region

  # CM-6 / CM-8: required tags on every taggable resource.
  default_tags {
    tags = {
      Project         = var.project_name
      Environment     = "shared"
```

```
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
}

data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

resource "random_id" "suffix" {
  byte_length = 4
}

locals {
  account_id  = data.aws_caller_identity.current.account_id
  partition   = data.aws_partition.current.partition
  bucket_name = "${var.project_name}-tfstate-${random_id.suffix.hex}"
}

# ------------------------------------------------------------------------------
# SC-12 / SC-13: customer-managed key, rotation enabled.
# State is a secrets store. It gets a key you control, not an AWS-managed one.
# ------------------------------------------------------------------------------
resource "aws_kms_key" "tfstate" {
  description             = "CMK for Terraform state at rest"
  enable_key_rotation     = true
  deletion_window_in_days = var.kms_deletion_window_days
  policy                  = data.aws_iam_policy_document.kms.json
}

resource "aws_kms_alias" "tfstate" {
  name          = "alias/${var.project_name}-tfstate"
  target_key_id = aws_kms_key.tfstate.key_id
}

data "aws_iam_policy_document" "kms" {
  # Without a root-admin statement the key becomes unmanageable. AWS requires
  # this; omitting it is the single most common way to brick a CMK.
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket" "tfstate" {
  bucket = local.bucket_name
}

# AC-3 / AC-6: ACLs disabled entirely. AWS default since April 2023, and an
# ACL-enabled bucket is itself a Security Hub finding (S3.12).
```

```
resource "aws_s3_bucket_ownership_controls" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
}

# SC-28: encryption at rest with our CMK. bucket_key_enabled cuts KMS request
# charges by up to 99%; state files are written on every apply, so this matters.
resource "aws_s3_bucket_server_side_encryption_configuration" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.tfstate.arn
    }
    bucket_key_enabled = true
  }
}

# CP-9 / SI-7: versioning is mandatory here, not optional. A corrupted or
# truncated state file is recoverable only if the prior version survived.
resource "aws_s3_bucket_versioning" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AC-3: all four flags. Two axes (ACL vs policy) by two states (new vs existing).
resource "aws_s3_bucket_public_access_block" "tfstate" {
  bucket                  = aws_s3_bucket.tfstate.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# AU-11: keep old state versions long enough to recover, not forever.
resource "aws_s3_bucket_lifecycle_configuration" "tfstate" {
  bucket     = aws_s3_bucket.tfstate.id
  depends_on = [aws_s3_bucket_versioning.tfstate]

  rule {
    id     = "expire-noncurrent-state"
    status = "Enabled"

    # An empty filter applies the rule to every object. Provider 5.x errors
    # if a rule has neither filter nor prefix.
    filter {}

    noncurrent_version_expiration {
      noncurrent_days = var.state_version_retention_days
    }

    abort_incomplete_multipart_upload {
```

```
      days_after_initiation = 7
    }
  }
}

# SC-8: refuse any request that did not arrive over TLS.
data "aws_iam_policy_document" "tfstate" {
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.tfstate.arn, "${aws_s3_bucket.tfstate.arn}/*"]

    principals {
      type        = "*"
      identifiers = ["*"]
    }

    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }
}

resource "aws_s3_bucket_policy" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  policy = data.aws_iam_policy_document.tfstate.json
}
```

```
# variables.tf
variable "project_name" {
  type        = string
  description = "Short project identifier. Becomes part of the bucket name."
  default     = "cgep-lab"
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_name))
    error_message = "project_name must be 3-21 lowercase alphanumerics or hyphens, starting with a
letter."
  }
}

variable "aws_region" {
  type        = string
  description = "Region for the state backend. Every workspace must use the same one."
  default     = "us-east-1"
}

variable "kms_deletion_window_days" {
  type        = number
  description = "Waiting period before a scheduled key deletion completes. AWS allows 7-30."
  default     = 7
  validation {
```

```
    condition     = var.kms_deletion_window_days >= 7 && var.kms_deletion_window_days <= 30
    error_message = "kms_deletion_window_days must be between 7 and 30."
  }
}

variable "state_version_retention_days" {
  type        = number
  description = "How long superseded state versions are kept before expiry."
  default     = 90
}
```

```
# outputs.tf
output "state_bucket" {
  value       = aws_s3_bucket.tfstate.id
  description = "Bucket name. Paste into every other workspace's backend block."
}

output "state_kms_key_arn" {
  value       = aws_kms_key.tfstate.arn
  description = "CMK ARN protecting state at rest."
}

output "backend_block" {
  description = "Copy-paste backend configuration for every other lab."
  value       = <<-EOT
    backend "s3" {
      bucket       = "${aws_s3_bucket.tfstate.id}"
      key          = "CHANGE-ME/terraform.tfstate"
      region       = "${var.aws_region}"
      encrypt      = true
      kms_key_id   = "${aws_kms_key.tfstate.arn}"
      use_lockfile = true
    }
  EOT
}
```

That last output is worth stealing as a pattern. A module that emits its own consumer configuration removes an entire category of transcription error.

Step 3 Apply the bootstrap

```
export AWS_PROFILE=cgep
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
```

Expected tail:

```
Apply complete! Resources: 10 added, 0 changed, 0 destroyed.
```

Ten resources: the random suffix, the key, the alias, the bucket, and its six configuration resources (ownership controls, encryption, versioning, public access block, lifecycle, policy).

The lifecycle configuration is the slow one and routinely takes **40 to 60 seconds** while the rest finish in under two. That is S3 settling, not a hang. The KMS key takes about 15 seconds.

Capture the backend block:

```
terraform output -raw backend_block
```

Step 4 Commit the bootstrap state

```
git add -f terraform/bootstrap/terraform.tfstate
git commit -m "Bootstrap: state backend created"
```

The `-f` is required because your `.gitignore` excludes `*.tfstate`, correctly, for every other workspace.

Read this before you object. Committing a state file is normally wrong, and it is right here for exactly one reason: this state describes a bucket, a KMS key, an alias, and five configuration resources. There is no credential, no password, no key material in it. A KMS key's *ARN* is not sensitive; its key material never leaves AWS and never appears in state.

Confirm that for yourself rather than trusting the paragraph:

```
terraform state pull | jq '[.resources[].instances[].attributes | keys] | flatten | unique'
```

If you ever add a resource to this workspace that has a secret attribute, this decision becomes wrong and you must revisit it. Note it in the workspace README so the next person knows the constraint.

Step 5 Native locking, and why the DynamoDB table is optional

Two concurrent applies against one state file corrupt it. Terraform prevents that with a lock.

Terraform 1.10 added `use_lockfile = true`, which uses an S3 conditional write to hold the lock. No second service, nothing extra to pay for, nothing extra to secure. **This is the current recommended approach and what this lab uses.**

The older mechanism is a DynamoDB table with a `LockID` string hash key, referenced by the backend's `dynamodb_table` argument. You need it only if you are pinned below Terraform 1.10. If that is you, add:

```
resource "aws_dynamodb_table" "tflock" {
  name         = "${var.project_name}-tflock"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  server_side_encryption {
    enabled = true
  }

  point_in_time_recovery {
    enabled = true
  }
}
```

and `dynamodb_table = "<name>"` in the backend block. Running both mechanisms at once is supported and harmless.

Step 6 Point a workspace at the backend

In any lab workspace, add the backend block inside `terraform { }`, giving each workspace a distinct `key`:

```
terraform {
  required_version = ">= 1.10"

  backend "s3" {
    bucket       = "cgep-lab-tfstate-XXXXXXXX"
    key          = "labs/2-3/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    kms_key_id   = "arn:aws:kms:us-east-1:ACCOUNT:key/KEY-ID"
    use_lockfile = true
  }

  required_providers { /* ... */ }
}
```

The `key` is a path inside the bucket, and it is how workspaces stay isolated. Reuse one `key` across two labs and the second lab will believe it owns the first lab's resources, then offer to destroy them. Use `labs/<lab-number>/terraform.tfstate` throughout.

Step 7 Migrate existing local state

If you already applied Lab 2.3 locally, do not start over:

```
cd terraform/primitives/compliant-s3   # reference layout: cd reference/lab-2-3
# add the backend block first, then:
terraform init -migrate-state
```

Terraform detects the local state, asks to copy it up, and answering `yes` uploads it. Verify before deleting anything local:

```
terraform state list
aws s3 ls "s3://<state-bucket>/labs/2-3/"
```

Keep the local `terraform.tfstate.backup` until you have confirmed a clean `plan` against the remote state. "No changes" is the signal that migration worked.

Step 8 Record the outputs in `cgep.env`

Lab 4.3 needs this bucket and key by name, and both strings contain your account ID and a random suffix. Read them out rather than retyping them:

```
bucket=$(terraform output -raw state_bucket) &&
kms=$(terraform output -raw state_kms_key_arn) &&
sed -i '/^export TF_VAR_state_bucket=/d;/^export TF_VAR_state_kms_arn=/d' ../../cgep.env &&
{
  echo "export TF_VAR_state_bucket=$bucket"
  echo "export TF_VAR_state_kms_arn=$kms"
} >> ../../cgep.env
source ../../cgep.env
```

The `sed` deletes any previous value before the append, so re-running the lab updates `cgep.env` instead of stacking a second `export` that silently shadows the first. It runs **before** the redirect rather than inside the braces: `sed -i` replaces the file, and a `>>` that is already open would carry on writing to the old inode, which is now unlinked. The lines would simply vanish.

The `&&` chain is not decoration. Run this before the apply and `terraform output` exits non-zero with a warning on stderr, and a naive version would capture that warning text, append it to `cgep.env`, and then execute it when you source the file. Nothing appends unless both values actually exist.

If you have not created `cgep.env`, Lab 0.1 Step 15 sets it up. It is the file you source at the start of every session, and it is gitignored because it names your account.

Verification

Every check below asserts a value, which the describe calls this section used to print could not. `aws s3api get-bucket-encryption` exits 0 whether the answer is `aws:kms` or `AES256`, and `get-bucket-versioning` prints nothing and still exits 0 on a bucket where versioning was never enabled. The state backend is the one bucket every later lab writes to, so a control that quietly is not there costs you more here than anywhere else.

Pass the workspace, so it does not matter where you run it from:

```
bash scripts/verify.sh terraform/bootstrap
```

Every line names a control and either passes or fails with what it expected and what the account actually returned. The run ends in `VERIFIED` or `NOT VERIFIED` and sets the exit status accordingly.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 2.2. Every check asserts a value and exits non-zero if
# the account disagrees.
#
# Run it from the workspace that owns the outputs.
#
# What this replaces: `aws s3api get-bucket-encryption` printed a
# configuration and exited 0 whether the algorithm was aws:kms or AES256, and
# `get-bucket-versioning` printed nothing at all and still exited 0 when
# versioning had never been turned on. Both looked like verification.
set -uo pipefail

# Point this at the workspace whose `terraform output` describes the resources
# you want checked, so it does not matter where you run it from. Guessing the
# working directory is how a verification script cheerfully checks the wrong
# account and passes.
WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0

pass() { printf '  PASS  %-8s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:-(nothing)}'"
  fi
}

BUCKET=$(terraform output -raw state_bucket)
KMS=$(terraform output -raw state_kms_key_arn)

echo "=== configuration ==="

check SC-28 "state encryption" "aws:kms" \
  "$(aws s3api get-bucket-encryption --bucket "$BUCKET" \
       --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.SSEAlgor
ithm' \
       --output text 2>/dev/null)"

# The key has to be yours, not the AWS managed aws/s3 key, or "encrypted with
# KMS" is true and meaningless.
check SC-28 "state CMK" "$KMS" \
```

```
  "$(aws s3api get-bucket-encryption --bucket "$BUCKET" \
       --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.KMSMaste
rKeyID' \
       --output text 2>/dev/null)"

check CP-9 "versioning" "Enabled" \
  "$(aws s3api get-bucket-versioning --bucket "$BUCKET" \
       --query 'Status' --output text 2>/dev/null)"

PAB=$(aws s3api get-public-access-block --bucket "$BUCKET" --output json 2>/dev/null)
[[ -z "$PAB" ]] && PAB='{}'
for flag in BlockPublicAcls IgnorePublicAcls BlockPublicPolicy RestrictPublicBuckets; do
  check AC-3 "$flag" "true" \
    "$(jq -r ".PublicAccessBlockConfiguration.${flag} // empty" <<<"$PAB")"
done

# SC-8. The statement has to exist AND deny, so both are asserted rather than
# eyeballed out of a jq dump.
POLICY=$(aws s3api get-bucket-policy --bucket "$BUCKET" --query Policy --output text 2>/dev/null)
[[ -z "$POLICY" ]] && POLICY='{"Statement":[]}'
check SC-8 "TLS deny effect" "Deny" \
  "$(jq -r '[.Statement[] | select(.Sid=="DenyInsecureTransport")][0].Effect // empty' <<<"$POLICY")"
check SC-8 "TLS deny condition" "false" \
  "$(jq -r '[.Statement[] | select(.Sid=="DenyInsecureTransport")]
[0].Condition.Bool."aws:SecureTransport" // empty' <<<"$POLICY")"

echo "=== enforcement ==="

# A control you have not watched deny something is a control you are guessing
# about. This is the statement above, actually refusing.
OUT=$(aws s3api list-objects-v2 --bucket "$BUCKET" \
  --endpoint-url "http://s3.us-east-1.amazonaws.com" 2>&1)
RC=$?
if [[ $RC -ne 0 && "$OUT" == *AccessDenied* ]]; then
  pass "SC-8" "plain HTTP was refused"
else
  fail "SC-8" "plain HTTP was NOT refused (exit $RC)"
fi

echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the account."
else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-2-2"
mkdir -p "$EVIDENCE"

bash scripts/verify.sh terraform/bootstrap 2>&1 \
  | tee "$EVIDENCE/backend-verification.txt"
```

Capture it after it passes. What goes in the file is a list of controls with a verdict against each, which is a stronger artifact than three raw API dumps a reader has to interpret for themselves.

Portfolio submission checklist

- [ ] `terraform/bootstrap/{main.tf,variables.tf,outputs.tf,README.md}` committed.
- [ ] `terraform/bootstrap/terraform.tfstate` committed, with the README explaining why that is safe here.
- [ ] At least one lab workspace has a `backend "s3"` block with its own `key`.
- [ ] `evidence/lab-2-2/backend-verification.txt`, the output of the four verification commands.
- [ ] README notes the region, because the backend and every workspace must agree on it.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`Error: Failed to get existing workspaces: S3 bucket does not exist.`** The backend block runs before anything else, including provider configuration. The bucket must already exist. That is the entire reason for the bootstrap workspace.
- **`AccessDenied` on `kms:GenerateDataKey` during `init`.** The identity running Terraform needs `kms:Encrypt`, `kms:Decrypt`, and `kms:GenerateDataKey` on the state CMK. The root-admin statement in the key policy covers an admin principal; a scoped CI role needs an explicit grant. Lab 4.3 adds it.
- **`Error acquiring the state lock.`** An interrupted run left a lock. Confirm nobody else is applying, then `terraform force-unlock <lock-id>`. With `use_lockfile`, the lock is an object at `<key>.tflock`; you can see it with `aws s3 ls`.

- `use_lockfile` **unrecognized.** You are below Terraform 1.10. Either upgrade or use the DynamoDB table from Step 5.
- **Backend changed and now `init` fails.** Any edit to the backend block requires `terraform init -reconfigure` (discard the old backend) or `-migrate-state` (copy it across). Terraform will not guess which you meant.
- `BucketAlreadyExists` **on bootstrap.** The `random_id` suffix should prevent it. If you hardcoded a name, change it. Bucket names are globally unique across every AWS account.

Cleanup

Do not clean this up until the course is finished. Everything downstream stores state here.

At the end of the course, after Lab 6.1:

```
# Every other workspace must be destroyed first, or you will strand resources
# with no state describing them.
BUCKET=$(cd terraform/bootstrap && terraform output -raw state_bucket)

aws s3api list-object-versions --bucket "$BUCKET" \
  --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' --output json \
  | aws s3api delete-objects --bucket "$BUCKET" \
    --delete file:///dev/stdin || true

cd terraform/bootstrap && terraform destroy -auto-approve
```

The KMS key enters its deletion waiting period rather than disappearing, and **bills at $1/month for the whole window**. Set `kms_deletion_window_days = 7` (the minimum) if that bothers you.

Stranding is the real risk here. If you delete the state bucket while another lab's resources still exist, those resources have no state and Terraform can no longer destroy them. You will be deleting them by hand in the console, which is exactly the failure mode this course exists to argue against.

How this feeds the capstone

- **Ch 4**, your pipeline's `terraform init` finds real state, so `plan` shows an accurate diff instead of proposing to create everything. The OIDC role gets scoped read/write to this bucket and key.
- **Capstone Layer 3**, `Apply` on merge to `main` works, because the runner and your laptop share one state. Without this lab that step cannot be completed.
- **Your write-up**, the state backend is a defensible answer to "where do your secrets live and who can read them," which is a question every assessor asks and most candidates have not considered.

Revision history

These notes

- New lab. Without a remote backend a fresh CI runner plans against empty state, which makes the capstone's apply-on-merge step uncompletable.
- Uses `use_lockfile` for native S3 locking, with the DynamoDB table documented as the pre-1.10 fallback.
- The bootstrap workspace keeps local state deliberately, with the reasoning and a command to verify it holds no secrets.

The official labs

Did not exist. State was local everywhere.

CGE-P LAB NOTES

Lab 2.3: Building Your First Compliant Resource (AWS S3)

You've spent enough time in spreadsheets pretending they're controls. This lab ends that. You'll write Terraform for an S3 primitive that satisfies eleven NIST 800-53 controls, and produce machine-readable evidence of every one. No screenshots.

Every control in the table below is enforced by a line of code you will write. If you cannot point at the line, it does not go in the table. That discipline is the whole subject of this course.

For reading. Code lines wrap to fit the page; copy code from docs/02_03_first_compliant_resource.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/02_03_first_compliant_resource.md` in your clone, not from the PDF.

Learning objectives

- Express NIST 800-53 controls as Terraform resources, citing each control at the line that enforces it.
- Enforce encryption rather than defaulting to it, in transit as well as at rest.
- Distinguish a control you implemented from a control you merely claimed.
- Build the primitive that the rest of the CGE-P labs reuse and verify automatically.

Controls implemented

Every row here is enforced by a line of code you will write. If you cannot point at the line, it does not go in the table.

Control	Enforced by	Note
SC-8, SC-8(1)	Bucket policy denying `aws:SecureTransport = false`	Transit encryption, the companion to SC-28.
SC-12, SC-13	`aws_kms_key` with `enable_key_rotation`	Key custody and rotation are yours.
SC-28, SC-28(1)	SSE-KMS default + policy denying wrong-key uploads	Enforced, not defaulted.
AC-3	Public access block, all four flags	
AC-6	`BucketOwnerEnforced` ownership	ACLs disabled.
AU-3	`aws_s3_bucket_logging` + delivery grant	Content of audit records.
AU-9	Versioning on the **log** bucket	Log objects cannot be silently overwritten.
AU-11	Lifecycle rules	Retention is a control and a budget.
CM-6, CM-8	Provider `default_tags`	Boundary enumeration by API.
CP-9, SI-7	Versioning on the primary bucket	

AU-6 is deliberately absent. AU-6 is audit *review, analysis, and reporting*. Shipping logs into a bucket is AU-3 plus AU-11; nothing in this lab reads them. Lab 5.2 earns AU-6 by standing up Athena over the trail and running a query. Noticing that a control has been collected rather than reviewed is a more valuable skill than closing the gap.

Prerequisites

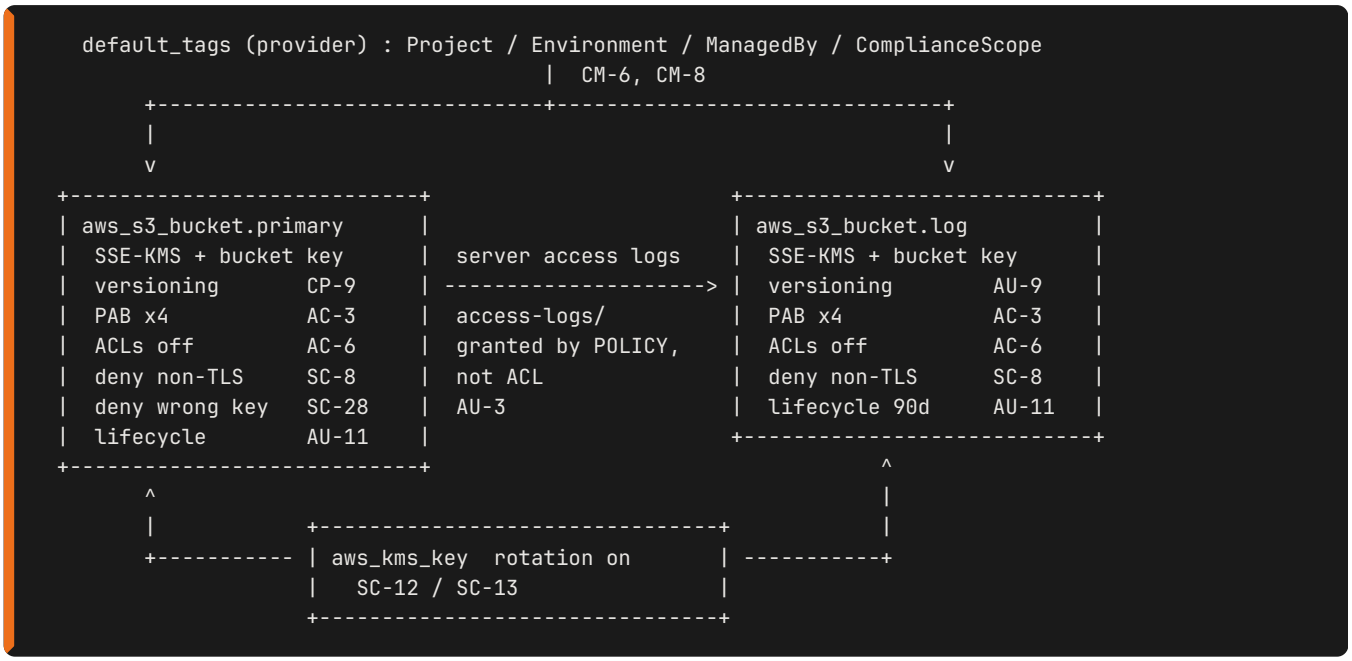
- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 2.2 completed.** You need the state bucket name and the state CMK ARN.
- AWS account with permissions for S3, KMS, and IAM policy reads in `us-east-1`.
- Terraform `>= 1.10`.
- AWS CLI v2, with the `credential_process` profile from Lab 0.1 and `export AWS_PROFILE=cgep` set in this shell.

Estimated time & cost

- Time: 60 to 75 minutes the first time, 15 on repeats. The KMS key policy deserves reading rather than pasting, and that is most of the difference.
- Cost: **$1/month for the KMS key**, prorated to fractions of a cent if you destroy same-day, and it continues billing through the deletion waiting period. Empty S3 buckets have no idle cost. Under $0.02 total for a same-day run.

That KMS dollar is what buys SC-12 and SC-13. An AWS-managed key costs nothing and cannot satisfy either, because you can neither set its rotation policy nor read its key policy.

Architecture



Two things in that diagram are worth noticing now. The log bucket is versioned exactly like the primary, and log delivery is granted by a bucket policy rather than an ACL.

Step-by-step walkthrough

Step 1 Create the project structure

```
mkdir -p terraform/primitives/compliant-s3 && cd terraform/primitives/compliant-s3
touch main.tf variables.tf outputs.tf README.md
```

Step 2 Terraform block, backend, provider

```
# main.tf
terraform {
  required_version = ">= 1.10"

  # A PARTIAL backend configuration. The account-specific values are
  # supplied at init time from a file you do not commit; see below.
  backend "s3" {
    key          = "labs/2-3/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }

  required_providers {
    aws    = { source = "hashicorp/aws", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
}

provider "aws" {
  region = var.aws_region

  # CM-6 / CM-8: required compliance tags on every taggable resource.
  # CM-6 is the configuration-settings argument; CM-8 is the stronger one,
  # because ComplianceScope is how you enumerate an authorization boundary
  # with an API call instead of a spreadsheet.
  default_tags {
    tags = {
      Project         = var.project_name
      Environment     = var.environment
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
}
```

```
data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

resource "random_id" "bucket_suffix" {
  byte_length = 4
}

locals {
  account_id = data.aws_caller_identity.current.account_id
  partition  = data.aws_partition.current.partition

  effective_suffix = coalesce(var.bucket_suffix, random_id.bucket_suffix.hex)
  primary_name     = "${var.project_name}-${var.environment}-data-${local.effective_suffix}"
  log_name         = "${var.project_name}-${var.environment}-logs-${local.effective_suffix}"
}
```

`coalesce` skips both `null` and the empty string, which is why the variable defaults to `null` and the intent reads directly. The longhand alternative is a `var.bucket_suffix != "" ? ... : ...` ternary.

The bucket-name arithmetic is not arbitrary, and it constrains the validation you write next. Worst case: `project_name` at 21, plus `-staging-data-`, plus 8 hex characters, is 43 characters. S3's limit is 63. **The regex ceiling in `variables.tf` was reverse-engineered from this naming scheme.** Change the scheme, move the regex.

Why the backend block is incomplete. Your bucket name and key ARN both carry your AWS account ID, and the capstone requires your repository to be **public**. Hardcoding them here would teach you to commit account identifiers, which is the opposite of the habit this course is building.

Terraform supports partial backend configuration: leave the account-specific keys out of the block and supply them at init. Build `backend.hcl` from Lab 2.2's outputs rather than transcribing them:

```
cd ../lab-2-2
printf 'bucket     = "%s"\nkms_key_id = "%s"\n' \
  "$(terraform output -raw state_bucket)" \
  "$(terraform output -raw state_kms_key_arn)" > ../lab-2-3/backend.hcl
```

Add `backend.hcl` to `.gitignore`, then initialize with it:

```
terraform init -backend-config=backend.hcl
```

Omit `-backend-config` and Terraform prompts for the two missing values, so the workspace still runs for someone who has not made the file. Commit a `backend.hcl.example` holding placeholders so the next person knows what is expected. Every later lab uses the same pattern with its own `key`.

Step 3 variables.tf

```
# variables.tf
variable "project_name" {
  type        = string
  description = "Short project identifier. Becomes part of bucket names and the Project tag."
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_name))
    error_message = "project_name must be 3-21 lowercase alphanumerics or hyphens, starting with a
letter."
  }
}

variable "environment" {
  type        = string
  description = "Deployment environment. Drives the Environment tag and downstream policy decisions."
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be one of: dev, staging, prod."
  }
}

variable "aws_region" {
  type        = string
  description = "Region. Must match the region of the Lab 2.2 state backend."
  default     = "us-east-1"
}

variable "bucket_suffix" {
  type        = string
  description = "Optional suffix to force a specific bucket name. Defaults to a random_id."
  default     = null
}

variable "kms_deletion_window_days" {
  type        = number
  description = "Waiting period before a scheduled CMK deletion completes. AWS allows 7-30."
  default     = 7
  validation {
    condition     = var.kms_deletion_window_days >= 7 && var.kms_deletion_window_days <= 30
    error_message = "kms_deletion_window_days must be between 7 and 30."
  }
}

variable "log_retention_days" {
  type        = number
  description = "AU-11. How long access-log objects are kept before expiry."
  default     = 90
  validation {
    condition     = var.log_retention_days >= 1
    error_message = "log_retention_days must be at least 1."
  }
}

variable "noncurrent_version_retention_days" {
```

```
  type        = number
  description = "How long superseded object versions survive before expiry."
  default     = 30
}
```

Validation blocks are **preventive** controls. They reject bad input at plan time, before anything reaches AWS. Compare that to detecting a mistyped `Environment` tag in a Config rule three hours after deploy, which is **detective** and strictly weaker. That distinction is worth more to you than the regex.

Step 4 The KMS key

```
# main.tf (continued)

# ------------------------------------------------------------------------------
# SC-12 (key establishment and management) + SC-13 (cryptographic protection)
# A customer-managed key with rotation. This is the control an AWS-managed key
# cannot satisfy: you cannot set rotation policy or read the key policy of one.
# ------------------------------------------------------------------------------
resource "aws_kms_key" "bucket" {
  description             = "CMK for ${local.primary_name} and its access-log bucket"
  enable_key_rotation     = true
  deletion_window_in_days = var.kms_deletion_window_days
  policy                  = data.aws_iam_policy_document.kms.json
}

resource "aws_kms_alias" "bucket" {
  name          = "alias/${var.project_name}-${var.environment}-s3"
  target_key_id = aws_kms_key.bucket.key_id
}

data "aws_iam_policy_document" "kms" {
  # Omitting this statement makes the key permanently unmanageable. AWS will
  # let you do it. Do not.
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }

  # S3 server access log delivery encrypts each log object with this key, so
  # the logging service principal needs to use it. The SourceAccount condition
  # is confused-deputy protection: it stops another account from inducing the
  # S3 service to use your key on their behalf.
  statement {
    sid       = "AllowS3LogDelivery"
    effect    = "Allow"
    actions   = ["kms:GenerateDataKey", "kms:Decrypt"]
```

```
    resources = ["*"]
    principals {
      type        = "Service"
      identifiers = ["logging.s3.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceAccount"
      values   = [local.account_id]
    }
  }
}
```

`resources = ["*"]` inside a *key policy* means "this key," not "every key." Key policies are attached to one key and scoped to it. This confuses everybody once.

Constraint worth knowing before you hit it. An S3 server-access-log *destination* bucket encrypted with SSE-KMS requires **S3 Bucket Keys enabled** on that bucket. This is why `bucket_key_enabled = true` appears on the log bucket below and is not optional there. If log objects never appear, this is the first thing to check.

Step 5 The two buckets and the shared baseline

```
# main.tf (continued)

resource "aws_s3_bucket" "primary" {
  bucket = local.primary_name
}

resource "aws_s3_bucket" "log" {
  bucket = local.log_name
}

# AC-6: ACLs disabled entirely. AWS's default for new buckets since April 2023.
# The tempting alternative, BucketOwnerPreferred plus a log-delivery-write ACL,
# re-enables ACLs on the log bucket and trips Security Hub control S3.12. Log
# delivery is granted by bucket policy instead; see Step 7.
resource "aws_s3_bucket_ownership_controls" "primary" {
  bucket = aws_s3_bucket.primary.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
}

resource "aws_s3_bucket_ownership_controls" "log" {
  bucket = aws_s3_bucket.log.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
```

```
}

# SC-28: encryption at rest with the CMK, not the AWS-managed default.
resource "aws_s3_bucket_server_side_encryption_configuration" "primary" {
  bucket = aws_s3_bucket.primary.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.bucket.arn
    }
    bucket_key_enabled = true
  }
}

# bucket_key_enabled is REQUIRED here, not merely economical: an SSE-KMS
# server-access-log destination bucket must have S3 Bucket Keys on.
resource "aws_s3_bucket_server_side_encryption_configuration" "log" {
  bucket = aws_s3_bucket.log.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.bucket.arn
    }
    bucket_key_enabled = true
  }
}

# CP-9 / SI-7 on the primary: prior object states survive deletion and overwrite.
resource "aws_s3_bucket_versioning" "primary" {
  bucket = aws_s3_bucket.primary.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AU-9 on the log bucket: protection of audit information.
# Easy to omit, and the omission is quiet. Without it, log objects can be
# overwritten in place, which leaves the bucket holding your audit records
# less protected than the bucket holding your data.
resource "aws_s3_bucket_versioning" "log" {
  bucket = aws_s3_bucket.log.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AC-3: all four flags. Three is not enough.
resource "aws_s3_bucket_public_access_block" "primary" {
  bucket                  = aws_s3_bucket.primary.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_public_access_block" "log" {
```

```
  bucket                  = aws_s3_bucket.log.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Those four flags are not four copies of one idea. They are a 2x2:

	Blocks *new*	**Neutralizes** *existing*
ACLs	`block_public_acls`	`ignore_public_acls`
Policies	`block_public_policy`	`restrict_public_buckets`

Set three and you have either left an existing public grant live or left the door open for a new one.

Step 6 Lifecycle, for AU-11 and for your bill

```
# main.tf (continued)

# AU-11: audit record retention. Also the difference between a lab and an
# unbounded S3 invoice.
resource "aws_s3_bucket_lifecycle_configuration" "log" {
  bucket     = aws_s3_bucket.log.id
  depends_on = [aws_s3_bucket_versioning.log]

  rule {
    id     = "expire-access-logs"
    status = "Enabled"

    # An empty filter applies the rule to all objects. Provider 5.x rejects a
    # rule that has neither filter nor prefix.
    filter {}

    expiration {
      days = var.log_retention_days
    }

    noncurrent_version_expiration {
      noncurrent_days = var.noncurrent_version_retention_days
    }

    abort_incomplete_multipart_upload {
      days_after_initiation = 7
    }
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "primary" {
  bucket     = aws_s3_bucket.primary.id
```

```
  depends_on = [aws_s3_bucket_versioning.primary]

  rule {
    id     = "expire-noncurrent-versions"
    status = "Enabled"
    filter {}

    noncurrent_version_expiration {
      noncurrent_days = var.noncurrent_version_retention_days
    }

    abort_incomplete_multipart_upload {
      days_after_initiation = 7
    }
  }
}
```

Note what the primary bucket's rule does *not* do: it never expires current object versions. Expiring live data is a business decision, not a compliance default. Superseded versions and abandoned multipart uploads are pure carrying cost, so those go.

Step 7 Bucket policies

```
# main.tf (continued)

data "aws_iam_policy_document" "primary" {
  # SC-8: transmission confidentiality. Refuse anything not over TLS.
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.primary.arn, "${aws_s3_bucket.primary.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  # SC-28 enforcement: encryption becomes a condition of upload, not a default
  # applied on your behalf. Read the warning below before you apply this.
  statement {
    sid       = "DenyUnencryptedObjectUploads"
    effect    = "Deny"
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.primary.arn}/*"]
    principals {
      type        = "*"
```

```
      identifiers = ["*"]
    }
    condition {
      test     = "StringNotEquals"
      variable = "s3:x-amz-server-side-encryption"
      values   = ["aws:kms"]
    }
  }

  # SC-28(1): the right algorithm with the wrong key is not the control.
  statement {
    sid       = "DenyWrongKmsKey"
    effect    = "Deny"
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.primary.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "StringNotEquals"
      variable = "s3:x-amz-server-side-encryption-aws-kms-key-id"
      values   = [aws_kms_key.bucket.arn]
    }
  }
}

resource "aws_s3_bucket_policy" "primary" {
  bucket = aws_s3_bucket.primary.id
  policy = data.aws_iam_policy_document.primary.json
}

data "aws_iam_policy_document" "log" {
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.log.arn, "${aws_s3_bucket.log.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  # AU-3: the modern replacement for the log-delivery-write ACL.
  # Both conditions matter. SourceArn scopes the grant to exactly one source
  # bucket; SourceAccount stops a different account from using the S3 logging
  # service as a confused deputy to write into your bucket.
  statement {
    sid       = "AllowS3ServerAccessLogDelivery"
    effect    = "Allow"
```

```
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.log.arn}/access-logs/*"]
    principals {
      type        = "Service"
      identifiers = ["logging.s3.amazonaws.com"]
    }
    condition {
      test     = "ArnLike"
      variable = "aws:SourceArn"
      values   = [aws_s3_bucket.primary.arn]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceAccount"
      values   = [local.account_id]
    }
  }
}

resource "aws_s3_bucket_policy" "log" {
  bucket = aws_s3_bucket.log.id
  policy = data.aws_iam_policy_document.log.json
}

# AU-3: wire the primary bucket's access logs into the log bucket.
# depends_on is genuinely required: the delivery grant must exist before S3
# will accept the logging configuration, and nothing here references the
# policy, so Terraform cannot infer the order. Most depends_on in the wild is
# cargo cult; this one is not.
resource "aws_s3_bucket_logging" "primary" {
  bucket        = aws_s3_bucket.primary.id
  target_bucket = aws_s3_bucket.log.id
  target_prefix = "access-logs/"

  depends_on = [aws_s3_bucket_policy.log]
}
```

Read this before you apply. `DenyUnencryptedObjectUploads` uses `StringNotEquals`, and in IAM a `StringNotEquals` condition evaluates to **true when the key is absent**. So this Deny fires on any `PutObject` that does not explicitly carry the `x-amz-server-side-encryption` header, including uploads that the bucket's own default encryption would have encrypted anyway.

That is the intended behavior and it has a real usability cost. Uploads must be explicit:

```
aws s3 cp ./file.txt "s3://$BUCKET/file.txt" \
  --sse aws:kms --sse-kms-key-id "$KMS_ARN" --profile <your-sandbox>
```

Deciding whether to accept that friction is a genuine control-design trade-off, and defending your answer is exactly the kind of reasoning the capstone write-up scores. If you decide against it, delete the statement and say why in your README. Do not leave it in and route around it.

Note what is deliberately **not** on the log bucket: the SSE-enforcement denies. The S3 log delivery service does not send an `x-amz-server-side-encryption` header, so adding those statements there silently breaks log delivery. The bucket's default encryption still applies. This asymmetry is the single easiest way to break this lab.

Step 8 outputs.tf

```
# outputs.tf
output "bucket_name" {
  value       = aws_s3_bucket.primary.bucket
  description = "Primary bucket name."
}

output "bucket_arn" {
  value       = aws_s3_bucket.primary.arn
  description = "Primary bucket ARN."
}

output "log_bucket_name" {
  value       = aws_s3_bucket.log.bucket
  description = "Access-log bucket name."
}

output "log_bucket_arn" {
  value       = aws_s3_bucket.log.arn
  description = "Access-log bucket ARN."
}

output "kms_key_arn" {
  value       = aws_kms_key.bucket.arn
  description = "CMK protecting both buckets (SC-12 / SC-13 attestation)."
}

# The attestation is the module's evidence surface. Lab 2.4 builds the same
# thing on GCP; Lab 6.1's OSCAL component points at the JSON this lands in.
output "compliance_attestation" {
  description = "Computed attestation of the controls this primitive enforces."
  value = {
    encryption_algorithm = one([
      for rule in aws_s3_bucket_server_side_encryption_configuration.primary.rule :
      rule.apply_server_side_encryption_by_default[0].sse_algorithm
    ])
    kms_key_arn             = aws_kms_key.bucket.arn
    kms_rotation_enabled    = aws_kms_key.bucket.enable_key_rotation
    primary_versioning      = aws_s3_bucket_versioning.primary.versioning_configuration[0].status
    log_versioning          = aws_s3_bucket_versioning.log.versioning_configuration[0].status
    public_access_blocked   = alltrue([
      aws_s3_bucket_public_access_block.primary.block_public_acls,
      aws_s3_bucket_public_access_block.primary.block_public_policy,
      aws_s3_bucket_public_access_block.primary.ignore_public_acls,
      aws_s3_bucket_public_access_block.primary.restrict_public_buckets,
    ])
    acls_disabled = one([
```

```
      for rule in aws_s3_bucket_ownership_controls.primary.rule : rule.object_ownership
    ]) == "BucketOwnerEnforced"
    tls_required          = true
    access_logging_target = aws_s3_bucket_logging.primary.target_bucket
    # Derived, like every other field here. Reading var.log_retention_days
    # would attest to what we asked for rather than what the bucket enforces.
    log_retention_days = one([
      for r in aws_s3_bucket_lifecycle_configuration.log.rule :
      r.expiration[0].days if r.id == "expire-access-logs"
    ])
  }
}
```

Why the `one(...)` expression. Terraform models the `rule` block of `aws_s3_bucket_server_side_encryption_configuration` as a **set**, and sets are not index-addressable. The `for` expression flattens it to a tuple, then `one()` extracts the single element.

The alternative, `tolist(...)[0]`, works and is worse. If there were ever two rules, it returns an arbitrary one silently; `one()` raises an error. **This output is an attestation, so failing loudly beats attesting confidently to the wrong value.** That instinct is the difference between an evidence pipeline and a liability.

Step 9 init / plan / apply

`project_name` and `environment` are the two variables with no default, which is deliberate: they name your buckets and drive the `Environment` tag, and a default would let you deploy to the wrong environment by forgetting a flag. Give them values in a `terraform.tfvars`, which Terraform reads automatically:

```
cat > terraform.tfvars <<'EOF'
project_name = "cgep-lab"
environment  = "dev"
EOF
```

`*.tfvars` is gitignored, so this stays on your machine. Skip this step and Terraform silently prompts for both values on every `plan`, `apply` and `destroy`, and fails outright under `-input=false`, which is how CI runs.

If you set up `cgep.env` in Lab 0.1 Step 15, you already have these as `TF_VAR_project_name` and `TF_VAR_environment`, and you can skip the file entirely. `source cgep.env` and Terraform finds them on its own.

```
export AWS_PROFILE=cgep
terraform init
terraform fmt
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
```

Run `terraform fmt` from **this directory**. It rewrites in place and prints only the names of files it changed, so silence means either "already formatted" or "no `.tf` files here." Use `terraform fmt -check` and read the exit code if you want certainty: `0` formatted, `3` changes needed, `1` error.

Expected tail:

```
Apply complete! Resources: 18 added, 0 changed, 0 destroyed.

Outputs:

bucket_arn = "arn:aws:s3:::cgep-lab-dev-data-XXXXXXXX"
bucket_name = "cgep-lab-dev-data-XXXXXXXX"
compliance_attestation = {
  "access_logging_target" = "cgep-lab-dev-logs-XXXXXXXX"
  "acls_disabled" = true
  "encryption_algorithm" = "aws:kms"
  "kms_key_arn" = "arn:aws:kms:us-east-1:...:key/..."
  "kms_rotation_enabled" = true
  "log_retention_days" = 90
  "log_versioning" = "Enabled"
  "primary_versioning" = "Enabled"
  "public_access_blocked" = true
  "tls_required" = true
}
kms_key_arn = "arn:aws:kms:us-east-1:...:key/..."
log_bucket_arn = "arn:aws:s3:::cgep-lab-dev-logs-XXXXXXXX"
log_bucket_name = "cgep-lab-dev-logs-XXXXXXXX"
```

Eighteen resources: the random suffix, the KMS key and its alias, the two buckets, and the configuration resources that harden each of them.

Step 10 Capture evidence

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-2-3"
mkdir -p "$EVIDENCE"
terraform show -json tfplan > "$EVIDENCE/plan.json"
terraform show -json        > "$EVIDENCE/state.json"
terraform output -json compliance_attestation > "$EVIDENCE/attestation.json"
```

Open `state.json` and find each control rather than taking the table's word for it:

- SC-28 at `server_side_encryption_configuration[].rule[].apply_server_side_encryption_by_default[].sse_algorithm`
- SC-8 in the primary bucket policy's `DenyInsecureTransport` statement
- AC-3 in the four `true`s
- AU-9 in `aws_s3_bucket_versioning.log`
- CM-6 in `tags_all`

`terraform show -json` output **is** machine-readable compliance evidence. No screenshots.

One caution the pipeline chapters depend on. `state.json` for other workspaces can contain secrets in plaintext. This workspace's state does not, but the `capture-evidence.sh` script in Lab 2.5 runs `terraform state pull` and uploads the result into an Object Lock vault. Uploading a secret into immutable storage means you cannot delete it. Lab 2.5 addresses this directly.

Verification

Every check below asserts a value. That is the whole difference from the describe calls this section used to print. `aws s3api get-bucket-encryption` exits 0 whether the answer is `aws:kms` or `AES256`, and since S3 turned on default encryption for every bucket it essentially never fails at all. `get-bucket-versioning` on a bucket where versioning was never enabled prints nothing and also exits 0. A verification step that cannot fail is the same mistake as a control that cannot fail, one page later in your own repository.

```
bash scripts/verify.sh terraform/primitives/compliant-s3
```

You get a `PASS` or `FAIL` line naming the control, and a final `VERIFIED` or `NOT VERIFIED` that sets the exit status. Read the failures rather than the passes: a `FAIL` line tells you the control, what it expected, and what the account actually returned.

The last three checks are the ones an assessor cares about, because they make the controls deny something rather than merely describe themselves. Plain HTTP is refused, an upload with no encryption header is refused, and the same upload naming the key succeeds. **The success matters as much as the denials**: a policy that refuses everything is broken, not strict.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 2.3. Every check asserts a value and exits non-zero if
# the cloud disagrees.
#
# The point is that this script CAN fail. The describe calls it replaces could
# not. `aws s3api get-bucket-encryption` exits 0 whether the answer is aws:kms
# or AES256, and since S3 turned on default encryption for every bucket it
# essentially never fails at all. `get-bucket-versioning` on a bucket where
# versioning was never enabled prints nothing and also exits 0. Reading that
# output and nodding is not verification, it just feels like it.
set -uo pipefail

# Point this at the workspace whose `terraform output` describes the resources
# you want checked, so it does not matter where you run it from. Guessing the
# working directory is how a verification script cheerfully checks the wrong
# account and passes.
WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0
```

```
pass() { printf '  PASS  %-8s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:-(nothing)}'"
  fi
}

refuse() { # refuse CONTROL LABEL COMMAND...
  local control=$1 label=$2 out rc
  shift 2
  out=$("$@" 2>&1)
  rc=$?
  if [[ $rc -ne 0 && "$out" == *AccessDenied* ]]; then
    pass "$control" "$label was refused"
  else
    fail "$control" "$label was NOT refused (exit $rc). A control you have not watched deny is a
guess."
  fi
}

BUCKET=$(terraform output -raw bucket_name)
LOGS=$(terraform output -raw log_bucket_name)
KMS=$(terraform output -raw kms_key_arn)

echo "=== configuration ==="

# SC-28. The algorithm, not merely the presence of a configuration.
check SC-28 "bucket encryption" "aws:kms" \
  "$(aws s3api get-bucket-encryption --bucket "$BUCKET" \
       --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.SSEAlgor
ithm' \
       --output text 2>/dev/null)"

# SC-12 / SC-13. AWS renders booleans as True.
check SC-12 "key rotation" "True" \
  "$(aws kms get-key-rotation-status --key-id "$KMS" \
       --query 'KeyRotationEnabled' --output text 2>/dev/null)"

# CP-9 and AU-9. An unversioned bucket has no Status field at all, so this
# reads as empty and fails, where the bare describe printed nothing and passed.
check CP-9 "primary versioning" "Enabled" \
  "$(aws s3api get-bucket-versioning --bucket "$BUCKET" \
       --query 'Status' --output text 2>/dev/null)"
check AU-9 "log versioning" "Enabled" \
  "$(aws s3api get-bucket-versioning --bucket "$LOGS" \
       --query 'Status' --output text 2>/dev/null)"

# AC-3. All four flags, named individually so a failure says which one.
PAB=$(aws s3api get-public-access-block --bucket "$BUCKET" --output json 2>/dev/null)
# A bucket with no public access block at all fails the call, and an empty
# string is not JSON, so jq would error instead of reporting the missing flag.
```

```
[[ -z "$PAB" ]] && PAB='{}'
for flag in BlockPublicAcls IgnorePublicAcls BlockPublicPolicy RestrictPublicBuckets; do
  check AC-3 "$flag" "true" \
    "$(jq -r ".PublicAccessBlockConfiguration.${flag} // empty" <<<"$PAB")"
done

# AC-6.
check AC-6 "object ownership" "BucketOwnerEnforced" \
  "$(aws s3api get-bucket-ownership-controls --bucket "$BUCKET" \
       --query 'OwnershipControls.Rules[0].ObjectOwnership' --output text 2>/dev/null)"

# AU-3. The target has to be the log bucket, not merely some bucket.
check AU-3 "log target" "$LOGS" \
  "$(aws s3api get-bucket-logging --bucket "$BUCKET" \
       --query 'LoggingEnabled.TargetBucket' --output text 2>/dev/null)"

# AU-11. Compared against the module's own attestation, so this catches drift
# between what the code claims and what the account actually holds.
EXPECTED_DAYS=$(terraform output -json compliance_attestation | jq -r '.log_retention_days')
check AU-11 "log expiry days" "$EXPECTED_DAYS" \
  "$(aws s3api get-bucket-lifecycle-configuration --bucket "$LOGS" \
       --query 'Rules[?ID==`expire-access-logs`].Expiration.Days | [0]' \
       --output text 2>/dev/null)"

echo "=== enforcement ==="

# SC-8. Configuration says TLS is required; this is the bucket saying no.
refuse SC-8 "plain HTTP" \
  aws s3api list-objects-v2 --bucket "$BUCKET" \
  --endpoint-url "http://s3.us-east-1.amazonaws.com"

echo test > /tmp/cgep-verify.txt

# SC-28. An upload with no encryption header.
refuse SC-28 "unencrypted upload" \
  aws s3 cp /tmp/cgep-verify.txt "s3://$BUCKET/verify.txt"

# The same upload done correctly has to succeed, or the policy is simply
# broken rather than strict.
if aws s3 cp /tmp/cgep-verify.txt "s3://$BUCKET/verify.txt" \
     --sse aws:kms --sse-kms-key-id "$KMS" >/dev/null 2>&1; then
  pass "SC-28" "correctly encrypted upload succeeded"
else
  fail "SC-28"
"correctly encrypted upload was refused. The policy denies everything, which is not the control."
fi
rm -f /tmp/cgep-verify.txt

echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the account."
else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-2-3"
mkdir -p "$EVIDENCE"

bash scripts/verify.sh terraform/primitives/compliant-s3 2>&1 \
  | tee "$EVIDENCE/enforcement-test.txt"
```

Capture it after it passes. A transcript full of `FAIL` lines is a record of an environment that does not hold its controls, which is worth having while you fix it and is not what the checklist is asking for.

Portfolio submission checklist

- [ ] `terraform/primitives/compliant-s3/{main.tf,variables.tf,outputs.tf,README.md}`
- [ ] `README.md`, one paragraph naming the eleven controls and stating plainly that AU-6 is **not** among them and why.
- [ ] `evidence/lab-2-3/plan.json`
- [ ] `evidence/lab-2-3/state.json`
- [ ] `evidence/lab-2-3/attestation.json`
- [ ] `evidence/lab-2-3/enforcement-test.txt`, a transcript of the three denials and one success above.

That last item is new here, and it is the one an assessor would actually want.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`BucketAlreadyExists`.** S3 names are globally unique across every AWS account. The `random_id` suffix prevents this unless you set `bucket_suffix` by hand.
- **Access log objects never appear.** Three causes, in order of likelihood: (1) `bucket_key_enabled` is not `true` on the SSE-KMS log bucket, which server access logging requires; (2) the KMS key policy is missing the

`logging.s3.amazonaws.com` grant; (3) you have been waiting less than a few hours. Server access log delivery is best-effort and slow. Generate traffic against the primary bucket first, or there is nothing to log.

- **`AccessDenied` on every upload after apply.** Working as designed. See the `DenyUnencryptedObjectUploads` warning in Step 7 and pass `--sse aws:kms --sse-kms-key-id`.
- **`MalformedPolicy: Policy has invalid action`** on the log bucket. `logging.s3.amazonaws.com` needs `s3:PutObject` on the object path (`arn/access-logs/*`), not on the bucket ARN.
- **`InvalidArgument` on the lifecycle rule.** Provider 5.x requires each rule to have `filter` or `prefix`. An empty `filter {}` means "all objects."
- **`failed to find SSO session section`.** Terraform's AWS provider does not always parse SSO config the way the CLI does. The `credential_process` profile sidesteps it entirely, because resolution happens in the CLI rather than in the provider.
- **`Error acquiring the state lock`.** Leftover from an interrupted run. With Lab 2.2's `use_lockfile`, the lock is an object at `labs/2-3/terraform.tflock`. Confirm nobody else is applying, then `terraform force-unlock <lock-id>`.
- **Region mismatch on verification.** The provider wins for resource creation; `aws s3api` reads fall back to your profile's region. Pass `--region us-east-1` if a check returns `NoSuchBucket`.

Cleanup

Versioned buckets refuse to be destroyed while they hold any object version, and now **both** buckets are versioned. Empty both, including delete markers:

```
for B in "$(terraform output -raw bucket_name)" "$(terraform output -raw log_bucket_name)"; do
  aws s3api list-object-versions --bucket "$B" --output json \
    | python3 -c 'import sys,json; d=json.load(sys.stdin); items=[*d.get("Versions",
[]),*d.get("DeleteMarkers",[])]; print(json.dumps({"Objects":
[{"Key":o["Key"],"VersionId":o["VersionId"]} for o in items]}))' > /tmp/del.json
  # delete-objects rejects an empty Objects list, so skip when there is nothing to do
  if [ "$(jq '.Objects | length' /tmp/del.json)" -gt 0 ]; then
    aws s3api delete-objects --bucket "$B" --delete file:///tmp/del.json
  fi
done

terraform destroy -auto-approve
```

The `jq` length guard matters. `delete-objects` rejects an empty `Objects` list, and the tempting fix is a trailing `|| true`, which swallows that error along with every real failure.

The KMS key does not disappear. It enters the deletion waiting period you set with `kms_deletion_window_days` (minimum 7) and **bills the full $1/month throughout**. That is unavoidable; AWS deliberately makes key destruction slow.

Where this lands in the capstone. The starter application ships eight named gaps in its `GAPS.md`, and the pattern you just built closes four of them on a bucket you did not write:

Gap	What is missing in the starter	What closes it here
GAP-01	`aws_s3_bucket.uploads` uses AWS-managed SSE-S3, so PHI keys are not in customer custody	your CMK plus `aws_s3_bucket_server_side_encryption_configuration`
GAP-03	no policy denying non-TLS requests	the `aws:SecureTransport` deny, SC-8
GAP-04	no versioning, so a PHI overwrite is unrecoverable	`aws_s3_bucket_versioning`, CP-9 and SI-7
GAP-07	the Lambda role holds `s3:*` on the workload bucket	the least-privilege argument behind AC-6

Worth reading `GAPS.md` now rather than at the capstone. These stop being abstract controls once you have seen the resource they are missing from.

How this feeds the capstone

This is your first compliant primitive, and by the end of the course you will have a dozen.

- **Ch 3**, your Rego policies read this exact `plan.json`. Because these notes add SC-8 and CMK enforcement, Lab 3.4's policy library gains rules for both, and the `compliance_attestation` output gives the policies a stable shape to assert against.
- **Ch 4**, the `plan + show -json` ritual becomes a CI step, and `evidence/lab-2-3/` seeds the signed bundles. Worth knowing now: Chapter 4's gate runs Trivy, which flags both a missing HTTPS-enforcement policy and non-CMK encryption at HIGH. Build the bucket the way this lab does and the gate stays quiet; take either shortcut and your own pipeline will tell you so two chapters later.
- **Ch 6**, you write an OSCAL Component Definition for this module. SC-28's `implemented-requirement` points at `evidence/lab-2-3/state.json`. An assessor reads the OSCAL, follows the link, sees the same JSON you generated. The audit becomes a traversal.
- **Capstone Layer 1**, the required "hardening overrides" on the starter's uploads bucket are this pattern applied to a bucket you did not create. The capstone also requires a customer-managed key with rotation, which is the key you build in Step 4.

Revision history

These notes

- Encryption moved from SSE-S3 (`AES256`) to a customer-managed KMS key with rotation and bucket keys, adding SC-12 and SC-13.
- Added a bucket policy denying `aws:SecureTransport = false`, adding SC-8 and SC-8(1).
- Added policy statements denying uploads that do not name the correct key, making SC-28 enforced rather than defaulted.
- Added versioning to the access-log bucket, adding AU-9. Previously only the primary bucket was versioned.
- Added lifecycle configuration to both buckets, adding AU-11.
- Switched log delivery from a `log-delivery-write` ACL to a bucket policy grant with `aws:SourceArn` and `aws:SourceAccount` conditions, and set `BucketOwnerEnforced` on both buckets, adding AC-6.
- Dropped the AU-6 claim. Collecting logs is AU-3 and AU-11; AU-6 requires review, which Lab 5.2 now provides.
- Remapped versioning from CM-6 to CP-9 and SI-7, and added CM-8 alongside CM-6 for the tagging control.
- Added `compliance_attestation` output as a machine-readable evidence surface.
- Resource count 11 to 18.

The official labs

Initial release: five controls claimed (SC-28, AU-3, AU-6, CM-6, AC-3), SSE-S3 encryption, one versioned bucket, ACL-based log delivery.

CGE-P LAB NOTES

Lab 2.4: Terraform Modules for Compliance (GCP)

Lab 2.3 built one bucket. This lab builds a pattern. The shift to feel: you don't deploy buckets, you deploy a module that deploys buckets, and the security floor sits inside the module where consumers cannot reach it.

It is on GCP on purpose. Doing the same controls in a second cloud is what proves the control IDs are portable and the resource types are not.

For reading. Code lines wrap to fit the page; copy code from docs/02_04_terraform_modules_for_compliance.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/02_04_terraform_modules_for_compliance.md` in your clone, not from the PDF.

Learning objectives

- Compose a module with a clear interface: inputs, outputs, and hardcoded compliance defaults.
- Encode SC-8, SC-12, SC-13, SC-28, AU-3, AU-11, AC-3, and CM-6 so a consumer cannot switch them off.
- Distinguish a control you **implement** from one you **inherit** from the platform, and express the difference honestly.
- Emit a compliance attestation that Chapter 3 asserts against and Chapter 6 cites.

Controls implemented

Control	Enforced by	Note
SC-12	`google_kms_crypto_key.rotation_period`	90-day rotation.
SC-13, SC-28	`encryption.default_kms_key_name` (CMEK)	Your key, not Google's.
AC-3	`uniform_bucket_level_access` + `public_access_prevention = "enforced"`	
AU-3	`logging.log_bucket`	Logs go somewhere other than the bucket being logged.
AU-9	Versioning on the log bucket	
AU-11	`retention_policy + lifecycle_rule`	Two different mechanisms; see Step 4.
CM-6, CM-8	Merged required labels	Consumers may add, never suppress.
SC-8	**Inherited from GCP**	See below. This is the interesting one.

The inherited control

On AWS you write a bucket policy denying `aws:SecureTransport = false`, because S3 will happily serve plain HTTP if you let it. On GCP there is no equivalent resource, because the Cloud Storage JSON and XML APIs are TLS-only. The control is satisfied, and you did not implement it.

That distinction has a name in OSCAL: an `implementation-status` of `inherited`, with the provider named as the responsible party. It is not a loophole and it is not a gap. It is the correct answer, and writing "implemented" where you mean "inherited" is the kind of small dishonesty that collapses an assessment when someone asks you to show the code.

Lab 6.1 encodes this. Keep it in mind while you write the module.

Prerequisites

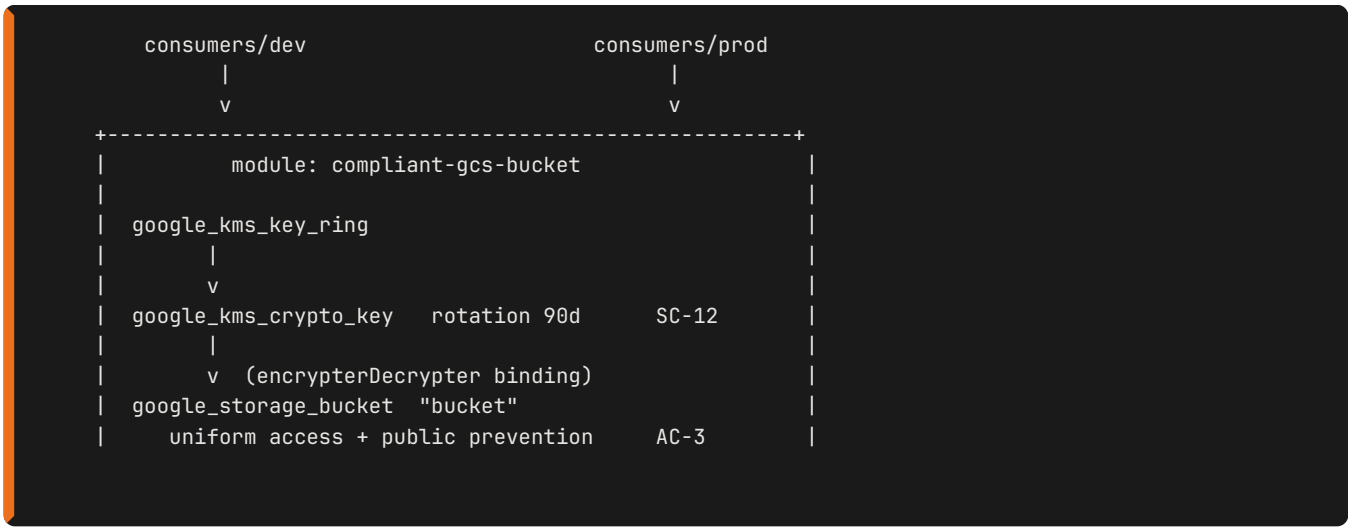
- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 0.1 Part 2** for the GCP side: creating the project, attaching billing, enabling the APIs, and the two separate authentications. This is the first GCP lab, so do that first if you have only set up AWS so far.
- A GCP project you control with billing enabled. Examples use `your-gcp-project`.
- `gcloud auth login` **and** `gcloud auth application-default login`. The google provider uses ADC.
- Roles: `roles/storage.admin`, `roles/cloudkms.admin`.
- `gcloud services enable cloudkms.googleapis.com`.
- Terraform `>= 1.9`. The cross-variable validation in Step 3 needs it.

Estimated time & cost

- Time: 60 to 75 minutes.
- Cost: KMS key versions bill about **$0.06 per active version per month**, and 90-day rotation means versions accumulate. Storage is free while the buckets are empty. Destroy same-day and the prorated cost is fractions of a cent.

Read the Cleanup section before you apply. GCP KMS keys cannot be truly deleted, and one setting in this module can make a bucket undestroyable for a year.

Architecture



```
    |     CMEK                                  SC-13/SC-28 |
    |     versioning + retention_policy         AU-11       |
    |     lifecycle_rule (noncurrent expiry)    AU-11       |
    |     logging -> log bucket                 AU-3        |
    |     merged required labels                CM-6/CM-8   |
    |                                                       |
    |  google_storage_bucket  "log"                         |
    |     versioning                            AU-9        |
    |     uniform access + public prevention    AC-3        |
    +-------------------------------------------------------+
```

Two buckets now, not one. The access-log bucket is the v2 addition, and it exists for the same reason it exists in Lab 2.3.

Step-by-step walkthrough

Concept: designing the interface

A module is a directory with an interface. The body decides what is hardcoded; the interface decides what consumers can change. Three rules:

1. `main.tf` hardcodes anything compliance-relevant: encryption, uniform access, versioning, logging, required labels.
2. `variables.tf` exposes only what the business actually changes: project, environment, retention duration, names.
3. `outputs.tf` returns evidence, including a computed attestation.

Notice what the module does not contain: a `provider` block. Providers are configured by the root module and inherited by children. This is the single most important structural difference between this lab and Lab 2.3, and Lab 2.3 gets it wrong on purpose. Lab 2.3's "primitive" declares its own provider, which makes it a root module wearing the word "primitive." The moment you try to call it twice for two regions, that provider block is what stops you.

If you take one thing from this lab into your capstone, take that.

Step 1 `modules/compliant-gcs-bucket/main.tf`

```
# main.tf
terraform {
  required_version = ">= 1.9"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
  # No provider block. Consumers configure it; this module inherits it.
```

```
}

locals {
  # CM-6 / CM-8: the four required labels. Merged ON TOP of consumer labels,
  # which is the asymmetry that makes them non-negotiable.
  required_labels = {
    project          = var.project_label
    environment      = var.environment
    managed_by       = "terraform"
    compliance_scope = "cge-p-lab"
  }

  effective_labels = merge(var.labels, local.required_labels)

  # GCS bucket names sit in ONE namespace shared by every Google customer, so
  # a fixed name is a name a stranger may already hold. Lab 2.3 met the same
  # wall on S3 and answers it the same way.
  effective_suffix = coalesce(var.bucket_suffix, random_id.bucket_suffix.hex)

  bucket_name = "${var.project_label}-${var.environment}-${var.bucket_name_suffix}-$
{local.effective_suffix}"
  log_name    = "${var.project_label}-${var.environment}-${var.bucket_name_suffix}-logs-$
{local.effective_suffix}"

  # KMS names are scoped to this project and location, not to Google, so they
  # need no random suffix and deliberately do not carry one. A key ring cannot
  # be deleted from a project, so a name that churns leaves an orphan forever.
  keyring_id = "${var.environment}-${var.bucket_name_suffix}-ring"
  key_id     = "${var.environment}-${var.bucket_name_suffix}-key"
}

resource "random_id" "bucket_suffix" {
  byte_length = 4
}

data "google_storage_project_service_account" "gcs" {
  project = var.gcp_project
}

# SC-12: cryptographic key establishment and management. We own the key.
resource "google_kms_key_ring" "ring" {
  name     = local.keyring_id
  location = var.kms_location
  project  = var.gcp_project
}

# SC-12 / SC-13: 90-day rotation, expressed in seconds because the API is
# not sorry about it.
resource "google_kms_crypto_key" "key" {
  name            = local.key_id
  key_ring        = google_kms_key_ring.ring.id
  rotation_period = "7776000s" # 90 days

  lifecycle {
    prevent_destroy = false # set true in production
  }
```

```
}

resource "google_kms_crypto_key_iam_member" "gcs_encrypter" {
  crypto_key_id = google_kms_crypto_key.key.id
  role          = "roles/cloudkms.cryptoKeyEncrypterDecrypter"
  member        = "serviceAccount:${data.google_storage_project_service_account.gcs.email_address}"
}

# ------------------------------------------------------------------------
# AU-3 / AU-9: the access-log bucket. Logs belong somewhere other than the
# thing they are logging.
# ------------------------------------------------------------------------
resource "google_storage_bucket" "log" {
  name     = local.log_name
  project  = var.gcp_project
  location = var.location

  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  # AU-9: audit records must survive overwrite.
  versioning {
    enabled = true
  }

  # AU-11: logs expire on a schedule instead of accumulating forever.
  lifecycle_rule {
    condition {
      age = var.log_retention_days
    }
    action {
      type = "Delete"
    }
  }

  labels = local.effective_labels
}

# GCS writes access logs as the Cloud Storage Analytics group. This binding is
# the GCP analogue of the AWS log-delivery bucket-policy grant in Lab 2.3.
resource "google_storage_bucket_iam_member" "log_writer" {
  bucket = google_storage_bucket.log.name
  role   = "roles/storage.objectCreator"
  member = "group:cloud-storage-analytics@google.com"
}

# ------------------------------------------------------------------------
# The primary bucket. AC-3 + SC-28 + AU-3 + AU-11 + CM-6 in one declaration.
# ------------------------------------------------------------------------
resource "google_storage_bucket" "bucket" {
  name     = local.bucket_name
  project  = var.gcp_project
  location = var.location

  # AC-3: uniform access removes per-object ACLs entirely (the GCP equivalent
  # of BucketOwnerEnforced), and public_access_prevention refuses public
```

```
  # grants even if someone with IAM rights tries to add one.
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  versioning {
    enabled = true
  }

  # SC-13 / SC-28: CMEK. Without this block GCS still encrypts at rest with a
  # Google-managed key, which is encryption you cannot rotate, scope, or audit.
  encryption {
    default_kms_key_name = google_kms_crypto_key.key.id
  }

  # AU-11, mechanism one: a retention policy is WORM. Objects cannot be
  # deleted until they reach this age. See Step 4 before setting is_locked.
  retention_policy {
    retention_period = var.retention_days * 86400
    is_locked        = var.lock_retention_policy
  }

  # AU-11, mechanism two: lifecycle expires superseded versions. Versioning
  # without this is an unbounded bill.
  lifecycle_rule {
    condition {
      num_newer_versions = var.keep_noncurrent_versions
      with_state         = "ARCHIVED"
    }
    action {
      type = "Delete"
    }
  }

  # AU-3: access logs to the dedicated bucket.
  logging {
    log_bucket        = google_storage_bucket.log.name
    log_object_prefix = "access-logs/"
  }

  labels = local.effective_labels

  depends_on = [google_kms_crypto_key_iam_member.gcs_encrypter]
}
```

The `depends_on` is genuine. The GCS service account must hold encrypt/decrypt on the key **before** the bucket is created, and there is no data dependency between the bucket and the IAM binding for Terraform to infer it from. Same shape as the legitimate `depends_on` in Lab 2.3.

Why the bucket name has a random tail, and the key ring does not. A GCS bucket name is not unique to your project, it is unique across every Google customer alive. `cgep-lab-dev-data-001` is exactly the name every other student on this lab would pick, and the first one to run it owns it forever. Terraform reports that as `Error 409` partway through an apply, after the key ring and log bucket already exist, which is the worst moment to discover a naming problem.

KMS key rings and keys are scoped to your project and location, so they have no such contest and get no suffix. That is deliberate rather than an oversight: **a key ring cannot be deleted from a Google project**, so a name that changes on every apply leaves a permanent orphan behind each time. Put randomness where the namespace is shared, and nowhere else.

Step 2 The two-mechanism retention lesson

`retention_policy` and `lifecycle_rule` both appear above, and students routinely think one is redundant. They do opposite jobs:

	`retention_policy`	`lifecycle_rule`
Effect	Objects **cannot be deleted** before this age	Objects **are** deleted at this age
Control	AU-9, AU-11 (preservation)	AU-11 (retention limit), cost
Direction	A floor	A ceiling
Reversible	Only if `is_locked = false`	Yes

A compliance regime usually wants both: keep for at least N (you cannot destroy evidence early), delete after M (you do not hold data longer than the policy allows). Holding data past its retention limit is itself a finding under most privacy regimes.

One asymmetry worth noticing before you are asked about it. The data bucket sets a CMEK; the log bucket does not, so its objects are encrypted with a Google-managed key. Lab 2.3 did give the S3 log bucket the same CMK, and the mismatch is real. The reason is that GCS delivers usage logs through a Google-owned writer, `cloud-storage-analytics@google.com`, which would need encrypt rights on your key before it could write into a CMEK bucket, and granting a shared Google group access to your key is a trade rather than an obvious win. Both positions are defensible. **What is not defensible is leaving it undecided and unwritten**, which is how an assessor finds it first. If you take this to a capstone, say which you chose and why.

Step 3 `variables.tf` with validation

```
# variables.tf
variable "gcp_project" {
  type        = string
  description = "GCP project ID where the bucket and KMS resources live."
}

variable "location" {
  type        = string
  description = "GCS bucket location. Multi-regions like US and EU are valid."
  default     = "us-central1"
}

variable "kms_location" {
  type        = string
  description = "KMS keyring location. Must be a single region; multi-regions are not supported."
  default     = "us-central1"
}

variable "project_label" {
  type        = string
  description = "Short project identifier."
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_label))
    error_message = "project_label must be 3-21 lowercase alphanumerics or hyphens, starting with a
letter."
  }
}

variable "environment" {
  type        = string
  description = "Deployment environment."
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be one of: dev, staging, prod."
  }
}

variable "retention_days" {
  type        = number
  description = "Minimum object retention in days. Production must be >= 365."

  validation {
    condition     = var.retention_days >= 1 && var.retention_days <= 3650
    error_message = "retention_days must be between 1 and 3650."
  }

  validation {
    condition     = var.environment != "prod" || var.retention_days >= 365
    error_message = "retention_days must be >= 365 when environment == \"prod\"."
  }
}

variable "log_retention_days" {
```

```
  type        = number
  description = "AU-11. Age at which access-log objects are deleted."
  default     = 90
}

variable "keep_noncurrent_versions" {
  type        = number
  description = "How many superseded versions to retain before lifecycle deletes them."
  default     = 3
}

variable "lock_retention_policy" {
  type        = bool
  description = "IRREVERSIBLE. Locking a retention policy prevents shortening or removing it, and the
bucket cannot be deleted until every object ages out."
  default     = false
}

variable "bucket_name_suffix" {
  type        = string
  description = "Name of this bucket instance, e.g. data-001. Not globally unique on its own; see
bucket_suffix."
  validation {
    condition     = can(regex("^[a-z0-9-]{3,30}$", var.bucket_name_suffix))
    error_message = "bucket_name_suffix must be 3-30 lowercase alphanumerics or hyphens."
  }
}

# Mirrors var.bucket_suffix in Lab 2.3, same job and same escape hatch: leave
# it null and the module generates one, set it when a name must stay stable.
variable "bucket_suffix" {
  type        = string
  default     = null
  description = "Fixed suffix for global uniqueness. Null generates one."

  validation {
    condition     = var.bucket_suffix == null || can(regex("^[a-z0-9-]{3,20}$",
coalesce(var.bucket_suffix, "gen")))
    error_message = "bucket_suffix must be 3-20 lowercase alphanumerics or hyphens."
  }
}

variable "labels" {
  type        = map(string)
  description = "Optional additional labels. Required compliance labels merge on top."
  default     = {}
}
```

Why two location variables. GCS buckets accept multi-region names like `US` and `EU`. KMS keyrings do not. Setting both from one variable fails with `KMS_RESOURCE_NOT_FOUND_IN_LOCATION` the first time you try `US`. Splitting them keeps the lesson honest.

`lock_retention_policy` **is an input, not a hardcoded** `false`**.** An irreversible choice belongs in the consumer's code where a reviewer sees it, rather than buried in a module the consumer never opens. The loud description is part of the control.

Step 4 `outputs.tf` as evidence

```
# outputs.tf
output "bucket_url" {
  value       = google_storage_bucket.bucket.url
  description = "gs:// URL of the compliant bucket."
}

output "bucket_name" {
  value       = google_storage_bucket.bucket.name
  description = "Name of the compliant bucket."
}

output "log_bucket_name" {
  value       = google_storage_bucket.log.name
  description = "Name of the access-log bucket."
}

output "kms_key_id" {
  value       = google_kms_crypto_key.key.id
  description = "Resource ID of the CMEK protecting this bucket."
}

output "compliance_attestation" {
  description = "Computed attestation of the controls this module enforces."
  value = {
    # Read back from the resource, never from the variable that set it. An
    # attestation that repeats its own input cannot be wrong, which means it
    # cannot be right either: delete the encryption block and a hardcoded
    # "cmek" still says cmek.
    encryption_type          = length(google_storage_bucket.bucket.encryption) > 0 ? "cmek" : "google-
managed"
    kms_key_id               = one([for e in google_storage_bucket.bucket.encryption :
e.default_kms_key_name])
    kms_rotation_period      = google_kms_crypto_key.key.rotation_period
    versioning_enabled       = google_storage_bucket.bucket.versioning[0].enabled
    log_versioning_enabled   = google_storage_bucket.log.versioning[0].enabled
    public_access_prevention = google_storage_bucket.bucket.public_access_prevention
    uniform_access_enforced  = google_storage_bucket.bucket.uniform_bucket_level_access
    access_logging_target    = google_storage_bucket.bucket.logging[0].log_bucket
    # is_locked is the sharp case. Locking is a one-way API call that can be
    # made from the console without Terraform, so var.lock_retention_policy
    # would attest false on a bucket that is genuinely, permanently locked.
    retention_period_days    = one([for r in google_storage_bucket.bucket.retention_policy :
r.retention_period / 86400])
    retention_policy_locked = one([for r in google_storage_bucket.bucket.retention_policy :
r.is_locked])
```

```
    # condition and action are SETS, not lists, so [0] is a type error that
    # terraform validate accepts and terraform plan rejects. Lab 2.3's own
    # note on this applies verbatim: one() raises where tolist(...)[0] would
    # silently return an arbitrary element.
    log_retention_days = one([
      for r in google_storage_bucket.log.lifecycle_rule :
      one(r.condition).age
      if one(r.action).type == "Delete"
    ])

    required_labels_present = alltrue([
      for k in keys(local.required_labels) :
      contains(keys(google_storage_bucket.bucket.labels), k)
    ])

    # SC-8 is satisfied by the platform, not by this module. Saying so in the
    # attestation is the honest form; Lab 6.1 maps this to an OSCAL
    # implementation-status of "inherited".
    transit_encryption = "inherited-gcp-tls-only"
  }
}
```

Compare this shape to Lab 2.3's `compliance_attestation`. Different cloud, different resource types, same evidence surface. That similarity is what lets one Rego library and one OSCAL component describe both.

Step 5 Consumer: dev

```
# consumers/dev/main.tf
terraform {
  required_version = ">= 1.9"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}

provider "google" {
  project = var.gcp_project
  region  = "us-central1"
}

variable "gcp_project" { type = string }

module "data_bucket" {
  source = "../../modules/compliant-gcs-bucket"

  gcp_project        = var.gcp_project
  project_label      = "cgep-lab"
  environment        = "dev"
  retention_days     = 30
  bucket_name_suffix = "data-001"
}
```

```
output "compliance_attestation" { value = module.data_bucket.compliance_attestation }
output "bucket_name" { value = module.data_bucket.bucket_name }
output "log_bucket_name" { value = module.data_bucket.log_bucket_name }
output "bucket_url"  { value = module.data_bucket.bucket_url }
```

Six lines of business configuration. Twenty-plus controls. The provider lives here, in the root module, exactly where it belongs.

Step 6 Consumer: prod

Same module, two values changed:

```
module "data_bucket" {
  source = "../../modules/compliant-gcs-bucket"

  gcp_project        = var.gcp_project
  project_label      = "cgep-lab"
  environment        = "prod"
  retention_days     = 365
  bucket_name_suffix = "data-001"
}
```

Step 7 Apply dev

Plan prod, but do not apply it. A 365-day retention takes a year to expire.

```
cd consumers/dev
terraform init
terraform fmt && terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
```

Tail:

```
compliance_attestation = {
  "access_logging_target"    = "cgep-lab-dev-data-001-logs-9f2ac41b"
  "encryption_type"          = "cmek"
  "kms_rotation_period"      = "7776000s"
  "log_retention_days"       = 90
  "log_versioning_enabled"   = true
  "public_access_prevention" = "enforced"
  "required_labels_present"  = true
  "retention_period_days"    = 30
  "retention_policy_locked"  = false
  "transit_encryption"       = "inherited-gcp-tls-only"
  "uniform_access_enforced"  = true
  "versioning_enabled"       = true
}
```

That output is your SC-12 / SC-13 / SC-28 / AC-3 / AU-3 / AU-9 / AU-11 / CM-6 attestation in machine-readable form.

Step 8 The negative test

Copy `consumers/dev` to `consumers/negative-test`, set `environment = "prod"` and leave `retention_days = 30`:

```
Plan: 6 to add, 0 to change, 0 to destroy.

Error: Invalid value for variable

  on main.tf line 29, in module "data_bucket":
  29:   retention_days     = 30

  var.environment is "prod"
  var.retention_days is 30

retention_days must be >= 365 when environment == "prod".

This was checked by the validation rule at
../../modules/compliant-gcs-bucket/variables.tf:54,3-13.
```

This is the lesson. The compliance check ran at `terraform plan`, before any resource existed, with a message specific enough that the developer fixes it without filing a ticket. Preventive beats detective, and it is cheaper.

Two things about that output that surprise people.

Terraform renders the whole plan first, then fails. You will see six resources listed under `+ create` and a line reading `Plan: 6 to add` above the error. Nothing was created. A plan is a proposal, and this one was refused before anything acted on it, so the `negative-test.txt` in your evidence folder is a plan that never ran. Say that in your write-up, because a reviewer skimming for `6 to add` will otherwise read it as a near miss.

`terraform validate` says `Success!` on this exact configuration. Both values are literals sitting in `main.tf`, so it looks like something a static check should catch, and it is not: `validate` checks syntax, types and references, never values. Variable validation rules are evaluated during `plan`. The practical consequence is worth carrying into Lab 4.3: **a pipeline whose only Terraform gate is `terraform validate` passes this configuration.** The pipeline you build there runs `validate` and then `plan` under `set -euo pipefail`, which is why it catches what validate alone would wave through.

Run a second negative test: try to suppress a required label.

```
labels = { compliance_scope = "not-in-scope" }
```

Plan it. The label comes back as `cge-p-lab`, because `merge(var.labels, local.required_labels)` puts the required set last and last wins. **The consumer can add labels and cannot override the compliance ones.** That asymmetry is the entire design, and watching it refuse is better than reading about it.

Verification

> **gcloud silently drops field names it does not recognize.** The projection in `--format` is a filter, not a query: ask for a field that does not exist and gcloud prints the fields it did understand and says nothing at all about the one it did not. An earlier draft of this section asked for `logging`, whose real name is `logging_config`, and the output simply had no logging section in it.
>
> Read what that means for evidence. **A control that is genuinely missing and a field name you got wrong produce identical output.** If you are verifying a control and its section is absent, your first move is to confirm the field name, not to conclude the control failed. The field names come from gcloud's own resource model rather than from the JSON API, so they are `logging_config` and `default_kms_key` even though the REST API calls them `logging` and `encryption.defaultKmsKeyName`.

That trap is exactly why this section asserts rather than prints. Run it from `consumers/dev`:

```
bash scripts/verify.sh consumers/dev
```

Every line names a control and either passes or fails with what it expected and what the project actually returned. The run ends in `VERIFIED` or `NOT VERIFIED` and sets the exit status accordingly.

On the one enforcement check. It is a read, not a write, and that is deliberate. The obvious way to prove public access prevention works is to try adding `allUsers` as a viewer and watch it be refused. If the control were missing, that test would succeed and leave your bucket public. A check that damages the thing it is checking when it fails is not worth running, so this one asks for the bucket listing anonymously and requires a 401 or 403.

The same reasoning rules out watching the retention policy deny a delete. Doing that means putting an object in the bucket, and a 30 day retention floor then prevents you from deleting it, which prevents you from destroying the bucket, for 30 days. The configuration assertion is the honest trade here, and the cleanup section explains the rest.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 2.4. Every check asserts a value and exits non-zero if
# the project disagrees.
#
# Run it from consumers/dev, where the outputs live.
#
# One trap this replaces: gcloud's --format projection is a filter, not a
# query. Ask for a field it does not recognize and it prints the fields it did
# understand and says nothing about the one it did not, exit code 0. An earlier
# draft asked for `logging` instead of `logging_config` and simply had no
# logging section in its output. A control that is genuinely missing and a
# field name you typed wrong look identical.
set -uo pipefail
```

```
# Point this at the workspace whose `terraform output` describes the resources
# you want checked, so it does not matter where you run it from. Guessing the
# working directory is how a verification script cheerfully checks the wrong
# account and passes.
WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0

pass() { printf '  PASS  %-8s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:-(nothing)}'"
  fi
}

BUCKET=$(terraform output -raw bucket_name)
LOGS=$(terraform output -raw log_bucket_name)
ATT=$(terraform output -json compliance_attestation)
KMS_ID=$(jq -r '.kms_key_id' <<<"$ATT")

# projects/P/locations/L/keyRings/R/cryptoKeys/K
KEY_PROJECT=$(awk -F/ '{print $2}' <<<"$KMS_ID")
KEY_LOCATION=$(awk -F/ '{print $4}' <<<"$KMS_ID")
KEY_RING=$(awk -F/ '{print $6}' <<<"$KMS_ID")
KEY_NAME=$(awk -F/ '{print $8}' <<<"$KMS_ID")

bucket_field() { # bucket_field BUCKET FIELD
  gcloud storage buckets describe "gs://$1" --format="value($2)" 2>/dev/null
}

# Booleans are read as JSON rather than through value(), which renders them
# with Python's capitalisation. Asserting against "True" when the tool happens
# to emit "true" is a failure that teaches nothing.
bucket_json() { # bucket_json BUCKET JQ_PATH
  gcloud storage buckets describe "gs://$1" --format=json 2>/dev/null \
    | jq -r "$2 // empty"
}

echo "=== configuration ==="

# SC-13 / SC-28. The key itself, not merely that some key is set.
check SC-28 "default CMEK" "$KMS_ID" "$(bucket_field "$BUCKET" default_kms_key)"

# SC-12. Ninety days, in seconds, because the API is not sorry about it.
check SC-12 "key rotation" "7776000s" \
  "$(gcloud kms keys describe "$KEY_NAME" --keyring="$KEY_RING" \
       --location="$KEY_LOCATION" --project="$KEY_PROJECT" \
       --format="value(rotationPeriod)" 2>/dev/null)"

# CP-9 and AU-9.
```

```
check CP-9 "data versioning" "true" "$(bucket_json "$BUCKET" .versioning_enabled)"
check AU-9 "log versioning"  "true" "$(bucket_json "$LOGS" .versioning_enabled)"

# AC-3 and AC-6.
check AC-3 "public access prevention" "enforced" \
  "$(bucket_field "$BUCKET" public_access_prevention)"
check AC-6 "uniform bucket access" "true" \
  "$(bucket_json "$BUCKET" .uniform_bucket_level_access)"

# AU-3. The target has to be the log bucket, not merely some bucket. This is
# the field that was silently absent when the name was wrong.
check AU-3 "log target" "$LOGS" "$(bucket_field "$BUCKET" logging_config.logBucket)"

# AU-11. Asserted against the module's own attestation, so a disagreement
# between the code and the project shows up as a failure rather than as two
# numbers nobody compared.
check AU-11 "retention seconds" \
  "$(( $(jq -r '.retention_period_days' <<<"$ATT") * 86400 ))" \
  "$(bucket_field "$BUCKET" retention_policy.retentionPeriod)"

echo "=== enforcement ==="

# AC-3, without touching anything. An unauthenticated read of the bucket
# listing must be refused. This is deliberately a read: the obvious GCP denial
# test is to try adding allUsers as a viewer, and if the control were missing
# that test would succeed and leave your bucket public. A check that damages
# the thing it is checking when it fails is not worth running.
CODE=$(curl -s -o /dev/null -w '%{http_code}' \
  "https://storage.googleapis.com/storage/v1/b/${BUCKET}/o" 2>/dev/null)
if [[ "$CODE" == "401" || "$CODE" == "403" ]]; then
  pass "AC-3" "anonymous listing refused with HTTP $CODE"
else
  fail "AC-3" "anonymous listing returned HTTP ${CODE:-(no response)}, expected 401 or 403"
fi

echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the project."
else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-2-4"
mkdir -p "$EVIDENCE"
terraform show -json tfplan > "$EVIDENCE/plan.json"
terraform output -json compliance_attestation > "$EVIDENCE/attestation.json"
```

The negative test is the one worth keeping, because it is the proof the module refuses rather than merely defaults:

```
( cd ../negative-test && terraform init -input=false >/dev/null && terraform plan 2>&1 ) \
  | tee "$EVIDENCE/negative-test.txt"
```

The subshell is deliberate: the `cd` ends with it, so the `tee` path stays relative to where you started and you are not left in a different directory. `negative-test` is a workspace you have never initialized, so it needs its own `terraform init` before it can plan.

Portfolio submission checklist

- [ ] `terraform/modules/compliant-gcs-bucket/{main.tf,variables.tf,outputs.tf,README.md}`
- [ ] `terraform/consumers/{dev,prod}/` committed.
- [ ] Module README lists each control by family **and marks SC-8 as inherited, with the reason.**
- [ ] `evidence/lab-2-4/plan.json`
- [ ] `evidence/lab-2-4/attestation.json` from `terraform output -json compliance_attestation`
- [ ] `evidence/lab-2-4/negative-test.txt` capturing both refusals: the prod retention rule and the label-override attempt.

Troubleshooting

- **`KMS_RESOURCE_NOT_FOUND_IN_LOCATION`** when `location` is `US`. Keyrings need a single region. The two-variable split is the fix.
- **`Permission cloudkms.cryptoKeyEncrypterDecrypter denied`** during bucket creation. The GCS service agent needs encrypt/decrypt on the key before the bucket exists. The `depends_on` sequences it; keep it if you refactor.
- **Access logs never appear.** GCS access logs are delivered roughly hourly, and only when there is traffic. Confirm the `cloud-storage-analytics@google.com` binding exists on the log bucket.
- **Retention policy cannot be shortened.** It can be lengthened or removed only while `is_locked = false`. Once locked, neither.
- **`googleapi: Error 409: The requested bucket name is not available.`** GCS names are global, so this used to be routine. The module now generates a random suffix, so a 409 means you pinned `bucket_suffix` to a value someone else holds. Unset it and let the module choose.
- **`reauth related error (invalid_rapt)`.** Run `gcloud auth application-default login` again. ADC tokens expire and Terraform will not refresh them.

Cleanup

```
cd consumers/dev
terraform destroy -auto-approve
```

Three things that will surprise you:

1. **A locked retention policy makes the bucket undestroyable** until every object ages out. With `retention_days = 365` and `lock_retention_policy = true`, that is a year. This is why the variable defaults to `false` and says so loudly.
2. **KMS crypto keys are never truly deleted.** Destroying a key version enters a 30-day soft-delete. The key and keyring objects remain indefinitely; keyrings cannot be deleted at all. They are free, so this is tidiness rather than cost, but a `terraform destroy` that reports success has not removed them.
3. **The log bucket may hold objects** and refuse to destroy. Empty it first if the bucket saw traffic.

How this feeds the capstone

- **The module discipline is the deliverable**, more than the module. Your capstone's KMS and S3 hardening should be a module both dev and prod consume, so the floor is enforced once.
- **Ch 3**, Rego reads `compliance_attestation` and refuses to merge a plan that cannot produce it.
- **Ch 6**, the OSCAL component for `compliant-gcs-bucket-v1` cites this module path as its implementation and the attestation JSON as evidence. The `transit_encryption` field becomes an `inherited` implementation status, which is the honest OSCAL and the thing most candidates get wrong.

Revision history

v2 (current)

- Added an access-log bucket, with versioning and a lifecycle rule, adding AU-3 and AU-9. The module previously logged nowhere.
- `lock_retention_policy` exposed as an input rather than hardcoded, so an irreversible choice appears in the consumer's code.
- Added a second lifecycle mechanism for superseded versions, and documented why `retention_policy` (a floor) and `lifecycle_rule` (a ceiling) are both needed.
- The attestation now records `transit_encryption` as inherited from the platform, rather than leaving SC-8 unstated.
- Added a second negative test showing that a consumer cannot suppress a required label.

v1

Initial release: single bucket, no access logging, `is_locked` hardcoded.

CGE-P LAB NOTES

Lab 2.5: IaC as Compliance Evidence (AWS)

A reviewed, signed, immutably-stored Terraform commit is stronger evidence than a screenshot. This lab builds the vault that holds your evidence and the script that puts it there.

It also confronts a trap that is easy to walk into: capturing raw Terraform state into an immutable bucket. If that state contains a secret, you have just made the secret permanently undeletable. Step 3 deals with it.

For reading. Code lines wrap to fit the page; copy code from docs/02_05_iac_as_compliance_evidence.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/02_05_iac_as_compliance_evidence.md` in your clone, not from the PDF.

Learning objectives

- Define chain of custody in terms of integrity, attribution, and reproducibility.
- Build an S3 Object Lock vault that refuses deletion by design.
- Capture a workspace's evidence, hash it, bundle it, and upload it with a recorded VersionId.
- Recognize that evidence collection is itself a data-handling risk, and mitigate it before it becomes permanent.

Controls implemented

Control	Enforced by
AU-9, AU-9(3)	Object Lock retention; the vault refuses deletion
AU-11	Retention mode and duration, chosen explicitly
SC-8	Bucket policy denying non-TLS
SC-12, SC-13, SC-28	Vault CMK with rotation, SSE-KMS
AC-3, AC-6	Public access block, ACLs disabled
CP-9	Versioning, required by Object Lock
CM-6, CM-8	`default_tags`

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 2.2** state backend, and **Lab 2.3** completed with `evidence/lab-2-3/` populated and the workspace still on disk.
- AWS CLI v2, plus `jq` and either `sha256sum` or `shasum`.
- Terraform `>= 1.10`.
- Optional: Cosign, for the signing preview at the end. Lab 4.4 does it properly.

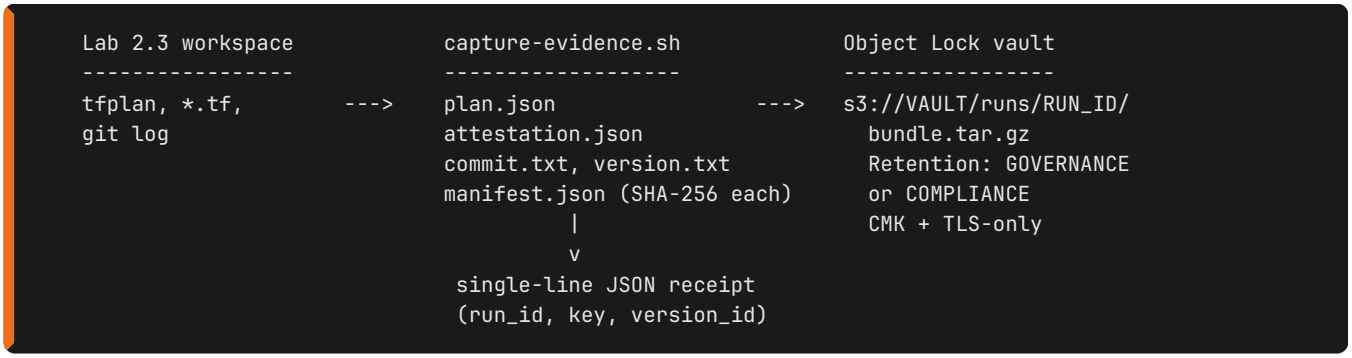
> **Read the Cleanup section before you apply this one.** The choice between `GOVERNANCE` and `COMPLIANCE` is not reversible and not symmetric: `GOVERNANCE` retention can be bypassed by a privileged caller, and `COMPLIANCE` cannot be bypassed by anyone, including the account root. Choose `COMPLIANCE` with a long retention and the bucket stays until every object expires, whatever you or AWS support would prefer.
>
> The defaults here are the safe ones. Change them on purpose, not on the way past.

Estimated time & cost

- Time: 45 to 60 minutes.
- Cost: Object Lock itself is free; you pay S3 storage on a few KB of bundles. **$1/month for the vault CMK.** Under $0.02 if you destroy same-day.

Choose `GOVERNANCE` mode with a 1-day retention for lab work. `COMPLIANCE` mode cannot be shortened or bypassed by anyone, including the account root, until retention expires. Pick `COMPLIANCE` with a long retention in a lab and you have created a bucket you will be paying for, and unable to empty, for the duration you chose. That is the correct behavior. It is also a genuinely expensive mistake.

Architecture



Note what is **not** in that list, and see Step 3: raw `terraform state pull` output.

Step-by-step walkthrough

Concept: why code is evidence

A screenshot of a console says "I once saw this." A Terraform plan committed to git, reviewed in a pull request, applied in CI, and stored in a vault that refuses deletion says "this is what was deployed, who reviewed it, when, and the artifact is unchanged since."

Auditors want three properties: integrity, attribution, reproducibility. Code-as-evidence delivers all three. Screenshots deliver none.

Step 1 Build the vault

Object Lock has one hard constraint: **it must be enabled at bucket creation.** There is no retrofit. If you get this wrong you destroy and recreate.

```
# terraform/primitives/evidence-vault/main.tf
terraform {
  required_version = ">= 1.10"

  backend "s3" {
    bucket       = "cgep-lab-tfstate-XXXXXXXX"
    key          = "labs/2-5/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    kms_key_id   = "arn:aws:kms:us-east-1:ACCOUNT:key/KEY-ID"
    use_lockfile = true
  }

  required_providers {
    aws    = { source = "hashicorp/aws", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project         = var.project_name
      Environment     = "evidence"
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
}

data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

resource "random_id" "suffix" {
  byte_length = 4
}

locals {
  account_id = data.aws_caller_identity.current.account_id
  partition  = data.aws_partition.current.partition
  vault_name = "${var.project_name}-grc-evidence-vault-${random_id.suffix.hex}"
}

# SC-12 / SC-13: the vault gets its own key, separate from the workload key
```

```
# in Lab 2.3. Separation of duties: whoever can read the data should not
# automatically be able to read the evidence about the data.
resource "aws_kms_key" "vault" {
  description             = "CMK for the GRC evidence vault"
  enable_key_rotation     = true
  deletion_window_in_days = var.kms_deletion_window_days
  policy                  = data.aws_iam_policy_document.kms.json
}

resource "aws_kms_alias" "vault" {
  name          = "alias/${var.project_name}-evidence-vault"
  target_key_id = aws_kms_key.vault.key_id
}

data "aws_iam_policy_document" "kms" {
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket" "vault" {
  bucket              = local.vault_name
  object_lock_enabled = true # MUST be set at bucket creation. No retrofit exists.
}

resource "aws_s3_bucket_ownership_controls" "vault" {
  bucket = aws_s3_bucket.vault.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
}

# Object Lock requires versioning. This is not optional.
resource "aws_s3_bucket_versioning" "vault" {
  bucket = aws_s3_bucket.vault.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AU-9 / AU-11: the control that makes this a vault rather than a bucket.
resource "aws_s3_bucket_object_lock_configuration" "vault" {
  bucket     = aws_s3_bucket.vault.id
  depends_on = [aws_s3_bucket_versioning.vault]

  rule {
    default_retention {
      mode = var.lock_mode
      days = var.retention_days
```

```
    }
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "vault" {
  bucket = aws_s3_bucket.vault.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.vault.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_public_access_block" "vault" {
  bucket                  = aws_s3_bucket.vault.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

data "aws_iam_policy_document" "vault" {
  # SC-8. Easily skipped on an internal bucket, and this is the one whose
  # contents are the audit record itself.
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.vault.arn, "${aws_s3_bucket.vault.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  # AU-9: nobody but the account root deletes this bucket. Object Lock protects
  # the objects; this protects the container.
  statement {
    sid       = "DenyBucketDeletion"
    effect    = "Deny"
    actions   = ["s3:DeleteBucket"]
    resources = [aws_s3_bucket.vault.arn]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "StringNotEquals"
      variable = "aws:PrincipalArn"
```

```
      values   = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket_policy" "vault" {
  bucket = aws_s3_bucket.vault.id
  policy = data.aws_iam_policy_document.vault.json
}
```

```
# terraform/primitives/evidence-vault/variables.tf
variable "project_name" {
  type    = string
  default = "cgep-lab"
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_name))
    error_message = "project_name must be 3-21 lowercase alphanumerics or hyphens, starting with a
letter."
  }
}

variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "lock_mode" {
  type        = string
  description = "GOVERNANCE for lab work; COMPLIANCE for real evidence."
  default     = "GOVERNANCE"
  validation {
    condition     = contains(["GOVERNANCE", "COMPLIANCE"], var.lock_mode)
    error_message = "lock_mode must be GOVERNANCE or COMPLIANCE."
  }
}

variable "retention_days" {
  type        = number
  description = "Default retention applied to every uploaded object."
  default     = 1

  validation {
    condition     = var.retention_days >= 1 && var.retention_days <= 3650
    error_message = "retention_days must be between 1 and 3650."
  }

  # COMPLIANCE mode is unbypassable by anyone including root, so a long
  # retention has to be a deliberate act rather than a default.
  validation {
    condition     = var.lock_mode != "COMPLIANCE" || var.retention_days <= 7 ||
var.acknowledge_compliance_mode
    error_message = "COMPLIANCE mode beyond 7 days requires acknowledge_compliance_mode = true. You
will not be able to delete these objects, ever, until retention expires."
  }
```

```
}

variable "acknowledge_compliance_mode" {
  type        = bool
  description = "Set true only if you intend evidence to outlive your ability to delete it."
  default     = false
}

variable "kms_deletion_window_days" {
  type    = number
  default = 7
  validation {
    condition     = var.kms_deletion_window_days >= 7 && var.kms_deletion_window_days <= 30
    error_message = "kms_deletion_window_days must be between 7 and 30."
  }
}
```

```
# terraform/primitives/evidence-vault/outputs.tf
output "vault_name" {
  value       = aws_s3_bucket.vault.id
  description = "Vault bucket name. Feed to capture-evidence.sh --vault."
}

output "vault_kms_key_arn" {
  value       = aws_kms_key.vault.arn
  description = "CMK protecting the vault."
}

output "lock_mode" {
  value       = var.lock_mode
  description = "Retention mode in force (AU-11 attestation)."
}

output "retention_days" {
  value       = var.retention_days
  description = "Default retention window in days (AU-11 attestation)."
}
```

That cross-variable validation on `retention_days` needs Terraform 1.9 or later. It exists because the most common way to hurt yourself in this lab is a `COMPLIANCE` vault with a 365-day retention created by accident.

GOVERNANCE vs COMPLIANCE. GOVERNANCE retention can be bypassed by a caller holding `s3:BypassGovernanceRetention` using `--bypass-governance-retention`. COMPLIANCE cannot be bypassed by anyone, including the account root, until the window expires. Use GOVERNANCE for labs so you can clean up. Use COMPLIANCE for real evidence, and mean it.

Step 2 Apply

```
export AWS_PROFILE=cgep
cd terraform/primitives/evidence-vault   # reference layout: cd reference/lab-2-5/terraform
terraform init
terraform fmt && terraform validate
terraform apply -auto-approve
VAULT=$(terraform output -raw vault_name)
```

Expected: `Apply complete! Resources: 10 added, 0 changed, 0 destroyed.`

Step 3 The problem with capturing state

The obvious capture script does this:

```
terraform state pull > "$BUNDLE_DIR/state.json"
```

then uploaded the bundle into an Object Lock vault.

Think about what that means. State holds every attribute of every resource in plaintext, including values marked `sensitive`: RDS passwords, IAM secret access keys, Lambda environment variables. Object Lock means the upload **cannot be deleted** until retention expires. In `COMPLIANCE` mode, not even by the account root.

So the happy path is: capture a secret, then make it permanently irretrievable-but-undeletable, in a bucket whose entire purpose is to be handed to an auditor.

Three ways out, and you should be able to defend your choice:

Approach	Trade-off
Capture the plan, not the state	`plan.json` describes intent and configuration, which is what policy engines and assessors actually read. Loses "what is currently deployed." **This is our default.**
Capture state, redacted	Keeps the deployed view. Redaction is a denylist, and denylists are wrong eventually.
Capture state, and never put secrets in Terraform	Correct in principle. Requires every secret to live in Secrets Manager with only ARNs in state. The right long-term answer, and not achievable by Chapter 2.

The script below takes the first, and provides `--include-state` for when you have consciously decided the workspace is secret-free. It refuses to run that flag against `COMPLIANCE` mode without an explicit override, because that combination is the irreversible one.

Step 4 Write `capture-evidence.sh`

```
#!/usr/bin/env bash
# scripts/capture-evidence.sh
# Usage:
#   capture-evidence.sh --workspace <path> --run-id <id> --vault <bucket>
#                       [--profile <p>] [--include-state] [--i-understand-compliance-mode]

set -euo pipefail

PROFILE_ARG=""
WORKSPACE=""
RUN_ID=""
VAULT=""
INCLUDE_STATE=0
ACK_COMPLIANCE=0

while [[ $# -gt 0 ]]; do
  case "$1" in
    --workspace) WORKSPACE="$2"; shift 2 ;;
    --run-id)    RUN_ID="$2";    shift 2 ;;
    --vault)     VAULT="$2";     shift 2 ;;
    --profile)   PROFILE_ARG="--profile $2"; shift 2 ;;
    --include-state) INCLUDE_STATE=1; shift ;;
    --i-understand-compliance-mode) ACK_COMPLIANCE=1; shift ;;
    *) echo "Unknown arg: $1" >&2; exit 2 ;;
  esac
done

[[ -z "$WORKSPACE" || -z "$RUN_ID" || -z "$VAULT" ]] && {
  echo "Usage: $0 --workspace <path> --run-id <id> --vault <bucket> [--profile <p>]" >&2
  exit 2
}

# Refuse the one irreversible combination: raw state into unbypassable storage.
if [[ $INCLUDE_STATE -eq 1 && $ACK_COMPLIANCE -eq 0 ]]; then
  MODE=$(aws $PROFILE_ARG s3api get-object-lock-configuration --bucket "$VAULT" \
           --query 'ObjectLockConfiguration.Rule.DefaultRetention.Mode' \
           --output text 2>/dev/null || echo "NONE")
  if [[ "$MODE" == "COMPLIANCE" ]]; then
    echo "REFUSING: --include-state against a COMPLIANCE-mode vault." >&2
    echo "Terraform state contains every attribute in plaintext, including secrets." >&2
    echo "COMPLIANCE retention cannot be bypassed by anyone, including root." >&2
    echo "Pass --i-understand-compliance-mode if this is genuinely what you want." >&2
    exit 3
  fi
fi

WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT

if command -v sha256sum >/dev/null 2>&1; then SHASUM="sha256sum"
elif command -v shasum   >/dev/null 2>&1; then SHASUM="shasum -a 256"
else echo "Need sha256sum or shasum" >&2; exit 2; fi
```

```
CAPTURED_AT=$(date -u +%Y-%m-%dT%H:%M:%SZ)
BUNDLE_DIR="$WORK/bundle-$RUN_ID"
mkdir -p "$BUNDLE_DIR"

# plan.json: the configuration and intent. The primary artifact.
( cd "$WORKSPACE" && [[ -f tfplan ]] \
    && terraform show -json tfplan > "$BUNDLE_DIR/plan.json" 2>/dev/null || true )

# The module's own attestation, if it emits one (Lab 2.3 does).
( cd "$WORKSPACE" && terraform output -json compliance_attestation \
    > "$BUNDLE_DIR/attestation.json" 2>/dev/null || true )

if [[ $INCLUDE_STATE -eq 1 ]]; then
  ( cd "$WORKSPACE" && terraform state pull > "$BUNDLE_DIR/state.json" 2>/dev/null || true )
fi

( cd "$WORKSPACE" && git log -1 --pretty=full > "$BUNDLE_DIR/commit.txt" 2>/dev/null \
    || echo "no git commit available" > "$BUNDLE_DIR/commit.txt" )
terraform version > "$BUNDLE_DIR/version.txt"

# manifest.json: filename, sha256, size, captured_at per file.
{
  echo "["
  FIRST=1
  for f in "$BUNDLE_DIR"/*; do
    base=$(basename "$f")
    [[ "$base" == "manifest.json" ]] && continue
    HASH=$($SHASUM "$f" | awk '{print $1}')
    SIZE=$(wc -c < "$f" | tr -d ' ')
    [[ $FIRST -eq 1 ]] && FIRST=0 || printf ","
    printf '\n  {"filename":"%s","sha256":"%s","size":%s,"captured_at_utc":"%s"}' \
      "$base" "$HASH" "$SIZE" "$CAPTURED_AT"
  done
  echo
  echo "]"
} > "$BUNDLE_DIR/manifest.json"

BUNDLE_TGZ="$WORK/bundle-$RUN_ID.tar.gz"
( cd "$WORK" && tar czf "$BUNDLE_TGZ" "bundle-$RUN_ID" )

KEY="runs/$RUN_ID/bundle.tar.gz"
VERSION_ID=$(aws $PROFILE_ARG s3api put-object \
  --bucket "$VAULT" --key "$KEY" --body "$BUNDLE_TGZ" \
  --query VersionId --output text)

printf '{"run_id":"%s","vault":"%s","key":"%s","version_id":"%s","captured_at_utc":"%s","includes_stat
e":%s}\n' \
  "$RUN_ID" "$VAULT" "$KEY" "$VERSION_ID" "$CAPTURED_AT" \
  "$([[ $INCLUDE_STATE -eq 1 ]] && echo true || echo false)"
```

Three deliberate details beyond the state guard. The bundle tarball is built inside the `mktemp` directory so the `trap` cleans it up, instead of being left in `/tmp`. The VersionId comes from `--query VersionId` rather than an `awk` parse of JSON. And the receipt records whether state was included, because that is a fact a future reader needs.

Step 5 Run it against Lab 2.3

```
chmod +x scripts/capture-evidence.sh

bash scripts/capture-evidence.sh \
  --workspace ../lab-2-3 \
  --run-id    test-001 \
  --vault     "$VAULT"
```

`../lab-2-3` is the workspace you applied in Lab 2.3. If you have built your own repository following the capstone layout, the same workspace lives at `terraform/primitives/compliant-s3`, and you pass that path instead. The script cares about finding a Terraform workspace, not about where you keep it.

Receipt:

```
{"run_id":"test-001","vault":"cgep-lab-grc-evidence-vault-XXXXXXXX","key":"runs/test-001/
bundle.tar.gz","version_id":"<base64-version-id>","captured_at_utc":"<iso-
utc>","includes_state":false}
```

The VersionId is the durable handle. Anything that points at this evidence later, including your OSCAL component's evidence URI in Lab 6.1, uses `s3://VAULT/KEY?versionId=...`. Without the version, you are pointing at "whatever is at this key now," which in a versioned bucket is not the same claim.

Step 6 Verify retention applied

```
aws s3api get-object-retention --bucket "$VAULT" \
  --key runs/test-001/bundle.tar.gz
```

```
{ "Retention": { "Mode": "GOVERNANCE", "RetainUntilDate": "<retain-until-utc>" } }
```

You did not set that explicitly. The bucket's default rule applied it at upload.

Step 7 The destructive test

This is the lesson.

The receipt printed a `version_id`. Read it back rather than transcribing it, for the same reason you read `backend.hcl` out of Lab 2.2's outputs:

```
VERSION_ID=$(aws s3api list-object-versions --bucket "$VAULT" \
  --prefix runs/test-001/ --query 'Versions[0].VersionId' --output text)
echo "$VERSION_ID"

aws s3api delete-object --bucket "$VAULT" \
  --key runs/test-001/bundle.tar.gz \
  --version-id "$VERSION_ID"
```

```
An error occurred (AccessDenied) when calling the DeleteObject operation:
Access Denied because object protected by object lock.
```

That rejection is the proof. The lesson is not that S3 has a feature called Object Lock. The lesson is that your evidence now resists silent tampering by an administrator who would prefer it not exist, and that resistance is a property of the storage rather than a promise in a policy document.

Try one more thing, and notice the difference:

```
aws s3api delete-object --bucket "$VAULT" --key runs/test-001/bundle.tar.gz
```

That one **succeeds**, because without `--version-id` it writes a delete marker rather than removing the object. The object is still there, still locked, still retrievable by version. Delete markers are how a versioned bucket says "hidden," not "gone." An auditor who does not know this will believe evidence was destroyed; an engineer who does knows where to look.

Step 8 Optional preview: sign the bundle

Read this before running it. Most people should skip this step.

Object Lock makes the bundle immutable. It does not prove who made it or when. That is what signing adds, and it is the whole subject of Lab 4.4.

The command below signs as **you**, and that has a consequence you cannot take back:

```
cosign sign-blob --yes --bundle bundle.sig.bundle /tmp/bundle-test-001.tar.gz
```

Cosign opens a browser, you log in with GitHub, Google or Microsoft, and Fulcio issues a certificate naming **the email address of the account you used**. That certificate and its signature are written to Rekor, a **public transparency log that is append-only by design**. Cosign says so before it proceeds:

```
This information will be used for signing this artifact and will be stored in
public transparency logs and cannot be removed later
```

That is not a warning about a preference. It means your personal email address becomes a permanent public record, attached to a throwaway lab artifact, and no one can delete it afterward. Transparency logs are immutable for the same reason your evidence vault is, which is a good property in the right place and a poor trade here.

Lab 4.4 does it the way it should be done. There, signing happens inside the pipeline, and Fulcio issues a certificate whose subject is the **repository, workflow and ref** rather than a human:

```
https://github.com/OWNER/REPO/.github/workflows/grc-gate.yml@refs/heads/main
```

Nothing personal enters the log, the identity is more useful for auditing because it names the process rather than whoever happened to be at a keyboard, and there is no long-lived credential involved either. Waiting for 4.4 costs you nothing and teaches the correct shape.

If you do want to see a signature today, use a throwaway account, or sign something you do not mind being publicly associated with forever. Not your lab evidence, and not with the address you use for work.

Record the vault in `cgep.env`

Lab 5.2 scopes CloudTrail data events to this vault, and needs its ARN:

```
vault=$(terraform output -raw vault_name) &&
  # Delete any previous value before appending, so re-running this lab updates
  # cgep.env rather than stacking a second export that silently shadows the
  # first.
  sed -i '/^export TF_VAR_evidence_vault_arn=/d' ../../../cgep.env
  echo "export TF_VAR_evidence_vault_arn=arn:aws:s3:::$vault" >> ../../../cgep.env
source ../../../cgep.env
```

Do this after the apply, not after the plan. Outputs do not exist until the resources do. The `&&` guard is what stops a premature run appending Terraform's "No outputs found" warning into `cgep.env`, where sourcing the file then tries to execute it.

Verification

Every check below asserts a value, which the describe calls this section used to print could not. `aws s3api get-bucket-encryption` exits 0 whether the answer is `aws:kms` or `AES256`, and `get-bucket-versioning` prints nothing and still exits 0 on a bucket where versioning was never enabled. Object Lock in COMPLIANCE mode means nobody deletes the object, including you, including the account root, until the retention date passes. Reading a describe call that says so is not the same as watching the delete fail.

Pass the workspace, so it does not matter where you run it from:

```
bash scripts/verify.sh terraform
```

Every line names a control and either passes or fails with what it expected and what the account actually returned. The run ends in `VERIFIED` or `NOT VERIFIED` and sets the exit status accordingly.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 2.5. Every check asserts a value and exits non-zero if
# the account disagrees.
#
# This lab's controls are the ones most worth watching refuse. Object Lock in
# COMPLIANCE mode means nobody deletes the object, including you, including
# the account root, until the retention date passes. Reading a describe call
# that says so is not the same as watching the delete fail.
set -uo pipefail

# Point this at the workspace whose `terraform output` describes the resources
# you want checked, so it does not matter where you run it from. Guessing the
# working directory is how a verification script cheerfully checks the wrong
# account and passes.
WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0

pass() { printf '  PASS  %-8s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:-(nothing)}'"
  fi
}

refuse() { # refuse CONTROL LABEL COMMAND...
  local control=$1 label=$2 out rc
  shift 2
  out=$("$@" 2>&1)
  rc=$?
  if [[ $rc -ne 0 && "$out" == *AccessDenied* ]]; then
    pass "$control" "$label was refused"
  else
    fail "$control" "$label was NOT refused (exit $rc)"
  fi
}

VAULT=$(terraform output -raw vault_name)
MODE=$(terraform output -raw lock_mode)
KEY="runs/test-001/bundle.tar.gz"

# capture-evidence.sh sets VERSION_ID inside its own process, which never
# reaches yours, so derive it here rather than assuming it is set.
VERSION_ID=$(aws s3api list-object-versions --bucket "$VAULT" \
  --prefix "runs/test-001/" --query 'Versions[0].VersionId' --output text 2>/dev/null)

echo "=== configuration ==="

check SI-7 "object lock" "Enabled" \
  "$(aws s3api get-object-lock-configuration --bucket "$VAULT" \
```

```
       --query 'ObjectLockConfiguration.ObjectLockEnabled' --output text 2>/dev/null)"

check SC-28 "vault encryption" "aws:kms" \
  "$(aws s3api get-bucket-encryption --bucket "$VAULT" \
       --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.SSEAlgor
ithm' \
       --output text 2>/dev/null)"

# The mode is the whole lesson. GOVERNANCE can be bypassed by anyone holding
# s3:BypassGovernanceRetention; COMPLIANCE cannot be bypassed by anyone.
check AU-9 "retention mode" "$MODE" \
  "$(aws s3api get-object-retention --bucket "$VAULT" --key "$KEY" \
       --query 'Retention.Mode' --output text 2>/dev/null)"

if [[ -z "$VERSION_ID" || "$VERSION_ID" == "None" ]]; then
  fail "AU-9" "no object version found under runs/test-001/. Run capture-evidence.sh first."
else
  pass "AU-9" "object version present: $VERSION_ID"
fi

echo "=== enforcement ==="

refuse SC-8 "plain HTTP" \
  aws s3api list-objects-v2 --bucket "$VAULT" \
  --endpoint-url "http://s3.us-east-1.amazonaws.com"

# The versioned delete is the real test. Deleting without a version id only
# writes a delete marker, which succeeds and proves nothing: the object is
# still there underneath. Naming the version is what Object Lock refuses.
if [[ -n "$VERSION_ID" && "$VERSION_ID" != "None" ]]; then
  refuse AU-9 "versioned delete under $MODE retention" \
    aws s3api delete-object --bucket "$VAULT" --key "$KEY" --version-id "$VERSION_ID"
fi

# The misleading success, asserted rather than described. A delete with no
# version id writes a delete marker: the call succeeds, the object disappears
# from listings, and nothing was destroyed. It is the most misread result in
# this lab, so the script both proves it and then puts the vault back.
if aws s3api delete-object --bucket "$VAULT" --key "$KEY" >/dev/null 2>&1; then
  pass "AU-9" "unversioned delete succeeded, having written only a delete marker"
  MARKER=$(aws s3api list-object-versions --bucket "$VAULT" --prefix "$KEY" \
    --query 'DeleteMarkers[0].VersionId' --output text 2>/dev/null)
  if [[ -n "$MARKER" && "$MARKER" != "None" ]]; then
    # Delete markers carry no retention of their own, so this is permitted
    # even under COMPLIANCE mode, and it leaves the object visible again.
    if aws s3api delete-object --bucket "$VAULT" --key "$KEY" \
         --version-id "$MARKER" >/dev/null 2>&1; then
      pass "AU-9" "delete marker removed, the object is listed again"
    else
      fail "AU-9" "delete marker $MARKER could not be removed. Remove it by hand."
    fi
  fi
else
  fail "AU-9" "unversioned delete was refused, which is not how Object Lock behaves"
fi
```

```
echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the account."
else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-2-5"
mkdir -p "$EVIDENCE"

# The receipt: re-run the capture and keep its output this time.
scripts/capture-evidence.sh \
  --workspace ../lab-2-3 \
  --run-id    test-001 \
  --vault     "$VAULT" | tee "$EVIDENCE/receipt.json"

# The lock test: both refusals, the misleading success, and the cleanup.
bash scripts/verify.sh terraform 2>&1 | tee "$EVIDENCE/lock-test.txt"
```

Portfolio submission checklist

- [ ] `terraform/primitives/evidence-vault/` deploys the vault as shown.
- [ ] `scripts/capture-evidence.sh` committed and executable.
- [ ] At least one bundle uploaded, VersionId recorded in `evidence/lab-2-5/receipt.json`.
- [ ] `evidence/lab-2-5/lock-test.txt`, a transcript of the failed versioned delete.
- [ ] README states your `lock_mode` and `retention_days` **and defends them**.
- [ ] README states whether you capture state, and why. This is new here, and it is the question a data-protection reviewer asks first.

Troubleshooting

- **`Saved plan is stale`** when you apply. Something changed the state between your `plan -out=tfplan` and your `apply`, and a saved plan is a promise about one specific state. Often it is innocuous: a `terraform output` or `refresh` recording a data source counts. Plan again, read the new plan, apply that. Nothing is lost and nothing is broken; Terraform cannot distinguish a harmless change from a dangerous one, so it refuses instead of guessing.
- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache

settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.

- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`InvalidBucketState: Object Lock configuration cannot be enabled on existing buckets`.** Object Lock is creation-time only. There is no upgrade path. Destroy and recreate.
- **`COMPLIANCE` mode with too long a retention.** You cannot shorten or remove it. The objects sit until they expire, billing the whole time. The `acknowledge_compliance_mode` variable exists to make you type something before this can happen.
- **`--include-state` refused.** Working as designed. Read Step 3 and decide deliberately.
- **Clock drift.** `RetainUntilDate` is wall-clock UTC. If your laptop or runner clock is skewed, retention math will surprise you. Trust the server's date.
- **Cosign keyless from a laptop** needs a browser for the OIDC flow. In GitHub Actions it is automatic with `permissions: id-token: write`.
- **`delete-objects` fails with `MalformedXML`.** You passed an empty `Objects` list. Guard on length; see Cleanup.

Cleanup

`GOVERNANCE` allows bypass. `COMPLIANCE` does not, and if you chose it, the bucket stays until every retention expires. For a lab, that is the wrong choice; for production evidence, it is the point.

```
VAULT=$(terraform output -raw vault_name)

aws s3api list-object-versions --bucket "$VAULT" --output json \
  | jq '{Objects: [(.Versions // []), (.DeleteMarkers // [])] | flatten | .[] | {Key, VersionId}} ' \
  | jq -s '{Objects: map(.Objects)}' > /tmp/del.json

if [ "$(jq '.Objects | length' /tmp/del.json)" -gt 0 ]; then
  aws s3api delete-objects --bucket "$VAULT" --delete file:///tmp/del.json \
    --bypass-governance-retention
fi

terraform destroy -auto-approve
```

The length guard is there instead of a trailing `|| true`, which would swallow the empty-list error and every other error along with it. A cleanup script that cannot fail is a cleanup script that cannot tell you it failed.

The vault CMK enters its deletion waiting period and bills $1/month throughout.

How this feeds the capstone

This vault **is** the capstone's evidence vault.

- **Ch 4.3 and 4.4**, every PR runs the pipeline, which signs the bundle and uploads it here with this exact pattern.
- **Ch 5.2**, you turn on CloudTrail S3 data events for this bucket specifically. It is the one bucket where "who read the evidence" is itself an audit question.
- **Ch 6.1**, your OSCAL component's `links[rel=evidence].href` resolves to an object in here, by VersionId.
- **The grader** downloads from here and runs `verify-evidence.sh`. Build it once, well.

Revision history

These notes

- Vault encryption moved from SSE-S3 to a dedicated customer-managed KMS key, separate from the workload key, adding SC-12 and SC-13.
- Added a bucket policy denying `aws:SecureTransport = false`, adding SC-8.
- `capture-evidence.sh` no longer captures raw Terraform state by default. It bundles the plan and the module attestation instead, and refuses `--include-state` against a COMPLIANCE-mode vault without an explicit acknowledgment.
- Added an `acknowledge_compliance_mode` guard so COMPLIANCE retention beyond 7 days must be deliberate.
- The receipt records whether state was included, and the VersionId is read with `--query` rather than parsed out of JSON by hand.
- Cleanup guards on object-list length instead of relying on `|| true`.

The official labs

Initial release: AES256 vault, `terraform state pull` captured into every bundle by default, no TLS-deny policy.

CGE-P LAB NOTES

Lab 3.3: Writing Compliance Policies in Rego (GCP)

You wrote Terraform that satisfies controls. Now you write the policy that proves a plan satisfies them before it ever applies. Five policies, five controls, one library that survives a cloud change.

The `# METADATA` blocks are not decoration. Lab 6.1 extracts them to build the OSCAL component, so each control mapping is authored once and consumed twice.

For reading. Code lines wrap to fit the page; copy code from docs/03_03_writing_compliance_policies_rego.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/03_03_writing_compliance_policies_rego.md` in your clone, not from the PDF.

Learning objectives

- Author Rego policies with structured annotations that map each rule to a NIST control.
- Write `_test.rego` fixtures asserting both passing and failing behavior.
- **Mutation-test every policy**, so you know it can fail, not merely that it passed.
- Extract policy metadata programmatically with `opa inspect`, so the mapping feeds Chapter 6 instead of decorating a comment.

The five policies

Control	File	Enforces
SC-28	`sc28_encryption.rego`	Every `google_storage_bucket` has a CMEK `encryption` block.
AC-3	`ac3_no_public.rego`	Uniform access + `public_access_prevention = "enforced"`; firewalls do not expose 22 or 3389 to `0.0.0.0/0`.
CM-6	`cm6_required_tags.rego`	Four required labels on every taggable resource.
AU-3	`au3_access_logging.rego`	**New in v2.** Every bucket has a `logging` block naming a target.
AU-9	`au9_log_immutability.rego`	**New in v2.** The bucket named as a logging target is itself versioned.

Every deny message carries the resource address **and** the control ID. The developer fixes their own violation without filing a GRC ticket.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- OPA `>= 1.0` (tested on 1.19.1). Policies declare `import rego.v1`, so they also run on 0.x `>= 0.60`.
- Terraform `>= 1.9`. The `google` provider does **not** need real credentials for this lab; see the note at Step 2.
- Lab 2.4 completed. Its module is what the fixture consumes, and its `compliance_attestation` is what the AU-9 rule leans on.

Estimated time & cost

- 75 to 90 minutes. The mutation-testing section accounts for much of that, and it is the part worth the time.
- Free. Everything runs against `terraform plan` output. Nothing is applied.

Step-by-step walkthrough

Step 1 Structure

```
mkdir -p policies/tests terraform fixtures
```

Step 2 The mixed fixture

A fixture with compliant and non-compliant resources gives the suite something concrete to flag. This is Lab 2.4's world, deliberately broken in five specific ways.

```
# terraform/main.tf
terraform {
  required_version = ">= 1.9"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}

provider "google" {
  project = var.gcp_project
  region  = "us-central1"
}

variable "gcp_project" { type = string }

locals {
  labels = {
    project          = "lab33"
    environment      = "dev"
    managed_by       = "terraform"
    compliance_scope = "cge-p-lab"
  }
}

resource "google_kms_key_ring" "ring" {
  name     = "lab33-ring"
  location = "us-central1"
}

resource "google_kms_crypto_key" "key" {
```

```
  name     = "lab33-key"
  key_ring = google_kms_key_ring.ring.id
}

# A compliant log bucket: versioned, so AU-9 is satisfiable.
resource "google_storage_bucket" "logs" {
  name                        = "${var.gcp_project}-lab33-logs"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  versioning { enabled = true }

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }

  labels = local.labels
}

# An UNVERSIONED log bucket. AU-9 should fire on anything logging here.
resource "google_storage_bucket" "bad_logs" {
  name                        = "${var.gcp_project}-lab33-badlogs"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }

  labels = local.labels
}

# Fully compliant.
resource "google_storage_bucket" "good" {
  name                        = "${var.gcp_project}-lab33-good"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  versioning { enabled = true }
  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging {
    log_bucket        = google_storage_bucket.logs.name
    log_object_prefix = "access-logs/"
  }

  labels = local.labels
}

# SC-28 should fire: no encryption block.
resource "google_storage_bucket" "bad_no_cmek" {
  name                        = "${var.gcp_project}-lab33-no-cmek"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  logging { log_bucket = google_storage_bucket.logs.name }
  labels  = local.labels
```

```
}

# AC-3 should fire: uniform access off, prevention not enforced.
resource "google_storage_bucket" "bad_public" {
  name                        = "${var.gcp_project}-lab33-public"
  location                    = "us-central1"
  uniform_bucket_level_access = false
  public_access_prevention    = "inherited"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging    { log_bucket = google_storage_bucket.logs.name }
  labels     = local.labels
}

# CM-6 should fire: no labels.
resource "google_storage_bucket" "bad_no_labels" {
  name                        = "${var.gcp_project}-lab33-no-labels"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging    { log_bucket = google_storage_bucket.logs.name }
}

# AU-3 should fire: no logging block at all.
resource "google_storage_bucket" "bad_no_logging" {
  name                        = "${var.gcp_project}-lab33-no-logging"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  labels     = local.labels
}

# AU-9 should fire: logs to a bucket that is not versioned.
resource "google_storage_bucket" "bad_log_target" {
  name                        = "${var.gcp_project}-lab33-badtarget"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging    { log_bucket = google_storage_bucket.bad_logs.name }
  labels     = local.labels
}

resource "google_compute_network" "demo" {
  name                    = "lab33-demo"
  auto_create_subnetworks = false
}

# AC-3 should fire.
resource "google_compute_firewall" "open_ssh" {
  name          = "lab33-open-ssh"
```

```
  network       = google_compute_network.demo.name
  direction     = "INGRESS"
  source_ranges = ["0.0.0.0/0"]

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }
}
```

```
cd terraform
terraform init
terraform plan -out=tfplan && terraform show -json tfplan > plan.json
```

The `&&` is load bearing. Written as two separate lines, a failed plan still runs the redirect, and `plan.json` is created as an empty file. Everything downstream then reads it without complaint.

You never apply. The policies operate on `plan.json`.

This lab needs no Google account. The provider refuses to start without credentials, which reads like a hard prerequisite and is not one. The fixture declares no data sources and is never applied, so nothing here ever calls the API. A placeholder token satisfies the provider and the plan is produced entirely offline:

```
export GOOGLE_OAUTH_ACCESS_TOKEN=offline-fixture
export TF_VAR_gcp_project=cgep-lab-fixture
terraform plan -out=tfplan
```

Use your real project if you have one; the plan is identical either way, because the resource names in `plan.json` are what the policies read.

Do not carry this trick into a lab that applies anything. It works here for two specific reasons, no data sources and no apply, and a fake token in Lab 2.4 buys you a confusing authentication failure partway through creating real infrastructure.

Step 3 SC-28

```
# policies/sc28_encryption.rego
# METADATA
# title: SC-28 - Encryption at Rest (GCS)
# description: "Every google_storage_bucket must encrypt at rest with a customer-managed encryption
key (CMEK)."
# authors:
#   - CGE-P
# custom:
```

```
#   control_id: SC-28
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add an encryption { default_kms_key_name = ... } block referencing a
google_kms_crypto_key you control."
package compliance.sc28

import rego.v1

deny contains msg if {
    some resource in all_buckets
    not has_cmek(resource)
    msg := sprintf(
        "[SC-28] %s: missing customer-managed encryption key. Remediation: add encryption
{ default_kms_key_name = ... }.",
        [resource.address],
    )
}

all_buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}

all_buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

has_cmek(resource) if {
    count(resource.values.encryption) > 0
    not empty_kms_key(resource.values.encryption[0])
}

empty_kms_key(enc) if enc.default_kms_key_name == ""
empty_kms_key(enc) if enc.default_kms_key_name == null
```

Why `has_cmek` checks the block, not the value. At plan time the KMS key ID is "(known after apply)" because the key does not exist yet, and the plan JSON omits unknown values entirely. Requiring a populated string would fail every compliant configuration. Accept a non-empty block; fail only when it is missing or explicitly empty. This is correct policy semantics, not a workaround.

Note the `all_buckets` helper. The obvious alternative is to duplicate the whole `deny` rule to recurse into `child_modules`, once per policy. Factoring the traversal into one set comprehension halves the length of every rule, and means a bug in the recursion is fixed in one place.

Step 4 AU-3 and AU-9, the two new ones

```
# policies/au3_access_logging.rego
# METADATA
# title: AU-3 - Content of Audit Records (GCS access logging)
# description: "Every google_storage_bucket must emit access logs to a named target bucket."
# custom:
#   control_id: AU-3
#   framework: nist-800-53-rev5
#   severity: medium
#   remediation: "Add a logging { log_bucket = ... } block naming a dedicated log bucket."
package compliance.au3

import rego.v1

deny contains msg if {
    some resource in buckets
    not is_log_sink(resource)
    not has_logging(resource)
    msg := sprintf(
        "[AU-3] %s: no access logging configured. Remediation: add logging { log_bucket = ... }.",
        [resource.address],
    )
}

buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}

buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

has_logging(resource) if {
    count(resource.values.logging) > 0
    resource.values.logging[0].log_bucket != ""
}

# A bucket that is itself a logging target is exempt. Without this exemption
# the rule demands infinite regress: every log bucket needs a log bucket.
is_log_sink(resource) if {
    some other in buckets
    count(other.values.logging) > 0
    other.values.logging[0].log_bucket == resource.values.name
}
```

That exemption is the interesting part, and it is the kind of thing you only discover by running the policy against a real plan. A naive AU-3 rule flags your log bucket for not having a log bucket, forever. Deciding where the recursion stops is a policy decision, and stating it in the rule is better than suppressing the finding later.

```
# policies/au9_log_immutability.rego
# METADATA
# title: AU-9 - Protection of Audit Information (log target versioning)
# description: "A bucket named as a logging target must have versioning enabled so audit records
cannot be silently overwritten."
# custom:
#   control_id: AU-9
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Enable versioning { enabled = true } on the bucket used as a logging target."
package compliance.au9

import rego.v1

deny contains msg if {
    some target_name in log_target_names
    some bucket in buckets
    bucket.values.name == target_name
    not versioned(bucket)
    msg := sprintf(
        "[AU-9] %s: used as a logging target but versioning is disabled. Audit records can be silently
overwritten. Remediation: versioning { enabled = true }.",
        [bucket.address],
    )
}

buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}

buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

log_target_names contains name if {
    some b in buckets
    count(b.values.logging) > 0
    name := b.values.logging[0].log_bucket
}

versioned(bucket) if {
    count(bucket.values.versioning) > 0
    bucket.values.versioning[0].enabled == true
}
```

Write the rule that catches your own past mistake. It is the most reliable source of good policy ideas you will ever have, and this one catches a mistake almost everybody makes once: versioning the bucket that holds the data and forgetting the one that holds the record of who touched it.

Step 5 AC-3 and CM-6

Both use the shared `buckets` helper rather than repeating the traversal.

```
# policies/ac3_no_public.rego
# METADATA
# title: AC-3 - Access Enforcement (no public GCS, no open management ports)
# description: "GCS buckets must enforce uniform_bucket_level_access AND
public_access_prevention=enforced. Firewalls must not allow 0.0.0.0/0 on ports 22 or 3389."
# custom:
#   control_id: AC-3
#   framework: nist-800-53-rev5
#   severity: critical
#   remediation: "Set uniform_bucket_level_access = true and public_access_prevention = \"enforced\".
For firewalls, narrow source_ranges."
package compliance.ac3

import rego.v1

deny contains msg if {
    some r in buckets
    not locked_down(r)
    msg := sprintf(
        "[AC-3] %s: bucket allows public access. Remediation: set uniform_bucket_level_access=true and
public_access_prevention=\"enforced\".",
        [r.address],
    )
}

deny contains msg if {
    some r in input.planned_values.root_module.resources
    r.type == "google_compute_firewall"
    r.values.direction == "INGRESS"
    some src in r.values.source_ranges
    public_range(src)
    some allow in r.values.allow
    some port in allow.ports
    mgmt_port(port)
    msg := sprintf(
        "[AC-3] %s: management port %s open to %s. Remediation: narrow source_ranges or remove the
rule.",
        [r.address, port, src],
    )
}

buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}

buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

locked_down(r) if {
    r.values.uniform_bucket_level_access == true
    r.values.public_access_prevention == "enforced"
}
```

```
mgmt_port(p) if p == "22"
mgmt_port(p) if p == "3389"

public_range(s) if s == "0.0.0.0/0"
public_range(s) if s == "*"
```

```
# policies/cm6_required_tags.rego
# METADATA
# title: CM-6 - Configuration Settings (required compliance labels)
# description: "Every taggable resource must carry the four required labels."
# custom:
#   control_id: CM-6
#   framework: nist-800-53-rev5
#   severity: medium
#   remediation: "Add the four required labels, or consume the Lab 2.4 module which merges them."
package compliance.cm6

import rego.v1

required := {"project", "environment", "managed_by", "compliance_scope"}

labelable_type(t) if t == "google_storage_bucket"
labelable_type(t) if t == "google_compute_instance"
labelable_type(t) if t == "google_compute_disk"

deny contains msg if {
    some resource in all_resources
    labelable_type(resource.type)
    missing := required - provided_labels(resource)
    count(missing) > 0
    msg := sprintf(
        "[CM-6] %s: missing required labels %v. Remediation: add them, or consume the compliant
module.",
        [resource.address, sort_array(missing)],
    )
}

all_resources contains r if { some r in input.planned_values.root_module.resources }

all_resources contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
}

provided_labels(resource) := keys if {
    resource.values.labels
    keys := {k | resource.values.labels[k]}
}

provided_labels(resource) := set() if not resource.values.labels

sort_array(s) := sort([x | some x in s])
```

Set subtraction requires sets on both sides. `required - provided` fails if `provided_labels` returns an array. That is exactly why it is a set comprehension `{k | ...}` and not a list.

Step 6 Tests

One passing and one failing fixture per policy, minimum.

```
# policies/tests/au9_log_immutability_test.rego
package compliance.au9_test

import rego.v1
import data.compliance.au9

compliant := {"planned_values": {"root_module": {"resources": [
    {
        "address": "google_storage_bucket.app",
        "type": "google_storage_bucket",
        "values": {"name": "app", "logging": [{"log_bucket": "logs"}], "versioning": []},
    },
    {
        "address": "google_storage_bucket.logs",
        "type": "google_storage_bucket",
        "values": {"name": "logs", "logging": [], "versioning": [{"enabled": true}]},
    },
]}}}

noncompliant := {"planned_values": {"root_module": {"resources": [
    {
        "address": "google_storage_bucket.app",
        "type": "google_storage_bucket",
        "values": {"name": "app", "logging": [{"log_bucket": "logs"}], "versioning": []},
    },
    {
        "address": "google_storage_bucket.logs",
        "type": "google_storage_bucket",
        "values": {"name": "logs", "logging": [], "versioning": [{"enabled": false}]},
    },
]}}}

test_versioned_log_target_passes if {
    count(au9.deny) == 0 with input as compliant
}

test_unversioned_log_target_fails if {
    some msg in au9.deny with input as noncompliant
    contains(msg, "AU-9")
    contains(msg, "google_storage_bucket.logs")
}
```

Write the equivalent for the other four. Then:

```
opa test -v policies/
```

Step 7 Mutation testing, or: prove the policy can fail

This section is new here, and it is the most important part of the lab.

A passing test suite proves your policy returns the answer you expected on the inputs you thought of. It does not prove the policy is *load-bearing*. A rule with a typo in the resource type matches nothing, denies nothing, and passes every "compliant input produces zero denials" test you wrote. It is a control that cannot fail, which is to say it is not a control.

The check is mechanical: break the policy on purpose and confirm the tests notice. The script repeats that reasoning in its own header rather than relying on this page, because it is a file you commit and a reviewer opens on its own.

```
#!/usr/bin/env bash
# scripts/mutation-test.sh
# For each policy, apply a mutation that should break it, and require that
# `opa test` FAILS. A mutation that survives means no test constrains that
# rule, and the rule is decorative.
#
# A passing suite proves the policy returns the answer you expected on the
# inputs you thought of. It does not prove the policy is load-bearing. A rule
# with a typo in the resource type matches nothing, denies nothing, and passes
# every "compliant input produces zero denials" test you wrote. It is a control
# that cannot fail, which is to say it is not a control.
set -uo pipefail

POLICY_DIR="${1:-policies}"
FAILED=0

for f in "$POLICY_DIR"/*.rego; do
  cp "$f" "$f.bak"

  # Mutation: invert every equality comparison in the rule bodies.
  sed -i 's/ == / != /g' "$f"

  if opa test "$POLICY_DIR" >/dev/null 2>&1; then
    echo "SURVIVED: $f"
    echo "          tests still pass with the logic inverted."
    echo "          This rule is not constrained by any test."
    FAILED=1
  else
    echo "killed:   $f"
  fi

  mv "$f.bak" "$f"
done

if [[ $FAILED -eq 0 ]]; then
  echo "All policies are constrained by at least one test."
```

```
else
  echo "At least one policy is unconstrained. Treat it as enforcing nothing." >&2
fi
exit $FAILED
```

```
chmod +x scripts/mutation-test.sh
bash scripts/mutation-test.sh
```

Every policy must report `killed`. A `SURVIVED` line means that policy has no test which actually depends on its logic, and you should assume it is not enforcing anything until you have written one that does.

This is worth doing once by hand to feel it. Comment out the body of your SC-28 `has_cmek` rule so it always returns true, run `opa test`, and watch which tests go red. If none do, your fixtures are not exercising the rule.

The general principle transfers well beyond Rego: **every compliance check must be mutation-tested before it is trusted.** An unfired control and an absent control look identical in a green pipeline.

Step 8 Metadata as data, not decoration

It is tempting to treat `# METADATA` as a comment. OPA parses it as structured annotations, and you can extract them:

```
opa inspect --annotations --format json policies/ \
  | jq '[.annotations[] | {
      control: .annotations.custom.control_id,
      severity: .annotations.custom.severity,
      title: .annotations.title,
      package: (.path | map(.value) | join(".")),
      remediation: .annotations.custom.remediation
    }]' > "$EVIDENCE/policy-catalog.json"

cat "$EVIDENCE/policy-catalog.json"
```

```
[
  { "control": "SC-28", "severity": "high",     "package": "data.compliance.sc28", "...": "..." },
  { "control": "AC-3",  "severity": "critical", "package": "data.compliance.ac3",  "...": "..." },
  { "control": "CM-6",  "severity": "medium",   "package": "data.compliance.cm6",  "...": "..." },
  { "control": "AU-3",  "severity": "medium",   "package": "data.compliance.au3",  "...": "..." },
  { "control": "AU-9",  "severity": "high",     "package": "data.compliance.au9",  "...": "..." }
]
```

That file is the bridge to Chapter 6. Lab 6.1 reads it to generate the `implemented-requirement` entries in your OSCAL component, so **the control mapping is authored once, in the policy that enforces it, and never retyped.** Every place a mapping is retyped is a place it can drift.

Step 9 Run the library against the real plan

```
opa test -v policies/
bash scripts/mutation-test.sh

for ns in sc28 ac3 cm6 au3 au9; do
  echo "=== $ns ==="
  opa eval -d policies -i terraform/plan.json "data.compliance.$ns.deny" --format=pretty
done
```

Expected, abridged:

```
=== sc28 ===
["[SC-28] google_storage_bucket.bad_no_cmek: missing customer-managed encryption key. ..."]

=== ac3 ===
["[AC-3] google_compute_firewall.open_ssh: management port 22 open to 0.0.0.0/0. ...",
 "[AC-3] google_storage_bucket.bad_public: bucket allows public access. ..."]

=== cm6 ===
["[CM-6] google_storage_bucket.bad_no_labels: missing required labels [...]. ..."]

=== au3 ===
["[AU-3] google_storage_bucket.bad_no_logging: no access logging configured. ..."]

=== au9 ===
["[AU-9] google_storage_bucket.bad_logs: used as a logging target but versioning is disabled. ..."]
```

Each broken resource flagged exactly once by the right control. The good bucket is quiet, and so is the compliant log bucket, because the AU-3 exemption works.

Step 10 Close the loop: make the fixture compliant

Watching the policies fire proves they can deny. It does not prove they stop denying once the problem is fixed, and a rule that fires on everything is as useless as one that fires on nothing. Six edits in `terraform/main.tf`:

Resource	Control	Change
`google_storage_bucket.bad_public`	AC-3	`uniform_bucket_level_access = true` and `public_access_prevention = "enforced"`
`google_compute_firewall.open_ssh`	AC-3	narrow `source_ranges` to something like `["10.0.0.0/8"]`
`google_storage_bucket.bad_no_cmek`	SC-28	add an `encryption` block naming `google_kms_crypto_key.key.id`
`google_storage_bucket.bad_no_labels`	CM-6	add `labels = local.labels`
`google_storage_bucket.bad_no_logging`	AU-3	add a `logging` block targeting `google_storage_bucket.logs.name`
`google_storage_bucket.bad_logs`	AU-9	add `versioning { enabled = true }`

Copy the shapes from `google_storage_bucket.good`, which is the compliant one and is right there in the same file.

Then replan and ask for the opposite result:

```
cd terraform
terraform plan -out=tfplan && terraform show -json tfplan > plan.json
cd ..
bash scripts/verify.sh --expect-clean
```

`bad_logs` is the one worth pausing on. Nothing about that bucket is insecure on its own; it fails only because `bad_log_target` names it as a logging destination, and a log sink that can be overwritten is not evidence. The rule reads the relationship, not the resource, which is why AU-9 needed its own policy rather than a versioning check bolted onto AU-3.

Put the fixture back when you are done. It is the artifact you re-run against, and a repository where every policy passes teaches nobody anything:

```
git checkout -- terraform/main.tf
```

Verification

An empty deny set is what a broken run looks like too. That is the trap in this lab, and it is worse than the usual kind because emptiness is also the goal: once you fix the fixture, every namespace is supposed to return `[]`. Run `opa eval` against a `plan.json` that is zero bytes, because `terraform plan` failed a few commands earlier, and you get exactly the same five empty sets and exit status 0. Nothing anywhere reports a problem.

So this script asserts the **input** before it believes anything the policies say about it: the file exists, parses, and describes resources. Only then does it check the deny sets, against the count each control should produce.

```
bash scripts/verify.sh
```

Expect one denial each from SC-28, CM-6, AU-3 and AU-9, and two from AC-3, which owns both the public bucket and the open SSH rule. After you fix the fixture, ask for the opposite and the input guard still applies:

```
bash scripts/verify.sh --expect-clean
```

That difference is the point. `--expect-clean` passing means the fixture is compliant **and** there was a real plan to judge it against. Reading `[]` off the screen proves only the first, and only if you already know the second.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 3.3.
#
# The failure this exists to prevent: `opa eval` against a zero byte plan.json
# returns [] for every namespace and exits 0. An empty deny set is also what
# success looks like once you have fixed the fixture, so a pipeline that broke
# three commands earlier is indistinguishable from a clean run. This asserts
# the INPUT is real before it believes anything the policies say about it.
#
#   bash scripts/verify.sh                 # the shipped fixture, expect denials
#   bash scripts/verify.sh --expect-clean  # after you fix it, expect none
set -uo pipefail

EXPECT_CLEAN=0
[[ "${1:-}" == "--expect-clean" ]] && EXPECT_CLEAN=1

FAILED=0
pass() { printf '  PASS  %-8s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

PLAN=terraform/plan.json

echo "=== the input ==="

if [[ ! -s "$PLAN" ]]; then
  fail INPUT "$PLAN is missing or empty. terraform plan probably failed, and every deny set below
would read [] regardless."
  echo; echo "NOT VERIFIED: no plan to evaluate." >&2
  exit 1
fi
pass INPUT "$PLAN is $(wc -c < "$PLAN") bytes"

if ! jq -e . "$PLAN" >/dev/null 2>&1; then
  fail INPUT "$PLAN is not valid JSON."
  echo; echo "NOT VERIFIED: no plan to evaluate." >&2
  exit 1
fi

# A plan that parses can still describe nothing. Count what the policies read.
RESOURCES=$(jq '[.planned_values.root_module.resources[]?] | length' "$PLAN" 2>/dev/null)
if [[ "${RESOURCES:-0}" -gt 0 ]]; then
  pass INPUT "plan describes $RESOURCES resources"
else
  fail INPUT "plan parses but describes no resources, so every policy has nothing to judge."
  echo; echo "NOT VERIFIED: nothing to evaluate." >&2
  exit 1
fi

echo "=== deny sets ==="

# Expected counts for the fixture as shipped. Each broken resource is flagged
# exactly once by the control that owns it, and AC-3 owns two of them: the
# public bucket and the open SSH rule.
declare -A EXPECTED=( [sc28]=1 [ac3]=2 [cm6]=1 [au3]=1 [au9]=1 )
```

```
for ns in sc28 ac3 cm6 au3 au9; do
  count=$(opa eval -d policies -i "$PLAN" "data.compliance.$ns.deny" --format=json 2>/dev/null \
          | jq '[.result[]?.expressions[]?.value[]?] | length')
  count=${count:-0}
  if [[ $EXPECT_CLEAN -eq 1 ]]; then
    want=0
  else
    want=${EXPECTED[$ns]}
  fi
  if [[ "$count" == "$want" ]]; then
    pass "$ns" "$count denial(s)"
  else
    fail "$ns" "expected $want denial(s), got $count"
  fi
done

echo
if [[ $FAILED -eq 0 ]]; then
  if [[ $EXPECT_CLEAN -eq 1 ]]; then
    echo "VERIFIED: the fixture is clean, and the plan it was judged against was real."
  else
    echo "VERIFIED: every control fired exactly where expected."
  fi
else
  echo "NOT VERIFIED: the deny sets do not match. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-3-3"
mkdir -p "$EVIDENCE"
opa test policies/ --format=json > "$EVIDENCE/opa-test-results.json"
bash scripts/mutation-test.sh 2>&1 | tee "$EVIDENCE/mutation-results.txt"
bash scripts/verify.sh 2>&1 | tee "$EVIDENCE/deny-sets.txt"
```

The mutation output matters more than the test results. Passing tests show the suite runs; the mutation log shows each rule was broken on purpose and the suite noticed.

Portfolio submission checklist

- [ ] `policies/` with five policies, each carrying a `# METADATA` block.
- [ ] `policies/tests/` with passing and failing fixtures for each.
- [ ] `scripts/mutation-test.sh` committed and executable.
- [ ] `evidence/lab-3-3/opa-test-results.json` from `opa test --format=json policies/`.
- [ ] `evidence/lab-3-3/mutation-results.txt`.

- [ ] `evidence/lab-3-3/deny-sets.txt`, showing each control firing where expected.
- [ ] `evidence/lab-3-3/policy-catalog.json`.
- [ ] `policies/README.md` listing each policy, control, severity, and remediation.

Troubleshooting

- `rego_parse_error: yaml: mapping values are not allowed` in METADATA. The YAML parser rejects unquoted colons inside a value. Quote `description` and `remediation`.
- **A passing fixture surprises you with a deny.** Module-wrapped resources live under `child_modules[]`, not `root_module.resources`. Use the shared `buckets` / `all_resources` helpers.
- `encryption: [{}]` **on a CMEK bucket.** Plan-time unknowns are omitted from the JSON. Require the block, not the value.
- **Set subtraction returns the wrong type.** Both operands must be sets. Use `{k | ...}`, not `[k | ...]`.
- **Mutation test reports SURVIVED on a rule you are sure works.** The `sed` mutation only touches `==`, so rules built purely from `count()` or `not` need a hand-written mutation. Add one, or accept that this rule needs a targeted test instead.
- `opa inspect` **shows no annotations.** The `# METADATA` block must sit immediately above the `package` declaration or a rule, with no blank line between.

Cleanup

Plan-only. Nothing deployed unless you applied the fixture, which the lab does not require. If you did:

```
cd terraform && terraform destroy -auto-approve
```

How this feeds the capstone

- **Ch 3.4** adds AWS variants targeting `aws_s3_bucket` and friends, and runs the combined suite through Conftest.
- **Ch 4.3** calls that gate on every PR.
- **Ch 6.1** reads `policy-catalog.json` to generate OSCAL `implemented-requirement` entries. Authored once, consumed twice.
- **The capstone** requires five or more Rego policies with tests, each citing a control from your declared framework. You have five, tested, mutation-checked, and self-describing. The remaining work is retargeting the control IDs from NIST 800-53 to HIPAA, SOC 2, or CMMC, which is a metadata edit rather than a rewrite. That portability is the entire argument for putting the control ID in the annotation instead of the filename.

Revision history

These notes

- Two policies added: AU-3 (access logging configured) and AU-9 (the logging target is itself versioned).
- Added `scripts/mutation-test.sh`. A policy no test constrains cannot fail, so each one is broken on purpose and the tests must notice.
- Refactored the `child_modules` traversal into a shared helper rather than duplicating the `deny` rule per policy.
- `# METADATA` annotations are now consumed rather than decorative: Lab 6.1 extracts them with `opa inspect` to generate the OSCAL component.

The official labs

Initial release: three policies (SC-28, AC-3, CM-6), metadata blocks present but unused, traversal duplicated per rule.

CGE-P LAB NOTES

Lab 3.4: Integrating PaC with Terraform via Conftest (AWS)

You wrote five Rego policies in Lab 3.3 against GCP fixtures. This lab does two things. It runs them against an AWS Terraform plan via Conftest, and it forces you to add AWS variants, because rules that hardcode `google_storage_bucket` match nothing in an AWS plan.

The library survives the cloud change because the control IDs do. The rules do not.

For reading. Code lines wrap to fit the page; copy code from docs/03_04_integrating_pac_with_terraform.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/03_04_integrating_pac_with_terraform.md` in your clone, not from the PDF.

Learning objectives

- Wire Conftest into the plan workflow as a fail-closed gate.
- Add AWS variants for SC-8, SC-28, AC-3, AU-9, and CM-6, preserving control IDs.
- Understand why some controls cannot be fully asserted at plan time, and what to do about it.
- Demonstrate a blocked merge by feeding a deliberately broken plan to the gate.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- Lab 2.3 workspace on disk. Its plan is the input, and it contains a KMS key and two bucket policies the rules below depend on.
- Lab 3.3 policy library carried forward. Five rules, five controls.
- Conftest `>= 0.50`.
- OPA `>= 0.60`, for `opa test` and the mutation script.

Estimated time & cost

- 60 minutes.
- Free. No AWS resources beyond Lab 2.3.

Step-by-step walkthrough

Step 1 Carry the library forward and re-test

```
cp -r ../lab-3-3/policies ./policies
cp -r ../lab-3-3/scripts  ./scripts
opa test -v policies/
bash scripts/mutation-test.sh
```

Sanity check before extending. If the mutation script reports a survivor now, fix it now; adding five more rules on top of an untrusted base compounds the problem.

Step 2 Generate the AWS plan

```
cd ../lab-2-3
export AWS_PROFILE=cgep
terraform init
terraform plan -out=tfplan
terraform show -json tfplan > plan.json
```

Step 3 The cross-cloud lesson

Run the GCP policies against the AWS plan:

```
conftest test --policy policies --namespace compliance.sc28 plan.json
conftest test --policy policies --namespace compliance.ac3  plan.json
conftest test --policy policies --namespace compliance.au9  plan.json
```

They pass, with zero coverage. There are no `google_storage_bucket` resources here, so the rules match nothing and deny nothing.

Sit with that for a moment, because it is the same failure mode as an untested policy. A rule that matches nothing is indistinguishable from a rule that found nothing wrong. Conftest reports `0 failures` for both. The only difference is whether your infrastructure is compliant or your policy is inert.

That is why Lab 3.3 ends with mutation testing, and it is why the AWS variants below are separate files rather than a generalized rule. Two readable rules beat one clever rule whose coverage you cannot eyeball.

Step 4 SC-28 for AWS, now requiring a CMK

Asking whether *any* encryption configuration exists is the weaker question. Lab 2.3 uses a customer-managed key and the capstone requires one, so this rule asks whether the right kind is in place.

```
# policies/sc28_encryption_aws.rego
# METADATA
# title: SC-28 - Encryption at Rest (AWS S3, customer-managed key)
# description: "Every aws_s3_bucket must have a server-side encryption configuration using aws:kms
with a customer-managed key."
# custom:
#   control_id: SC-28
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add aws_s3_bucket_server_side_encryption_configuration with sse_algorithm =
\"aws:kms\", kms_master_key_id referencing your aws_kms_key, and bucket_key_enabled = true."
package compliance.sc28_aws

import rego.v1

deny contains msg if {
    some bucket in bucket_addresses
```

```
    not has_encryption(bucket)
    msg := sprintf(
        "[SC-28] %s: no aws_s3_bucket_server_side_encryption_configuration references this bucket.
Remediation: add one.",
        [bucket],
    )
}

deny contains msg if {
    some bucket in bucket_addresses
    has_encryption(bucket)
    not has_kms_encryption(bucket)
    msg := sprintf(
        "[SC-28] %s: encrypted with SSE-S3 (AES256), not a customer-managed key. Remediation:
sse_algorithm = \"aws:kms\" with kms_master_key_id.",
        [bucket],
    )
}

bucket_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket"
    addr := sprintf("aws_s3_bucket.%s", [r.name])
}

sse_configs contains r if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_server_side_encryption_configuration"
}

has_encryption(bucket_addr) if {
    some r in sse_configs
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
}

has_kms_encryption(bucket_addr) if {
    some r in sse_configs
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
    r.expressions.rule[_].apply_server_side_encryption_by_default[_].sse_algorithm.constant_value ==
"aws:kms"
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.arn", [addr])
```

Why match by reference, not by value. At plan time the bucket name is "(known after apply)" because the `random_id` suffix has not been generated. Both `aws_s3_bucket.values.bucket` and the encryption resource's `values.bucket` are `null` in the JSON. Use `configuration.root_module.resources[].expressions.bucket.references`, which holds strings like `"aws_s3_bucket.primary.id"` that Terraform resolves at apply.

Two separate `deny` rules rather than one compound condition. A missing configuration and a weak configuration are different findings with different remediations, and merging them produces a message that is wrong half the time.

Step 5 SC-8 for AWS, and the limits of plan-time policy

This is the hardest of the five to assert, and the reason is worth understanding.

A bucket policy is a JSON string. In Lab 2.3 it is produced by an `aws_iam_policy_document` data source whose inputs include bucket ARNs, which depend on names, which depend on `random_id`. **At plan time the rendered policy JSON is unknown.** You cannot parse what Terraform has not computed.

So the rule works in two tiers, and says so:

```
# policies/sc8_tls_required_aws.rego
# METADATA
# title: SC-8 - Transmission Confidentiality (S3 TLS enforcement)
# description: "Every aws_s3_bucket must have a bucket policy, and that policy must deny requests
where aws:SecureTransport is false."
# custom:
#   control_id: SC-8
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add an aws_s3_bucket_policy with a Deny statement on s3:* when Bool
aws:SecureTransport = false."
#   plan_time_limitation: "Rendered policy JSON is unknown at plan time when bucket ARNs are computed.
Tier 1 (a policy exists) always evaluates; tier 2 (it denies non-TLS) evaluates only when the JSON is
known. Post-apply verification closes the gap."
package compliance.sc8_aws

import rego.v1

# Tier 1: structural. Always evaluable. A bucket with no policy at all cannot
# be enforcing TLS.
deny contains msg if {
    some bucket in bucket_addresses
    not has_policy(bucket)
    msg := sprintf(
        "[SC-8] %s: no aws_s3_bucket_policy references this bucket, so non-TLS requests are permitted.
Remediation: add a policy denying aws:SecureTransport = false.",
        [bucket],
    )
}

# Tier 2: semantic. Only fires when the rendered JSON is known, which happens
# for hardcoded bucket names or on a re-plan against existing state.
deny contains msg if {
    some r in input.planned_values.root_module.resources
    r.type == "aws_s3_bucket_policy"
    is_string(r.values.policy)
    doc := json.unmarshal(r.values.policy)
    not denies_insecure_transport(doc)
    msg := sprintf(
```

```
        "[SC-8] %s: bucket policy is present but contains no Deny on aws:SecureTransport = false.",
        [r.address],
    )
}

bucket_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket"
    addr := sprintf("aws_s3_bucket.%s", [r.name])
}

has_policy(bucket_addr) if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_policy"
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
}

denies_insecure_transport(doc) if {
    some stmt in doc.Statement
    stmt.Effect == "Deny"
    stmt.Condition.Bool["aws:SecureTransport"] == "false"
}

denies_insecure_transport(doc) if {
    some stmt in doc.Statement
    stmt.Effect == "Deny"
    some v in stmt.Condition.Bool["aws:SecureTransport"]
    v == "false"
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
```

Two `denies_insecure_transport` definitions because IAM condition values may be a bare string or an array, and both are valid JSON policy. Rego's multiple-definition semantics handle that cleanly: either matches.

Teach the limitation explicitly. A student who believes tier 2 always runs will trust a gate that is only doing tier 1. The honest statement is: at plan time we prove a policy exists; that it says the right thing is verified after apply, by the `aws s3api get-bucket-policy` check in Lab 2.3's verification section, and continuously by Security Hub in Lab 5.2. Policy-as-code is one layer of three, and pretending otherwise is how people end up with confident dashboards and open buckets.

Step 6 AC-3, AU-9, and CM-6 for AWS

```
# policies/ac3_no_public_aws.rego
# METADATA
# title: AC-3 - Access Enforcement (AWS S3 public access block)
# description: "Every aws_s3_bucket must have an aws_s3_bucket_public_access_block with all four flags
true."
```

```
# custom:
#   control_id: AC-3
#   framework: nist-800-53-rev5
#   severity: critical
#   remediation: "Add aws_s3_bucket_public_access_block with all four flags set to true. Three is not
enough."
package compliance.ac3_aws

import rego.v1

deny contains msg if {
    some bucket in bucket_addresses
    not has_complete_pab(bucket)
    msg := sprintf(
        "[AC-3] %s: missing or incomplete aws_s3_bucket_public_access_block. All four flags must be
true.",
        [bucket],
    )
}

bucket_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket"
    addr := sprintf("aws_s3_bucket.%s", [r.name])
}

has_complete_pab(bucket_addr) if {
    pab := pab_for(bucket_addr)
    v := pab_planned_values(pab)
    v.block_public_acls == true
    v.block_public_policy == true
    v.ignore_public_acls == true
    v.restrict_public_buckets == true
}

pab_for(bucket_addr) := addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_public_access_block"
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
    addr := sprintf("aws_s3_bucket_public_access_block.%s", [r.name])
}

pab_planned_values(addr) := values if {
    some r in input.planned_values.root_module.resources
    r.address == addr
    values := r.values
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
```

```
# policies/au9_log_immutability_aws.rego
# METADATA
# title: AU-9 - Protection of Audit Information (S3 log bucket versioning)
# description: "A bucket named as an aws_s3_bucket_logging target must itself have versioning
enabled."
# custom:
#   control_id: AU-9
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add aws_s3_bucket_versioning for the log bucket with status = \"Enabled\"."
package compliance.au9_aws

import rego.v1

deny contains msg if {
    some target in log_target_addresses
    not versioned(target)
    msg := sprintf(
        "[AU-9] %s: used as an access-log target but has no aws_s3_bucket_versioning. Audit records
can be silently overwritten.",
        [target],
    )
}

# The target_bucket expression references the log bucket's address.
log_target_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_logging"
    some ref in r.expressions.target_bucket.references
    addr := trim_suffix(trim_suffix(ref, ".id"), ".bucket")
    startswith(addr, "aws_s3_bucket.")
}

versioned(bucket_addr) if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_versioning"
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
```

Run that one against a plan where only the primary bucket is versioned. It fires. That rule is the mechanical form of a mistake almost everybody makes once: an architecture that promises versioning on both buckets, and code that versions the one holding data. A policy suite catches it on the first pull request, which is the argument for policy-as-code stated more persuasively than any slide.

```
# policies/cm6_required_tags_aws.rego
# METADATA
# title: CM-6 - Configuration Settings (AWS required tags)
# description: "Every taggable resource must carry the four required tags."
```

```
# custom:
#   control_id: CM-6
#   framework: nist-800-53-rev5
#   severity: medium
#   remediation: "Add the four required tags, or set them once via provider default_tags."
package compliance.cm6_aws

import rego.v1

required := {"Project", "Environment", "ManagedBy", "ComplianceScope"}

taggable_type(t) if t == "aws_s3_bucket"
taggable_type(t) if t == "aws_kms_key"
taggable_type(t) if t == "aws_dynamodb_table"
taggable_type(t) if t == "aws_lambda_function"
taggable_type(t) if t == "aws_cloudtrail"

deny contains msg if {
    some resource in all_resources
    taggable_type(resource.type)
    missing := required - tag_keys(resource)
    count(missing) > 0
    msg := sprintf(
        "[CM-6] %s: missing required tags %v. Remediation: add them, or use provider default_tags.",
        [resource.address, sort_array(missing)],
    )
}

all_resources contains r if { some r in input.planned_values.root_module.resources }

all_resources contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
}

# With provider default_tags, the merged set lives in tags_all.
tag_keys(resource) := keys if {
    resource.values.tags_all
    keys := {k | resource.values.tags_all[k]}
}

tag_keys(resource) := keys if {
    not resource.values.tags_all
    resource.values.tags
    keys := {k | resource.values.tags[k]}
}

tag_keys(resource) := set() if {
    not resource.values.tags_all
    not resource.values.tags
}

sort_array(s) := sort([x | some x in s])
```

Note `aws_kms_key` in the taggable list. Lab 2.3 creates one, and an untagged key is a resource outside your boundary enumeration.

Step 7 Run the gate against the compliant plan

```
for ns in sc8_aws sc28_aws ac3_aws au9_aws cm6_aws; do
  echo "=== compliance.$ns ==="
  conftest test --policy policies --namespace "compliance.$ns" plan.json
done
```

Expected: five namespaces, zero failures. Lab 2.3 now has full AWS coverage from your library.

Step 8 Break it three ways

One broken plan per control class, because a gate you have only watched pass is a gate you are guessing about.

```
mkdir -p broken && cp ../lab-2-3/*.tf broken/
```

1. **SC-28**: change `sse_algorithm` to `"AES256"` and drop `kms_master_key_id`.
2. **SC-8**: delete `aws_s3_bucket_policy.primary`.
3. **AU-9**: delete `aws_s3_bucket_versioning.log`.

Then:

```
( cd broken && terraform init && terraform plan -out=tfplan && terraform show -json tfplan >
plan.json )

for ns in sc8_aws sc28_aws au9_aws; do
  conftest test --policy policies --namespace "compliance.$ns" broken/plan.json
done
```

```
FAIL - broken/plan.json - compliance.sc28_aws - [SC-28] aws_s3_bucket.primary: encrypted with SSE-S3
(AES256), not a customer-managed key. ...
FAIL - broken/plan.json - compliance.sc8_aws  - [SC-8] aws_s3_bucket.primary: no aws_s3_bucket_policy
references this bucket, ...
FAIL - broken/plan.json - compliance.au9_aws  - [AU-9] aws_s3_bucket.log: used as an access-log target
but has no aws_s3_bucket_versioning. ...
```

Non-zero exit. Each message names the resource, the control, and the fix.

Step 9 The wrapper script

```
#!/usr/bin/env bash
# scripts/policy-gate.sh
# Fail-closed Conftest gate over a Terraform plan.
#
# Usage:
```

```
#   policy-gate.sh --workspace <dir>   # runs `terraform show -json tfplan` there
#   policy-gate.sh --plan <plan.json>  # uses an existing plan JSON
set -euo pipefail

POLICY_DIR="policies"
WORKSPACE=""
PLAN=""
EVIDENCE_DIR="evidence/lab-3-4"

NAMESPACES=(
  compliance.sc8_aws
  compliance.sc28_aws
  compliance.ac3_aws
  compliance.au9_aws
  compliance.cm6_aws
)

while [[ $# -gt 0 ]]; do
  case "$1" in
    --workspace) WORKSPACE="$2"; shift 2 ;;
    --plan)      PLAN="$2";      shift 2 ;;
    --policy)    POLICY_DIR="$2"; shift 2 ;;
    --evidence)  EVIDENCE_DIR="$2"; shift 2 ;;
    *) echo "Unknown arg: $1" >&2; exit 2 ;;
  esac
done

if [[ -z "$PLAN" ]]; then
  [[ -z "$WORKSPACE" ]] && { echo "Usage: $0 --workspace <dir> | --plan <plan.json>" >&2; exit 2; }
  ( cd "$WORKSPACE" && terraform show -json tfplan > plan.json )
  PLAN="$WORKSPACE/plan.json"
fi

mkdir -p "$EVIDENCE_DIR"
RESULTS="$EVIDENCE_DIR/conftest-results.json"
EXIT=0

{
  echo "["
  FIRST=1
  for ns in "${NAMESPACES[@]}"; do
    [[ $FIRST -eq 1 ]] && FIRST=0 || printf ","
    OUT=$(conftest test --policy "$POLICY_DIR" --namespace "$ns" --output=json "$PLAN" || true)
    echo "${OUT:-[]}"
    if ! echo "${OUT:-[]}" | jq -e 'all(.[]; (.failures // []) | length == 0)' >/dev/null 2>&1; then
      EXIT=1
    fi
  done
  echo "]"
} > "$RESULTS"

# Fail closed if the gate produced nothing. An empty result set is not a
# pass: a rule that matches nothing and a rule that found nothing wrong are
# indistinguishable in the output. This is the check that makes the gate a
# gate rather than a decoration.
if ! jq -e 'flatten | length > 0' "$RESULTS" >/dev/null 2>&1; then
```

```
  echo "policy-gate: FAIL (no policy results produced; gate did not run)" >&2
  exit 2
fi

if [[ $EXIT -eq 0 ]]; then
  echo "policy-gate: PASS"
else
  echo "policy-gate: FAIL"
  jq -r 'flatten | .[] | (.failures // [])[] | .msg' "$RESULTS" >&2
fi
exit $EXIT
```

Three details worth naming:

- `jq` **instead of an inline** `python3` **heredoc.** Same job, one dependency you already have, and no indentation hazard inside YAML later.
- **The empty-results guard.** Without it the gate returns `PASS` when Conftest fails to load any policy at all, because zero failures out of zero tests is zero failures. That is the inert-policy failure mode from Step 3, caught by the harness.
- **Failure messages echoed to stderr.** A developer reading a red PR should not have to download an artifact to learn what broke.

Verification

A gate has three outcomes and they have to be told apart: pass, fail on real findings, and **fail closed** because it had nothing to evaluate. The third is the one worth building carefully, and it is also the one this lab used to test incorrectly.

```
bash scripts/verify.sh
```

Exit 2 is not proof of a working guard. The old test for fail-closed was

```
policy-gate.sh --policy /tmp/no-policies-here plan.json
```

which exits 2 because `plan.json` is not a recognized argument, the script taking `--plan`. It never loads a policy and never reads the plan. That is the same exit code the guard produces, so the guard could have been deleted outright and this test would still have passed.

It was checked: removing the empty-results guard makes an empty policy directory exit **0**, a gate reporting success while enforcing nothing. That is precisely the decoration this lab exists to prevent, and the test that was supposed to catch it could not.

So the script asserts the **message**, not just the status, and then makes the argument mistake on purpose to prove the two are distinguishable.

Expect the compliant fixture to pass, the degraded one to exit 1 citing SC-8, SC-28 and AU-9, and an empty policy directory to exit 2 saying the gate did not run.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 3.4.
#
# A gate has three outcomes and they must be told apart. Pass, fail on real
# findings, and fail closed because it had nothing to evaluate. The third is
# the one that gets faked: the guide used to test it with
#
#   policy-gate.sh --policy /tmp/no-policies-here plan.json
#
# which exits 2 because `plan.json` is not a recognized argument, before a
# single policy is loaded or the plan is read. Same exit code as the guard it
# claimed to be testing, so the guard could have been deleted entirely and
# that test would still have passed.
set -uo pipefail

WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0
pass() { printf '  PASS  %-10s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-10s %s\n' "$1" "$2" >&2; FAILED=1; }

EV=$(mktemp -d)
trap 'rm -rf "$EV"' EXIT
EMPTY=$(mktemp -d)

run() { # run POLICY_DIR PLAN -> sets RC and OUT
  OUT=$(bash scripts/policy-gate.sh --policy "$1" --plan "$2" --evidence "$EV" 2>&1)
  RC=$?
}

echo "=== a compliant plan passes ==="
run policies fixtures/plan-compliant.json
if [[ $RC -eq 0 && "$OUT" == *"policy-gate: PASS"* ]]; then
  pass "compliant" "exit 0 and PASS"
else
  fail "compliant" "expected exit 0 with PASS, got exit $RC"
fi

echo "=== a degraded plan fails, citing the right controls ==="
run policies fixtures/plan-degraded.json
if [[ $RC -eq 1 ]]; then
  pass "degraded" "exit 1"
else
  fail "degraded" "expected exit 1, got $RC"
fi

CONTROLS=$(jq -r '[.[][]?.failures[]?.msg] | .[] | capture("\\[(?<c>[A-Z]+-[0-9.]+)\\]").c' \
  "$EV/conftest-results.json" 2>/dev/null | sort -u | paste -sd, -)
if [[ "$CONTROLS" == "AU-9,SC-28,SC-8" ]]; then
  pass "degraded" "cites AU-9, SC-28, SC-8"
```

```
else
  fail "degraded" "expected AU-9,SC-28,SC-8; got '${CONTROLS:-(none)}'"
fi

echo "=== a gate with no policies fails CLOSED, for the stated reason ==="
run "$EMPTY" fixtures/plan-compliant.json
if [[ $RC -eq 2 ]]; then
  pass "fail-closed" "exit 2"
else
  fail "fail-closed" "expected exit 2, got $RC"
fi

# Exit 2 alone proves nothing here: an argument typo produces it too. The
# message is what distinguishes a gate that refused to run from a gate that
# was never invoked.
if [[ "$OUT" == *"gate did not run"* ]]; then
  pass "fail-closed" "reported that the gate did not run"
else
  fail "fail-closed" "exit 2 for the wrong reason. Output was: $(head -1 <<<"$OUT")"
fi

# And prove the distinction is real, by making the mistake on purpose.
BAD=$(bash scripts/policy-gate.sh --policy policies fixtures/plan-compliant.json 2>&1)
BADRC=$?
if [[ $BADRC -eq 2 && "$BAD" == *"Unknown arg"* ]]; then
  pass "fail-closed" "a bad argument also exits 2, which is why the message is checked"
else
  fail "fail-closed" "expected a positional plan to be rejected with exit 2"
fi

rmdir "$EMPTY" 2>/dev/null
echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: the gate passes, fails, and fails closed, each for its own reason."
else
  echo "NOT VERIFIED: the gate does not behave as described. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-3-4"
mkdir -p "$EVIDENCE"

# Both runs, kept as separate artifacts. The gate always writes
# conftest-results.json, so each run is renamed before the next overwrites it.
bash scripts/policy-gate.sh --plan fixtures/plan-compliant.json --evidence "$EVIDENCE"
mv "$EVIDENCE/conftest-results.json" "$EVIDENCE/conftest-pass.json"

# The degraded run exits 1 by design, so it needs `|| true` or the mv is
# skipped and you lose the more interesting of the two files.
bash scripts/policy-gate.sh --plan fixtures/plan-degraded.json --evidence "$EVIDENCE" || true
```

```
mv "$EVIDENCE/conftest-results.json" "$EVIDENCE/conftest-fail.json"

# The three outcomes, each proven for its own reason.
bash scripts/verify.sh 2>&1 | tee "$EVIDENCE/inert-gate-test.txt"
```

Portfolio submission checklist

- [ ] `policies/` holds GCP and AWS variants: ten files, six control IDs.
- [ ] `scripts/policy-gate.sh` committed and executable.
- [ ] `evidence/lab-3-4/conftest-pass.json`
- [ ] `evidence/lab-3-4/conftest-fail.json`
- [ ] `evidence/lab-3-4/inert-gate-test.txt`, showing the gate exits non-zero with no policies present.
- [ ] `policies/README.md` noting which file targets which cloud, and the SC-8 plan-time limitation.

Troubleshooting

- **`policies: no such file or directory`.** `--policy` resolves relative to your shell. Pass an absolute or canonically-relative path.
- **`no policies matched`.** The `package` declaration must equal the `--namespace` string exactly.
- **A passing fixture fires anyway.** Module-wrapped resources live under `child_modules[]`.
- **Bucket name comparisons return undefined.** Plan-time IDs are unknown. Match by reference in `configuration...expressions.<arg>.references`.
- **SC-8 tier 2 never fires.** Expected. The rendered policy JSON is unknown when bucket ARNs are computed. Tier 1 still runs. Verify the semantics post-apply.
- **`jq: error: Cannot iterate over null`** in the gate. Conftest emits `null` rather than `[]` for a namespace with no results. The `// []` fallbacks handle it; keep them if you refactor.

Cleanup

Local. Nothing in the cloud beyond Lab 2.3.

These policies already detect capstone gaps. The starter's `GAPS.md` lists eight, and three of the AWS variants you just wrote catch one each, against `aws_s3_bucket.uploads` in the starter's own Terraform:

Policy	Catches
`sc28_encryption_aws`	GAP-01, SSE-S3 rather than a customer CMK
`sc8_tls_required_aws`	GAP-03, no `aws:SecureTransport` deny
`au9_log_immutability_aws`	GAP-04, no versioning

The capstone asks for **at least five** policies detecting the most material gaps, and requires that the grader can re-introduce a gap and watch your suite fail closed. Three of the five already exist. Point them at the starter's plan and see what happens before you write more.

How this feeds the capstone

`scripts/policy-gate.sh` is the exact script CI calls in Lab 4.3. The capstone's workflow shells out with `--workspace ./terraform`, and the plan is checked before apply.

Two things to carry forward. The empty-results guard is what makes the capstone's "we re-introduce a gap and confirm the gate fires" check survivable, because a gate that silently loads no policies passes that test in the worst possible way. And the SC-8 two-tier pattern is the honest shape for any control whose enforcement lives in a computed string: assert what you can at plan time, verify the rest after apply, and write down which is which.

Revision history

These notes

- Two AWS variants added: SC-8 (a bucket policy exists and denies non-TLS) and AU-9 (the access-log target is versioned).
- SC-28's AWS variant now distinguishes missing encryption from weak encryption, and requires a customer-managed key rather than any configuration.
- `policy-gate.sh` gained an empty-results guard, so a gate that loads no policies exits non-zero instead of reporting success.
- Gate failure messages are echoed to stderr rather than only written to the evidence artifact.

The official labs

Initial release: three AWS variants (SC-28, AC-3, CM-6), gate could report `PASS` having evaluated nothing.

CGE-P LAB NOTES

Lab 4.3: Building a GRC Evidence Pipeline (AWS + GitHub Actions)

The local Conftest gate from Lab 3.4 catches violations on your laptop. CI catches them across the whole team. This lab wires the gate into GitHub Actions, runs it on every pull request, and uploads a named evidence artifact for every run.

The YAML file you commit **is** your CM-3, CM-6, CA-2, CA-7, RA-5, SR-3, and AU-9 evidence.

For reading. Code lines wrap to fit the page; copy code from docs/04_03_grc_evidence_pipeline.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/04_03_grc_evidence_pipeline.md` in your clone, not from the PDF.

Learning objectives

- Wire AWS OIDC trust so the workflow assumes a role with no long-lived keys.
- Scope that role to exactly what it needs, including the Lab 2.2 state backend.
- Run plan, Conftest, and a vulnerability scan on every PR, failing closed.
- Pin the supply chain: actions by commit SHA, tools by version and checksum.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 2.2** state backend. The pipeline needs it; see "Why the backend is a prerequisite" below.
- Lab 2.3, Lab 3.3, Lab 3.4 artifacts committed into the repo.
- A GitHub repository you own.
- AWS permissions to create an IAM OIDC provider and role.

Estimated time & cost

- 90 minutes.
- Free. GitHub Actions' free tier covers this, and the workflow only plans.

Why the backend is a prerequisite

Run `terraform init` on a fresh runner with a local backend and the runner has no state, so `plan` reports every resource as "to be created."

For a plan-only workflow that is merely misleading. For the capstone, which requires `Apply` on merge to `main`, it is fatal: the first apply collides with your existing resources or silently builds a second copy of everything.

Lab 2.2 exists to prevent this. If you skipped it, stop and do it now, because every diff this pipeline produces is fiction until you have.

Architecture

```
  PR opened
      |
      v
  workflow run
      |-- Configure AWS creds (OIDC, no keys on disk)          IA-5
      |-- terraform init  (reads Lab 2.2 remote state)
      |-- terraform fmt -check                                 CM-6
      |-- terraform plan  (a REAL diff, because state exists)
      |-- Conftest gate via scripts/policy-gate.sh             CA-2, fail closed
      |-- Trivy config scan                                    RA-5, fail closed
      |-- Upload evidence artifact                             AU-9
      +-- Comment on PR with the summary
```

Lab 4.4 adds Cosign signing and uploads the bundle to the Lab 2.5 vault.

Step-by-step walkthrough

Step 1 OIDC trust, scoped properly

```
# oidc/main.tf
terraform {
  required_version = ">= 1.10"

  backend "s3" {
    bucket       = "cgep-lab-tfstate-XXXXXXXX"
    key          = "labs/4-3-oidc/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }

  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region = "us-east-1"
  default_tags {
    tags = {
      Project         = "cgep-lab"
      Environment     = "shared"
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
```

```
}

variable "github_org"    { type = string }
variable "github_repo"   { type = string }
variable "state_bucket"  { type = string }
variable "state_kms_arn" { type = string }

data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

# AWS now maintains the trust chain for this provider internally, so the
# thumbprint is no longer load-bearing. It remains a required argument.
resource "aws_iam_openid_connect_provider" "github" {
  url             = "https://token.actions.githubusercontent.com"
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

resource "aws_iam_role" "grc_gate" {
  name = "cgep-grc-gate"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Federated = aws_iam_openid_connect_provider.github.arn }
      Action    = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          "token.actions.githubusercontent.com:sub" = "repo:${var.github_org}/${var.github_repo}:*"
        }
      }
    }]
  })
}

# AC-6: read-only for the plan itself.
resource "aws_iam_role_policy_attachment" "readonly" {
  role       = aws_iam_role.grc_gate.name
  policy_arn = "arn:${data.aws_partition.current.partition}:iam::aws:policy/ReadOnlyAccess"
}

# The state backend needs more than read. `terraform plan` writes a lock
# object and refreshes state. Without this the job fails at init.
resource "aws_iam_role_policy" "state_access" {
  name = "tfstate-access"
  role = aws_iam_role.grc_gate.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect   = "Allow"
```

```
        Action   = ["s3:ListBucket", "s3:GetBucketLocation"]
        Resource = "arn:${data.aws_partition.current.partition}:s3:::${var.state_bucket}"
      },
      {
        Effect = "Allow"
        Action = ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"]
        Resource = "arn:${data.aws_partition.current.partition}:s3:::${var.state_bucket}/*"
      },
      {
        Effect   = "Allow"
        Action   = ["kms:Decrypt", "kms:GenerateDataKey"]
        Resource = var.state_kms_arn
      },
    ]
  })
}

output "role_arn" { value = aws_iam_role.grc_gate.arn }
```

```
cd oidc
terraform init
terraform apply
```

The `StringLike` on `sub` binds this role to one repository. **Do not loosen it.** A role trusted by `repo:*:*` is trusted by every public repository on GitHub, which is not a subtle failure. If you want it tighter still, bind to a specific ref or environment:

```
"repo:YourOrg/YourRepo:ref:refs/heads/main"
"repo:YourOrg/YourRepo:environment:production"
```

The environment form is what you want for the capstone's apply job, because it lets a GitHub environment protection rule gate the credential itself.

If the OIDC provider already exists in the account:

```
terraform import aws_iam_openid_connect_provider.github \
  arn:aws:iam::ACCOUNT:oidc-provider/token.actions.githubusercontent.com
```

Step 2 Repository variables

`cgep.env` holds these values on your machine, and CI cannot read that file. It needs its own copy. Derive them from the environment rather than retyping, so the two cannot drift apart:

```
source ../../cgep.env
gh variable set AWS_ROLE_ARN     --body "arn:aws:iam::$(aws sts get-caller-identity --query Account --
output text):role/cgep-grc-gate"
gh variable set TF_STATE_BUCKET  --body "$TF_VAR_state_bucket"
```

Run that from inside your repository and `gh` targets it automatically; add `--repo OWNER/NAME` only if you are somewhere else.

> **Variables, not secrets, and the distinction is the lesson.** A role ARN and a bucket name are configuration. They identify things; they do not grant access to them. GitHub **secrets** are write-only and masked in logs, which is right for a credential and actively unhelpful for a value you want to read in a failed run. Putting non-secrets in `secrets` is a common habit and it quietly trains people that everything is equally sensitive, which is how real credentials end up treated casually.
>
> Notice what is **not** here: no access key, no secret key, no long-lived credential of any kind. That is the point of the OIDC trust you just built. The role ARN being public costs you nothing, because assuming it requires a token that only your repository, on your ref, can obtain. If this lab had you paste an access key into `secrets`, it would be teaching the opposite of what it claims to teach.

Step 3 The workflow

```
# .github/workflows/grc-gate.yml
name: grc-gate

on:
  pull_request:
    branches: [main]
  workflow_dispatch:

# AC-6: the minimum token scope that works.
permissions:
  id-token: write      # AWS OIDC + Cosign keyless (Lab 4.4)
  contents: read
  pull-requests: write # PR comment

env:
  AWS_REGION: us-east-1
  TF_WORKING_DIR: terraform/primitives/compliant-s3
  TERRAFORM_VERSION: 1.15.8
  CONFTEST_VERSION: 0.69.0
  OPA_VERSION: 1.19.1
  TRIVY_VERSION: 0.74.0

jobs:
  grc-gate:
    runs-on: ubuntu-latest
    steps:
      # SR-3 / SR-11: actions pinned by commit SHA, not by tag.
      # A tag is a mutable pointer. Pinning to v4 means "whatever the
      # maintainer, or whoever compromises their account, decides v4 means."
      - name: Checkout
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1

      - name: Configure AWS credentials (OIDC)
```

```
        uses: aws-actions/configure-aws-credentials@e6de054238d6b7531b4efff3b6587d9aade6a06c # v6.2.3
        with:
          role-to-assume: ${{ vars.AWS_ROLE_ARN }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@dfe3c3f87815947d99a8997f908cb6525fc44e9e # v4.0.1
        with:
          terraform_version: ${{ env.TERRAFORM_VERSION }}
          terraform_wrapper: false

      - name: Install Conftest and OPA
        run: |
          set -euo pipefail
          curl -fsSL "https://github.com/open-policy-agent/conftest/releases/download/v$
{CONFTEST_VERSION}/conftest_${CONFTEST_VERSION}_Linux_x86_64.tar.gz" \
            | tar -xz -C /usr/local/bin conftest
          curl -fsSL "https://github.com/open-policy-agent/opa/releases/download/v${OPA_VERSION}/
opa_linux_amd64_static" \
            -o /usr/local/bin/opa
          chmod +x /usr/local/bin/opa
          conftest --version && opa version

      - name: Install Trivy
        run: |
          set -euo pipefail
          curl -fsSL "https://github.com/aquasecurity/trivy/releases/download/v${TRIVY_VERSION}/
trivy_${TRIVY_VERSION}_Linux-64bit.tar.gz" \
            | tar -xz -C /usr/local/bin trivy
          trivy --version

      # Policy unit tests run before the policies are trusted to gate anything.
      - name: opa test (the gate's own tests)
        run: opa test -v policies/

      - name: Mutation-test the policy suite
        run: bash scripts/mutation-test.sh

      - name: terraform fmt
        run: terraform fmt -check -recursive terraform/

      - name: Terraform plan
        working-directory: ${{ env.TF_WORKING_DIR }}
        run: |
          set -euo pipefail
          terraform init -input=false
          terraform validate
          terraform plan -out=tfplan -no-color | tee plan.txt
          terraform show -json tfplan > plan.json

      - name: Conftest policy gate
        id: conftest
        run: |
          mkdir -p evidence
          bash scripts/policy-gate.sh \
            --workspace "${TF_WORKING_DIR}" \
```

```
            --policy policies \
            --evidence evidence

      - name: Trivy config scan
        id: trivy
        if: always()
        run: |
          set -euo pipefail
          trivy config "${TF_WORKING_DIR}" \
            --format sarif --output evidence/trivy.sarif \
            --severity HIGH,CRITICAL --exit-code 0
          COUNT=$(jq '[.runs[].results[]] | length' evidence/trivy.sarif)
          echo "trivy HIGH+CRITICAL: $COUNT"
          jq -r '.runs[].results[] | "  " + .ruleId + ": " + .message.text' evidence/trivy.sarif ||
true
          test "$COUNT" -eq 0

      - name: Copy plan into evidence
        if: always()
        run: |
          cp "${TF_WORKING_DIR}/plan.json" evidence/plan.json || true
          cp "${TF_WORKING_DIR}/plan.txt"  evidence/plan.txt  || true

      - name: Upload evidence artifact
        if: always()
        uses: actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a # v7.0.1
        with:
          name: grc-evidence-${{ github.run_id }}
          path: evidence/
          retention-days: 365
```

Choices worth understanding:

- `permissions: id-token: write` is required for OIDC. Without it the credential step fails in a way that looks like an AWS problem and is not.
- `if: always()` on the scan, evidence copy, and upload. Without it a Conftest failure aborts the job before evidence is captured, and the entire point of CI evidence is that it survives the failure.
- **Actions pinned by SHA.** "Pin your actions" is common advice usually illustrated with mutable tags, which misses the point. `@v4` is a pointer the upstream maintainer can move, and has moved, including during at least one widely-reported supply chain incident. The comment after the SHA keeps it readable.

A pin is a maintenance obligation, not a one-time decision. This is the half of the story most guidance leaves out, and it is worth internalizing before you adopt the practice.

A tag floats, so it silently picks up fixes and silently picks up compromises. A SHA is frozen, so it does neither. What it also does is stop receiving the upstream's runtime migrations, and eventually you are running something the platform is deprecating under you.

This curriculum's own CI demonstrated it. Every job passed, and every job carried:

```
Node.js 20 is deprecated. The following actions target Node.js 20 but are
being forced to run on Node.js 24: actions/checkout@11bd719...
```

Nothing was broken. The pins were simply old enough that GitHub was force-migrating them off their target runtime, and the only signal was a warning in a green run that nobody reads.

So pin, and pair it with a refresh process: Dependabot or Renovate raising a PR per bump, a quarterly review, or at minimum a CI job that fails when a pinned action is more than N releases behind. **A pin with no refresh process converts a supply chain risk into an obsolescence risk.** You have not removed the problem, you have changed which problem you have, and the second one is quieter.

Trading one risk for a quieter one is still usually the right trade. Just make it knowingly, and write down who checks.

- `retention-days: 365`, not the 90-day default. Lab 4.4 mirrors the bundle into the vault, but the artifact should outlive the audit cycle regardless.
- **Policy tests run before the gate.** A suite that fails its own unit tests should not be trusted to decide whether your infrastructure ships.

Step 4 Trivy, not tfsec

Much older guidance reaches for `tfsec`. It has been folded into Trivy by its maintainer, Aqua Security, and the misconfiguration checks now live there. `trivy config` runs the same rule set, is actively maintained, and covers Terraform, CloudFormation, Kubernetes, and Dockerfiles with one binary.

Point it at your own Lab 2.3 workspace before you point it at the starter. Trivy flags missing HTTPS enforcement on a bucket policy and SSE-S3 instead of a customer-managed key, both at HIGH. Lab 2.3 closes both, so the gate stays quiet. Take either shortcut and this gate, configured to fail closed on HIGH, will fail your own Chapter 2 artifact.

That agreement between your baseline and your gate is not automatic. It is the check to run whenever you change the baseline.

Step 5 Open a PR and watch it run

```
git checkout -b add-grc-gate
git add .github/workflows/grc-gate.yml policies/ scripts/ oidc/ terraform/
git commit -m "Add GRC evidence pipeline"
git push -u origin add-grc-gate
gh pr create --title "Add GRC evidence pipeline" --body "Reference pipeline."
gh run watch
```

Against the compliant Lab 2.3 workspace:

```
policy-gate: PASS
trivy HIGH+CRITICAL: 0
```

Against the `cgep-app-starter`, which ships eight intentional gaps, both gates fire. That is the expected outcome and the starting point for the capstone.

Step 6 The two-PR demonstration

The capstone wants a green PR and a red PR in your history.

1. **Red**: branch that introduces a violation. Delete `aws_s3_bucket_policy.primary` (SC-8), or set `block_public_acls = false` (AC-3), or drop `aws_s3_bucket_versioning.log` (AU-9). Open the PR. The gate fails, the merge is blocked.
2. **Green**: fix it. The gate passes, the PR merges.

Both runs leave evidence artifacts. Both URLs go in your write-up.

Turn on the branch protection that makes this mean something:

```
gh api -X PUT "repos/YourOrg/YourRepo/branches/main/protection" \
  --input - <<'EOF'
{
  "required_status_checks": { "strict": true, "contexts": ["grc-gate"] },
  "enforce_admins": true,
  "required_pull_request_reviews": { "required_approving_review_count": 1 },
  "restrictions": null
}
EOF
```

Without branch protection the gate is advisory. A red check that anyone can merge past is not CM-3; it is a suggestion. `enforce_admins: true` is what makes it a control rather than a habit, and it is the first thing an assessor will check.

Step 7 The inert-gate test

Before you trust this, prove it can fail for the right reason:

```
git checkout -b test-inert-gate
git rm -r policies/
git commit -m "TEST: remove policies (expect gate failure, do not merge)"
git push -u origin test-inert-gate
gh pr create --title "TEST: inert gate" --body "Expect failure."
```

The run must go **red**, with `policy-gate: FAIL (no policy results produced; gate did not run)`. If it goes green, your gate reports success when it is evaluating nothing, which is the worst possible failure mode: a confident dashboard over an unexamined system.

Close the PR without merging and keep the run URL. It is the best single piece of evidence in your portfolio that the gate is real.

Takeaway: the YAML is the control statement

Workflow content	Control
`on: pull_request` + branch protection requiring this check	CM-3 (configuration change control)
`terraform fmt -check` and the Conftest tag rule	CM-6 (configuration settings)
The workflow as a recurring assessment	CA-2, CA-7
Trivy scanning every change	RA-5
Actions pinned by SHA, tools by version	SR-3, SR-11 (supply chain)
OIDC instead of stored access keys	IA-5, AC-6
Evidence artifacts retained 365 days, signed in Lab 4.4	AU-9

The workflow file is checked in, the history is preserved, and an assessor traversing your OSCAL component in Lab 6.1 follows an evidence URI that lands on this workflow's output.

Verification

- A PR triggers the workflow and it appears in the Actions tab.
- `grc-evidence-<run-id>` is attached with `plan.json`, `plan.txt`, `conftest-results.json`, `trivy.sarif`.
- Compliant code: green. Non-compliant: red, with control IDs in the log.
- Removing `policies/` makes it red, not green.
- Branch protection blocks merge on red, including for admins.

Portfolio submission checklist

- [ ] `oidc/` module committed, with the state-access policy.
- [ ] `.github/workflows/grc-gate.yml` committed, every action SHA-pinned.
- [ ] `vars.AWS_ROLE_ARN` set.
- [ ] One green PR and one red PR in history.
- [ ] The inert-gate test run URL, closed unmerged.
- [ ] Branch protection enabled with `enforce_admins: true`, screenshot or `gh api` output in `evidence/lab-4-3/`.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`Could not assume role with OIDC: invalid identity token`.** The `sub` condition does not match. PR runs use `repo:OWNER/REPO:pull_request`; branch pushes use `repo:OWNER/REPO:ref:refs/heads/BRANCH`. `StringLike` with `repo:OWNER/REPO:*` covers both.
- **`AccessDenied` on `s3:PutObject` during `terraform init`.** The role needs write to the state bucket and `kms:GenerateDataKey` on the state key. `ReadOnlyAccess` alone is not enough; that is what `aws_iam_role_policy.state_access` is for.
- **`Error acquiring the state lock` on concurrent PRs.** Two runs against one state `key`. Serialize with a concurrency group, or give PR runs a separate key.
- **Conftest finds no policies.** `--policy` resolves relative to the step's working directory. The gate script takes it as an argument for exactly this reason.
- **Trivy findings you disagree with.** Add `.trivyignore` with a dated justification per rule ID. Do not scatter `--skip` flags through the workflow; an exclusion nobody can find is an exclusion nobody reviews.
- **The plan shows every resource as new.** Your backend block is missing or points at the wrong key. See the prerequisite note at the top.

Cleanup

Delete test branches. The workflow file stays; it is the deliverable. The IAM role and OIDC provider stay; they are free.

How this feeds the capstone

This is the capstone's pipeline. Lab 4.4 adds Cosign signing and the vault upload; the capstone adds the apply step.

For that apply step, use a separate job gated on `github.ref == 'refs/heads/main'`, a GitHub environment with a protection rule, and a **second** IAM role whose trust policy binds to `repo:OWNER/REPO:environment:production`. The plan role stays read-only. Splitting plan credentials from apply credentials is the single highest-value thing you can say in your write-up about this layer, and it is only possible because Lab 2.2 gave you somewhere for the two jobs to share state.

Revision history

These notes

- Requires the Lab 2.2 remote state backend. Without it a fresh runner plans against empty state, which makes the capstone's apply-on-merge step uncompletable.
- The OIDC role gained an explicit state-access policy alongside `ReadOnlyAccess`, because a plan writes a lock object and refreshes state.
- `tfsec` replaced by `trivy config`, its maintained successor.
- Actions pinned by commit SHA rather than by tag, with a note that a pin is a maintenance obligation.
- Added policy unit tests and the mutation test as pipeline steps, ahead of the gate that depends on them.
- Added the inert-gate test: removing `policies/` must turn the run red.
- Branch protection with `enforce_admins: true` documented as the thing that makes the gate a control rather than a suggestion.
- Evidence artifact retention raised from 90 to 365 days.

The official labs

Initial release: local state, `tfsec`, actions pinned by tag, no mutation test and no inert-gate test.

CGE-P LAB NOTES

Lab 4.4: Evidence Management & Chain of Custody (AWS)

You have a pipeline. You have an immutable vault. This lab connects them with cryptographic signing, so the evidence is provably yours, provably untampered, and provably timestamped. The auditor does not need to trust you. They verify.

Two things about verification scripts are worth knowing before you write one, and both are the kind that pass every happy-path test. Steps 3 and 4 cover them.

For reading. Code lines wrap to fit the page; copy code from docs/04_04_evidence_chain_of_custody.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/04_04_evidence_chain_of_custody.md` in your clone, not from the PDF.

Learning objectives

- Define chain of custody as four properties: **authenticity, integrity, timeliness, completeness**, and implement all four rather than three.
- Extend the Lab 4.3 pipeline with keyless Cosign signing via GitHub OIDC.
- Verify a bundle end-to-end, including a certificate identity check that actually constrains who signed it.
- Break the chain deliberately, four ways, and watch each break get caught.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- Lab 2.5 vault deployed. You have its bucket name.
- Lab 4.3 pipeline working on every PR.
- Cosign `>= 3.0` locally (`cosign version`).
- `jq` and `date` (GNU coreutils; macOS users see the Troubleshooting note).

Estimated time & cost

- 75 minutes.
- Free. Sigstore is free; marginal S3 cost.

Concept: why signing matters

Lab 2.5 made the bundle immutable. Object Lock prevents deletion, but it does not prove **who** created the bundle or **when**.

A determined insider with admin in your AWS account can stand up a different bucket, drop a tampered bundle in it, and point a sloppy auditor at that. Cosign closes the loop. The signature ties the bundle to a specific GitHub Actions run, on a specific repository, at a specific moment. The certificate Sigstore's Fulcio CA issues carries the OIDC subject, and Rekor timestamps it in a public transparency log.

None of that is bypassable by someone with admin in your AWS account, because none of it lives in your AWS account. That is the property you are buying.

Four ways the chain breaks. Close all four or you have a story, not a chain:

Property	Broken by	Closed by
Integrity	Byte-level tampering	SHA-256 recompute
Authenticity	Anyone can fabricate a bundle	Cosign signature + **strict cert identity**
Timeliness	Backdated evidence	Rekor transparency log entry
Completeness	Quietly dropping an inconvenient file	Manifest with a file list, checked

Most pipelines implement the first two and stop.

Architecture

```
PR opens
   |
   v
workflow run --+--> plan / policy / scan           (Lab 4.3)
               |
               +--> tar czf evidence bundle
               |
               +--> cosign sign-blob --bundle      (keyless, GitHub OIDC)
               |         |
               |         +--> Fulcio issues a short-lived cert
               |         |     carrying repo + workflow + ref as the subject
               |         +--> Rekor logs the signature with a timestamp
               |
               +--> aws s3 cp bundle + .sha256 + .sig.bundle + receipt.json
                         to s3://VAULT/runs/<run_id>/   (Object Lock applies)
               |
               v
auditor: scripts/verify-evidence.sh <run_id>
         |-- recompute SHA-256               integrity
         |-- cosign verify-blob, strict id   authenticity + timeliness
         |-- manifest file list matches      completeness
         +-- get-object-retention in future  preservation
         "CHAIN INTACT"
```

Step-by-step walkthrough

Step 1 Add Cosign to the workflow

```
      - name: Install Cosign
        uses: sigstore/cosign-installer@6f9f17788090df1f26f669e9d70d6ae9567deba6 # v4.1.2
        with:
          cosign-release: 'v3.1.3'
```

Then, after the evidence copy step:

```
      - name: Bundle, sign, and upload to vault
        id: sign
        if: always()
        env:
          VAULT:  ${{ vars.EVIDENCE_VAULT }}
          RUN_ID: ${{ github.run_id }}
          SHA:    ${{ github.sha }}
        run: |
          set -euo pipefail
          BUNDLE="evidence-${RUN_ID}-${SHA}.tar.gz"

          # Completeness: record what SHOULD be in the bundle before we build it.
          ( cd evidence && ls -1 > .manifest.txt && sha256sum * > .sha256sums.txt )

          ( cd evidence && tar czf "../${BUNDLE}" . )
          sha256sum "${BUNDLE}" | awk '{print $1}' > "${BUNDLE}.sha256"

          cosign sign-blob --yes --bundle "${BUNDLE}.sig.bundle" "${BUNDLE}"

          KEY_PREFIX="runs/${RUN_ID}"
          for f in "${BUNDLE}" "${BUNDLE}.sha256" "${BUNDLE}.sig.bundle"; do
            aws s3 cp "$f" "s3://${VAULT}/${KEY_PREFIX}/$f"
          done

          VERSION_ID=$(aws s3api head-object --bucket "${VAULT}" \
            --key "${KEY_PREFIX}/${BUNDLE}" --query VersionId --output text)

          jq -n \
            --arg run_id     "${RUN_ID}" \
            --arg vault      "${VAULT}" \
            --arg bundle_key "${KEY_PREFIX}/${BUNDLE}" \
            --arg version_id "${VERSION_ID}" \
            --arg sha256     "$(cat "${BUNDLE}.sha256")" \
            --arg commit     "${SHA}" \
            --arg repo       "${GITHUB_REPOSITORY}" \
            --arg workflow   "${GITHUB_WORKFLOW_REF}" \
            '$ARGS.named' > receipt.json

          aws s3 cp receipt.json "s3://${VAULT}/${KEY_PREFIX}/receipt.json"
```

```
      # The pass/fail decision moves to the LAST step, so evidence is signed
      # and stored even when the gate failed. A failed run is exactly the run
      # whose evidence you will want later.
      - name: Enforce gate result
        if: always()
        run: |
          test "${{ steps.conftest.outcome }}" = "success" || exit 1
          test "${{ steps.trivy.outcome }}"    = "success" || exit 1
```

Two things to notice. `jq -n '$ARGS.named'` builds the receipt instead of a bash heredoc, which means values containing quotes cannot corrupt the JSON. And the receipt now records `repo` and `workflow`, because Step 4's strict verification needs to know what identity to expect.

Step 2 Grant the role write access to the vault

The Lab 4.3 role has `ReadOnlyAccess` plus state access. Add a narrow vault write:

```
export AWS_PROFILE=cgep
VAULT=YOUR_VAULT_BUCKET
VAULT_KMS=YOUR_VAULT_CMK_ARN

aws iam put-role-policy --role-name cgep-grc-gate --policy-name vault-write \
  --policy-document "$(jq -n --arg v "$VAULT" --arg k "$VAULT_KMS" '{
    Version: "2012-10-17",
    Statement: [
      { Effect: "Allow",
        Action: ["s3:PutObject","s3:GetObject","s3:GetBucketLocation","s3:ListBucket"],
        Resource: ["arn:aws:s3:::\($v)","arn:aws:s3:::\($v)/*"] },
      { Effect: "Allow",
        Action: ["kms:GenerateDataKey","kms:Decrypt"],
        Resource: $k }
    ]}')"

gh variable set EVIDENCE_VAULT --body "$VAULT" --repo OWNER/REPO
```

Note the KMS statement. The Lab 2.5 vault is SSE-KMS with a customer-managed key, so `s3:PutObject` alone returns `AccessDenied` with a message about KMS that does not obviously say "add a KMS permission." An AES256 vault would not need it, which is why this is easy to miss.

No `s3:DeleteObject`. The pipeline writes evidence and must never be able to remove it. Object Lock would refuse anyway; not granting the permission means the attempt never reaches the bucket, and defense in depth is cheap here.

Step 3 The verify script

```
#!/usr/bin/env bash
# scripts/verify-evidence.sh <run_id> [--vault B] [--profile P] [--identity REGEX]
set -euo pipefail

RUN_ID="${1:?usage: verify-evidence.sh <run_id> [--vault B] [--profile P] [--identity REGEX]}"
shift || true

VAULT="${EVIDENCE_VAULT:-}"
PROFILE_ARG=""
IDENTITY="${COSIGN_IDENTITY:-}"

while [[ $# -gt 0 ]]; do
  case "$1" in
    --vault)    VAULT="$2";   shift 2 ;;
    --profile)  PROFILE_ARG="--profile $2"; shift 2 ;;
    --identity) IDENTITY="$2"; shift 2 ;;
    *) echo "Unknown arg: $1" >&2; exit 2 ;;
  esac
done

[[ -z "$VAULT" ]] && { echo "Set --vault or EVIDENCE_VAULT" >&2; exit 2; }

if command -v sha256sum >/dev/null 2>&1; then SHA256="sha256sum"
else SHA256="shasum -a 256"; fi

WORK=$(mktemp -d); trap 'rm -rf "$WORK"' EXIT; cd "$WORK"
PREFIX="runs/${RUN_ID}"

aws $PROFILE_ARG s3 cp "s3://${VAULT}/${PREFIX}/" . --recursive \
  --exclude "*" --include "evidence-*.tar.gz*" --include "receipt.json"

BUNDLE=$(ls evidence-*.tar.gz 2>/dev/null | head -1)
[[ -z "$BUNDLE" ]] && { echo "FAIL: no bundle found at ${PREFIX}" >&2; exit 1; }

echo "=== 1. Integrity (SHA-256) ==="
EXPECTED=$(cat "${BUNDLE}.sha256")
ACTUAL=$($SHA256 "${BUNDLE}" | awk '{print $1}')
[[ "$EXPECTED" == "$ACTUAL" ]] || { echo "FAIL: SHA mismatch"; exit 1; }
echo "  OK (${ACTUAL})"

echo "=== 2. Authenticity + timeliness (Cosign / Sigstore Rekor) ==="
# Derive the expected signer identity from the receipt unless overridden.
if [[ -z "$IDENTITY" && -f receipt.json ]]; then
  REPO=$(jq -r '.repo // empty' receipt.json)
  [[ -n "$REPO" ]] && IDENTITY="^https://github.com/${REPO}/\\.github/workflows/.*@refs/.*$"
fi
[[ -z "$IDENTITY" ]] && { echo "FAIL: no signer identity to check against." >&2; exit 1; }

cosign verify-blob \
  --bundle "${BUNDLE}.sig.bundle" \
  --certificate-identity-regexp "$IDENTITY" \
  --certificate-oidc-issuer 'https://token.actions.githubusercontent.com' \
  "${BUNDLE}"
```

```
echo "  OK (signed by an identity matching ${IDENTITY})"

echo "=== 3. Completeness (manifest) ==="
mkdir -p extracted && tar xzf "${BUNDLE}" -C extracted
if [[ -f extracted/.manifest.txt ]]; then
  MISSING=0
  while IFS= read -r f; do
    [[ -e "extracted/$f" ]] || { echo "  MISSING: $f"; MISSING=1; }
  done < extracted/.manifest.txt
  [[ $MISSING -eq 0 ]] || { echo "FAIL: bundle is missing files listed in its manifest"; exit 1; }
  ( cd extracted && $SHA256 -c .sha256sums.txt --quiet ) \
    || { echo "FAIL: a bundled file does not match its recorded hash"; exit 1; }
  echo "  OK ($(wc -l < extracted/.manifest.txt) files present and matching)"
else
  echo "FAIL: no manifest in bundle; completeness cannot be established"; exit 1
fi

echo "=== 4. Preservation (Object Lock retention) ==="
RETAIN_UNTIL=$(aws $PROFILE_ARG s3api get-object-retention \
  --bucket "${VAULT}" --key "${PREFIX}/${BUNDLE}" \
  --query 'Retention.RetainUntilDate' --output text)
# Compare as epoch seconds: correct by construction rather than by
# coincidence. See "the second finding" below for why the string test
# happens to work and why that is not a reason to keep it.
RETAIN_EPOCH=$(date -d "$RETAIN_UNTIL" +%s 2>/dev/null || date -jf "%Y-%m-%dT%H:%M:%S" "$
{RETAIN_UNTIL%.*}" +%s)
NOW_EPOCH=$(date -u +%s)
[[ "$RETAIN_EPOCH" -gt "$NOW_EPOCH" ]] || { echo "FAIL: retention expired"; exit 1; }
echo "  OK (retain until ${RETAIN_UNTIL})"

echo
echo "CHAIN INTACT for run ${RUN_ID}"
```

Step 4 The two defects this fixes

Worth stopping on, because both are the sort that never show up in a passing test.

The first: identity verification that verifies nothing. `--certificate-identity-regexp '.*'` is the path of least resistance, and it accepts a certificate issued to *any* GitHub Actions workflow in *any* repository on GitHub. Anyone with a public repo can produce a bundle that passes.

The signature was real. The question "whose signature" was never asked. Authenticity is not "a valid signature exists," it is "a valid signature from the identity I expect," and the difference is the entire control. These notes derive the expected identity from the receipt and refuses to run without one.

The second: a check that is right for the wrong reason. The obvious way to test retention is:

```
NOW=$(date -u +%Y-%m-%dT%H:%M:%SZ)
[[ "$RETAIN_UNTIL" > "$NOW" ]]
```

`RETAIN_UNTIL` comes back from AWS as `2026-04-27T18:30:33.696000+00:00`, `NOW` is `2026-04-27T18:30:33Z`, and `>` in bash is a lexicographic string comparison. Different formats compared as text looks like an obvious bug.

It is not. Generate two hundred thousand retention dates spread across a day either side of now and the string test agrees with an epoch comparison every single time. Both strings share a fixed-width, zero-padded ISO prefix, so comparing `YYYY-MM-DDTHH:MM:SS` as text *is* comparing it chronologically. The formats diverge only at the character after the seconds, and that character decides the outcome only when both sides land in the same second, where "not in the future" is the correct answer either way.

What does break it is a **non-UTC offset**:

Retention	Now	String test	Reality
`2026-08-18T16:00:00-05:00`	`2026-08-18T20:00:00Z`	expired	**an hour in the future**
`2026-08-19T00:00:00+05:00`	`2026-08-18T20:00:00Z`	valid	**an hour in the past**

AWS returns UTC for `get-object-retention`, so the string test is correct. It is correct **by coincidence**, resting on an upstream formatting choice nobody wrote down and nobody would notice changing.

Comparing epoch seconds is correct by construction, for any format that arrives. Prefer it, and understand that you are buying robustness rather than fixing a live defect.

There is a lesson here about verification work itself, and it cuts toward humility. "These strings have different formats, therefore the comparison is broken" is a plausible inference that survives review, sounds authoritative, and is wrong. It took a fuzz loop to find that out. **If you are going to claim a check is broken, make it fail in front of you first.** That is the same standard this lab applies to the controls; it applies to the criticism too.

Step 5 Break it four ways

```
EVIDENCE_VAULT=YOUR_VAULT bash scripts/verify-evidence.sh RUN_ID
```

Expect `CHAIN INTACT`. Now break each property in turn and confirm the right check catches it.

1. Integrity.

```
aws s3 cp "s3://${VAULT}/runs/${RUN_ID}/${BUNDLE}" /tmp/b.tar.gz
echo junk >> /tmp/b.tar.gz
$SHA256 /tmp/b.tar.gz   # differs from the .sha256 sidecar
```

Step 1 fails. And note you cannot write the tampered file *back*: Object Lock refuses to overwrite the existing key. The tampered copy exists only on your laptop.

2. Authenticity.

```
cosign sign-blob --yes --bundle /tmp/fake.sig.bundle /tmp/b.tar.gz
```

Sign it yourself with your own identity. Substitute that bundle and re-verify. Step 2 fails on certificate identity, because your personal OIDC subject is not the workflow subject in the receipt. **Under a `.*` identity regex this attack succeeds.**

3. Completeness.

```
tar xzf "${BUNDLE}" && rm plan.json && tar czf tampered.tar.gz .
```

Repackage without `plan.json` and re-sign it with the workflow identity, which an insider with repo write could do. Steps 1 and 2 pass. Step 3 fails, because `plan.json` is listed in `.manifest.txt` and is not there. Integrity and authenticity alone cannot detect this: the bundle is intact and correctly signed, it is simply missing the file that showed the finding.

4. Preservation. Wait for a GOVERNANCE 1-day retention to lapse, or point the script at an object in an unlocked bucket. Step 4 fails.

Four properties, four breaks, four catches. That table is the best page in your write-up.

Verification

- The vault holds `bundle.tar.gz`, `.sha256`, `.sig.bundle`, and `receipt.json` for at least one run.
- `verify-evidence.sh <run_id>` exits 0 with `CHAIN INTACT`.
- Each of the four tampering scenarios exits non-zero, at the expected step.
- A bundle signed by a different identity is rejected.

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-4-4"
mkdir -p "$EVIDENCE"
{
  echo "### unmodified bundle, expect CHAIN INTACT"
  bash scripts/verify-evidence.sh test-001 2>&1

  echo "### tampered bundle, expect failure"
  cp bundle.tar.gz /tmp/tampered.tar.gz && echo "x" >> /tmp/tampered.tar.gz
  sha256sum /tmp/tampered.tar.gz 2>&1
} | tee "$EVIDENCE/tamper-tests.txt"
```

Portfolio submission checklist

- [ ] `.github/workflows/grc-gate.yml` with Cosign install and the bundle/sign/upload step.
- [ ] `scripts/verify-evidence.sh` committed and executable.
- [ ] At least one run's full bundle in the vault.
- [ ] `evidence/lab-4-4/tamper-tests.txt` capturing all four failures.

- [ ] `WRITEUP.md` section mapping each chain property to the artifact that proves it, and naming which step catches which break.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`cosign sign-blob: failed to get OIDC token`.** The job needs `permissions: id-token: write`.
- **`cosign verify-blob` fails with certificate identity mismatch.** Working as designed if the bundle came from a different repo or workflow. If it is your own run, print the actual subject with `cosign verify-blob --bundle ... --certificate-identity-regexp '.*' ... 2>&1 | head` once, read it, then write the correct pattern. Do not leave `.*` in place.
- **`AccessDenied` on upload to the vault, mentioning KMS.** The role needs `kms:GenerateDataKey` on the vault CMK, not just `s3:PutObject`.
- **Rekor propagation race.** The public log can lag the signing call by about a second. CI naturally waits; this is a laptop-only race.
- **Object Lock rejects an overwrite.** Keys include `runs/<run_id>`, so each run is unique. A re-run that reuses a run ID gets a 403. Trigger a fresh run.
- **`date: illegal option -- d` on macOS.** BSD `date` differs from GNU. The script falls back to `date -jf`; or `brew install coreutils` and use `gdate`.
- **`sha256sum: command not found` on macOS.** The script detects and falls back to `shasum -a 256`.

Cleanup

Do not clean the vault. A 365-day-retention vault exists so evidence outlives the PR that produced it. For lab purposes Lab 2.5 deployed GOVERNANCE with 1-day retention, so bundles become deletable tomorrow. Production is COMPLIANCE, longer retention, no clean.

How this feeds the capstone

Every push now leaves a signed, timestamped, immutably-stored record of what was tested and what happened. An assessor who never meets you reconstructs it in minutes:

1. Read your OSCAL component (Lab 6.1).
2. Follow the evidence URI to a specific object version in the vault.
3. Run `verify-evidence.sh <run_id>`.
4. See `CHAIN INTACT`.

The capstone grader does exactly this and checks three things: the Cosign signature against the public Sigstore log, a SHA-256 recompute, and Object Lock retention. All three are in the script. The fourth check, completeness, is the one that will distinguish your submission, because most candidates will not have implemented it.

Revision history

These notes

- Verification now constrains the signer identity, deriving it from the receipt and refusing to run without one. A permissive `.*` identity regex accepts a signature from any repository on GitHub.
- Added a completeness check: the bundle carries a manifest and per-file hashes, so removing a file from a correctly-signed bundle is detected.
- Retention compared as epoch seconds rather than as ISO strings, for robustness against non-UTC offsets.
- The pass/fail decision moved to the last workflow step, so evidence is signed and stored even when the gate fails.
- The receipt is built with `jq -n` rather than a heredoc, and records the repository and workflow so verification knows what identity to expect.
- The vault write policy gained `kms:GenerateDataKey`, needed now the vault uses a customer-managed key.

The official labs

Initial release: three of the four chain properties, with authenticity checked against a permissive identity regex and no completeness check.

CGE-P LAB NOTES

Lab 5.2: AWS Security Services Baseline

CloudTrail records what happened. Config records what the resource looked like. Security Hub aggregates findings into one normalized view. Athena is what turns any of it into an answer.

That last sentence is the whole reason this lab matters. **This is where AU-6 is earned.** Lab 2.3 ships logs into a bucket, which is AU-3 and AU-11. It stops short of AU-6, audit review and analysis, because nothing there reads a log. Here, something does.

For reading. Code lines wrap to fit the page; copy code from docs/05_02_aws_security_services.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/05_02_aws_security_services.md` in your clone, not from the PDF.

Learning objectives

- Deploy a multi-region CloudTrail with log-file validation and KMS encryption.
- Enable S3 **data events** on the evidence vault, and understand why that bucket specifically.
- Enable Security Hub with the NIST 800-53 Rev 5 standard.
- **Query the trail with Athena and produce an answer**, which is what AU-6 costs.
- Defend a control mapping you disagree with.

Controls implemented

Control	Service	Note
AU-2, AU-12	CloudTrail management events	Audit event selection and generation.
AU-3	CloudTrail data events on the vault	Object-level, JSON, guaranteed delivery.
AU-6	**Athena queries over the trail**	The control most often claimed on the strength of collection alone.
AU-9	Log-file validation digests + trail bucket versioning	
AU-11	Lifecycle on the trail bucket	
SC-28	Trail encrypted with a CMK	
RA-5, SI-4	Security Hub	
CM-2, CM-6, CM-8	AWS Config, where deployable	

A mapping to argue with

Log-file validation is commonly mapped to **AU-10 (non-repudiation)**. Push back on that.

Log-file validation emits an hourly digest file signed by an AWS-managed key, letting you detect modification or deletion of log files. That is integrity protection of audit records: **AU-9**, and specifically AU-9(3) when the mechanism is cryptographic, with a strong SI-7 argument alongside it.

AU-10 is about binding an action to an individual so they cannot later deny performing it. CloudTrail's `userIdentity` block does contribute to that. The *digest file* does not; it proves the log was not altered, not that a particular human did a particular thing.

Both mappings can be argued. AU-9 is the stronger argument, and the reason to teach this is not the answer. It is that **a control mapping is a claim you defend, and an assessor who disagrees with yours will ask you why**. Being able to say "we mapped it to AU-9 because the digest protects the record rather than binding the actor, and here is where AU-10 is covered instead" is the skill.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 2.5** vault deployed. Data events target it.
- AWS account with admin or near-admin rights.
- Terraform `>= 1.10`, using the Lab 2.2 backend.
- Check for an existing Security Hub subscription you do not want to disturb: `aws securityhub describe-hub`.

Estimated time & cost

THIS LAB COSTS REAL MONEY. Read this before applying.

- CloudTrail **management** events: the first trail is free.
- CloudTrail **data** events: about **$0.10 per 100,000 events**. Scoped to the evidence vault only, a lab account generates a handful. Point them at a busy bucket and this is the line item that surprises people.
- Security Hub: about $0.001 per check per month. The NIST 800-53 standard runs roughly 300 checks, so under $1/month in a quiet single-account lab.
- AWS Config: about $2/month per recorder plus $0.001 per rule evaluation. Many Security Hub controls need Config to evaluate.
- Trail CMK: **$1/month**.
- S3 storage for logs: pennies, and bounded by the lifecycle rule.

Destroy within an hour and expect under $2. Leave Security Hub, Config, and two CMKs running for a month and it is **$10 to $15**. Estimates that quote $5 to $8 are usually leaving out Config and the second key.

Architecture

```
every region
------------
CloudTrail "cgep-lab-mgmt"  multi-region
  management events                       AU-2 / AU-12
  data events: s3://EVIDENCE_VAULT/*      AU-3
```

```
    log-file validation -> digests           AU-9 / SI-7
    KMS CMK                                  SC-28
        |
        v
  S3 trail bucket: versioned, KMS, TLS-only, lifecycle 400d
        |
        +--> Athena external table  ----> AU-6 queries
        |
  AWS Config recorder  ---> same bucket      CM-2 / CM-6 / CM-8   (SCP may block)
        |
        v
  Security Hub (NIST 800-53 Rev 5 + FSBP)    RA-5 / SI-4
```

Step-by-step walkthrough

Concept: why baseline services beat point tools

Every cloud security vendor will sell you a slicker console. CloudTrail, Config, and Security Hub are inside the platform you already pay for, mapped to the controls assessors ask about, and emitting JSON you can pipe into the pipeline you already built. They are not pretty. They are durable. Pick the durable one.

Step 1 The trail bucket, on the shared baseline

It is easy to hold this bucket to a lower standard than a workload bucket, on the reasoning that nothing but CloudTrail writes to it. That is exactly backward: it carries the audit record of the entire account.

```
resource "aws_kms_key" "trail" {
  description             = "CMK for CloudTrail logs"
  enable_key_rotation     = true
  deletion_window_in_days = 7
  policy                  = data.aws_iam_policy_document.trail_kms.json
}

data "aws_iam_policy_document" "trail_kms" {
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }

  # CloudTrail must be able to encrypt log files with this key.
  statement {
    sid       = "AllowCloudTrailEncrypt"
```

```
    effect    = "Allow"
    actions   = ["kms:GenerateDataKey*", "kms:DescribeKey"]
    resources = ["*"]
    principals {
      type        = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    condition {
      test     = "StringLike"
      variable = "aws:SourceArn"
      values   = ["arn:${local.partition}:cloudtrail:${var.aws_region}:${local.account_id}:trail/cgep-
lab-mgmt"]
    }
  }

  # Athena and your analysts must be able to read them back.
  statement {
    sid       = "AllowAccountDecrypt"
    effect    = "Allow"
    actions   = ["kms:Decrypt", "kms:DescribeKey"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket" "trail" {
  bucket = "cgep-lab-cloudtrail-${random_id.suffix.hex}"
}

resource "aws_s3_bucket_ownership_controls" "trail" {
  bucket = aws_s3_bucket.trail.id
  rule { object_ownership = "BucketOwnerEnforced" }
}

resource "aws_s3_bucket_versioning" "trail" {
  bucket = aws_s3_bucket.trail.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "trail" {
  bucket = aws_s3_bucket.trail.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.trail.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_public_access_block" "trail" {
  bucket                  = aws_s3_bucket.trail.id
  block_public_acls       = true
```

```
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# AU-11: 400 days covers a full annual audit cycle plus the audit itself.
resource "aws_s3_bucket_lifecycle_configuration" "trail" {
  bucket     = aws_s3_bucket.trail.id
  depends_on = [aws_s3_bucket_versioning.trail]

  rule {
    id     = "expire-trail-logs"
    status = "Enabled"
    filter {}

    transition {
      days          = 90
      storage_class = "GLACIER_IR"
    }

    expiration { days = var.trail_retention_days }

    noncurrent_version_expiration { noncurrent_days = 30 }
  }
}
```

The Glacier Instant Retrieval transition at 90 days is new. Trail logs are written constantly, read rarely, and must be readable immediately when they are read. That is exactly the storage class's use case, and it cuts the dominant cost of a long retention.

Step 2 The trail bucket policy

```
data "aws_iam_policy_document" "trail" {
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.trail.arn, "${aws_s3_bucket.trail.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  statement {
    sid       = "AWSCloudTrailAclCheck"
    effect    = "Allow"
```

```
    actions   = ["s3:GetBucketAcl"]
    resources = [aws_s3_bucket.trail.arn]
    principals {
      type        = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceArn"
      values   = ["arn:${local.partition}:cloudtrail:${var.aws_region}:${local.account_id}:trail/cgep-
lab-mgmt"]
    }
  }

  statement {
    sid       = "AWSCloudTrailWrite"
    effect    = "Allow"
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.trail.arn}/AWSLogs/${local.account_id}/*"]
    principals {
      type        = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "s3:x-amz-acl"
      values   = ["bucket-owner-full-control"]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceArn"
      values   = ["arn:${local.partition}:cloudtrail:${var.aws_region}:${local.account_id}:trail/cgep-
lab-mgmt"]
    }
  }
}

resource "aws_s3_bucket_policy" "trail" {
  bucket = aws_s3_bucket.trail.id
  policy = data.aws_iam_policy_document.trail.json
}
```

The `aws:SourceArn` conditions are confused-deputy protection: without them, someone else's trail in another account could be pointed at your bucket. Note that this is the same pattern Lab 2.3 uses for S3 access-log delivery, which is why these notes use a policy there rather than an ACL.

Step 3 The trail, with data events

```
resource "aws_cloudtrail" "mgmt" {
  name                          = "cgep-lab-mgmt"
  s3_bucket_name                = aws_s3_bucket.trail.id
  is_multi_region_trail         = true
```

```
  include_global_service_events = true
  enable_log_file_validation    = true
  kms_key_id                    = aws_kms_key.trail.arn

  # AU-3: object-level events on the evidence vault ONLY.
  # This is the one bucket where "who read the evidence" is itself an audit
  # question. Scoping it here keeps the bill in cents instead of dollars.
  advanced_event_selector {
    name = "S3 data events on the evidence vault"

    field_selector {
      field  = "eventCategory"
      equals = ["Data"]
    }
    field_selector {
      field  = "resources.type"
      equals = ["AWS::S3::Object"]
    }
    field_selector {
      field       = "resources.ARN"
      starts_with = ["${var.evidence_vault_arn}/"]
    }
  }

  advanced_event_selector {
    name = "All management events"

    field_selector {
      field  = "eventCategory"
      equals = ["Management"]
    }
  }

  depends_on = [aws_s3_bucket_policy.trail]
}
```

Why the vault and nothing else. Skipping data events entirely leaves object-level access to the evidence vault unrecorded, and that is the gap an insider uses: read the evidence, learn what the auditor will see, and no record exists that you looked. Management events show the bucket being *created*, never that it was *read*.

Scoping data events to one bucket is the honest middle. Turning them on account-wide in a real environment is a real budget decision, and saying so is better than pretending the choice is free.

Step 4 Security Hub

```
resource "aws_securityhub_account" "this" {}

resource "aws_securityhub_standards_subscription" "nist_800_53" {
  standards_arn = "arn:${local.partition}:securityhub:${var.aws_region}::standards/nist-800-53/v/
5.0.0"
  depends_on    = [aws_securityhub_account.this]
}
```

```
resource "aws_securityhub_standards_subscription" "fsbp" {
  standards_arn = "arn:${local.partition}:securityhub:${var.aws_region}::standards/aws-foundational-
security-best-practices/v/1.0.0"
  depends_on    = [aws_securityhub_account.this]
}
```

If Security Hub is already enabled:

```
terraform import aws_securityhub_account.this "$(aws sts get-caller-identity --query Account --output
text)"
```

A prediction to check. Point Security Hub at an account holding buckets built the easy way, with ACLs enabled, no TLS-deny policy and versioning off, and expect findings S3.12, S3.5 and S3.14 respectively. Point it at the buckets from Lab 2.3 and those three are absent.

Watching that happen costs one screenshot and is the most direct evidence you will get that the Chapter 2 baseline and the Chapter 5 monitoring agree with each other.

Step 5 AWS Config, and the SCP wall

Config is included, and in many org-managed accounts it is centrally administered and blocked by a service control policy:

```
AccessDeniedException: ... is not authorized to perform: config:PutConfigurationRecorder
... with an explicit deny in a service control policy
```

If that is you, comment out the Config resources. Then note what Security Hub does next: it raises a CRITICAL finding titled "AWS Config should be enabled and use the service-linked role for resource recording."

That finding is itself the evidence. Your account is reporting its own gap, in a machine-readable format, without you writing anything. Capture it, cite it in your write-up as a known and accepted limitation with the compensating control named (Config is managed at the org level), and you have handled it the way a GRC engineer should. Silence would have been the wrong answer; so would pretending you deployed it.

Step 6 Apply and wait

```
export AWS_PROFILE=cgep
terraform init && terraform apply -auto-approve
```

Wait 10 to 20 minutes. Security Hub populates its first findings slowly.

Step 7 AU-6, for real: Athena over the trail

This is the step that separates "we collect logs" from "we review them."

```
resource "aws_athena_workgroup" "grc" {
  name = "cgep-grc"

  configuration {
    enforce_workgroup_configuration = true

    result_configuration {
      output_location = "s3://${aws_s3_bucket.trail.id}/athena-results/"

      encryption_configuration {
        encryption_option = "SSE_KMS"
        kms_key_arn       = aws_kms_key.trail.arn
      }
    }
  }
}

resource "aws_glue_catalog_database" "trail" {
  name = "cgep_grc"
}
```

Create the table. CloudTrail's Athena DDL is long and AWS generates it for you from the console; the partition-projection version below is worth using because it avoids running `MSCK REPAIR TABLE` forever:

```
CREATE EXTERNAL TABLE cgep_grc.cloudtrail_logs (
  eventVersion STRING,
  userIdentity STRUCT<
    type: STRING, principalId: STRING, arn: STRING, accountId: STRING,
    userName: STRING,
    sessionContext: STRUCT<
      attributes: STRUCT<mfaAuthenticated: STRING, creationDate: STRING>>>,
  eventTime STRING,
  eventSource STRING,
  eventName STRING,
  awsRegion STRING,
  sourceIPAddress STRING,
  userAgent STRING,
  errorCode STRING,
  errorMessage STRING,
  requestParameters STRING,
  resources ARRAY<STRUCT<ARN: STRING, accountId: STRING, type: STRING>>,
  readOnly STRING,
  eventType STRING
)
PARTITIONED BY (region STRING, dt STRING)
ROW FORMAT SERDE 'com.amazon.emr.hive.serde.CloudTrailSerde'
STORED AS INPUTFORMAT 'com.amazon.emr.cloudtrail.CloudTrailInputFormat'
OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION 's3://<TRAIL_BUCKET>/AWSLogs/<ACCOUNT_ID>/CloudTrail/'
TBLPROPERTIES (
  'projection.enabled' = 'true',
  'projection.region.type' = 'enum',
  'projection.region.values' = 'us-east-1,us-west-2',
  'projection.dt.type' = 'date',
```

```
  'projection.dt.range' = '2026/01/01,NOW',
  'projection.dt.format' = 'yyyy/MM/dd',
  'projection.dt.interval' = '1',
  'projection.dt.interval.unit' = 'DAYS',
  'storage.location.template' =
    's3://<TRAIL_BUCKET>/AWSLogs/<ACCOUNT_ID>/CloudTrail/${region}/${dt}/'
);
```

Now the three queries that constitute an audit review. Save them; they are the deliverable.

```
-- AU-6.1: who touched the evidence vault, and what did they do?
SELECT eventTime, userIdentity.arn, eventName, sourceIPAddress, errorCode
FROM cgep_grc.cloudtrail_logs
WHERE dt >= date_format(current_date - interval '7' day, '%Y/%m/%d')
  AND eventSource = 's3.amazonaws.com'
  AND element_at(resources, 1).ARN LIKE '%evidence-vault%'
ORDER BY eventTime DESC;

-- AU-6.2: console sign-ins without MFA. AC-2 / IA-2 evidence.
SELECT eventTime, userIdentity.arn, sourceIPAddress
FROM cgep_grc.cloudtrail_logs
WHERE dt >= date_format(current_date - interval '30' day, '%Y/%m/%d')
  AND eventName = 'ConsoleLogin'
  AND userIdentity.sessionContext.attributes.mfaAuthenticated = 'false'
ORDER BY eventTime DESC;

-- AU-6.3: denied actions, clustered by principal. The signal that someone
-- is probing the edges of their permissions.
SELECT userIdentity.arn, eventName, count(*) AS attempts
FROM cgep_grc.cloudtrail_logs
WHERE dt >= date_format(current_date - interval '7' day, '%Y/%m/%d')
  AND errorCode IN ('AccessDenied', 'UnauthorizedOperation')
GROUP BY userIdentity.arn, eventName
HAVING count(*) > 5
ORDER BY attempts DESC;
```

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-5-2"
mkdir -p "$EVIDENCE"
aws athena start-query-execution \
  --work-group cgep-grc \
  --query-string "$(cat queries/au6-vault-access.sql)" \
  --query-execution-context Database=cgep_grc
```

AU-6 is a process, not a resource. Running the query once satisfies nothing. What satisfies AU-6 is: the query exists, it runs on a defined cadence, a named human reviews the output, and the review is recorded. Schedule it (EventBridge on a weekly rule is enough for the capstone), write the output into your evidence vault, and name the reviewer in your write-up.

If you do not schedule it, say so honestly and mark AU-6 as `partial` in your OSCAL. A partial you can defend beats an `implemented` you cannot.

Step 8 Capture findings as evidence

```
aws securityhub get-findings --region us-east-1 --max-results 50 \
  > "$EVIDENCE/security-hub-findings.json"

aws cloudtrail get-trail-status --name cgep-lab-mgmt --region us-east-1 \
  > "$EVIDENCE/trail-status.json"

# AU-9: prove the digest chain validates
aws cloudtrail validate-logs \
  --trail-arn "$(terraform output -raw trail_arn)" \
  --start-time "$(date -u -d '1 day ago' +%Y-%m-%dT%H:%M:%SZ)" \
  | tee "$EVIDENCE/log-validation.txt"
```

That last command is the AU-9 evidence, and it is the one most often skipped. It walks the digest chain and reports whether any log file was modified or deleted. Enabling validation and never validating is the same shape of mistake as writing a policy and never testing it.

Verification

- `aws cloudtrail get-trail-status` returns `"IsLogging": true`.
- `aws cloudtrail get-event-selectors` shows both advanced selectors.
- `aws cloudtrail validate-logs` reports no invalid files.
- `aws securityhub describe-hub` returns the hub ARN.
- At least one finding within 30 minutes.
- All three Athena queries return results, or an explainable empty set.
- Read an object from the evidence vault, wait, then confirm the read appears in the AU-6.1 query output. **That round trip is the proof that data events work.**

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-5-2"
mkdir -p "$EVIDENCE"
aws athena get-query-results --query-execution-id "$QID" \
  > "$EVIDENCE/au6-review-output.json"
```

That file is what makes your AU-6 claim defensible. A query you ran once and did not keep is indistinguishable from a query you never ran.

Portfolio submission checklist

- [ ] `terraform/baselines/aws/` with `cloudtrail.tf`, `security_hub.tf`, `athena.tf`, optionally `config.tf`.

- [ ] `queries/*.sql`, all three AU-6 queries.
- [ ] README mapping services to controls, **including the AU-9-versus-AU-10 argument**.
- [ ] `evidence/lab-5-2/security-hub-findings.json`
- [ ] `evidence/lab-5-2/log-validation.txt`
- [ ] `evidence/lab-5-2/au6-review-output.json`, plus a note on cadence and reviewer.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`InsufficientS3BucketPolicyException`.** The bucket policy is missing an `aws:SourceArn` condition, or the trail is being created before the policy. The `depends_on` sequences it.
- **`InsufficientEncryptionPolicyException`.** The trail CMK policy lacks the `cloudtrail.amazonaws.com` `GenerateDataKey*` grant.
- **`ResourceConflictException: Account is already subscribed`.** Import: `terraform import aws_securityhub_account.this <ACCOUNT_ID>`.
- **`AccessDeniedException ... explicit deny in a service control policy`** for Config. Centrally managed. Comment it out and cite the Security Hub finding as evidence of the gap.
- **Athena returns zero rows.** Check the `LOCATION` includes the account ID and that `projection.dt.range` starts before your trail did. Partition projection silently returns nothing for a range with no data.
- **Athena `HIVE_CURSOR_ERROR` on decryption.** The querying principal needs `kms:Decrypt` on the trail CMK. That is the `AllowAccountDecrypt` statement.
- **No findings after 30 minutes.** Check `aws securityhub get-enabled-standards`.
- **Config recorder name conflict.** One recorder per region. Delete the existing one first.

Cleanup

```
# Capture evidence BEFORE destroying.
aws securityhub get-findings --region us-east-1 --max-results 50 \
  > "$EVIDENCE/security-hub-findings.json"

# Keep Security Hub enabled but drop it from state, if you want it running.
terraform state rm aws_securityhub_account.this

terraform destroy -auto-approve
```

The trail bucket holds objects and is versioned, so add `force_destroy = true` or empty it with the version-aware loop from Lab 2.3. Both CMKs enter their deletion windows and bill throughout.

If ongoing cost worries you, the highest-impact action is unsubscribing the Security Hub standards. The hub is free; the checks bill.

The starter's audit gaps. `GAPS.md` closes with two the account-level controls here bear on directly: **GAP-08**, an API Gateway stage with no access logging, no throttling and no WAF, and **GAP-06**, a Lambda with no DLQ and no X-Ray. CloudTrail gives you who called the API; neither replaces the per-service logging those gaps are actually missing, and saying so precisely is the difference between a real control narrative and a hopeful one.

How this feeds the capstone

Capstone Layer 1 requires a multi-region CloudTrail with log-file validation writing to a dedicated bucket. That is Step 3, and the trail bucket is Step 1.

Your OSCAL component declares `implemented-requirement` entries for AU-2, AU-3, AU-6, AU-9, RA-5, and SI-4. The implementation statements name CloudTrail, Athena, and Security Hub. The evidence URIs point at signed copies of `security-hub-findings.json`, `log-validation.txt`, and your AU-6 query output in the vault.

Be careful with AU-6 specifically. If you scheduled the review, claim `implemented` and cite the EventBridge rule. If you ran it once by hand, claim `partial` and say what is missing. The grader is reading for exactly that distinction, because claiming AU-6 on the strength of collection alone is the single most common overclaim in this space.

Revision history

These notes

- AU-6 is now implemented rather than claimed: a Glue database, an Athena workgroup and three review queries, with a note that running them once by hand is `partial`, not `implemented`.
- CloudTrail data events enabled for the evidence vault, scoped by an advanced event selector. Object-level reads of the vault were previously unrecorded.
- The trail bucket moved to a customer-managed key with versioning, lifecycle including a Glacier Instant Retrieval transition, and a TLS-deny policy.
- Log-file validation remapped from AU-10 to AU-9(3) and SI-7, with the argument for the change stated rather than assumed.
- Added `aws cloudtrail validate-logs` to the evidence steps, so validation is exercised rather than merely enabled.
- Cost estimate corrected upward to $10 to $15 per month running, from $5 to $8.

The official labs

Initial release: management events only, AES256 trail bucket, AU-6 claimed without a review mechanism, log-file validation mapped to AU-10 and never run.

CGE-P LAB NOTES

Lab 5.4: GCP Security Services Baseline

GCP leads with identity and data. Where AWS gives you a flat IAM policy and a Security Hub aggregator, GCP gives you Org Policy that rejects misconfigurations at the API call, Workload Identity Federation that replaces service account keys with short-lived OIDC tokens, and Data Access logs that, deliberately, you have to turn on.

This lab does all three for one project, including the part most guidance skips: a safe way to introduce a preventive control without breaking production on the day you enable it.

For reading. Code lines wrap to fit the page; copy code from docs/05_04_gcp_security_services.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/05_04_gcp_security_services.md` in your clone, not from the PDF.

Learning objectives

- Enforce Org Policy at the API so non-compliant creation is rejected before the resource exists.
- **Roll out a preventive control in audit mode first**, then enforce. This is new in v2.
- Replace long-lived service account keys with Workload Identity Federation.
- Enable Data Access audit logs, the single most-cited GCP audit finding.

Controls implemented

Control	Mechanism
CM-6	`storage.uniformBucketLevelAccess`, `storage.publicAccessPrevention`
AC-2, IA-5	`iam.disableServiceAccountKeyCreation` + WIF
AC-3	`compute.requireOsLogin`
AU-2, AU-3, AU-12	Data Access audit logs per service
SC-7	VPC Service Controls (discussed, not deployed; see Step 6)
SC-8	**Inherited from GCP.** Cloud Storage and the Google APIs are TLS-only.

Same inherited-control note as Lab 2.4. On AWS you write a bucket policy to get SC-8; on GCP the platform provides it. Record it as `inherited` in your OSCAL rather than claiming you implemented it.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 0.1 Part 2** for project, billing, APIs and both authentications. `orgpolicy` and `sts` are the two APIs people forget, and Step 8 enables them.
- A GCP project you own with billing enabled, and APIs `cloudkms`, `iam`, `cloudresourcemanager`, and `orgpolicy` enabled.
- Roles: `roles/orgpolicy.policyAdmin`, `roles/iam.workloadIdentityPoolAdmin`, `roles/logging.admin`. At project scope your project owner can grant these; at org scope you need org-level rights.
- `gcloud auth login` **and** `gcloud auth application-default login`.

- Terraform `>= 1.10`.

The lab uses project scope so it works in any environment. If your project sits in an Organization and you want org or folder scope, the same resources take a different `parent`.

> **Read the Cleanup section before you apply this one.** Three things here do not undo the way you would expect:
>
> - **Workload Identity Federation pools enter a 30-day soft delete.** You cannot recreate a pool with the same ID until it expires, unless you undelete and then purge it.
> - **Disabling an Org Policy does not retroactively un-enforce it.** Buckets created under it keep the shape it required.
> - **Removing an audit config is not retroactive either.** Logs already ingested keep billing until their retention expires.
>
> None of that is a lab artifact. It is what "enforced at the platform" costs, and knowing it before you apply is the difference between a decision and a surprise.

Estimated time & cost

- 90 minutes, mostly waiting for Org Policy propagation (5 to 10 minutes per change).
- Org Policy: free. WIF: free. Security Command Center Standard: free at org level.
- **Data Access logs: $0.50/GB ingested plus Cloud Logging storage.** Pennies for an empty project, gigabytes per day in a busy one. Start with one service.

Architecture

```
project (your-gcp-project)
--------------------------
Org Policy (project scope, REJECT at the API):
  storage.uniformBucketLevelAccess       = TRUE     CM-6
  storage.publicAccessPrevention         = TRUE     AC-3
  iam.disableServiceAccountKeyCreation   = TRUE     AC-2 / IA-5
  compute.requireOsLogin                 = TRUE     AC-3

Workload Identity Federation:
  pool     : github-actions
  provider : token.actions.githubusercontent.com
  condition: assertion.repository == "GRCEngClub/cgep-app-starter"
  SA       : cgep-grc-gate-sa@...  (roles/viewer)
```

```
Data Access audit logs (per service):        AU-2 / AU-3 / AU-12
  storage.googleapis.com    DATA_READ + DATA_WRITE + ADMIN_READ
  cloudkms.googleapis.com   DATA_READ + DATA_WRITE + ADMIN_READ
  iam.googleapis.com        DATA_READ + DATA_WRITE + ADMIN_READ
```

Step-by-step walkthrough

Concept: why identity-first

GCP's bet is that the smallest unit of security is the principal, not the resource.

Org Policy enforces at the API call: a bucket creation that violates `uniformBucketLevelAccess` is **rejected**, not flagged. WIF replaces "create a service account, download a JSON key, paste it into GitHub Secrets, hope nobody leaks it" with "the runtime presents an OIDC token, GCP swaps it for a short-lived access token, the token expires by itself."

Together they make whole categories of attack uneconomical. Compare the layers you now have:

Layer	Example	When it acts
Preventive, at the API	Org Policy	The action never happens
Preventive, at the pipeline	Conftest gate (Ch 3)	Before merge
Detective	Security Hub, SCC, Config	Minutes to hours after

Org Policy is the strongest layer, and it is the one that will page you at 2am if you enable it carelessly. Hence Step 2.

Step 1 Org Policy in audit mode first

This is an addition to the official lab, and the most useful habit in it.

Going straight to `enforce = "TRUE"` is fine in a lab account. In a real project, turning on a rejection at the API without knowing what currently violates it is how you break the deploy pipeline of a team that has never heard of you.

The Org Policy v2 API supports a dry-run specification: the constraint evaluates and logs violations without rejecting anything.

```
# Phase 1: observe. Violations are logged, nothing is blocked.
resource "google_org_policy_policy" "uniform_bucket_access" {
  name   = "projects/${var.gcp_project}/policies/storage.uniformBucketLevelAccess"
  parent = "projects/${var.gcp_project}"
```

```
  dry_run_spec {
    rules {
      enforce = "TRUE"
    }
  }
}
```

Leave it for a week. Query the violations:

```
gcloud logging read \
  'protoPayload.status.details.violations.type="ORG_POLICY_DRY_RUN"' \
  --project="$TF_VAR_gcp_project" --limit=50 --format=json
```

Then, once the list is empty or every entry is understood, promote it:

```
# Phase 2: enforce.
resource "google_org_policy_policy" "uniform_bucket_access" {
  name   = "projects/${var.gcp_project}/policies/storage.uniformBucketLevelAccess"
  parent = "projects/${var.gcp_project}"

  spec {
    rules {
      enforce = "TRUE"
    }
  }
}
```

Do not audit by omitting the rules block. It is a natural guess and it is dangerously wrong: a policy with no rules is not auditing, it is **not in effect at all**. You would observe zero violations and conclude you were compliant. `dry_run_spec` is the mechanism that actually audits. Confirm your `google` provider supports it; it is present in recent 5.x.

The rest of the constraints, in their enforcing form:

```
resource "google_org_policy_policy" "public_access_prevention" {
  name   = "projects/${var.gcp_project}/policies/storage.publicAccessPrevention"
  parent = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
  }
}

resource "google_org_policy_policy" "disable_sa_keys" {
  name   = "projects/${var.gcp_project}/policies/iam.disableServiceAccountKeyCreation"
  parent = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
```

```
  }
}

resource "google_org_policy_policy" "require_oslogin" {
  name   = "projects/${var.gcp_project}/policies/compute.requireOsLogin"
  parent = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
  }
}
```

`storage.publicAccessPrevention` is new in v2 and pairs with Lab 2.4's per-bucket setting. The module sets it on buckets it creates; the org policy sets it on buckets **anyone** creates, including by hand in the console. That is the difference between a module standard and an organizational control, and it is worth saying out loud in your write-up.

Step 1b Apply it

Nothing above has reached GCP yet. Step 2 tests constraints that do not exist until you apply, and skipping this is easy because Step 1 ends in a wall of HCL rather than a command.

`gcp_project`, `github_org` and `github_repo` have no defaults, because a default project is precisely how you enforce an org policy on the wrong project. Put them in `terraform.tfvars`, which Terraform reads automatically and which is gitignored:

```
# Unquoted heredoc, so the values you set once in cgep.env are written in
# rather than retyped. A tfvars full of placeholders is a tfvars somebody
# eventually commits with a real project id in it.
cat > terraform.tfvars <<EOF
gcp_project = "$TF_VAR_gcp_project"
github_org  = "$TF_VAR_github_org"
github_repo = "$TF_VAR_github_repo"
EOF
```

```
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
```

Apply the **enforcing** form before testing. If `uniform_bucket_access` is still on `dry_run_spec`, Step 2's bucket test will succeed, and it will look like a broken control rather than a policy that is deliberately only watching.

Without the tfvars file Terraform prompts for all three values on every `plan`, `apply` and `destroy`, and fails outright under `-input=false`.

If `cgep.env` from Lab 0.1 Step 15 has `TF_VAR_gcp_project`, `TF_VAR_github_org` and `TF_VAR_github_repo` filled in, skip the file. Source it and Terraform picks them up.

Step 2 Test the enforcement

```
gcloud iam service-accounts keys create /tmp/key.json \
  --iam-account="$(terraform output -raw service_account_email)" \
  --project="$TF_VAR_gcp_project"
```

```
ERROR: (gcloud.iam.service-accounts.keys.create) FAILED_PRECONDITION:
Key creation is not allowed on this service account.
constraint iam.disableServiceAccountKeyCreation
```

This is the lesson. The control did not fire after the fact in a finding three hours later. **The action did not happen.** Capture that transcript; it is better evidence than any configuration dump, because it demonstrates enforcement rather than intent.

Do the same for the bucket constraint:

```
# The name carries a random suffix because GCS bucket names are one namespace
# shared by every Google customer, and this one is meant to be refused rather
# than to collide with a stranger's.
TESTBUCKET="cgep-nonuniform-test-$(openssl rand -hex 4)"
gcloud storage buckets create "gs://$TESTBUCKET" \
  --project="$TF_VAR_gcp_project" --no-uniform-bucket-level-access
# expect: rejected by constraint storage.uniformBucketLevelAccess

# If it was NOT rejected, the constraint is not enforcing and you have just
# created a bucket that allows ACLs. Remove it before moving on.
gcloud storage buckets delete "gs://$TESTBUCKET" --quiet 2>/dev/null || true
```

Step 3 Workload Identity Federation

The `attribute_condition` is the whole security of this construct. Without it, **any GitHub repository on the public internet** can present a token to your provider and impersonate your service account.

```
resource "google_iam_workload_identity_pool" "github" {
  workload_identity_pool_id = "github-actions"
  display_name              = "GitHub Actions"
}

resource "google_iam_workload_identity_pool_provider" "github" {
  workload_identity_pool_id          =
google_iam_workload_identity_pool.github.workload_identity_pool_id
  workload_identity_pool_provider_id = "github"

  attribute_mapping = {
    "google.subject"       = "assertion.sub"
    "attribute.repository" = "assertion.repository"
    "attribute.actor"      = "assertion.actor"
    "attribute.ref"        = "assertion.ref"
  }
```

```
  # AC-3. Without this line the provider trusts all of GitHub.
  attribute_condition = "assertion.repository == \"${var.github_org}/${var.github_repo}\""

  oidc {
    issuer_uri = "https://token.actions.githubusercontent.com"
  }
}

resource "google_service_account" "gha" {
  account_id   = "cgep-grc-gate-sa"
  display_name = "CGE-P GRC gate (read-only)"
}

# AC-6: read-only. The gate plans and inspects; it does not change anything.
resource "google_project_iam_member" "gha_viewer" {
  project = var.gcp_project
  role    = "roles/viewer"
  member  = "serviceAccount:${google_service_account.gha.email}"
}

resource "google_service_account_iam_binding" "wif_user" {
  service_account_id = google_service_account.gha.name
  role               = "roles/iam.workloadIdentityUser"

  members = [
    "principalSet://iam.googleapis.com/${google_iam_workload_identity_pool.github.name}/
attribute.repository/${var.github_org}/${var.github_repo}",
  ]
}
```

Defense in depth here is two layers, and students routinely build only one. The `attribute_condition` on the provider decides who may exchange a token at all. The `principalSet` in the binding decides who may impersonate this particular service account. Set the condition and use `principalSet://.../*` in the binding and you have one layer. Set both and a mistake in either is survivable.

For an apply pipeline, tighten further to a specific ref:

```
principalSet://iam.googleapis.com/POOL/attribute.ref/refs/heads/main
```

The workflow side, keyless:

```
permissions:
  id-token: write
  contents: read

steps:
  - uses: google-github-actions/auth@7c6bc770dae815cd3e89ee6cdf493a5fab2cc093 # v3
    with:
      workload_identity_provider: projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/
```

```
github-actions/providers/github
      service_account: cgep-grc-gate-sa@your-gcp-project.iam.gserviceaccount.com

  - run: gcloud storage ls
```

The token is minted at job start, expires in an hour, and never touches disk. Same posture as the AWS OIDC pattern in Lab 4.3, different cloud, identical argument. Note the action is SHA-pinned for the same supply chain reason.

Step 4 Data Access audit logs

Off by default, and the most-cited GCP audit finding because nobody turns them on.

```
resource "google_project_iam_audit_config" "storage" {
  project = var.gcp_project
  service = "storage.googleapis.com"
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
  audit_log_config { log_type = "ADMIN_READ" }
}

resource "google_project_iam_audit_config" "kms" {
  project = var.gcp_project
  service = "cloudkms.googleapis.com"
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
  audit_log_config { log_type = "ADMIN_READ" }
}

resource "google_project_iam_audit_config" "iam" {
  project = var.gcp_project
  service = "iam.googleapis.com"
  audit_log_config { log_type = "ADMIN_READ" }
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
}
```

Verify a read actually lands:

```
gcloud storage ls gs://your-test-bucket
sleep 30
gcloud logging read \
  'protoPayload.serviceName="storage.googleapis.com" AND
protoPayload.methodName=~"storage.objects.list"' \
  --limit 5 --format=json
```

Turning them on is AU-2 and AU-12. Reading them is AU-6, and the same caution from Lab 5.2 applies: a log nobody reviews is a cost center, not a control. Log Analytics or a BigQuery sink is the GCP equivalent of the Athena setup in Lab 5.2. Build one or claim `partial`.

Step 5 Security Command Center

If your project is in an Organization with SCC enabled, findings flow in automatically. Standard is free at org level; Premium is enterprise-priced and not used here. The lab does not provision SCC because it needs org admin.

```
gcloud scc findings list ORG_ID --source=- --format=json \
  > "$EVIDENCE/scc-findings.json"
```

For a standalone project without an Organization, SCC is unavailable. The Org Policy enforcements above are your preventive layer, and stating that substitution explicitly in your write-up is the right move: "detective coverage is absent at project scope; compensated by preventive enforcement at the API."

Step 6 What is missing, and naming it

Two real controls this lab does not build, worth knowing so you do not overclaim:

VPC Service Controls (SC-7). The genuine data-exfiltration boundary on GCP. A service perimeter stops a principal with valid credentials from copying data to a project outside the perimeter, which IAM alone cannot do. It requires org-level rights and careful planning, and misconfiguring one locks you out of your own project. Name it as a gap with a remediation plan.

Assured Workloads / CMEK org constraints. `gcp.restrictNonCmekServices` forces CMEK across services rather than relying on each module to do the right thing. It is the organizational form of what Lab 2.4's module does per-bucket, and it is the same relationship as `storage.publicAccessPrevention` in Step 1.

Verification

Every check below asserts a value. The commands this replaces printed a `gcloud org-policies list` and left you to scan it, which cannot fail and so proves nothing. They also spelled your project as a literal `your-gcp-project`, which you would have had to substitute in six places. `TF_VAR_gcp_project` is already exported by `cgep.env`, so the script reads it.

```
bash scripts/verify.sh .
```

On the last check, which writes. Proving `iam.disableServiceAccountKeyCreation` works means trying to create a key. If the constraint is missing that attempt succeeds, and you now hold a real long-lived service account private key, on disk and in the project. The script deletes it immediately, reports the failure, and tells you the key id to remove by hand if the rollback itself fails. **A test that can leave a credential behind has to clean up after itself, and has to say so when it cannot.**

This is the opposite of the choice made in Lab 2.4, where the equivalent GCP denial test would have made a bucket public and was replaced with an anonymous read instead. The difference is that a key can be revoked and a public object cannot be un-read.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 5.4. Every check asserts a value and exits non-zero if
# the project disagrees.
#
# Everything is read as JSON and compared with jq. gcloud's value() projection
# renders booleans with Python's capitalisation and silently omits any field
# name it does not recognize, so a typo and a missing control look the same.
set -uo pipefail

WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0

pass() { printf '  PASS  %-8s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:-(nothing)}'"
  fi
}

PROJECT="${TF_VAR_gcp_project:?set TF_VAR_gcp_project, or source cgep.env}"
SA=$(terraform output -raw service_account_email)
PROVIDER=$(terraform output -raw workload_identity_provider)
MODE=$(terraform output -raw enforce_mode)

# projects/NUMBER/locations/global/workloadIdentityPools/POOL/providers/NAME
POOL=$(awk -F/ '{print $6}' <<<"$PROVIDER")

# enforce writes spec; dry_run writes dryRunSpec. Asserting the wrong one
# passes a project where nothing is actually enforced.
if [[ "$MODE" == "enforce" ]]; then
  SPEC=".spec"
else
  SPEC=".dryRunSpec"
fi

echo "=== org policy constraints (mode: $MODE) ==="

for constraint in storage.uniformBucketLevelAccess storage.publicAccessPrevention \
                  iam.disableServiceAccountKeyCreation compute.requireOsLogin; do
  check ORG "$constraint" "true" \
    "$(gcloud org-policies describe "$constraint" --project="$PROJECT" --format=json 2>/dev/null \
       | jq -r "${SPEC}.rules[0].enforce // empty")"
done

echo "=== data access logging ==="

# AU-3 and AU-12. ADMIN_READ is on by default; DATA_READ and DATA_WRITE are
# the ones that cost money and are therefore the ones people quietly skip.
IAM=$(gcloud projects get-iam-policy "$PROJECT" --format=json 2>/dev/null)
```

```
[[ -z "$IAM" ]] && IAM='{}'
for service in storage.googleapis.com cloudkms.googleapis.com iam.googleapis.com; do
  for logtype in ADMIN_READ DATA_READ DATA_WRITE; do
    check AU-12 "$service $logtype" "$logtype" \
      "$(jq -r --arg s "$service" --arg t "$logtype" \
           '[.auditConfigs[]? | select(.service==$s) | .auditLogConfigs[]? | select(.logType==$t)
| .logType][0] // empty' \
           <<<"$IAM")"
  done
done

echo "=== workload identity ==="

check AC-3 "pool state" "ACTIVE" \
  "$(gcloud iam workload-identity-pools describe "$POOL" --location=global \
       --project="$PROJECT" --format=json 2>/dev/null | jq -r '.state // empty')"

# AC-6. The gate service account must not carry primitive roles. Anything
# matching roles/owner, roles/editor or roles/viewer here defeats the point of
# a purpose-built identity.
PRIMITIVE=$(jq -r --arg sa "serviceAccount:$SA" \
  '[.bindings[]? | select(.members[]? == $sa) | .role
    | select(. == "roles/owner" or . == "roles/editor" or . == "roles/viewer")] | length' \
  <<<"$IAM")
check AC-6 "primitive roles on gate SA" "0" "$PRIMITIVE"

echo "=== enforcement ==="

# The service account key constraint, actually refusing.
#
# This one attempts a WRITE, which needs care: if the constraint is missing the
# attempt succeeds and leaves a real, long-lived private key on disk and in the
# project. So it is cleaned up immediately and reported as a failure, which is
# what it is.
KEYFILE=$(mktemp /tmp/cgep-sa-key.XXXXXX.json)
if gcloud iam service-accounts keys create "$KEYFILE" \
     --iam-account="$SA" --project="$PROJECT" >/dev/null 2>&1; then
  KEY_ID=$(jq -r '.private_key_id // empty' "$KEYFILE" 2>/dev/null)
  if [[ -n "$KEY_ID" ]]; then
    gcloud iam service-accounts keys delete "$KEY_ID" --iam-account="$SA" \
      --project="$PROJECT" --quiet >/dev/null 2>&1 \
      && fail "AC-6" "key creation SUCCEEDED and was rolled back. The constraint is not enforcing." \
      || fail "AC-6" "key creation SUCCEEDED and could not be rolled back. Delete key
$KEY_ID by hand, now."
  else
    fail "AC-6" "key creation SUCCEEDED. The constraint is not enforcing."
  fi
else
  pass "AC-6" "service account key creation was refused"
fi
rm -f "$KEYFILE"

echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the project."
```

```
else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-5-4"
mkdir -p "$EVIDENCE"
gcloud projects get-iam-policy "$TF_VAR_gcp_project" --format=json \
  > "$EVIDENCE/iam-policy.json"

{
  echo "### service account key creation, expect FAILED_PRECONDITION"
  gcloud iam service-accounts keys create /tmp/key.json \
    --iam-account="$SA_EMAIL" --project="$TF_VAR_gcp_project" 2>&1
} | tee "$EVIDENCE/enforcement-denied.txt"
```

Portfolio submission checklist

- [ ] `terraform/baselines/gcp/` committed.
- [ ] At least one workflow using WIF. **No service account JSON key anywhere in the repo or its history.**
- [ ] `evidence/lab-5-4/iam-policy.json` capturing the Data Access log configuration.
- [ ] `evidence/lab-5-4/enforcement-denied.txt`, the transcripts of both rejected actions.
- [ ] README notes the "Data Access logs are off by default" lesson, and names VPC Service Controls as a known gap.

Troubleshooting

- **Org Policy propagation latency.** First-apply changes take 5 to 10 minutes. A test run immediately after `terraform apply` may briefly succeed. That is propagation, not a broken policy.
- **`PERMISSION_DENIED` on the WIF provider.** You need `roles/iam.workloadIdentityPoolAdmin` specifically. Being project Owner is not sufficient.
- **`attribute.repository` mismatch.** GitHub's `assertion.repository` is the literal `OWNER/REPO`. Case, spelling, and the slash all matter. An opaque `PERMISSION_DENIED` from `auth@v2` is almost always this.
- **`policySpec is not supported`.** Enable the v2 API: `gcloud services enable orgpolicy.googleapis.com`.
- **`dry_run_spec` unrecognized.** Your `google` provider is too old. Upgrade, or accept that you are enforcing without an observation period and say so.
- **Data Access log cost.** In a busy project these ingest gigabytes per day. Start with `storage.googleapis.com` alone before adding KMS and IAM.

Cleanup

```
terraform destroy -auto-approve
```

Three things that will surprise you:

1. **WIF pools enter a 30-day soft delete.** You cannot recreate a pool with the same ID until it expires, or you `gcloud iam workload-identity-pools undelete` and then delete again with `--purge`.
2. **Disabling Org Policy does not retroactively un-enforce.** Buckets created with uniform access stay that way. The policy only stops blocking new violations.
3. **Audit config removal is not retroactive either.** Logs already ingested continue to bill for storage until their retention expires.

How this feeds the capstone

The WIF pattern is your AWS-OIDC equivalent for anything GCP-touching. If your capstone pipeline reaches into GCP for any reason, it uses WIF, never a key.

The Org Policy layer is the preventive tier above your Rego, which is detective. Saying that clearly in your write-up, with the three-layer table from the Concept section, is one of the strongest paragraphs you can write about defense in depth, because it shows you know which of your controls acts when.

The Data Access logs feed your OSCAL component's AU-2 and AU-3 statements: enabled per service, with the IAM policy JSON as evidence. And the audit-mode-then-enforce rollout in Step 1 is the answer to the write-up question "how would you introduce this control without breaking the engineering team," which the capstone brief asks in the form "without slowing the engineering team down."

Revision history

These notes

- Org Policy constraints roll out in `dry_run_spec` first and promote to `spec`, so a preventive control can be introduced without breaking a team that has never heard of you.
- Corrected the audit-mode guidance: omitting the rules block does not audit, it leaves the policy not in effect at all.
- Added `storage.publicAccessPrevention` alongside the original three constraints.
- Workload Identity Federation documented as two layers, the provider `attribute_condition` and the binding `principalSet`, rather than one.
- Named VPC Service Controls and CMEK org constraints as known gaps rather than leaving them unmentioned.

The official labs

Initial release: three constraints, straight to enforce, WIF documented as a single layer.

CGE-P LAB NOTES

Lab 6.1: Introduction to OSCAL

OSCAL is NIST's machine-readable format for controls, profiles, components, system security plans, and assessments. The point is not the format. The point is that an assessor can traverse from a catalog, to a profile, to a component, to an evidence URI, and verify your control **without ever talking to you**. The audit becomes a graph traversal.

This lab writes the smallest useful piece of that graph, and it **generates** the control mappings from the policies that enforce them rather than retyping them by hand.

For reading. Code lines wrap to fit the page; copy code from docs/06_01_introduction_to_oscal.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/06_01_introduction_to_oscal.md` in your clone, not from the PDF.

Learning objectives

- Author a valid Component Definition and a Profile, validated by `trestle`.
- Use `implementation-status` honestly: `implemented`, `partial`, `inherited`.
- Generate `implemented-requirement` entries from Lab 3.3's `policy-catalog.json`.
- **Verify that every evidence URI resolves**, because OSCAL will not check that for you.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- Python `>= 3.10`, `pip install compliance-trestle`.
- Lab 2.3 and/or Lab 2.4 module on disk. We describe them in OSCAL.
- Lab 2.5 vault exists, with at least one bundle, so evidence URIs resolve.
- Lab 3.3's `evidence/lab-3-3/policy-catalog.json`.

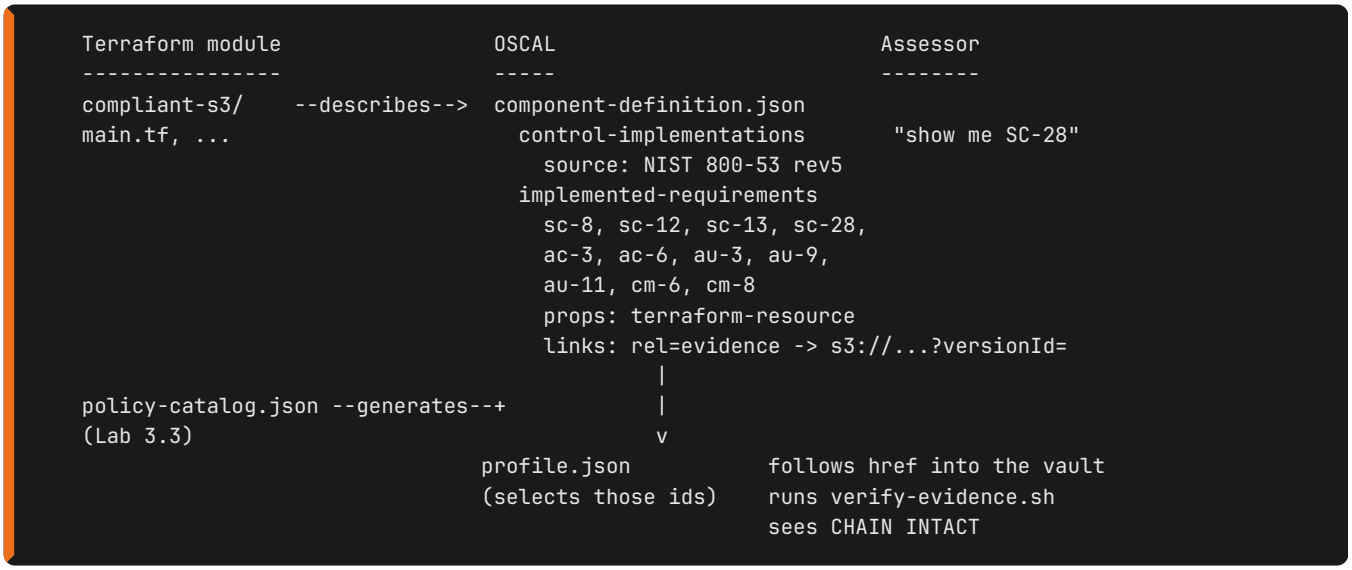
Estimated time & cost

- 75 to 90 minutes.
- Free. No cloud calls beyond fetching the NIST catalog and one `head-object` per evidence URI.

The five OSCAL models

Model	Describes	Built here
Catalog	A library of controls (NIST 800-53 Rev 5)	No. We link to NIST's.
Profile	A subset selected from one or more catalogs	Yes, minimal.
Component Definition	How a component implements specific controls	Yes, the centerpiece.
SSP	A whole system's controls and components	Capstone stretch goal.
Assessment Plan / Results	What the assessor planned and found	Out of scope.

Architecture



Step-by-step walkthrough

Step 1 Initialize a trestle workspace

```
pip install compliance-trestle
mkdir lab-6-1 && cd lab-6-1
trestle init
trestle version    # note the OSCAL version it pins
```

Write that version down. Catalog, profile, and component must all share an `oscal-version`, and mixing them is the single most common validation failure.

Step 2 Generate the requirements from the policy catalog

This is the change these notes make, and it is the whole idea.

The obvious approach is to hand-type each `implemented-requirement`, having already typed the same control ID into the Rego metadata in Lab 3.3. That is two copies of one mapping, in two languages, drifting apart the moment either changes.

Generate one from the other instead:

```
#!/usr/bin/env python3
# scripts/gen-oscal-requirements.py
"""Build OSCAL implemented-requirements from the Lab 3.3 policy catalog.

Each Rego policy declares its control_id in a # METADATA block. That is the
single source of truth; this script projects it into OSCAL so the mapping is
authored once and never retyped.
"""
import json, sys, uuid, pathlib

catalog_path = pathlib.Path(sys.argv[1])          # evidence/lab-3-3/policy-catalog.json
evidence_uri = sys.argv[2]                        # s3://VAULT/runs/ID/bundle.tar.gz?versionId=...

# Controls this component satisfies by means OTHER than a Rego policy, plus
# any whose status is not a plain "implemented". Everything here is a claim
# you must be able to defend out loud.
EXTRA = {
    "sc-12": ("implemented", "aws_kms_key.bucket with enable_key_rotation = true."),
    "sc-13": ("implemented", "SSE-KMS with a customer-managed key."),
    "ac-6":  ("implemented", "aws_s3_bucket_ownership_controls = BucketOwnerEnforced; ACLs disabled.")
,
    "au-11": ("implemented", "aws_s3_bucket_lifecycle_configuration expires logs at 90 days."),
    "cm-8":  ("implemented", "Provider default_tags applies ComplianceScope to every taggable
resource."),
    "au-6":  ("partial",     "Athena queries authored in Lab 5.2. Scheduled review not yet
automated."),
}

TERRAFORM_RESOURCE = {
    "sc-8":  "aws_s3_bucket_policy.primary",
    "sc-12": "aws_kms_key.bucket",
    "sc-13": "aws_s3_bucket_server_side_encryption_configuration.primary",
    "sc-28": "aws_s3_bucket_server_side_encryption_configuration.primary",
    "ac-3":  "aws_s3_bucket_public_access_block.primary",
    "ac-6":  "aws_s3_bucket_ownership_controls.primary",
    "au-3":  "aws_s3_bucket_logging.primary",
    "au-9":  "aws_s3_bucket_versioning.log",
    "au-11": "aws_s3_bucket_lifecycle_configuration.log",
    "cm-6":  "provider.aws.default_tags",
    "cm-8":  "provider.aws.default_tags",
}

policies = json.loads(catalog_path.read_text())
seen, reqs = set(), []

def add(control_id, description, status, policy_pkg=None):
    if control_id in seen:
        return
    seen.add(control_id)
    props = [{"name": "implementation-status", "value": status}]
    if control_id in TERRAFORM_RESOURCE:
        props.append({"name": "terraform-resource", "value": TERRAFORM_RESOURCE[control_id]})
    if policy_pkg:
        props.append({"name": "enforced-by-policy", "value": policy_pkg})
    reqs.append({
        "uuid": str(uuid.uuid4()),
```

```
        "control-id": control_id,
        "description": description,
        "props": props,
        "links": [{
            "rel": "evidence",
            "href": evidence_uri,
            "text": "Signed pipeline bundle containing terraform plan.json.",
        }],
    })

for p in policies:
    cid = (p.get("control") or "").lower()
    if not cid:
        continue
    add(cid,
        f"{p['title']} Enforced in the pipeline by Rego policy {p['package']}, "
        f"severity {p['severity']}. Remediation: {p.get('remediation', 'see policy')}",
        "implemented",
        p["package"])

for cid, (status, desc) in EXTRA.items():
    add(cid, desc, status)

json.dump(sorted(reqs, key=lambda r: r["control-id"]), sys.stdout, indent=2)
```

```
python3 scripts/gen-oscal-requirements.py \
  evidence/lab-3-3/policy-catalog.json \
  "s3://YOUR_VAULT/runs/RUN_ID/bundle.tar.gz?versionId=VERSION_ID" \
  > /tmp/requirements.json
```

Notice what the `EXTRA` dictionary is really for. It is the list of controls you claim on grounds **other** than an automated policy check, and every entry is something an assessor can ask you to justify. Keeping it small and explicit is the discipline. A component whose every claim came from a passing policy is a strong component; one whose claims are mostly hand-written prose is a document.

Step 3 The component definition

```
trestle create -t component-definition -o compliant-s3-v2 -x json
```

Then replace the skeleton. The shape, with the generated requirements spliced in:

```
{
  "component-definition": {
    "uuid": "GENERATED-UUID-V4",
    "metadata": {
      "title": "compliant-s3 module",
      "last-modified": "2026-08-17T18:00:00.000000+00:00",
      "version": "2.0.0",
      "oscal-version": "1.2.1",
```

```
      "parties": [
        {
          "uuid": "PARTY-UUID-V4",
          "type": "organization",
          "name": "Your organization"
        },
        {
          "uuid": "CSP-UUID-V4",
          "type": "organization",
          "name": "Amazon Web Services"
        }
      ]
    },
    "components": [
      {
        "uuid": "COMPONENT-UUID-V4",
        "type": "software",
        "title": "compliant-s3",
        "description": "Reusable Terraform pattern for an AWS S3 primary bucket plus a dedicated
access-log bucket. Enforces SSE-KMS with a customer-managed key, TLS-only access, versioning on both
buckets, full public access block, ACLs disabled, lifecycle retention, access logging, and required
compliance tags.",
        "purpose": "Provide a compliant-by-default S3 primitive that any team can adopt with three
lines of consumer Terraform.",
        "responsible-roles": [
          { "role-id": "provider", "party-uuids": ["PARTY-UUID-V4"] }
        ],
        "control-implementations": [
          {
            "uuid": "CI-UUID-V4",
            "source": "https://raw.githubusercontent.com/usnistgov/oscal-content/v1.2.1/nist.gov/
SP800-53/rev5/json/NIST_SP-800-53_rev5_catalog.json",
            "description": "NIST 800-53 Rev 5 controls satisfied by this module.",
            "implemented-requirements": [ "<<< generated by gen-oscal-requirements.py >>>" ]
          }
        ]
      }
    ]
  }
}
```

Splice them in:

```
jq --slurpfile reqs /tmp/requirements.json \

'.["component-definition"].components[0]["control-implementations"][0]["implemented-requirements"] =
$reqs[0]' \
  component-definitions/compliant-s3-v2/component-definition.json > /tmp/cd.json \
  && mv /tmp/cd.json component-definitions/compliant-s3-v2/component-definition.json
```

Anchor the catalog `source` to a tag, not `main`. NIST moves that branch, so an assessor opening your component a year from now would get whatever is current rather than the catalog you actually assessed against. Same argument as pinning a provider version, applied to a control catalog. Check the tag exists: several plausible-looking ones return 404.

Generate UUIDs properly. OSCAL requires v4 (`xxxxxxxx-xxxx-4xxx-yxxx-...` where `y` is 8/9/a/b). Run `python3 -c "import uuid; print(uuid.uuid4())"`. `trestle validate` rejects the wrong shape with a regex error that tells you nothing useful.

Step 4 Inherited controls, done honestly

If you are also describing Lab 2.4's GCP module, SC-8 is satisfied by the platform rather than your code. OSCAL has a way to say that, and using it is the difference between an accurate component and an overclaim:

```
{
  "uuid": "REQ-UUID-V4",
  "control-id": "sc-8",
  "description": "Transmission confidentiality is provided by the Google Cloud Storage API, which
accepts only TLS connections. No module-level control is implemented; this control is inherited from
the cloud service provider.",
  "props": [
    { "name": "implementation-status", "value": "inherited" },
    { "name": "leveraged-authorization", "value": "Google Cloud Platform" }
  ],
  "responsible-roles": [
    { "role-id": "cloud-service-provider", "party-uuids": ["CSP-UUID-V4"] }
  ]
}
```

Three statuses, three meanings, and students conflate them constantly:

Status	Means	The question it invites
`implemented`	Your code enforces it	"Show me the line."
`partial`	Some of it, and you can say which part	"What is missing, and when?"
`inherited`	The provider enforces it; you rely on that	"Where is their authorization?"

AU-6 in this component is `partial` on purpose. Lab 5.2 authored the Athena queries; nothing schedules them. Claiming `implemented` there would be claiming a control on the strength of having built the tooling for it.

Step 5 Validate

```
trestle validate -f component-definitions/compliant-s3-v2/component-definition.json
```

```
VALID: Model .../component-definition.json passed the Validator
```

Step 6 The profile

```
trestle create -t profile -o cge-p-minimum -x json
```

```
{
  "profile": {
    "uuid": "PROFILE-UUID-V4",
    "metadata": {
      "title": "CGE-P minimum control selection",
      "last-modified": "2026-08-17T18:00:00.000000+00:00",
      "version": "2.0.0",
      "oscal-version": "1.2.1"
    },
    "imports": [
      {
        "href": "https://raw.githubusercontent.com/usnistgov/oscal-content/v1.2.1/nist.gov/SP800-53/
rev5/json/NIST_SP-800-53_rev5_catalog.json",
        "include-controls": [
          {
            "with-ids": [
              "ac-3", "ac-6", "au-3", "au-6", "au-9", "au-11",
              "cm-6", "cm-8", "sc-8", "sc-12", "sc-13", "sc-28"
            ]
          }
        ]
      }
    ],
    "merge": { "as-is": true }
  }
}
```

```
trestle validate -f profiles/cge-p-minimum/profile.json
trestle profile-resolve -n cge-p-minimum -o cge-p-minimum-resolved
```

The profile and the component must agree: **every control in the component must appear in the profile.** Step 7 checks that mechanically, because doing it by eye stops working around control number six.

Step 7 Verify the graph, since OSCAL will not

This usually shows up as a footnote: OSCAL does not validate that hrefs resolve, so a broken URI is silently a useless attestation.

It belongs in the lab rather than the footnotes. A component definition whose evidence links 404 passes `trestle validate` cleanly. It is a perfectly valid document describing nothing.

```
#!/usr/bin/env bash
# scripts/verify-oscal-graph.sh <component-definition.json> <profile.json>
set -euo pipefail

CD="${1:?usage: verify-oscal-graph.sh <component-def> <profile>}"
PROFILE="${2:?}"
PROFILE_ARG="${AWS_PROFILE:+--profile $AWS_PROFILE}"
FAIL=0

echo "=== 1. Every evidence href resolves ==="
while IFS= read -r href; do
  case "$href" in
    s3://*)
      rest="${href#s3://}"
      bucket="${rest%%/*}"
      keypart="${rest#*/}"
      key="${keypart%%\?*}"
      version=""
      [[ "$keypart" == *"versionId="* ]] && version="--version-id ${keypart##*versionId=}"
      if aws $PROFILE_ARG s3api head-object --bucket "$bucket" --key "$key" $version >/dev/null 2>&1;
then
        echo "  OK      $href"
      else
        echo "  BROKEN  $href"; FAIL=1
      fi
      ;;
    http://*|https://*)
      if curl -fsSL -o /dev/null --max-time 20 "$href"; then
        echo "  OK      $href"
      else
        echo "  BROKEN  $href"; FAIL=1
      fi
      ;;
    *) echo "  SKIP    $href (unrecognized scheme)" ;;
  esac
done < <(jq -r '.. | objects | select(.rel? == "evidence") | .href' "$CD" | sort -u)

echo "=== 2. Catalog source resolves and is version-pinned ==="
SRC=$(jq -r '.. | objects | select(has("source")) | .source' "$CD" | head -1)
curl -fsSL -o /dev/null --max-time 30 "$SRC" \
  && echo "  OK      $SRC" || { echo "  BROKEN  $SRC"; FAIL=1; }
case "$SRC" in
  */main/*) echo "  WARN    catalog source tracks 'main'; pin to a tag"; ;;
esac

echo "=== 3. Every component control appears in the profile ==="
comm -23 \
  <(jq -r '.. | objects | select(has("control-id")) | .["control-id"]' "$CD" | sort -u) \
  <(jq -r '.profile.imports[]."include-controls"[]."with-ids"[]' "$PROFILE" | sort -u) \
  | while read -r missing; do
      echo "  MISSING from profile: $missing"; FAIL=1
    done
```

```
echo "=== 4. No placeholder UUIDs survived ==="
if grep -qE '(GENERATED|PARTY|COMPONENT|CI|REQ|PROFILE|CSP)-UUID-V4' "$CD" "$PROFILE"; then
  echo "  FAIL: placeholder UUIDs still present"; FAIL=1
else
  echo "  OK"
fi

[[ $FAIL -eq 0 ]] && echo && echo "OSCAL GRAPH INTACT" || { echo; echo "OSCAL GRAPH BROKEN"; exit 1; }
```

Run it, and put it in CI next to `trestle validate`. Validation proves the document is well-formed. This proves it is **true**.

Step 8 Demonstrate the traversal

Pick `sc-28` in the component. Follow `links[rel=evidence].href` into the vault. Run Lab 4.4's script:

```
EVIDENCE_VAULT=YOUR_VAULT bash scripts/verify-evidence.sh RUN_ID
```

`CHAIN INTACT`.

The catalog, the profile, the implementation statement, the evidence URI, and the signed bundle are now one connected graph. An assessor reading the OSCAL verifies your control without you in the room, which is the entire thesis of this course stated as a file format.

Verification

- `trestle validate` returns `VALID` for the component definition **and** the profile.
- `trestle profile-resolve` produces a resolved profile.
- `scripts/verify-oscal-graph.sh` prints `OSCAL GRAPH INTACT`.
- At least one evidence URI resolves to a real signed object, by version.
- `implementation-status` values are honest: nothing marked `implemented` that you cannot demonstrate.

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-6-1"
mkdir -p "$EVIDENCE"
trestle validate -f component-definitions/compliant-s3-v2/component-definition.json \
  2>&1 | tee "$EVIDENCE/trestle-validate.txt"
bash scripts/verify-oscal-graph.sh 2>&1 | tee "$EVIDENCE/graph-verify.txt"
```

`trestle validate` proves the document is well formed. Only the graph check proves it describes something real, which is why both go in.

Portfolio submission checklist

- [ ] `oscal/components/<your-component>.json`, trestle-validated.
- [ ] `oscal/profiles/cge-p-minimum.json`, validated.
- [ ] `scripts/gen-oscal-requirements.py` and `scripts/verify-oscal-graph.sh` committed.
- [ ] `evidence/lab-6-1/trestle-validate.txt` and `evidence/lab-6-1/graph-verify.txt`.
- [ ] `oscal/README.md` naming which module each component describes, where evidence lives, and **which controls are `partial` or `inherited` and why**.

Troubleshooting

- **`string does not match regex` on UUIDs.** OSCAL requires v4. Generate them; do not hand-write.
- **Missing required fields.** `trestle describe -t component-definition -n <name>` prints the schema requirements verbosely.
- **Evidence URIs that do not resolve.** OSCAL does not check. That is what Step 7 is for.
- **Catalog import fails.** NIST's URLs move. Anchor to a tag rather than `main`.
- **Version mismatch.** Catalog, profile, and component must share `oscal-version`. Check `trestle version`.
- **`jq --slurpfile` produced a nested array.** `$reqs[0]` unwraps it. Without the index you get `[[...]]` and trestle fails on the type.

Cleanup

OSCAL is JSON in your repo. Nothing in the cloud. Commit and move on.

How this feeds the capstone

```
oscal/
  components/YOUR_COMPONENT.json      # what you built
  profiles/cge-p-minimum.json           # the controls you claim
```

The evidence hrefs point at the latest signed bundle in your vault, written by the Lab 4.3 and 4.4 pipeline. The chain ends in the vault.

Two things will distinguish your submission. **`verify-oscal-graph.sh` in CI**, because the capstone brief warns that "an OSCAL file that doesn't actually describe your system is worse than no OSCAL file," and this script is the mechanical answer to that. And **honest `implementation-status` values**, because the graders check whether the OSCAL describes the system you built, and a component claiming twelve `implemented` controls where two are really `partial` is the exact failure the brief calls out as copy-paste OSCAL.

When you retarget from NIST 800-53 to HIPAA, SOC 2, or CMMC for the capstone, change the `control_id` in each Rego metadata block and the `source` catalog URI, then re-run the generator. The mapping was authored once. That is why it was worth generating.

Revision history

These notes

- `implemented-requirement` entries are generated from the Lab 3.3 policy catalog rather than hand-typed, so each control mapping is authored once in the Rego annotation that enforces it.
- Added `verify-oscal-graph.sh`: checks that every evidence href resolves, the catalog source is reachable and version-pinned, every component control appears in the profile, and no placeholder UUIDs survive. `trestle validate` proves the document is well-formed; this proves it is true.
- `implementation-status` is used honestly, including `partial` and `inherited`, with the distinction spelled out.
- `oscal-version` and the catalog tag aligned to the version `trestle` targets, and the guidance now says to confirm the tag exists.
- Control coverage grew from four to fourteen as the Chapter 2 baseline grew.

The official labs

Initial release: four controls, requirements typed by hand, catalog anchored to `main`, no graph verification.

CGE-P LAB NOTES

Capstone Project Brief

You've reached the capstone. Everything you've learned (IaC, policy as code, CI/CD, evidence management, OSCAL) comes together in one GitHub repo you build yourself.

You ship the repo, you pass the capstone.

For reading. Code lines wrap to fit the page; copy code from docs/07_01_capstone_brief.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/07_01_capstone_brief.md` in your clone, not from the PDF.

The on-ramp: labs are optional, encouraged, and the same skills

The capstone tests the skills the chapter labs taught. The labs are optional and strongly encouraged. Every lab produces an artifact you carry directly into your capstone repo.

If you skipped the labs you can still pass. This brief assumes you didn't. Read "What you've already built" at the bottom before deciding.

One exception, and it is not optional in practice: Lab 2.2, the remote state backend. Layer 3 requires the pipeline to apply on merge to `main`. A GitHub Actions runner with local state has no state, so the first apply after a merge either collides with your existing resources or builds a second copy of your infrastructure. If you skip every other lab, do 2.2. It takes twenty minutes and Layer 3 is not completable without it.

The system

You are the first GRC engineer at **Acme Health**, a 50-person telehealth company. Engineering shipped a Patient Intake API. It works. It is not audit-defensible. The CTO wants it audit-defensible in 30 days, without slowing engineering down.

The starter is real code in a real repo: `GRCEngClub/cgep-app-starter`. Fork it, deploy it, then govern it.

Step zero, the deploy gate:

```
git clone https://github.com/GRCEngClub/cgep-app-starter
cd cgep-app-starter
make deploy AWS_PROFILE=cgep
make test   AWS_PROFILE=cgep
```

If `make test` returns `{"submission_id": "...", "status": "received"}`, you have something to govern. If not, fix that first. Real GRC engineers inherit working systems; demonstrating you can stand one up is the floor.

The brief

Acme is pursuing three flags at once: **HIPAA Security Rule** (PHI is at stake), **SOC 2 Type II** (an enterprise customer is asking), and **CMMC Level 2** (a federal pilot is on the table). You will not satisfy all three. Pick one as your **primary framework**, defend the choice in your write-up, and structure every layer below around it.

The starter ships with eight named, intentional gaps. Design and build a system around it that closes them and produces evidence that the system stays closed.

See `GAPS.md` and `FRAMEWORKS.md` in the starter.

Four layers, one repo

Layer 0: the backend (twenty minutes, and everything depends on it)

An S3 state backend with KMS encryption, versioning, and locking. Lab 2.2. Every workspace below stores state here with its own `key`.

This is not scored as its own layer. It is scored by everything failing without it.

Layer 1: Terraform baseline (around the starter)

The starter gives you the workload. You add the GRC baseline that makes it defensible.

Required new resources:

- **KMS key(s) you own, with rotation enabled.** Bring the starter's S3 uploads bucket and DynamoDB table under your CMK.
- **S3 evidence bucket with Object Lock** (COMPLIANCE or GOVERNANCE, your call, defend it). Versioned, encrypted with your KMS key. Every pipeline run lands here.
- **CloudTrail** trail: multi-region, log-file validation on, writing management events to a dedicated bucket, and **S3 data events scoped to the evidence vault**.
- **Required hardening overrides** on the starter's resources to close `GAPS.md` (for example `aws_s3_bucket_server_side_encryption_configuration` with `sse_algorithm = "aws:kms"`, `aws_lambda_function.intake.vpc_config`, IAM tightened from `dynamodb:*`).

Every S3 bucket you create or harden must meet the curriculum baseline: CMK encryption, versioning, all four public-access-block flags, ACLs disabled, a bucket policy denying `aws:SecureTransport = false`, and lifecycle rules. That is Lab 2.3's `compliant-s3` pattern applied to buckets you did not create.

Use the starter's VPC. Do not build a second one. Closing GAP-05 means moving the Lambda *into* the VPC the starter already created.

Don't pad. A small Terraform that closes five gaps cleanly beats a large one that adds resources without governing the starter.

Layer 2: OPA policy suite

Five or more Rego policies enforcing controls from your **declared primary framework**.

Each policy:

- Has a metadata block naming the framework, control ID(s), severity, and remediation.
- Has its own `_test.rego` with passing and failing fixtures.

- Catches a real gap from `GAPS.md`. Not a generic tag check.
- Cites the control ID in the deny message so a developer reading the failed PR knows what to fix.

New requirement: your suite must be mutation-tested. Commit `scripts/mutation-test.sh` from Lab 3.3 and its output. A policy no test constrains is a policy that cannot fail, and a gate full of those passes every check while enforcing nothing. We will look for this.

New requirement: your gate must fail closed when it has nothing to evaluate. We run your pipeline with the policy directory removed. If it reports success, Layer 2 fails regardless of how good the policies are. Lab 3.4's empty-results guard is the fix.

Conftest runs the suite against your plan in the pipeline. We will also run your policies against a copy of the starter with one of your fixed gaps re-introduced, and confirm the gate fires.

Layer 3: GitHub Actions pipeline

One workflow. Five named steps, in order:

1. **Plan** the Terraform.
2. **Policy check** with Conftest.
3. **Apply** on merge to `main`.
4. **Sign** the evidence bundle with Cosign (keyless, GitHub OIDC).
5. **Upload** the signed bundle to the evidence vault from Layer 1.

Two pull requests must exist in your history: one that passed and merged, one that failed the policy gate and was blocked. Both are evidence the gate works.

Three requirements that separate a working pipeline from a demo:

- **Branch protection must be on, with `enforce_admins: true`.** A red check anyone can merge past is a suggestion, not CM-3. Include the `gh api` output or a screenshot.
- **Plan and apply must use different IAM roles.** The plan role is read-only plus state access. The apply role is scoped to what it changes and trusted only from `repo:OWNER/REPO:environment:production`, behind a GitHub environment protection rule. This is the single highest-value paragraph in most write-ups.
- **Actions pinned by commit SHA**, not by tag. Tags move.

Layer 4: OSCAL component

One `component-definition.json` describing what you actually built, the starter plus the controls you wrapped around it.

- Real v4 UUIDs.
- `control-implementation.source` points at your declared framework's catalog, **anchored to a tag, not `main`**.
- Implementation statements reference real Terraform addresses or ARNs as `props`.
- Evidence links resolve to real signed objects in your vault, by `versionId`.

- A profile selecting the controls your component implements.

New requirement: honest `implementation-status`. Use `implemented`, `partial`, and `inherited` accurately. A control you enforce with code is `implemented`. A control you have tooling for but no scheduled review is `partial`. A control the cloud provider supplies is `inherited`, with the provider named. A component claiming twelve `implemented` controls where three are really `partial` is the copy-paste OSCAL this brief has always warned about, just harder to spot.

New requirement: commit `scripts/verify-oscal-graph.sh` from Lab 6.1, and its output. `trestle validate` proves your document is well-formed. It does not prove a single href resolves. We check both.

If the OSCAL describes a system you didn't build, or cites a framework whose catalog you didn't declare, the layer fails.

The output

A working evidence vault. Every push to `main` produces a signed, timestamped artifact in immutable storage, automatically. That is the whole point.

Three deliverables, due in 30 days

Artifact	Format	What it proves
Public GitHub repo	Terraform + Rego + YAML	You can build the pipeline end to end.
Evidence bundle	Signed `.tar.gz` in S3	Your pipeline produces audit-grade evidence.
Write-up	`WRITEUP.md`	You can explain your design to a stakeholder.

Submission is the repo URL plus the commit SHA you want graded.

The write-up is not optional. Sections: design decisions, control coverage, trade-offs, what you'd do with another sprint, what you didn't get to. **Honest gaps don't lose points. Hand-waving does.**

Three things we score hard

1. **End-to-end integration.** Open a PR, the gate runs, the gate decides whether apply happens, apply triggers signing, signing uploads to the vault. Not four disconnected demos.
2. **Working evidence pipeline.** We pull a recent run and verify it: Cosign signature against the public Sigstore log **with a certificate identity that actually constrains the signer**, SHA-256 recompute, and Object Lock retention. All three must hold. A verify script using `--certificate-identity-regexp '.*'` accepts a bundle signed by any repository on GitHub and does not count as authenticity.

3. **Clear design reasoning.** Your write-up explains *why* you chose each tool and what trade-offs you accepted. Not just *what*.

Three mistakes to avoid

1. **Too much scope.** Thirty resources cleanly integrated beats two hundred bolted on.
2. **Copy-paste OSCAL.** A file that doesn't describe your system is worse than no file. Authenticity over completeness.
3. **Unsigned evidence.** A plan that isn't signed and immutably stored demonstrates no chain of custody.

And one more, added here because it is the most common way a good submission fails:

1. **Controls you claimed but did not implement.** The most frequent instance is AU-6, audit review and analysis, claimed because logs are being collected. Collecting is AU-3 and AU-11. Review requires something that reads them, on a cadence, with a named reviewer. Claim `partial` and say what's missing. **We would rather read an honest `partial` than a confident `implemented` that falls apart in one question.**

What you decide (and defend)

- **Primary framework**: HIPAA Security Rule, SOC 2 TSC, or CMMC Level 2. Pick one.
- AWS region.
- COMPLIANCE vs GOVERNANCE on the Object Lock vault.
- Whether the pipeline applies on merge to `main`, or after a manual approval gate post-merge.
- Single account vs separate evidence-vault account (cleaner: separate; acceptable for 30 days: single).
- Which gaps to close in Terraform vs enforce only in policy. Both valid; defend it.
- **Whether your evidence bundles include raw `terraform state`.** State holds every attribute in plaintext, including secrets, and Object Lock makes an upload undeletable. Lab 2.5 defaults to capturing the plan instead. Whatever you choose, say why in the write-up. A reviewer will ask.

Suggested 30-day plan

- **Week 1 · Design.** Pick your system. Map controls. Sketch the repo. Open a one-page design doc; it becomes the spine of `WRITEUP.md`. **Stand up the Lab 2.2 backend on day one**, before writing any other Terraform, so you never migrate state under a deadline.
- **Week 2 · Build infra.** Terraform baseline. Evidence bucket with Object Lock. KMS. CloudTrail. Apply once by hand from a feature branch. Don't start the pipeline until the baseline applies clean.
- **Week 3 · Policy + pipeline.** Five Rego policies, mutation-tested. The workflow. Signing wired. Open the green PR. Open a second PR that violates a policy and watch it go red. Run the inert-gate test.

- **Week 4 · OSCAL + write-up.** Author the component. Validate with `trestle`, then verify the graph. Wire evidence URIs to real vault objects by version. Write the reflection. Submit.

If you're in week three with no baseline running, cut scope. Trade workload resources for a working pipeline. Every time.

Submission checklist

- [ ] Your repo is a fork or clear derivative of `cgep-app-starter`. The starter's resources are still present and runnable.
- [ ] Remote state backend in use; no `terraform.tfstate` committed except a documented bootstrap.
- [ ] Declared **primary framework** named in `WRITEUP.md`'s first paragraph and in `control-implementation.source`.
- [ ] `terraform/` adds KMS keys, the Object Lock evidence bucket, CloudTrail with data events on the vault, and the gap-closing overrides.
- [ ] Every bucket meets the baseline: CMK, versioning, four PAB flags, ACLs disabled, TLS-deny policy, lifecycle.
- [ ] `policies/` has 5+ Rego policies with tests. `opa test ./policies` passes. Each cites a control ID from your framework.
- [ ] `scripts/mutation-test.sh` committed, output shows every policy killed.
- [ ] The gate fails closed with `policies/` removed. Evidence of that run included.
- [ ] `.github/workflows/grc-gate.yml` runs Plan, Policy check, Apply, Sign, Upload. Actions SHA-pinned.
- [ ] Branch protection on with `enforce_admins: true`.
- [ ] Separate IAM roles for plan and apply.
- [ ] One green PR and one red PR visible in history.
- [ ] At least one signed bundle in the vault. Cosign verifies **against a constrained identity**. SHA matches. Retention active.
- [ ] `oscal/components/<your-component>.json` validates with `trestle`.
- [ ] `scripts/verify-oscal-graph.sh` passes; every evidence href resolves.
- [ ] `WRITEUP.md` covers framework choice, gap remediation, trade-offs, and what you didn't get to.
- [ ] `README.md` is short, with verification instructions for the grader.

What you've already built (if you did the labs)

If you did them, you don't start from scratch, you assemble.

Lab	What it gives you
2.2 Remote State Backend	Layer 0 outright. Without it, Layer 3 cannot be completed.
2.3 First Compliant Resource	Your S3 hardening pattern: CMK, TLS deny, versioning on both buckets, PAB, ACLs off, lifecycle, tags. Drop onto the starter's uploads bucket.
2.4 Modules for Compliance	Module discipline, and the provider-block lesson. Wrap KMS + S3 hardening so dev and prod share one floor.
2.5 IaC as Compliance Evidence	The vault with Object Lock, plus `capture-evidence.sh` and its state-capture guard. The capstone vault IS this vault.
3.3 Writing Rego	Your policy library, the metadata format, the test pattern, and `mutation-test.sh`.
3.4 Conftest + Terraform	`scripts/policy-gate.sh` with the empty-results guard. The pipeline calls it directly.
4.3 GRC Evidence Pipeline	`.github/workflows/grc-gate.yml`, OIDC roles, SHA-pinned actions, branch protection.
4.4 Chain of Custody	Cosign + Object Lock, `verify-evidence.sh` with constrained identity and the completeness check.
5.2 AWS Security Services	CloudTrail with data events, Security Hub, and the Athena queries that make AU-6 claimable.
5.4 GCP Security Services	WIF, if your pipeline touches GCP. The audit-mode-then-enforce rollout answers "without slowing engineering down."
6.1 Introduction to OSCAL	Your component skeleton, the requirements generator, and `verify-oscal-graph.sh`.

If you did the labs as you went, the capstone is one week of design, two weeks of wiring, one week of writing.

If you skipped them, you have 30 days to learn what they teach **and** ship. Doable. Tighter.

Ship the repo. Earn the cert.

Revision history

These notes

- Added Layer 0, the remote state backend. Layer 3's apply-on-merge step is not completable without it.
- Requires the policy suite to be mutation-tested, and requires the gate to fail closed when it loads no policies.
- Requires branch protection with `enforce_admins: true`, and separate IAM roles for plan and apply.
- Requires honest `implementation-status` in the OSCAL, and `verify-oscal-graph.sh` to prove evidence links resolve.

- Cosign verification must constrain the signer identity; a permissive `.*` regex does not demonstrate authenticity.
- Added a fourth common mistake: claiming controls that were collected rather than implemented.

The official labs

Initial release: four layers, no state backend, no mutation-testing or inert-gate requirement.